

Essays, II

Paul Graham

Contents

1	A Version 1.0	1
2	Bradley's Ghost	13
3	It's Charisma, Stupid	17
4	Made in USA	23
5	What You'll Wish You'd Known	29
6	How to Start a Startup	43
7	A Unified Theory of VC Suckage	71
8	Undergraduation	77
9	Writing, Briefly	89
10	Return of the Mac	91
11	Why Smart People Have Bad Ideas	95
12	The Submarine	105
13	Hiring is Obsolete	113
14	What Business Can Learn from Open Source	129
15	After the Ladder	143
16	Inequality and Risk	147
17	What I Did this Summer	157
18	Ideas for Startups	167
19	The Venture Capital Squeeze	179

20	How to Fund a Startup	185
21	Web 2.0	211
22	Good and Bad Procrastination	221
23	How to Do What You Love	227
24	Why YC	241
25	6,631,372	243
26	Are Software Patents Evil?	247
27	See Randomness	261
28	The Hardest Lessons for Startups to Learn	265
29	How to Be Silicon Valley	279
30	Why Startups Condense in America	291
31	The Power of the Marginal	307
32	The Island Test	325
33	Copy What You Like	329
34	How to Present to Investors	333
35	A Student's Guide to Startups	343

Chapter 1

A Version 1.0

October 2004

As E. B. White said, “good writing is rewriting.” I didn’t realize this when I was in school. In writing, as in math and science, they only show you the finished product. You don’t see all the false starts. This gives students a misleading view of how things get made.

Part of the reason it happens is that writers don’t want people to see their mistakes. But I’m willing to let people see an early draft if it will show how much you have to rewrite to beat an essay into shape.

Below is the oldest version I can find of The Age of the Essay (probably the second or third day), with text that ultimately survived in red and text that later got deleted in gray. There seem to be several categories of cuts: things I got wrong, things that seem like bragging, flames, digressions, stretches of awkward prose, and unnecessary words.

I discarded more from the beginning. That’s not surprising; it takes a while to hit your stride. There are more digressions at the start, because I’m not sure where I’m heading.

The amount of cutting is about average. I probably write three to four words for every one that appears in the final version of an essay.

(Before anyone gets mad at me for opinions expressed here, remember that anything you see here that’s not in the final version is obviously

something I chose not to publish, often because I disagree with it.)

Recently a friend said that what he liked about my essays was that they weren't written the way we'd been taught to write essays in school. You remember: topic sentence, introductory paragraph, supporting paragraphs, conclusion. It hadn't occurred to me till then that those horrible things we had to write in school were even connected to what I was doing now. But sure enough, I thought, they did call them "essays," didn't they?

Well, they're not. Those things you have to write in school are not only not essays, they're one of the most pointless of all the pointless hoops you have to jump through in school. And I worry that they not only teach students the wrong things about writing, but put them off writing entirely.

So I'm going to give the other side of the story: what an essay really is, and how you write one. Or at least, how I write one. Students be forewarned: if you actually write the kind of essay I describe, you'll probably get bad grades. But knowing how it's really done should at least help you to understand the feeling of futility you have when you're writing the things they tell you to.

The most obvious difference between real essays and the things one has to write in school is that real essays are not exclusively about English literature. It's a fine thing for schools to teach students how to write. But for some bizarre reason (actually, a very specific bizarre reason that I'll explain in a moment), the teaching of writing has gotten mixed together with the study of literature. And so all over the country, students are writing not about how a baseball team with a small budget might compete with the Yankees, or the role of color in fashion, or what constitutes a good dessert, but about symbolism in Dickens.

With obvious results. Only a few people really care about symbolism in Dickens. The teacher doesn't. The students don't. Most of the people who've had to write PhD dissertations about Dickens don't. And

certainly Dickens himself would be more interested in an essay about color or baseball.

How did things get this way? To answer that we have to go back almost a thousand years. Between about 500 and 1000, life was not very good in Europe. The term “dark ages” is presently out of fashion as too judgemental (the period wasn’t dark; it was just *different*), but if this label didn’t already exist, it would seem an inspired metaphor. What little original thought there was took place in lulls between constant wars and had something of the character of the thoughts of parents with a new baby. The most amusing thing written during this period, Liudprand of Cremona’s Embassy to Constantinople, is, I suspect, mostly inadvertantly so.

Around 1000 Europe began to catch its breath. And once they had the luxury of curiosity, one of the first things they discovered was what we call “the classics.” Imagine if we were visited by aliens. If they could even get here they’d presumably know a few things we don’t. Immediately Alien Studies would become the most dynamic field of scholarship: instead of painstakingly discovering things for ourselves, we could simply suck up everything they’d discovered. So it was in Europe in 1200. When classical texts began to circulate in Europe, they contained not just new answers, but new questions. (If anyone proved a theorem in christian Europe before 1200, for example, there is no record of it.)

For a couple centuries, some of the most important work being done was intellectual archaeology. Those were also the centuries during which schools were first established. And since reading ancient texts was the essence of what scholars did then, it became the basis of the curriculum.

By 1700, someone who wanted to learn about physics didn’t need to start by mastering Greek in order to read Aristotle. But schools change slower than scholarship: the study of ancient texts had such prestige that it remained the backbone of education until the late 19th century. By then it was merely a tradition. It did serve some purposes: reading

a foreign language was difficult, and thus taught discipline, or at least, kept students busy; it introduced students to cultures quite different from their own; and its very uselessness made it function (like white gloves) as a social bulwark. But it certainly wasn't true, and hadn't been true for centuries, that students were serving apprenticeships in the hottest area of scholarship.

Classical scholarship had also changed. In the early era, philology actually mattered. The texts that filtered into Europe were all corrupted to some degree by the errors of translators and copyists. Scholars had to figure out what Aristotle said before they could figure out what he meant. But by the modern era such questions were answered as well as they were ever going to be. And so the study of ancient texts became less about ancientness and more about texts.

The time was then ripe for the question: if the study of ancient texts is a valid field for scholarship, why not modern texts? The answer, of course, is that the *raison d'être* of classical scholarship was a kind of intellectual archaeology that does not need to be done in the case of contemporary authors. But for obvious reasons no one wanted to give that answer. The archaeological work being mostly done, it implied that the people studying the classics were, if not wasting their time, at least working on problems of minor importance.

And so began the study of modern literature. There was some initial resistance, but it didn't last long. The limiting reagent in the growth of university departments is what parents will let undergraduates study. If parents will let their children major in x, the rest follows straightforwardly. There will be jobs teaching x, and professors to fill them. The professors will establish scholarly journals and publish one another's papers. Universities with x departments will subscribe to the journals. Graduate students who want jobs as professors of x will write dissertations about it. It may take a good long while for the more prestigious universities to cave in and establish departments in cheesier xes, but at the other end of the scale there are so many universities competing to attract students that the mere establishment of a discipline requires

little more than the desire to do it.

High schools imitate universities. And so once university English departments were established in the late nineteenth century, the 'riting component of the 3 Rs was morphed into English. With the bizarre consequence that high school students now had to write about English literature— to write, without even realizing it, imitations of whatever English professors had been publishing in their journals a few decades before. It's no wonder if this seems to the student a pointless exercise, because we're now three steps removed from real work: the students are imitating English professors, who are imitating classical scholars, who are merely the inheritors of a tradition growing out of what was, 700 years ago, fascinating and urgently needed work.

Perhaps high schools should drop English and just teach writing. The valuable part of English classes is learning to write, and that could be taught better by itself. Students learn better when they're interested in what they're doing, and it's hard to imagine a topic less interesting than symbolism in Dickens. Most of the people who write about that sort of thing professionally are not really interested in it. (Though indeed, it's been a while since they were writing about symbolism; now they're writing about gender.)

I have no illusions about how eagerly this suggestion will be adopted. Public schools probably couldn't stop teaching English even if they wanted to; they're probably required to by law. But here's a related suggestion that goes with the grain instead of against it: that universities establish a writing major. Many of the students who now major in English would major in writing if they could, and most would be better off.

It will be argued that it is a good thing for students to be exposed to their literary heritage. Certainly. But is that more important than that they learn to write well? And are English classes even the place to do it? After all, the average public high school student gets zero exposure to his artistic heritage. No disaster results. The people who are interested in art learn about it for themselves, and those who aren't

don't. I find that American adults are no better or worse informed about literature than art, despite the fact that they spent years studying literature in high school and no time at all studying art. Which presumably means that what they're taught in school is rounding error compared to what they pick up on their own.

Indeed, English classes may even be harmful. In my case they were effectively aversion therapy. Want to make someone dislike a book? Force him to read it and write an essay about it. And make the topic so intellectually bogus that you could not, if asked, explain why one ought to write about it. I love to read more than anything, but by the end of high school I never read the books we were assigned. I was so disgusted with what we were doing that it became a point of honor with me to write nonsense at least as good as the other students' without having more than glanced over the book to learn the names of the characters and a few random events in it.

I hoped this might be fixed in college, but I found the same problem there. It was not the teachers. It was English. We were supposed to read novels and write essays about them. About what, and why? That no one seemed to be able to explain. Eventually by trial and error I found that what the teacher wanted us to do was pretend that the story had really taken place, and to analyze based on what the characters said and did (the subtler clues, the better) what their motives must have been. One got extra credit for motives having to do with class, as I suspect one must now for those involving gender and sexuality. I learned how to churn out such stuff well enough to get an A, but I never took another English class.

And the books we did these disgusting things to, like those we mishandled in high school, I find still have black marks against them in my mind. The one saving grace was that English courses tend to favor pompous, dull writers like Henry James, who deserve black marks against their names anyway. One of the principles the IRS uses in deciding whether to allow deductions is that, if something is fun, it isn't work. Fields that are intellectually unsure of themselves rely on a

similar principle. Reading P.G. Wodehouse or Evelyn Waugh or Raymond Chandler is too obviously pleasing to seem like serious work, as reading Shakespeare would have been before English evolved enough to make it an effort to understand him. [sh] And so good writers (just you wait and see who's still in print in 300 years) are less likely to have readers turned against them by clumsy, self-appointed tour guides.

The other big difference between a real essay and the things they make you write in school is that a real essay doesn't take a position and then defend it. That principle, like the idea that we ought to be writing about literature, turns out to be another intellectual hangover of long forgotten origins. It's often mistakenly believed that medieval universities were mostly seminaries. In fact they were more law schools. And at least in our tradition lawyers are advocates: they are trained to be able to take either side of an argument and make as good a case for it as they can.

Whether or not this is a good idea (in the case of prosecutors, it probably isn't), it tended to pervade the atmosphere of early universities. After the lecture the most common form of discussion was the disputation. This idea is at least nominally preserved in our present-day thesis defense-- indeed, in the very word thesis. Most people treat the words thesis and dissertation as interchangeable, but originally, at least, a thesis was a position one took and the dissertation was the argument by which one defended it.

I'm not complaining that we blur these two words together. As far as I'm concerned, the sooner we lose the original sense of the word thesis, the better. For many, perhaps most, graduate students, it is stuffing a square peg into a round hole to try to recast one's work as a single thesis. And as for the disputation, that seems clearly a net lose. Arguing two sides of a case may be a necessary evil in a legal dispute, but it's not the best way to get at the truth, as I think lawyers would be the first to admit.

And yet this principle is built into the very structure of the essays they teach you to write in high school. The topic sentence is your thesis,

chosen in advance, the supporting paragraphs the blows you strike in the conflict, and the conclusion—uh, what is the conclusion? I was never sure about that in high school. If your thesis was well expressed, what need was there to restate it? In theory it seemed that the conclusion of a really good essay ought not to need to say any more than QED. But when you understand the origins of this sort of “essay”, you can see where the conclusion comes from. It’s the concluding remarks to the jury.

What other alternative is there? To answer that we have to reach back into history again, though this time not so far. To Michel de Montaigne, inventor of the essay. He was doing something quite different from what a lawyer does, and the difference is embodied in the name. *Essayer* is the French verb meaning “to try” (the cousin of our word *assay*), and an “*essai*” is an effort. An essay is something you write in order to figure something out.

Figure out what? You don’t know yet. And so you can’t begin with a thesis, because you don’t have one, and may never have one. An essay doesn’t begin with a statement, but with a question. In a real essay, you don’t take a position and defend it. You see a door that’s ajar, and you open it and walk in to see what’s inside.

If all you want to do is figure things out, why do you need to write anything, though? Why not just sit and think? Well, there precisely is Montaigne’s great discovery. Expressing ideas helps to form them. Indeed, *helps* is far too weak a word. 90% of what ends up in my essays was stuff I only thought of when I sat down to write them. That’s why I write them.

So there’s another difference between essays and the things you have to write in school. In school you are, in theory, explaining yourself to someone else. In the best case—if you’re really organized—you’re just writing it *down*. In a real essay you’re writing for yourself. You’re thinking out loud.

But not quite. Just as inviting people over forces you to clean up your

apartment, writing something that you know other people will read forces you to think well. So it does matter to have an audience. The things I've written just for myself are no good. Indeed, they're bad in a particular way: they tend to peter out. When I run into difficulties, I notice that I tend to conclude with a few vague questions and then drift off to get a cup of tea.

This seems a common problem. It's practically the standard ending in blog entries—with the addition of a “heh” or an emoticon, prompted by the all too accurate sense that something is missing.

And indeed, a lot of published essays peter out in this same way. Particularly the sort written by the staff writers of newsmagazines. Outside writers tend to supply editorials of the defend-a-position variety, which make a beeline toward a rousing (and foreordained) conclusion. But the staff writers feel obliged to write something more balanced, which in practice ends up meaning blurry. Since they're writing for a popular magazine, they start with the most radioactively controversial questions, from which (because they're writing for a popular magazine) they then proceed to recoil in terror. Gay marriage, for or against? This group says one thing. That group says another. One thing is certain: the question is a complex one. (But don't get mad at us. We didn't draw any conclusions.)

Questions aren't enough. An essay has to come up with answers. They don't always, of course. Sometimes you start with a promising question and get nowhere. But those you don't publish. Those are like experiments that get inconclusive results. Something you publish ought to tell the reader something he didn't already know.

But *what* you tell him doesn't matter, so long as it's interesting. I'm sometimes accused of meandering. In defend-a-position writing that would be a flaw. There you're not concerned with truth. You already know where you're going, and you want to go straight there, blustering through obstacles, and hand-waving your way across swampy ground. But that's not what you're trying to do in an essay. An essay is supposed to be a search for truth. It would be suspicious if it didn't meander.

The Meander is a river in Asia Minor (aka Turkey). As you might expect, it winds all over the place. But does it do this out of frivolity? Quite the opposite. Like all rivers, it's rigorously following the laws of physics. The path it has discovered, winding as it is, represents the most economical route to the sea.

The river's algorithm is simple. At each step, flow down. For the essayist this translates to: flow interesting. Of all the places to go next, choose whichever seems most interesting.

I'm pushing this metaphor a bit. An essayist can't have quite as little foresight as a river. In fact what you do (or what I do) is somewhere between a river and a roman road-builder. I have a general idea of the direction I want to go in, and I choose the next topic with that in mind. This essay is about writing, so I do occasionally yank it back in that direction, but it is not all the sort of essay I thought I was going to write about writing.

Note too that hill-climbing (which is what this algorithm is called) can get you in trouble. Sometimes, just like a river, you run up against a blank wall. What I do then is just what the river does: backtrack. At one point in this essay I found that after following a certain thread I ran out of ideas. I had to go back n paragraphs and start over in another direction. For illustrative purposes I've left the abandoned branch as a footnote.

Err on the side of the river. An essay is not a reference work. It's not something you read looking for a specific answer, and feel cheated if you don't find it. I'd much rather read an essay that went off in an unexpected but interesting direction than one that plodded dutifully along a prescribed course.

So what's interesting? For me, interesting means surprise. Design, as Matz has said, should follow the principle of least surprise. A button that looks like it will make a machine stop should make it stop, not speed up. Essays should do the opposite. Essays should aim for maximum surprise.

I was afraid of flying for a long time and could only travel vicariously. When friends came back from faraway places, it wasn't just out of politeness that I asked them about their trip. I really wanted to know. And I found that the best way to get information out of them was to ask what surprised them. How was the place different from what they expected? This is an extremely useful question. You can ask it of even the most unobservant people, and it will extract information they didn't even know they were recording.

Indeed, you can ask it in real time. Now when I go somewhere new, I make a note of what surprises me about it. Sometimes I even make a conscious effort to visualize the place beforehand, so I'll have a detailed image to diff with reality.

Surprises are facts you didn't already know. But they're more than that. They're facts that contradict things you thought you knew. And so they're the most valuable sort of fact you can get. They're like a food that's not merely healthy, but counteracts the unhealthy effects of things you've already eaten.

How do you find surprises? Well, therein lies half the work of essay writing. (The other half is expressing yourself well.) You can at least use yourself as a proxy for the reader. You should only write about things you've thought about a lot. And anything you come across that surprises you, who've thought about the topic a lot, will probably surprise most readers.

For example, in a recent essay I pointed out that because you can only judge computer programmers by working with them, no one knows in programming who the heroes should be. I certainly didn't realize this when I started writing the essay, and even now I find it kind of weird. That's what you're looking for.

So if you want to write essays, you need two ingredients: you need a few topics that you think about a lot, and you need some ability to ferret out the unexpected.

What should you think about? My guess is that it doesn't matter. Al-

most everything is interesting if you get deeply enough into it. The one possible exception are things like working in fast food, which have deliberately had all the variation sucked out of them. In retrospect, was there anything interesting about working in Baskin-Robbins? Well, it was interesting to notice how important color was to the customers. Kids a certain age would point into the case and say that they wanted yellow. Did they want French Vanilla or Lemon? They would just look at you blankly. They wanted yellow. And then there was the mystery of why the perennial favorite Pralines n' Cream was so appealing. I'm inclined now to think it was the salt. And the mystery of why Passion Fruit tasted so disgusting. People would order it because of the name, and were always disappointed. It should have been called In-sink-erator Fruit. And there was the difference in the way fathers and mothers bought ice cream for their kids. Fathers tended to adopt the attitude of benevolent kings bestowing largesse, and mothers that of harried bureaucrats, giving in to pressure against their better judgement. So, yes, there does seem to be material, even in fast food.

What about the other half, ferreting out the unexpected? That may require some natural ability. I've noticed for a long time that I'm pathologically observant.

[That was as far as I'd gotten at the time.]

[sh] In Shakespeare's own time, serious writing meant theological discourses, not the bawdy plays acted over on the other side of the river among the bear gardens and whorehouses.

The other extreme, the work that seems formidable from the moment it's created (indeed, is deliberately intended to be) is represented by Milton. Like the Aeneid, *Paradise Lost* is a rock imitating a butterfly that happened to get fossilized. Even Samuel Johnson seems to have balked at this, on the one hand paying Milton the compliment of an extensive biography, and on the other writing of *Paradise Lost* that "none who read it ever wished it longer."

Chapter 2

Bradley's Ghost



November 2004

A lot of people are writing now about why Kerry lost. Here I want to examine a more specific question: why were the exit polls so wrong?

In Ohio, which Kerry ultimately lost 49-51, exit polls gave him a 52-48 victory. And this wasn't just random error. In every swing state they overestimated the Kerry vote. In Florida, which Bush ultimately won 52-47, exit polls predicted a dead heat.

(These are not early numbers. They're from about midnight eastern time, long after polls closed in Ohio and Florida. And yet by the next afternoon the exit poll numbers online corresponded to the returns. The only way I can imagine this happening is if those in charge of the exit polls cooked the books after seeing the actual returns. But that's another issue.)

What happened? The source of the problem may be a variant of the Bradley Effect. This term was invented after Tom Bradley, the black mayor of Los Angeles, lost an election for governor of California despite a comfortable lead in the polls. Apparently voters were afraid

to say they planned to vote against him, lest their motives be (perhaps correctly) suspected.

It seems likely that something similar happened in exit polls this year. In theory, exit polls ought to be very accurate. You're not asking people what they would do. You're asking what they just did.

How can you get errors asking that? Because some people don't respond. To get a truly random sample, pollsters ask, say, every 20th person leaving the polling place who they voted for. But not everyone wants to answer. And the pollsters can't simply ignore those who won't, or their sample isn't random anymore. So what they do, apparently, is note down the age and race and sex of the person, and guess from that who they voted for.

This works so long as there is no *correlation* between who people vote for and whether they're willing to talk about it. But this year there may have been. It may be that a significant number of those who voted for Bush didn't want to say so.

Why not? Because people in the US are more conservative than they're willing to admit. The values of the elite in this country, at least at the moment, are NPR values. The average person, as I think both Republicans and Democrats would agree, is more socially conservative. But while some openly flaunt the fact that they don't share the opinions of the elite, others feel a little nervous about it, as if they had bad table manners.

For example, according to current NPR values, you can't say anything that might be perceived as disparaging towards homosexuals. To do so is "homophobic." And yet a large number of Americans are deeply religious, and the Bible is quite explicit on the subject of homosexuality. What are they to do? I think what many do is keep their opinions, but keep them to themselves.

They know what they believe, but they also know what they're supposed to believe. And so when a stranger (for example, a pollster) asks them their opinion about something like gay marriage, they will

not always say what they really think.

When the values of the elite are liberal, polls will tend to underestimate the conservativeness of ordinary voters. This seems to me the leading theory to explain why the exit polls were so far off this year. NPR values said one ought to vote for Kerry. So all the people who voted for Kerry felt virtuous for doing so, and were eager to tell pollsters they had. No one who voted for Kerry did it as an act of quiet defiance.

Chapter 3

It's Charisma, Stupid



November 2004, corrected June 2006

Occam's razor says we should prefer the simpler of two explanations. I begin by reminding readers of this principle because I'm about to propose a theory that will offend both liberals and conservatives. But Occam's razor means, in effect, that if you want to disagree with it, you have a hell of a coincidence to explain.

Theory: In US presidential elections, the more charismatic candidate wins.

People who write about politics, whether on the left or the right, have a consistent bias: they take politics seriously. When one candidate beats another they look for political explanations. The country is shifting to the left, or the right. And that sort of shift can certainly be the result of a presidential election, which makes it easy to believe it was the cause.

But when I think about why I voted for Clinton over the first George

Bush, it wasn't because I was shifting to the left. Clinton just seemed more dynamic. He seemed to want the job more. Bush seemed old and tired. I suspect it was the same for a lot of voters.

Clinton didn't represent any national shift leftward.¹ He was just more charismatic than George Bush or (God help us) Bob Dole. In 2000 we practically got a controlled experiment to prove it: Gore had Clinton's policies, but not his charisma, and he suffered proportionally.

² Same story in 2004. Kerry was smarter and more articulate than Bush, but rather a stiff. And Kerry lost.

As I looked further back, I kept finding the same pattern. Pundits said Carter beat Ford because the country distrusted the Republicans after Watergate. And yet it also happened that Carter was famous for his big grin and folksy ways, and Ford for being a boring klutz. Four years later, pundits said the country had lurched to the right. But Reagan, a former actor, also happened to be even more charismatic than Carter (whose grin was somewhat less cheery after four stressful years in office). In 1984 the charisma gap between Reagan and Mondale was like that between Clinton and Dole, with similar results. The first George Bush managed to win in 1988, though he would later be vanquished by one of the most charismatic presidents ever, because in 1988 he was up against the notoriously uncharismatic Michael Dukakis.

These are the elections I remember personally, but apparently the same pattern played out in 1964 and 1972. The most recent counterexample appears to be 1968, when Nixon beat the more charismatic Hubert Humphrey. But when you examine that election, it tends to support the charisma theory more than contradict it. As Joe

¹As Clinton himself discovered to his surprise when, in one of his first acts as president, he tried to shift the military leftward. After a bruising fight he escaped with a face-saving compromise.

²True, Gore won the popular vote. But politicians know the electoral vote decides the election, so that's what they campaign for. If Bush had been campaigning for the popular vote he would presumably have got more of it. (Thanks to judgmentalist for this point.)

McGinnis recounts in his famous book *The Selling of the President 1968*, Nixon knew he had less charisma than Humphrey, and thus simply refused to debate him on TV. He knew he couldn't afford to let the two of them be seen side by side.

Now a candidate probably couldn't get away with refusing to debate. But in 1968 the custom of televised debates was still evolving. In effect, Nixon won in 1968 because voters were never allowed to see the real Nixon. All they saw were carefully scripted campaign spots.

Oddly enough, the most recent true counterexample is probably 1960. Though this election is usually given as an example of the power of TV, Kennedy apparently would not have won without fraud by party machines in Illinois and Texas. But TV was still young in 1960; only 87% of households had it. ³ Undoubtedly TV helped Kennedy, so historians are correct in regarding this election as a watershed. TV required a new kind of candidate. There would be no more Calvin Coolidges.

The charisma theory may also explain why Democrats tend to lose presidential elections. The core of the Democrats' ideology seems to be a belief in government. Perhaps this tends to attract people who are earnest, but dull. Dukakis, Gore, and Kerry were so similar in that respect that they might have been brothers. Good thing for the Democrats that their screen lets through an occasional Clinton, even if some scandal results. ⁴

One would like to believe elections are won and lost on issues, if only

³Source: Nielsen Media Research. Of the remaining 13%, 11 didn't have TV because they couldn't afford it. I'd argue that the missing 11% were probably also the 11% most susceptible to charisma.

⁴One implication of this theory is that parties shouldn't be too quick to reject candidates with skeletons in their closets. Charismatic candidates will tend to have more skeletons than squeaky clean dullards, but in practice that doesn't seem to lose elections. The current Bush, for example, probably did more drugs in his twenties than any preceding president, and yet managed to get elected with a base of evangelical Christians. All you have to do is say you've reformed, and stonewall about the details.

fake ones like Willie Horton. And yet, if they are, we have a remarkable coincidence to explain. In every presidential election since TV became widespread, the apparently more charismatic candidate has won. Surprising, isn't it, that voters' opinions on the issues have lined up with charisma for 11 elections in a row?

The political commentators who come up with shifts to the left or right in their morning-after analyses are like the financial reporters stuck writing stories day after day about the random fluctuations of the stock market. Day ends, market closes up or down, reporter looks for good or bad news respectively, and writes that the market was up on news of Intel's earnings, or down on fears of instability in the Middle East. Suppose we could somehow feed these reporters false information about market closes, but give them all the other news intact. Does anyone believe they would notice the anomaly, and not simply write that stocks were up (or down) on whatever good (or bad) news there was that day? That they would say, hey, wait a minute, how can stocks be up with all this unrest in the Middle East?

I'm not saying that issues don't matter to voters. Of course they do. But the major parties know so well which issues matter how much to how many voters, and adjust their message so precisely in response, that they tend to split the difference on the issues, leaving the election to be decided by the one factor they can't control: charisma.

If the Democrats had been running a candidate as charismatic as Clinton in the 2004 election, he'd have won. And we'd be reading that the election was a referendum on the war in Iraq, instead of that the Democrats are out of touch with evangelical Christians in middle America.

During the 1992 election, the Clinton campaign staff had a big sign in their office saying "It's the economy, stupid." Perhaps it was even simpler than they thought.

Postscript

Opinions seem to be divided about the charisma theory. Some say it's

impossible, others say it's obvious. This seems a good sign. Perhaps it's in the sweet spot midway between.

As for it being impossible, I reply: here's the data; here's the theory; theory explains data 100%. To a scientist, at least, that means it deserves attention, however implausible it seems.

You can't believe voters are so superficial that they just choose the most charismatic guy? My theory doesn't require that. I'm not proposing that charisma is the only factor, just that it's the only one *left* after the efforts of the two parties cancel one another out.

As for the theory being obvious, as far as I know, no one has proposed it before. Election forecasters are proud when they can achieve the same results with much more complicated models.

Finally, to the people who say that the theory is probably true, but rather depressing: it's not so bad as it seems. The phenomenon is like a pricing anomaly; once people realize it's there, it will disappear. Once both parties realize it's a waste of time to nominate uncharismatic candidates, they'll tend to nominate only the most charismatic ones. And if the candidates are equally charismatic, charisma will cancel out, and elections will be decided on issues, as political commentators like to think they are now.

Thanks to Trevor Blackwell, Maria Daniels, Jessica Livingston, Jackie McDonough, and Robert Morris for reading drafts of this, and to Eric Raymond for pointing out that I was wrong about 1968.

Chapter 4

Made in USA



November 2004

(This is a new essay for the Japanese edition of Hackers & Painters. It tries to explain why Americans make some things well and others badly.)

A few years ago an Italian friend of mine travelled by train from Boston to Providence. She had only been in America for a couple weeks and hadn't seen much of the country yet. She arrived looking astonished. "It's so *ugly*!"

People from other rich countries can scarcely imagine the squalor of the man-made bits of America. In travel books they show you mostly natural environments: the Grand Canyon, whitewater rafting, horses in a field. If you see pictures with man-made things in them, it will be either a view of the New York skyline shot from a discreet distance, or a carefully cropped image of a seacoast town in Maine.

How can it be, visitors must wonder. How can the richest country in the world look like this?

Oddly enough, it may not be a coincidence. Americans are good at some things and bad at others. We're good at making movies and software, and bad at making cars and cities. And I think we may be good at what we're good at for the same reason we're bad at what we're bad at. We're impatient. In America, if you want to do something, you don't worry that it might come out badly, or upset delicate social balances, or that people might think you're getting above yourself. If you want to do something, as Nike says, *just do it*.

This works well in some fields and badly in others. I suspect it works in movies and software because they're both messy processes. "Systematic" is the last word I'd use to describe the way good programmers write software. Code is not something they assemble painstakingly after careful planning, like the pyramids. It's something they plunge into, working fast and constantly changing their minds, like a charcoal sketch.

In software, paradoxical as it sounds, good craftsmanship means working fast. If you work slowly and meticulously, you merely end up with a very fine implementation of your initial, mistaken idea. Working slowly and meticulously is premature optimization. Better to get a prototype done fast, and see what new ideas it gives you.

It sounds like making movies works a lot like making software. Every movie is a Frankenstein, full of imperfections and usually quite different from what was originally envisioned. But interesting, and finished fairly quickly.

I think we get away with this in movies and software because they're both malleable mediums. Boldness pays. And if at the last minute two parts don't quite fit, you can figure out some hack that will at least conceal the problem.

Not so with cars, or cities. They are all too physical. If the car business worked like software or movies, you'd surpass your competitors by

making a car that weighed only fifty pounds, or folded up to the size of a motorcycle when you wanted to park it. But with physical products there are more constraints. You don't win by dramatic innovations so much as by good taste and attention to detail.

The trouble is, the very word "taste" sounds slightly ridiculous to American ears. It seems pretentious, or frivolous, or even effeminate. Blue staters think it's "subjective," and red staters think it's for sissies. So anyone in America who really cares about design will be sailing upwind.

Twenty years ago we used to hear that the problem with the US car industry was the workers. We don't hear that any more now that Japanese companies are building cars in the US. The problem with American cars is bad design. You can see that just by looking at them.

All that extra sheet metal on the AMC Matador wasn't added by the workers. The problem with this car, as with American cars today, is that it was designed by marketing people instead of designers.

Why do the Japanese make better cars than us? Some say it's because their culture encourages cooperation. That may come into it. But in this case it seems more to the point that their culture prizes design and craftsmanship.

For centuries the Japanese have made finer things than we have in the West. When you look at swords they made in 1200, you just can't believe the date on the label is right. Presumably their cars fit together more precisely than ours for the same reason their joinery always has. They're obsessed with making things well.

Not us. When we make something in America, our aim is just to get the job done. Once we reach that point, we take one of two routes. We can stop there, and have something crude but serviceable, like a Vise-grip. Or we can improve it, which usually means encrusting it with gratuitous ornament. When we want to make a car "better," we stick tail fins on it, or make it longer, or make the windows smaller, depending on the current fashion.

Ditto for houses. In America you can have either a flimsy box banged together out of two by fours and drywall, or a McMansion— a flimsy box banged together out of two by fours and drywall, but larger, more dramatic-looking, and full of expensive fittings. Rich people don't get better design or craftsmanship; they just get a larger, more conspicuous version of the standard house.

We don't especially prize design or craftsmanship here. What we like is speed, and we're willing to do something in an ugly way to get it done fast. In some fields, like software or movies, this is a net win.

But it's not just that software and movies are malleable mediums. In those businesses, the designers (though they're not generally called that) have more power. Software companies, at least successful ones, tend to be run by programmers. And in the film industry, though producers may second-guess directors, the director controls most of what appears on the screen. And so American software and movies, and Japanese cars, all have this in common: the people in charge care about design—the former because the designers are in charge, and the latter because the whole culture cares about design.

I think most Japanese executives would be horrified at the idea of making a bad car. Whereas American executives, in their hearts, still believe the most important thing about a car is the image it projects. Make a good car? What's "good?" It's so *subjective*. If you want to know how to design a car, ask a focus group.

Instead of relying on their own internal design compass (like Henry Ford did), American car companies try to make what marketing people think consumers want. But it isn't working. American cars continue to lose market share. And the reason is that the customer doesn't want what he thinks he wants.

Letting focus groups design your cars for you only wins in the short term. In the long term, it pays to bet on good design. The focus group may say they want the meretricious feature du jour, but what they want even more is to imitate sophisticated buyers, and they,

though a small minority, really do care about good design. Eventually the pimps and drug dealers notice that the doctors and lawyers have switched from Cadillac to Lexus, and do the same.

Apple is an interesting counterexample to the general American trend. If you want to buy a nice CD player, you'll probably buy a Japanese one. But if you want to buy an MP3 player, you'll probably buy an iPod. What happened? Why doesn't Sony dominate MP3 players? Because Apple is in the consumer electronics business now, and unlike other American companies, they're obsessed with good design. Or more precisely, their CEO is.

I just got an iPod, and it's not just nice. It's *surprisingly* nice. For it to surprise me, it must be satisfying expectations I didn't know I had. No focus group is going to discover those. Only a great designer can.

Cars aren't the worst thing we make in America. Where the just-do-it model fails most dramatically is in our cities— or rather, exurbs. If real estate developers operated on a large enough scale, if they built whole towns, market forces would compel them to build towns that didn't suck. But they only build a couple office buildings or suburban streets at a time, and the result is so depressing that the inhabitants consider it a great treat to fly to Europe and spend a couple weeks living what is, for people there, just everyday life. ¹

But the just-do-it model does have advantages. It seems the clear winner for generating wealth and technical innovations (which are practically the same thing). I think speed is the reason. It's hard to create wealth by making a commodity. The real value is in things that are new, and if you want to be the first to make something, it helps to work fast. For better or worse, the just-do-it model is fast, whether you're Dan Bricklin writing the prototype of VisiCalc in a weekend, or a real

¹Japanese cities are ugly too, but for different reasons. Japan is prone to earthquakes, so buildings are traditionally seen as temporary; there is no grand tradition of city planning like the one Europeans inherited from Rome. The other cause is the notoriously corrupt relationship between the government and construction companies.

estate developer building a block of shoddy condos in a month.

If I had to choose between the just-do-it model and the careful model, I'd probably choose just-do-it. But do we have to choose? Could we have it both ways? Could Americans have nice places to live without undermining the impatient, individualistic spirit that makes us good at software? Could other countries introduce more individualism into their technology companies and research labs without having it metastasize as strip malls? I'm optimistic. It's harder to say about other countries, but in the US, at least, I think we can have both.

Apple is an encouraging example. They've managed to preserve enough of the impatient, hackerly spirit you need to write software. And yet when you pick up a new Apple laptop, well, it doesn't seem American. It's too perfect. It seems as if it must have been made by a Swedish or a Japanese company.

In many technologies, version 2 has higher resolution. Why not in design generally? I think we'll gradually see national characters superseded by occupational characters: hackers in Japan will be allowed to behave with a willfulness that would now seem unJapanese, and products in America will be designed with an insistence on taste that would now seem unAmerican. Perhaps the most successful countries, in the future, will be those most willing to ignore what are now considered national characters, and do each kind of work in the way that works best. Race you.

Thanks to Trevor Blackwell, Barry Eisler, Sarah Harlin, Shiro Kawai, Jessica Livingston, Jackie McDonough, Robert Morris, and Eric Raymond for reading drafts of this.

Chapter 5

What You'll Wish You'd Known

January 2005

(I wrote this talk for a high school. I never actually gave it, because the school authorities vetoed the plan to invite me.)

When I said I was speaking at a high school, my friends were curious. What will you say to high school students? So I asked them, what do you wish someone had told you in high school? Their answers were remarkably similar. So I'm going to tell you what we all wish someone had told us.

I'll start by telling you something you don't have to know in high school: what you want to do with your life. People are always asking you this, so you think you're supposed to have an answer. But adults ask this mainly as a conversation starter. They want to know what sort of person you are, and this question is just to get you talking. They ask it the way you might poke a hermit crab in a tide pool, to see what it does.

If I were back in high school and someone asked about my plans, I'd say that my first priority was to learn what the options were. You don't need to be in a rush to choose your life's work. What you need to do is discover what you like. You have to work on stuff you like if you want

to be good at what you do.

It might seem that nothing would be easier than deciding what you like, but it turns out to be hard, partly because it's hard to get an accurate picture of most jobs. Being a doctor is not the way it's portrayed on TV. Fortunately you can also watch real doctors, by volunteering in hospitals.¹

But there are other jobs you can't learn about, because no one is doing them yet. Most of the work I've done in the last ten years didn't exist when I was in high school. The world changes fast, and the rate at which it changes is itself speeding up. In such a world it's not a good idea to have fixed plans.

And yet every May, speakers all over the country fire up the Standard Graduation Speech, the theme of which is: don't give up on your dreams. I know what they mean, but this is a bad way to put it, because it implies you're supposed to be bound by some plan you made early on. The computer world has a name for this: premature optimization. And it is synonymous with disaster. These speakers would do better to say simply, don't give up.

What they really mean is, don't get demoralized. Don't think that you can't do what other people can. And I agree you shouldn't underestimate your potential. People who've done great things tend to seem as if they were a race apart. And most biographies only exaggerate this illusion, partly due to the worshipful attitude biographers inevitably sink into, and partly because, knowing how the story ends, they can't help streamlining the plot till it seems like the subject's life was a matter of destiny, the mere unfolding of some innate genius. In fact I suspect if you had the sixteen year old Shakespeare or Einstein in school with you, they'd seem impressive, but not totally unlike your other friends.

¹A doctor friend warns that even this can give an inaccurate picture. "Who knew how much time it would take up, how little autonomy one would have for endless years of training, and how unbelievably annoying it is to carry a beeper?"

Which is an uncomfortable thought. If they were just like us, then they had to work very hard to do what they did. And that's one reason we like to believe in genius. It gives us an excuse for being lazy. If these guys were able to do what they did only because of some magic Shakespeareanness or Einsteininess, then it's not our fault if we can't do something as good.

I'm not saying there's no such thing as genius. But if you're trying to choose between two theories and one gives you an excuse for being lazy, the other one is probably right.

So far we've cut the Standard Graduation Speech down from "don't give up on your dreams" to "what someone else can do, you can do." But it needs to be cut still further. There is *some* variation in natural ability. Most people overestimate its role, but it does exist. If I were talking to a guy four feet tall whose ambition was to play in the NBA, I'd feel pretty stupid saying, you can do anything if you really try.²

We need to cut the Standard Graduation Speech down to, "what someone else with your abilities can do, you can do; and don't underestimate your abilities." But as so often happens, the closer you get to the truth, the messier your sentence gets. We've taken a nice, neat (but wrong) slogan, and churned it up like a mud puddle. It doesn't make a very good speech anymore. But worse still, it doesn't tell you what to do anymore. Someone with your abilities? What are your abilities?

Upwind

I think the solution is to work in the other direction. Instead of working back from a goal, work forward from promising situations. This is what most successful people actually do anyway.

In the graduation-speech approach, you decide where you want to be in twenty years, and then ask: what should I do now to get there? I propose instead that you don't commit to anything in the future, but

²His best bet would probably be to become dictator and intimidate the NBA into letting him play. So far the closest anyone has come is Secretary of Labor.

just look at the options available now, and choose those that will give you the most promising range of options afterward.

It's not so important what you work on, so long as you're not wasting your time. Work on things that interest you and increase your options, and worry later about which you'll take.

Suppose you're a college freshman deciding whether to major in math or economics. Well, math will give you more options: you can go into almost any field from math. If you major in math it will be easy to get into grad school in economics, but if you major in economics it will be hard to get into grad school in math.

Flying a glider is a good metaphor here. Because a glider doesn't have an engine, you can't fly into the wind without losing a lot of altitude. If you let yourself get far downwind of good places to land, your options narrow uncomfortably. As a rule you want to stay upwind. So I propose that as a replacement for "don't give up on your dreams." Stay upwind.

How do you do that, though? Even if math is upwind of economics, how are you supposed to know that as a high school student?

Well, you don't, and that's what you need to find out. Look for smart people and hard problems. Smart people tend to clump together, and if you can find such a clump, it's probably worthwhile to join it. But it's not straightforward to find these, because there is a lot of faking going on.

To a newly arrived undergraduate, all university departments look much the same. The professors all seem forbiddingly intellectual and publish papers unintelligible to outsiders. But while in some fields the papers are unintelligible because they're full of hard ideas, in others they're deliberately written in an obscure way to seem as if they're saying something important. This may seem a scandalous proposition, but it has been experimentally verified, in the famous *Social Text* affair. Suspecting that the papers published by literary theorists were often just intellectual-sounding nonsense, a physicist deliberately wrote a

paper full of intellectual-sounding nonsense, and submitted it to a literary theory journal, which published it.

The best protection is always to be working on hard problems. Writing novels is hard. Reading novels isn't. Hard means worry: if you're not worrying that something you're making will come out badly, or that you won't be able to understand something you're studying, then it isn't hard enough. There has to be suspense.

Well, this seems a grim view of the world, you may think. What I'm telling you is that you should worry? Yes, but it's not as bad as it sounds. It's exhilarating to overcome worries. You don't see faces much happier than people winning gold medals. And you know why they're so happy? Relief.

I'm not saying this is the only way to be happy. Just that some kinds of worry are not as bad as they sound.

Ambition

In practice, "stay upwind" reduces to "work on hard problems." And you can start today. I wish I'd grasped that in high school.

Most people like to be good at what they do. In the so-called real world this need is a powerful force. But high school students rarely benefit from it, because they're given a fake thing to do. When I was in high school, I let myself believe that my job was to be a high school student. And so I let my need to be good at what I did be satisfied by merely doing well in school.

If you'd asked me in high school what the difference was between high school kids and adults, I'd have said it was that adults had to earn a living. Wrong. It's that adults take responsibility for themselves. Making a living is only a small part of it. Far more important is to take intellectual responsibility for oneself.

If I had to go through high school again, I'd treat it like a day job. I don't mean that I'd slack in school. Working at something as a day job doesn't mean doing it badly. It means not being defined by it. I mean

I wouldn't think of myself as a high school student, just as a musician with a day job as a waiter doesn't think of himself as a waiter.³ And when I wasn't working at my day job I'd start trying to do real work.

When I ask people what they regret most about high school, they nearly all say the same thing: that they wasted so much time. If you're wondering what you're doing now that you'll regret most later, that's probably it.⁴

Some people say this is inevitable — that high school students aren't capable of getting anything done yet. But I don't think this is true. And the proof is that you're bored. You probably weren't bored when you were eight. When you're eight it's called "playing" instead of "hanging out," but it's the same thing. And when I was eight, I was rarely bored. Give me a back yard and a few other kids and I could play all day.

The reason this got stale in middle school and high school, I now realize, is that I was ready for something else. Childhood was getting old.

I'm not saying you shouldn't hang out with your friends — that you

³A day job is one you take to pay the bills so you can do what you really want, like play in a band, or invent relativity.

Treating high school as a day job might actually make it easier for some students to get good grades. If you treat your classes as a game, you won't be demoralized if they seem pointless.

However bad your classes, you need to get good grades in them to get into a decent college. And that *is* worth doing, because universities are where a lot of the clumps of smart people are these days.

⁴The second biggest regret was caring so much about unimportant things. And especially about what other people thought of them.

I think what they really mean, in the latter case, is caring what random people thought of them. Adults care just as much what other people think, but they get to be more selective about the other people.

I have about thirty friends whose opinions I care about, and the opinion of the rest of the world barely affects me. The problem in high school is that your peers are chosen for you by accidents of age and geography, rather than by you based on respect for their judgement.

should all become humorless little robots who do nothing but work. Hanging out with friends is like chocolate cake. You enjoy it more if you eat it occasionally than if you eat nothing but chocolate cake for every meal. No matter how much you like chocolate cake, you'll be pretty queasy after the third meal of it. And that's what the malaise one feels in high school is: mental queasiness.⁵

You may be thinking, we have to do more than get good grades. We have to have *extracurricular activities*. But you know perfectly well how bogus most of these are. Collecting donations for a charity is an admirable thing to do, but it's not *hard*. It's not getting something done. What I mean by getting something done is learning how to write well, or how to program computers, or what life was really like in preindustrial societies, or how to draw the human face from life. This sort of thing rarely translates into a line item on a college application.

Corruption

It's dangerous to design your life around getting into college, because the people you have to impress to get into college are not a very discerning audience. At most colleges, it's not the professors who decide whether you get in, but admissions officers, and they are nowhere near as smart. They're the NCOs of the intellectual world. They can't tell how smart you are. The mere existence of prep schools is proof of that.

Few parents would pay so much for their kids to go to a school that didn't improve their admissions prospects. Prep schools openly say this is one of their aims. But what that means, if you stop to think about it, is that they can hack the admissions process: that they can take the

⁵The key to wasting time is distraction. Without distractions it's too obvious to your brain that you're not doing anything with it, and you start to feel uncomfortable. If you want to measure how dependent you've become on distractions, try this experiment: set aside a chunk of time on a weekend and sit alone and think. You can have a notebook to write your thoughts down in, but nothing else: no friends, TV, music, phone, IM, email, Web, games, books, newspapers, or magazines. Within an hour most people will feel a strong craving for distraction.

very same kid and make him seem a more appealing candidate than he would if he went to the local public school.⁶

Right now most of you feel your job in life is to be a promising college applicant. But that means you're designing your life to satisfy a process so mindless that there's a whole industry devoted to subverting it. No wonder you become cynical. The malaise you feel is the same that a producer of reality TV shows or a tobacco industry executive feels. And you don't even get paid a lot.

So what do you do? What you should not do is rebel. That's what I did, and it was a mistake. I didn't realize exactly what was happening to us, but I smelled a major rat. And so I just gave up. Obviously the world sucked, so why bother?

When I discovered that one of our teachers was herself using Cliff's Notes, it seemed par for the course. Surely it meant nothing to get a good grade in such a class.

In retrospect this was stupid. It was like someone getting fouled in a soccer game and saying, hey, you fouled me, that's against the rules, and walking off the field in indignation. Fouls happen. The thing to do when you get fouled is not to lose your cool. Just keep playing.

By putting you in this situation, society has fouled you. Yes, as you suspect, a lot of the stuff you learn in your classes is crap. And yes, as

⁶I don't mean to imply that the only function of prep schools is to trick admissions officers. They also generally provide a better education. But try this thought experiment: suppose prep schools supplied the same superior education but had a tiny (.001) negative effect on college admissions. How many parents would still send their kids to them?

It might also be argued that kids who went to prep schools, because they've learned more, *are* better college candidates. But this seems empirically false. What you learn in even the best high school is rounding error compared to what you learn in college. Public school kids arrive at college with a slight disadvantage, but they start to pull ahead in the sophomore year.

(I'm not saying public school kids are smarter than preppies, just that they are within any given college. That follows necessarily if you agree prep schools improve kids' admissions prospects.)

you suspect, the college admissions process is largely a charade. But like many fouls, this one was unintentional.⁷ So just keep playing.

Rebellion is almost as stupid as obedience. In either case you let yourself be defined by what they tell you to do. The best plan, I think, is to step onto an orthogonal vector. Don't just do what they tell you, and don't just refuse to. Instead treat school as a day job. As day jobs go, it's pretty sweet. You're done at 3 o'clock, and you can even work on your own stuff while you're there.

Curiosity

And what's your real job supposed to be? Unless you're Mozart, your first task is to figure that out. What are the great things to work on? Where are the imaginative people? And most importantly, what are you interested in? The word "aptitude" is misleading, because it implies something innate. The most powerful sort of aptitude is a consuming interest in some question, and such interests are often acquired tastes.

A distorted version of this idea has filtered into popular culture under the name "passion." I recently saw an ad for waiters saying they wanted people with a "passion for service." The real thing is not something one could have for waiting on tables. And passion is a bad word for it. A better name would be curiosity.

Kids are curious, but the curiosity I mean has a different shape from kid curiosity. Kid curiosity is broad and shallow; they ask why at random about everything. In most adults this curiosity dries up entirely. It has to: you can't get anything done if you're always asking why about everything. But in ambitious adults, instead of drying up, curiosity becomes narrow and deep. The mud flat morphs into a well.

⁷Why does society foul you? Indifference, mainly. There are simply no outside forces pushing high school to be good. The air traffic control system works because planes would crash otherwise. Businesses have to deliver because otherwise competitors would take their customers. But no planes crash if your school sucks, and it has no competitors. High school isn't evil; it's random; but random is pretty bad.

Curiosity turns work into play. For Einstein, relativity wasn't a book full of hard stuff he had to learn for an exam. It was a mystery he was trying to solve. So it probably felt like less work to him to invent it than it would seem to someone now to learn it in a class.

One of the most dangerous illusions you get from school is the idea that doing great things requires a lot of discipline. Most subjects are taught in such a boring way that it's only by discipline that you can flog yourself through them. So I was surprised when, early in college, I read a quote by Wittgenstein saying that he had no self-discipline and had never been able to deny himself anything, not even a cup of coffee.

Now I know a number of people who do great work, and it's the same with all of them. They have little discipline. They're all terrible procrastinators and find it almost impossible to make themselves do anything they're not interested in. One still hasn't sent out his half of the thank-you notes from his wedding, four years ago. Another has 26,000 emails in her inbox.

I'm not saying you can get away with zero self-discipline. You probably need about the amount you need to go running. I'm often reluctant to go running, but once I do, I enjoy it. And if I don't run for several days, I feel ill. It's the same with people who do great things. They know they'll feel bad if they don't work, and they have enough discipline to get themselves to their desks to start working. But once they get started, interest takes over, and discipline is no longer necessary.

Do you think Shakespeare was gritting his teeth and diligently trying to write Great Literature? Of course not. He was having fun. That's why he's so good.

If you want to do good work, what you need is a great curiosity about a promising question. The critical moment for Einstein was when he looked at Maxwell's equations and said, what the hell is going on here?

It can take years to zero in on a productive question, because it can take years to figure out what a subject is really about. To take an extreme example, consider math. Most people think they hate math, but the boring stuff you do in school under the name “mathematics” is not at all like what mathematicians do.

The great mathematician G. H. Hardy said he didn’t like math in high school either. He only took it up because he was better at it than the other students. Only later did he realize math was interesting — only later did he start to ask questions instead of merely answering them correctly.

When a friend of mine used to grumble because he had to write a paper for school, his mother would tell him: find a way to make it interesting. That’s what you need to do: find a question that makes the world interesting. People who do great things look at the same world everyone else does, but notice some odd detail that’s compellingly mysterious.

And not only in intellectual matters. Henry Ford’s great question was, why do cars have to be a luxury item? What would happen if you treated them as a commodity? Franz Beckenbauer’s was, in effect, why does everyone have to stay in his position? Why can’t defenders score goals too?

Now

If it takes years to articulate great questions, what do you do now, at sixteen? Work toward finding one. Great questions don’t appear suddenly. They gradually congeal in your head. And what makes them congeal is experience. So the way to find great questions is not to search for them — not to wander about thinking, what great discovery shall I make? You can’t answer that; if you could, you’d have made it.

The way to get a big idea to appear in your head is not to hunt for big ideas, but to put in a lot of time on work that interests you, and in the process keep your mind open enough that a big idea can take roost. Einstein, Ford, and Beckenbauer all used this recipe. They all knew

their work like a piano player knows the keys. So when something seemed amiss to them, they had the confidence to notice it.

Put in time how and on what? Just pick a project that seems interesting: to master some chunk of material, or to make something, or to answer some question. Choose a project that will take less than a month, and make it something you have the means to finish. Do something hard enough to stretch you, but only just, especially at first. If you're deciding between two projects, choose whichever seems most fun. If one blows up in your face, start another. Repeat till, like an internal combustion engine, the process becomes self-sustaining, and each project generates the next one. (This could take years.)

It may be just as well not to do a project "for school," if that will restrict you or make it seem like work. Involve your friends if you want, but not too many, and only if they're not flakes. Friends offer moral support (few startups are started by one person), but secrecy also has its advantages. There's something pleasing about a secret project. And you can take more risks, because no one will know if you fail.

Don't worry if a project doesn't seem to be on the path to some goal you're supposed to have. Paths can bend a lot more than you think. So let the path grow out the project. The most important thing is to be excited about it, because it's by doing that you learn.

Don't disregard unseemly motivations. One of the most powerful is the desire to be better than other people at something. Hardy said that's what got him started, and I think the only unusual thing about him is that he admitted it. Another powerful motivator is the desire to do, or know, things you're not supposed to. Closely related is the desire to do something audacious. Sixteen year olds aren't supposed to write novels. So if you try, anything you achieve is on the plus side of the ledger; if you fail utterly, you're doing no worse than expectations.

8

⁸And then of course there is money. It's not a big factor in high school, because you can't do much that anyone wants. But a lot of great things were created mainly to make money. Samuel Johnson said "no man but a blockhead ever wrote except

Beware of bad models. Especially when they excuse laziness. When I was in high school I used to write “existentialist” short stories like ones I’d seen by famous writers. My stories didn’t have a lot of plot, but they were very deep. And they were less work to write than entertaining ones would have been. I should have known that was a danger sign. And in fact I found my stories pretty boring; what excited me was the idea of writing serious, intellectual stuff like the famous writers.

Now I have enough experience to realize that those famous writers actually sucked. Plenty of famous people do; in the short term, the quality of one’s work is only a small component of fame. I should have been less worried about doing something that seemed cool, and just done something I liked. That’s the actual road to coolness anyway.

A key ingredient in many projects, almost a project on its own, is to find good books. Most books are bad. Nearly all textbooks are bad.⁹ So don’t assume a subject is to be learned from whatever book on it happens to be closest. You have to search actively for the tiny number of good books.

The important thing is to get out there and do stuff. Instead of waiting to be taught, go out and learn.

Your life doesn’t have to be shaped by admissions officers. It could be shaped by your own curiosity. It is for all ambitious adults. And you don’t have to wait to start. In fact, you don’t have to wait to be an adult. There’s no switch inside you that magically flips when you turn a certain age or graduate from some institution. You start being an adult when you decide to take responsibility for your life. You can do

for money.” (Many hope he was exaggerating.)

⁹Even college textbooks are bad. When you get to college, you’ll find that (with a few stellar exceptions) the textbooks are not written by the leading scholars in the field they describe. Writing college textbooks is unpleasant work, done mostly by people who need the money. It’s unpleasant because the publishers exert so much control, and there are few things worse than close supervision by someone who doesn’t understand what you’re doing. This phenomenon is apparently even worse in the production of high school textbooks.

that at any age.¹⁰

This may sound like bullshit. I'm just a minor, you may think, I have no money, I have to live at home, I have to do what adults tell me all day long. Well, most adults labor under restrictions just as cumbersome, and they manage to get things done. If you think it's restrictive being a kid, imagine having kids.

The only real difference between adults and high school kids is that adults realize they need to get things done, and high school kids don't. That realization hits most people around 23. But I'm letting you in on the secret early. So get to work. Maybe you can be the first generation whose greatest regret from high school isn't how much time you wasted.

Thanks to Ingrid Bassett, Trevor Blackwell, Rich Draves, Dan Giffin, Sarah Harlin, Jessica Livingston, Jackie McDonough, Robert Morris, Mark Nitzberg, Lisa Randall, and Aaron Swartz for reading drafts of this, and to many others for talking to me about high school.

¹⁰Your teachers are always telling you to behave like adults. I wonder if they'd like it if you did. You may be loud and disorganized, but you're very docile compared to adults. If you actually started acting like adults, it would be just as if a bunch of adults had been transposed into your bodies. Imagine the reaction of an FBI agent or taxi driver or reporter to being told they had to ask permission to go the bathroom, and only one person could go at a time. To say nothing of the things you're taught. If a bunch of actual adults suddenly found themselves trapped in high school, the first thing they'd do is form a union and renegotiate all the rules with the administration.

Chapter 6

How to Start a Startup

March 2005

(This essay is derived from a talk at the Harvard Computer Society.)

You need three things to create a successful startup: to start with good people, to make something customers actually want, and to spend as little money as possible. Most startups that fail do it because they fail at one of these. A startup that does all three will probably succeed.

And that's kind of exciting, when you think about it, because all three are doable. Hard, but doable. And since a startup that succeeds ordinarily makes its founders rich, that implies getting rich is doable too. Hard, but doable.

If there is one message I'd like to get across about startups, that's it. There is no magically difficult step that requires brilliance to solve.

The Idea

In particular, you don't need a brilliant idea to start a startup around. The way a startup makes money is to offer people better technology than they have now. But what people have now is often so bad that it doesn't take brilliance to do better.

Google's plan, for example, was simply to create a search site that didn't suck. They had three new ideas: index more of the Web, use links to rank search results, and have clean, simple web pages with

unintrusive keyword-based ads. Above all, they were determined to make a site that was good to use. No doubt there are great technical tricks within Google, but the overall plan was straightforward. And while they probably have bigger ambitions now, this alone brings them a billion dollars a year.¹

There are plenty of other areas that are just as backward as search was before Google. I can think of several heuristics for generating ideas for startups, but most reduce to this: look at something people are trying to do, and figure out how to do it in a way that doesn't suck.

For example, dating sites currently suck far worse than search did before Google. They all use the same simple-minded model. They seem to have approached the problem by thinking about how to do database matches instead of how dating works in the real world. An undergrad could build something better as a class project. And yet there's a lot of money at stake. Online dating is a valuable business now, and it might be worth a hundred times as much if it worked.

An idea for a startup, however, is only a beginning. A lot of would-be startup founders think the key to the whole process is the initial idea, and from that point all you have to do is execute. Venture capitalists know better. If you go to VC firms with a brilliant idea that you'll tell them about if they sign a nondisclosure agreement, most will tell you to get lost. That shows how much a mere idea is worth. The market price is less than the inconvenience of signing an NDA.

Another sign of how little the initial idea is worth is the number of startups that change their plan en route. Microsoft's original plan was to make money selling programming languages, of all things. Their current business model didn't occur to them until IBM dropped it in their lap five years later.

Ideas for startups are worth something, certainly, but the trouble is, they're not transferrable. They're not something you could hand to

¹Google's revenues are about two billion a year, but half comes from ads on other sites.

someone else to execute. Their value is mainly as starting points: as questions for the people who had them to continue thinking about.

What matters is not ideas, but the people who have them. Good people can fix bad ideas, but good ideas can't save bad people.

People

What do I mean by good people? One of the best tricks I learned during our startup was a rule for deciding who to hire. Could you describe the person as an animal? It might be hard to translate that into another language, but I think everyone in the US knows what it means. It means someone who takes their work a little too seriously; someone who does what they do so well that they pass right through professional and cross over into obsessive.

What it means specifically depends on the job: a salesperson who just won't take no for an answer; a hacker who will stay up till 4:00 AM rather than go to bed leaving code with a bug in it; a PR person who will cold-call *New York Times* reporters on their cell phones; a graphic designer who feels physical pain when something is two millimeters out of place.

Almost everyone who worked for us was an animal at what they did. The woman in charge of sales was so tenacious that I used to feel sorry for potential customers on the phone with her. You could sense them squirming on the hook, but you knew there would be no rest for them till they'd signed up.

If you think about people you know, you'll find the animal test is easy to apply. Call the person's image to mind and imagine the sentence "so-and-so is an animal." If you laugh, they're not. You don't need or perhaps even want this quality in big companies, but you need it in a startup.

For programmers we had three additional tests. Was the person genuinely smart? If so, could they actually get things done? And finally, since a few good hackers have unbearable personalities, could we

stand to have them around?

That last test filters out surprisingly few people. We could bear any amount of nerdiness if someone was truly smart. What we couldn't stand were people with a lot of attitude. But most of those weren't truly smart, so our third test was largely a restatement of the first.

When nerds are unbearable it's usually because they're trying too hard to seem smart. But the smarter they are, the less pressure they feel to act smart. So as a rule you can recognize genuinely smart people by their ability to say things like "I don't know," "Maybe you're right," and "I don't understand x well enough."

This technique doesn't always work, because people can be influenced by their environment. In the MIT CS department, there seems to be a tradition of acting like a brusque know-it-all. I'm told it derives ultimately from Marvin Minsky, in the same way the classic airline pilot manner is said to derive from Chuck Yeager. Even genuinely smart people start to act this way there, so you have to make allowances.

It helped us to have Robert Morris, who is one of the readiest to say "I don't know" of anyone I've met. (At least, he was before he became a professor at MIT.) No one dared put on attitude around Robert, because he was obviously smarter than they were and yet had zero attitude himself.

Like most startups, ours began with a group of friends, and it was through personal contacts that we got most of the people we hired. This is a crucial difference between startups and big companies. Being friends with someone for even a couple days will tell you more than companies could ever learn in interviews.²

²One advantage startups have over established companies is that there are no discrimination laws about starting businesses. For example, I would be reluctant to start a startup with a woman who had small children, or was likely to have them soon. But you're not allowed to ask prospective employees if they plan to have kids soon. Believe it or not, under current US law, you're not even allowed to discriminate on the basis of intelligence. Whereas when you're starting a company, you can discriminate on any basis you want about who you start it with.

It's no coincidence that startups start around universities, because that's where smart people meet. It's not what people learn in classes at MIT and Stanford that has made technology companies spring up around them. They could sing campfire songs in the classes so long as admissions worked the same.

If you start a startup, there's a good chance it will be with people you know from college or grad school. So in theory you ought to try to make friends with as many smart people as you can in school, right? Well, no. Don't make a conscious effort to schmooze; that doesn't work well with hackers.

What you should do in college is work on your own projects. Hackers should do this even if they don't plan to start startups, because it's the only real way to learn how to program. In some cases you may collaborate with other students, and this is the best way to get to know good hackers. The project may even grow into a startup. But once again, I wouldn't aim too directly at either target. Don't force things; just work on stuff you like with people you like.

Ideally you want between two and four founders. It would be hard to start with just one. One person would find the moral weight of starting a company hard to bear. Even Bill Gates, who seems to be able to bear a good deal of moral weight, had to have a co-founder. But you don't want so many founders that the company starts to look like a group photo. Partly because you don't need a lot of people at first, but mainly because the more founders you have, the worse disagreements you'll have. When there are just two or three founders, you know you have to resolve disputes immediately or perish. If there are seven or eight, disagreements can linger and harden into factions. You don't want mere voting; you need unanimity.

In a technology startup, which most startups are, the founders should include technical people. During the Internet Bubble there were a number of startups founded by business people who then went looking for hackers to create their product for them. This doesn't work well. Business people are bad at deciding what to do with technology,

because they don't know what the options are, or which kinds of problems are hard and which are easy. And when business people try to hire hackers, they can't tell which ones are good. Even other hackers have a hard time doing that. For business people it's roulette.

Do the founders of a startup have to include business people? That depends. We thought so when we started ours, and we asked several people who were said to know about this mysterious thing called "business" if they would be the president. But they all said no, so I had to do it myself. And what I discovered was that business was no great mystery. It's not something like physics or medicine that requires extensive study. You just try to get people to pay you for stuff.

I think the reason I made such a mystery of business was that I was disgusted by the idea of doing it. I wanted to work in the pure, intellectual world of software, not deal with customers' mundane problems. People who don't want to get dragged into some kind of work often develop a protective incompetence at it. Paul Erdos was particularly good at this. By seeming unable even to cut a grapefruit in half (let alone go to the store and buy one), he forced other people to do such things for him, leaving all his time free for math. Erdos was an extreme case, but most husbands use the same trick to some degree.

Once I was forced to discard my protective incompetence, I found that business was neither so hard nor so boring as I feared. There are esoteric areas of business that are quite hard, like tax law or the pricing of derivatives, but you don't need to know about those in a startup. All you need to know about business to run a startup are commonsense things people knew before there were business schools, or even universities.

If you work your way down the Forbes 400 making an x next to the name of each person with an MBA, you'll learn something important about business school. After Warren Buffett, you don't hit another MBA till number 22, Phil Knight, the CEO of Nike. There are only 5 MBAs in the top 50. What you notice in the Forbes 400 are a lot of people with technical backgrounds. Bill Gates, Steve Jobs, Larry

Ellison, Michael Dell, Jeff Bezos, Gordon Moore. The rulers of the technology business tend to come from technology, not business. So if you want to invest two years in something that will help you succeed in business, the evidence suggests you'd do better to learn how to hack than get an MBA.³

There is one reason you might want to include business people in a startup, though: because you have to have at least one person willing and able to focus on what customers want. Some believe only business people can do this— that hackers can implement software, but not design it. That's nonsense. There's nothing about knowing how to program that prevents hackers from understanding users, or about not knowing how to program that magically enables business people to understand them.

If you can't understand users, however, you should either learn how or find a co-founder who can. That is the single most important issue for technology startups, and the rock that sinks more of them than anything else.

What Customers Want

It's not just startups that have to worry about this. I think most businesses that fail do it because they don't give customers what they want. Look at restaurants. A large percentage fail, about a quarter in the first year. But can you think of one restaurant that had really good food and went out of business?

Restaurants with great food seem to prosper no matter what. A restaurant with great food can be expensive, crowded, noisy, dingy, out of the way, and even have bad service, and people will keep coming. It's true that a restaurant with mediocre food can sometimes attract customers through gimmicks. But that approach is very risky. It's more straightforward just to make the food good.

³Learning to hack is a lot cheaper than business school, because you can do it mostly on your own. For the price of a Linux box, a copy of K&R, and a few hours of advice from your neighbor's fifteen year old son, you'll be well on your way.

It's the same with technology. You hear all kinds of reasons why startups fail. But can you think of one that had a massively popular product and still failed?

In nearly every failed startup, the real problem was that customers didn't want the product. For most, the cause of death is listed as "ran out of funding," but that's only the immediate cause. Why couldn't they get more funding? Probably because the product was a dog, or never seemed likely to be done, or both.

When I was trying to think of the things every startup needed to do, I almost included a fourth: get a version 1 out as soon as you can. But I decided not to, because that's implicit in making something customers want. The only way to make something customers want is to get a prototype in front of them and refine it based on their reactions.

The other approach is what I call the "Hail Mary" strategy. You make elaborate plans for a product, hire a team of engineers to develop it (people who do this tend to use the term "engineer" for hackers), and then find after a year that you've spent two million dollars to develop something no one wants. This was not uncommon during the Bubble, especially in companies run by business types, who thought of software development as something terrifying that therefore had to be carefully planned.

We never even considered that approach. As a Lisp hacker, I come from the tradition of rapid prototyping. I would not claim (at least, not here) that this is the right way to write every program, but it's certainly the right way to write software for a startup. In a startup, your initial plans are almost certain to be wrong in some way, and your first priority should be to figure out where. The only way to do that is to try implementing them.

Like most startups, we changed our plan on the fly. At first we expected our customers to be Web consultants. But it turned out they didn't like us, because our software was easy to use and we hosted the site. It would be too easy for clients to fire them. We also thought we'd

be able to sign up a lot of catalog companies, because selling online was a natural extension of their existing business. But in 1996 that was a hard sell. The middle managers we talked to at catalog companies saw the Web not as an opportunity, but as something that meant more work for them.

We did get a few of the more adventurous catalog companies. Among them was Frederick's of Hollywood, which gave us valuable experience dealing with heavy loads on our servers. But most of our users were small, individual merchants who saw the Web as an opportunity to build a business. Some had retail stores, but many only existed online. And so we changed direction to focus on these users. Instead of concentrating on the features Web consultants and catalog companies would want, we worked to make the software easy to use.

I learned something valuable from that. It's worth trying very, very hard to make technology easy to use. Hackers are so used to computers that they have no idea how horrifying software seems to normal people. Stephen Hawking's editor told him that every equation he included in his book would cut sales in half. When you work on making technology easier to use, you're riding that curve up instead of down. A 10% improvement in ease of use doesn't just increase your sales 10%. It's more likely to double your sales.

How do you figure out what customers want? Watch them. One of the best places to do this was at trade shows. Trade shows didn't pay as a way of getting new customers, but they were worth it as market research. We didn't just give canned presentations at trade shows. We used to show people how to build real, working stores. Which meant we got to watch as they used our software, and talk to them about what they needed.

No matter what kind of startup you start, it will probably be a stretch for you, the founders, to understand what users want. The only kind of software you can build without studying users is the sort for which you are the typical user. But this is just the kind that tends to be open source: operating systems, programming languages, editors, and so

on. So if you're developing technology for money, you're probably not going to be developing it for people like you. Indeed, you can use this as a way to generate ideas for startups: what do people who are not like you want from technology?

When most people think of startups, they think of companies like Apple or Google. Everyone knows these, because they're big consumer brands. But for every startup like that, there are twenty more that operate in niche markets or live quietly down in the infrastructure. So if you start a successful startup, odds are you'll start one of those.

Another way to say that is, if you try to start the kind of startup that has to be a big consumer brand, the odds against succeeding are steeper. The best odds are in niche markets. Since startups make money by offering people something better than they had before, the best opportunities are where things suck most. And it would be hard to find a place where things suck more than in corporate IT departments. You would not believe the amount of money companies spend on software, and the crap they get in return. This imbalance equals opportunity.

If you want ideas for startups, one of the most valuable things you could do is find a middle-sized non-technology company and spend a couple weeks just watching what they do with computers. Most good hackers have no more idea of the horrors perpetrated in these places than rich Americans do of what goes on in Brazilian slums.

Start by writing software for smaller companies, because it's easier to sell to them. It's worth so much to sell stuff to big companies that the people selling them the crap they currently use spend a lot of time and money to do it. And while you can outhack Oracle with one frontal lobe tied behind your back, you can't outsell an Oracle salesman. So if you want to win through better technology, aim at smaller customers.

⁴

⁴Corollary: Avoid starting a startup to sell things to the biggest company of all, the government. Yes, there are lots of opportunities to sell them technology. But let someone else start those startups.

They're the more strategically valuable part of the market anyway. In technology, the low end always eats the high end. It's easier to make an inexpensive product more powerful than to make a powerful product cheaper. So the products that start as cheap, simple options tend to gradually grow more powerful till, like water rising in a room, they squash the "high-end" products against the ceiling. Sun did this to mainframes, and Intel is doing it to Sun. Microsoft Word did it to desktop publishing software like Interleaf and Framemaker. Mass-market digital cameras are doing it to the expensive models made for professionals. Avid did it to the manufacturers of specialized video editing systems, and now Apple is doing it to Avid. *Henry Ford* did it to the car makers that preceded him. If you build the simple, inexpensive option, you'll not only find it easier to sell at first, but you'll also be in the best position to conquer the rest of the market.

It's very dangerous to let anyone fly under you. If you have the cheapest, easiest product, you'll own the low end. And if you don't, you're in the crosshairs of whoever does.

Raising Money

To make all this happen, you're going to need money. Some startups have been self-funding— Microsoft for example— but most aren't. I think it's wise to take money from investors. To be self-funding, you have to start as a consulting company, and it's hard to switch from that to a product company.

Financially, a startup is like a pass/fail course. The way to get rich from a startup is to maximize the company's chances of succeeding, not to maximize the amount of stock you retain. So if you can trade stock for something that improves your odds, it's probably a smart move.

To most hackers, getting investors seems like a terrifying and mysterious process. Actually it's merely tedious. I'll try to give an outline of how it works.

The first thing you'll need is a few tens of thousands of dollars to pay

your expenses while you develop a prototype. This is called seed capital. Because so little money is involved, raising seed capital is comparatively easy— at least in the sense of getting a quick yes or no.

Usually you get seed money from individual rich people called “angels.” Often they’re people who themselves got rich from technology. At the seed stage, investors don’t expect you to have an elaborate business plan. Most know that they’re supposed to decide quickly. It’s not unusual to get a check within a week based on a half-page agreement.

We started Viaweb with \$10,000 of seed money from our friend Julian. But he gave us a lot more than money. He’s a former CEO and also a corporate lawyer, so he gave us a lot of valuable advice about business, and also did all the legal work of getting us set up as a company. Plus he introduced us to one of the two angel investors who supplied our next round of funding.

Some angels, especially those with technology backgrounds, may be satisfied with a demo and a verbal description of what you plan to do. But many will want a copy of your business plan, if only to remind themselves what they invested in.

Our angels asked for one, and looking back, I’m amazed how much worry it caused me. “Business plan” has that word “business” in it, so I figured it had to be something I’d have to read a book about business plans to write. Well, it doesn’t. At this stage, all most investors expect is a brief description of what you plan to do and how you’re going to make money from it, and the resumes of the founders. If you just sit down and write out what you’ve been saying to one another, that should be fine. It shouldn’t take more than a couple hours, and you’ll probably find that writing it all down gives you more ideas about what to do.

For the angel to have someone to make the check out to, you’re going to have to have some kind of company. Merely incorporating yourselves isn’t hard. The problem is, for the company to exist, you have to decide who the founders are, and how much stock they each

have. If there are two founders with the same qualifications who are both equally committed to the business, that's easy. But if you have a number of people who are expected to contribute in varying degrees, arranging the proportions of stock can be hard. And once you've done it, it tends to be set in stone.

I have no tricks for dealing with this problem. All I can say is, try hard to do it right. I do have a rule of thumb for recognizing when you have, though. When everyone feels they're getting a slightly bad deal, that they're doing more than they should for the amount of stock they have, the stock is optimally apportioned.

There is more to setting up a company than incorporating it, of course: insurance, business license, unemployment compensation, various things with the IRS. I'm not even sure what the list is, because we, ah, skipped all that. When we got real funding near the end of 1996, we hired a great CFO, who fixed everything retroactively. It turns out that no one comes and arrests you if you don't do everything you're supposed to when starting a company. And a good thing too, or a lot of startups would never get started.⁵

It can be dangerous to delay turning yourself into a company, because one or more of the founders might decide to split off and start another company doing the same thing. This does happen. So when you set up the company, as well as as apportioning the stock, you should get all the founders to sign something agreeing that everyone's ideas belong to this company, and that this company is going to be everyone's only job.

[If this were a movie, ominous music would begin here.]

While you're at it, you should ask what else they've signed. One of the worst things that can happen to a startup is to run into intellectual property problems. We did, and it came closer to killing us than any

⁵A friend who started a company in Germany told me they do care about the paperwork there, and that there's more of it. Which helps explain why there are not more startups in Germany.

competitor ever did.

As we were in the middle of getting bought, we discovered that one of our people had, early on, been bound by an agreement that said all his ideas belonged to the giant company that was paying for him to go to grad school. In theory, that could have meant someone else owned big chunks of our software. So the acquisition came to a screeching halt while we tried to sort this out. The problem was, since we'd been about to be acquired, we'd allowed ourselves to run low on cash. Now we needed to raise more to keep going. But it's hard to raise money with an IP cloud over your head, because investors can't judge how serious it is.

Our existing investors, knowing that we needed money and had nowhere else to get it, at this point attempted certain gambits which I will not describe in detail, except to remind readers that the word "angel" is a metaphor. The founders thereupon proposed to walk away from the company, after giving the investors a brief tutorial on how to administer the servers themselves. And while this was happening, the acquirers used the delay as an excuse to welch on the deal.

Miraculously it all turned out ok. The investors backed down; we did another round of funding at a reasonable valuation; the giant company finally gave us a piece of paper saying they didn't own our software; and six months later we were bought by Yahoo for much more than the earlier acquirer had agreed to pay. So we were happy in the end, though the experience probably took several years off my life.

Don't do what we did. Before you consummate a startup, ask everyone about their previous IP history.

Once you've got a company set up, it may seem presumptuous to go knocking on the doors of rich people and asking them to invest tens of thousands of dollars in something that is really just a bunch of guys with some ideas. But when you look at it from the rich people's point of view, the picture is more encouraging. Most rich people are looking for good investments. If you really think you have a chance of

succeeding, you're doing them a favor by letting them invest. Mixed with any annoyance they might feel about being approached will be the thought: are these guys the next Google?

Usually angels are financially equivalent to founders. They get the same kind of stock and get diluted the same amount in future rounds. How much stock should they get? That depends on how ambitious you feel. When you offer x percent of your company for y dollars, you're implicitly claiming a certain value for the whole company. Venture investments are usually described in terms of that number. If you give an investor new shares equal to 5% of those already outstanding in return for \$100,000, then you've done the deal at a pre-money valuation of \$2 million.

How do you decide what the value of the company should be? There is no rational way. At this stage the company is just a bet. I didn't realize that when we were raising money. Julian thought we ought to value the company at several million dollars. I thought it was preposterous to claim that a couple thousand lines of code, which was all we had at the time, were worth several million dollars. Eventually we settled on one million, because Julian said no one would invest in a company with a valuation any lower.⁶

What I didn't grasp at the time was that the valuation wasn't just the value of the code we'd written so far. It was also the value of our ideas, which turned out to be right, and of all the future work we'd do, which turned out to be a lot.

The next round of funding is the one in which you might deal with actual venture capital firms. But don't wait till you've burned through your last round of funding to start approaching them. VCs are slow to make up their minds. They can take months. You don't want to be running out of money while you're trying to negotiate with them.

⁶At the seed stage our valuation was in principle \$100,000, because Julian got 10% of the company. But this is a very misleading number, because the money was the least important of the things Julian gave us.

Getting money from an actual VC firm is a bigger deal than getting money from angels. The amounts of money involved are larger, millions usually. So the deals take longer, dilute you more, and impose more onerous conditions.

Sometimes the VCs want to install a new CEO of their own choosing. Usually the claim is that you need someone mature and experienced, with a business background. Maybe in some cases this is true. And yet Bill Gates was young and inexperienced and had no business background, and he seems to have done ok. Steve Jobs got booted out of his own company by someone mature and experienced, with a business background, who then proceeded to ruin the company. So I think people who are mature and experienced, with a business background, may be overrated. We used to call these guys “newscasters,” because they had neat hair and spoke in deep, confident voices, and generally didn’t know much more than they read on the teleprompter.

We talked to a number of VCs, but eventually we ended up financing our startup entirely with angel money. The main reason was that we feared a brand-name VC firm would stick us with a newscaster as part of the deal. That might have been ok if he was content to limit himself to talking to the press, but what if he wanted to have a say in running the company? That would have led to disaster, because our software was so complex. We were a company whose whole m.o. was to win through better technology. The strategic decisions were mostly decisions about technology, and we didn’t need any help with those.

This was also one reason we didn’t go public. Back in 1998 our CFO tried to talk me into it. In those days you could go public as a dog-food portal, so as a company with a real product and real revenues, we might have done well. But I feared it would have meant taking on a newscaster— someone who, as they say, “can talk Wall Street’s language.”

I’m happy to see Google is bucking that trend. They didn’t talk Wall Street’s language when they did their IPO, and Wall Street didn’t buy. And now Wall Street is collectively kicking itself. They’ll pay atten-

tion next time. Wall Street learns new languages fast when money is involved.

You have more leverage negotiating with VCs than you realize. The reason is other VCs. I know a number of VCs now, and when you talk to them you realize that it's a seller's market. Even now there is too much money chasing too few good deals.

VCs form a pyramid. At the top are famous ones like Sequoia and Kleiner Perkins, but beneath those are a huge number you've never heard of. What they all have in common is that a dollar from them is worth one dollar. Most VCs will tell you that they don't just provide money, but connections and advice. If you're talking to Vinod Khosla or John Doerr or Mike Moritz, this is true. But such advice and connections can come very expensive. And as you go down the food chain the VCs get rapidly dumber. A few steps down from the top you're basically talking to bankers who've picked up a few new vocabulary words from reading *Wired*. (Does your product use *XML*?) So I'd advise you to be skeptical about claims of experience and connections. Basically, a VC is a source of money. I'd be inclined to go with whoever offered the most money the soonest with the least strings attached.

You may wonder how much to tell VCs. And you should, because some of them may one day be funding your competitors. I think the best plan is not to be overtly secretive, but not to tell them everything either. After all, as most VCs say, they're more interested in the people than the ideas. The main reason they want to talk about your idea is to judge you, not the idea. So as long as you seem like you know what you're doing, you can probably keep a few things back from them.⁷

Talk to as many VCs as you can, even if you don't want their money, because a) they may be on the board of someone who will buy you,

⁷The same goes for companies that seem to want to acquire you. There will be a few that are only pretending to in order to pick your brains. But you can never tell for sure which these are, so the best approach is to seem entirely open, but to fail to mention a few critical technical secrets.

and b) if you seem impressive, they'll be discouraged from investing in your competitors. The most efficient way to reach VCs, especially if you only want them to know about you and don't want their money, is at the conferences that are occasionally organized for startups to present to them.

Not Spending It

When and if you get an infusion of real money from investors, what should you do with it? Not spend it, that's what. In nearly every startup that fails, the proximate cause is running out of money. Usually there is something deeper wrong. But even a proximate cause of death is worth trying hard to avoid.

During the Bubble many startups tried to "get big fast." Ideally this meant getting a lot of customers fast. But it was easy for the meaning to slide over into hiring a lot of people fast.

Of the two versions, the one where you get a lot of customers fast is of course preferable. But even that may be overrated. The idea is to get there first and get all the users, leaving none for competitors. But I think in most businesses the advantages of being first to market are not so overwhelmingly great. Google is again a case in point. When they appeared it seemed as if search was a mature market, dominated by big players who'd spent millions to build their brands: Yahoo, Lycos, Excite, Infoseek, Altavista, Inktomi. Surely 1998 was a little late to arrive at the party.

But as the founders of Google knew, brand is worth next to nothing in the search business. You can come along at any point and make something better, and users will gradually seep over to you. As if to emphasize the point, Google never did any advertising. They're like dealers; they sell the stuff, but they know better than to use it themselves.

The competitors Google buried would have done better to spend those millions improving their software. Future startups should learn from that mistake. Unless you're in a market where products are as

undifferentiated as cigarettes or vodka or laundry detergent, spending a lot on brand advertising is a sign of breakage. And few if any Web businesses are so undifferentiated. The dating sites are running big ad campaigns right now, which is all the more evidence they're ripe for the picking. (Fee, fie, fo, fum, I smell a company run by marketing guys.)

We were compelled by circumstances to grow slowly, and in retrospect it was a good thing. The founders all learned to do every job in the company. As well as writing software, I had to do sales and customer support. At sales I was not very good. I was persistent, but I didn't have the smoothness of a good salesman. My message to potential customers was: you'd be stupid not to sell online, and if you sell online you'd be stupid to use anyone else's software. Both statements were true, but that's not the way to convince people.

I was great at customer support though. Imagine talking to a customer support person who not only knew everything about the product, but would apologize abjectly if there was a bug, and then fix it immediately, while you were on the phone with them. Customers loved us. And we loved them, because when you're growing slow by word of mouth, your first batch of users are the ones who were smart enough to find you by themselves. There is nothing more valuable, in the early stages of a startup, than smart users. If you listen to them, they'll tell you exactly how to make a winning product. And not only will they give you this advice for free, they'll pay you.

We officially launched in early 1996. By the end of that year we had about 70 users. Since this was the era of "get big fast," I worried about how small and obscure we were. But in fact we were doing exactly the right thing. Once you get big (in users or employees) it gets hard to change your product. That year was effectively a laboratory for improving our software. By the end of it, we were so far ahead of our competitors that they never had a hope of catching up. And since all the hackers had spent many hours talking to users, we understood online commerce way better than anyone else.

That's the key to success as a startup. There is nothing more important than understanding your business. You might think that anyone in a business must, *ex officio*, understand it. Far from it. Google's secret weapon was simply that they understood search. I was working for Yahoo when Google appeared, and Yahoo didn't understand search. I know because I once tried to convince the powers that be that we had to make search better, and I got in reply what was then the party line about it: that Yahoo was no longer a mere "search engine." Search was now only a small percentage of our page views, less than one month's growth, and now that we were established as a "media company," or "portal," or whatever we were, search could safely be allowed to wither and drop off, like an umbilical cord.

Well, a small fraction of page views they may be, but they are an important fraction, because they are the page views that Web sessions start with. I think Yahoo gets that now.

Google understands a few other things most Web companies still don't. The most important is that you should put users before advertisers, even though the advertisers are paying and users aren't. One of my favorite bumper stickers reads "if the people lead, the leaders will follow." Paraphrased for the Web, this becomes "get all the users, and the advertisers will follow." More generally, design your product to please users first, and then think about how to make money from it. If you don't put users first, you leave a gap for competitors who do.

To make something users love, you have to understand them. And the bigger you are, the harder that is. So I say "get big slow." The slower you burn through your funding, the more time you have to learn.

The other reason to spend money slowly is to encourage a culture of cheapness. That's something Yahoo did understand. David Filo's title was "Chief Yahoo," but he was proud that his unofficial title was "Cheap Yahoo." Soon after we arrived at Yahoo, we got an email from Filo, who had been crawling around our directory hierarchy, asking if it was really necessary to store so much of our data on expensive RAID drives. I was impressed by that. Yahoo's market cap then was

already in the billions, and they were still worrying about wasting a few gigs of disk space.

When you get a couple million dollars from a VC firm, you tend to feel rich. It's important to realize you're not. A rich company is one with large revenues. This money isn't revenue. It's money investors have given you in the hope you'll be able to generate revenues. So despite those millions in the bank, you're still poor.

For most startups the model should be grad student, not law firm. Aim for cool and cheap, not expensive and impressive. For us the test of whether a startup understood this was whether they had Aeron chairs. The Aeron came out during the Bubble and was very popular with startups. Especially the type, all too common then, that was like a bunch of kids playing house with money supplied by VCs. We had office chairs so cheap that the arms all fell off. This was slightly embarrassing at the time, but in retrospect the grad-studenty atmosphere of our office was another of those things we did right without knowing it.

Our offices were in a wooden triple-decker in Harvard Square. It had been an apartment until about the 1970s, and there was still a claw-footed bathtub in the bathroom. It must once have been inhabited by someone fairly eccentric, because a lot of the chinks in the walls were stuffed with aluminum foil, as if to protect against cosmic rays. When eminent visitors came to see us, we were a bit sheepish about the low production values. But in fact that place was the perfect space for a startup. We felt like our role was to be impudent underdogs instead of corporate stuffed shirts, and that is exactly the spirit you want.

An apartment is also the right kind of place for developing software. Cube farms suck for that, as you've probably discovered if you've tried it. Ever notice how much easier it is to hack at home than at work? So why not make work more like home?

When you're looking for space for a startup, don't feel that it has to look professional. Professional means doing good work, not elevators

and glass walls. I'd advise most startups to avoid corporate space at first and just rent an apartment. You want to live at the office in a startup, so why not have a place designed to be lived in as your office?

Besides being cheaper and better to work in, apartments tend to be in better locations than office buildings. And for a startup location is very important. The key to productivity is for people to come back to work after dinner. Those hours after the phone stops ringing are by far the best for getting work done. Great things happen when a group of employees go out to dinner together, talk over ideas, and then come back to their offices to implement them. So you want to be in a place where there are a lot of restaurants around, not some dreary office park that's a wasteland after 6:00 PM. Once a company shifts over into the model where everyone drives home to the suburbs for dinner, however late, you've lost something extraordinarily valuable. God help you if you actually start in that mode.

If I were going to start a startup today, there are only three places I'd consider doing it: on the Red Line near Central, Harvard, or Davis Squares (Kendall is too sterile); in Palo Alto on University or California Aves; and in Berkeley immediately north or south of campus. These are the only places I know that have the right kind of vibe.

The most important way to not spend money is by not hiring people. I may be an extremist, but I think hiring people is the worst thing a company can do. To start with, people are a recurring expense, which is the worst kind. They also tend to cause you to grow out of your space, and perhaps even move to the sort of uncool office building that will make your software worse. But worst of all, they slow you down: instead of sticking your head in someone's office and checking out an idea with them, eight people have to have a meeting about it. So the fewer people you can hire, the better.

During the Bubble a lot of startups had the opposite policy. They wanted to get "staffed up" as soon as possible, as if you couldn't get anything done unless there was someone with the corresponding job title. That's big company thinking. Don't hire people to fill the gaps

in some a priori org chart. The only reason to hire someone is to do something you'd like to do but can't.

If hiring unnecessary people is expensive and slows you down, why do nearly all companies do it? I think the main reason is that people like the idea of having a lot of people working for them. This weakness often extends right up to the CEO. If you ever end up running a company, you'll find the most common question people ask is how many employees you have. This is their way of weighing you. It's not just random people who ask this; even reporters do. And they're going to be a lot more impressed if the answer is a thousand than if it's ten.

This is ridiculous, really. If two companies have the same revenues, it's the one with fewer employees that's more impressive. When people used to ask me how many people our startup had, and I answered "twenty," I could see them thinking that we didn't count for much. I used to want to add "but our main competitor, whose ass we regularly kick, has a hundred and forty, so can we have credit for the larger of the two numbers?"

As with office space, the number of your employees is a choice between seeming impressive, and being impressive. Any of you who were nerds in high school know about this choice. Keep doing it when you start a company.

Should You?

But should you start a company? Are you the right sort of person to do it? If you are, is it worth it?

More people are the right sort of person to start a startup than realize it. That's the main reason I wrote this. There could be ten times more startups than there are, and that would probably be a good thing.

I was, I now realize, exactly the right sort of person to start a startup. But the idea terrified me at first. I was forced into it because I was a Lisp hacker. The company I'd been consulting for seemed to be running into trouble, and there were not a lot of other companies using

Lisp. Since I couldn't bear the thought of programming in another language (this was 1995, remember, when "another language" meant C++) the only option seemed to be to start a new company using Lisp.

I realize this sounds far-fetched, but if you're a Lisp hacker you'll know what I mean. And if the idea of starting a startup frightened me so much that I only did it out of necessity, there must be a lot of people who would be good at it but who are too intimidated to try.

So who should start a startup? Someone who is a good hacker, between about 23 and 38, and who wants to solve the money problem in one shot instead of getting paid gradually over a conventional working life.

I can't say precisely what a good hacker is. At a first rate university this might include the top half of computer science majors. Though of course you don't have to be a CS major to be a hacker; I was a philosophy major in college.

It's hard to tell whether you're a good hacker, especially when you're young. Fortunately the process of starting startups tends to select them automatically. What drives people to start startups is (or should be) looking at existing technology and thinking, don't these guys realize they should be doing x, y, and z? And that's also a sign that one is a good hacker.

I put the lower bound at 23 not because there's something that doesn't happen to your brain till then, but because you need to see what it's like in an existing business before you try running your own. The business doesn't have to be a startup. I spent a year working for a software company to pay off my college loans. It was the worst year of my adult life, but I learned, without realizing it at the time, a lot of valuable lessons about the software business. In this case they were mostly negative lessons: don't have a lot of meetings; don't have chunks of code that multiple people own; don't have a sales guy running the company; don't make a high-end product; don't let your code get too big; don't leave finding bugs to QA people; don't go too long between

releases; don't isolate developers from users; don't move from Cambridge to Route 128; and so on.⁸ But negative lessons are just as valuable as positive ones. Perhaps even more valuable: it's hard to repeat a brilliant performance, but it's straightforward to avoid errors.⁹

The other reason it's hard to start a company before 23 is that people won't take you seriously. VCs won't trust you, and will try to reduce you to a mascot as a condition of funding. Customers will worry you're going to flake out and leave them stranded. Even you yourself, unless you're very unusual, will feel your age to some degree; you'll find it awkward to be the boss of someone much older than you, and if you're 21, hiring only people younger rather limits your options.

Some people could probably start a company at 18 if they wanted to. Bill Gates was 19 when he and Paul Allen started Microsoft. (Paul Allen was 22, though, and that probably made a difference.) So if you're thinking, I don't care what he says, I'm going to start a company now, you may be the sort of person who could get away with it.

The other cutoff, 38, has a lot more play in it. One reason I put it there is that I don't think many people have the physical stamina much past that age. I used to work till 2:00 or 3:00 AM every night, seven days a week. I don't know if I could do that now.

Also, startups are a big risk financially. If you try something that blows up and leaves you broke at 26, big deal; a lot of 26 year olds are broke. By 38 you can't take so many risks— especially if you have kids.

My final test may be the most restrictive. Do you actually want to start a startup? What it amounts to, economically, is compressing your working life into the smallest possible space. Instead of working at an ordinary rate for 40 years, you work like hell for four. And maybe end

⁸I was as bad an employee as this place was a company. I apologize to anyone who had to work with me there.

⁹You could probably write a book about how to succeed in business by doing everything in exactly the opposite way from the DMV.

up with nothing— though in that case it probably won't take four years.

During this time you'll do little but work, because when you're not working, your competitors will be. My only leisure activities were running, which I needed to do to keep working anyway, and about fifteen minutes of reading a night. I had a girlfriend for a total of two months during that three year period. Every couple weeks I would take a few hours off to visit a used bookshop or go to a friend's house for dinner. I went to visit my family twice. Otherwise I just worked.

Working was often fun, because the people I worked with were some of my best friends. Sometimes it was even technically interesting. But only about 10% of the time. The best I can say for the other 90% is that some of it is funnier in hindsight than it seemed then. Like the time the power went off in Cambridge for about six hours, and we made the mistake of trying to start a gasoline powered generator inside our offices. I won't try that again.

I don't think the amount of bullshit you have to deal with in a startup is more than you'd endure in an ordinary working life. It's probably less, in fact; it just seems like a lot because it's compressed into a short period. So mainly what a startup buys you is time. That's the way to think about it if you're trying to decide whether to start one. If you're the sort of person who would like to solve the money problem once and for all instead of working for a salary for 40 years, then a startup makes sense.

For a lot of people the conflict is between startups and graduate school. Grad students are just the age, and just the sort of people, to start software startups. You may worry that if you do you'll blow your chances of an academic career. But it's possible to be part of a startup and stay in grad school, especially at first. Two of our three original hackers were in grad school the whole time, and both got their degrees. There are few sources of energy so powerful as a procrastinating grad student.

If you do have to leave grad school, in the worst case it won't be for

too long. If a startup fails, it will probably fail quickly enough that you can return to academic life. And if it succeeds, you may find you no longer have such a burning desire to be an assistant professor.

If you want to do it, do it. Starting a startup is not the great mystery it seems from outside. It's not something you have to know about "business" to do. Build something users love, and spend less than you make. How hard is that?

Thanks to Trevor Blackwell, Sarah Harlin, Jessica Livingston, and Robert Morris for reading drafts of this essay, and to Steve Melendez and Gregory Price for inviting me to speak.

Chapter 7

A Unified Theory of VC Suckage

March 2005

A couple months ago I got an email from a recruiter asking if I was interested in being a “technologist in residence” at a new venture capital fund. I think the idea was to play Karl Rove to the VCs’ George Bush.

I considered it for about four seconds. Work for a VC fund? Ick.

One of my most vivid memories from our startup is going to visit Greylock, the famous Boston VCs. They were the most arrogant people I’ve met in my life. And I’ve met a lot of arrogant people.¹

I’m not alone in feeling this way, of course. Even a VC friend of mine dislikes VCs. “Assholes,” he says.

But lately I’ve been learning more about how the VC world works, and a few days ago it hit me that there’s a reason VCs are the way they are. It’s not so much that the business attracts jerks, or even that the power they wield corrupts them. The real problem is the way they’re paid.

¹After Greylock booted founder Philip Greenspun out of ArsDigita, he wrote a hilarious but also very informative essay about it.

The problem with VC funds is that they're *funds*. Like the managers of mutual funds or hedge funds, VCs get paid a percentage of the money they manage: about 2% a year in management fees, plus a percentage of the gains. So they want the fund to be huge—hundreds of millions of dollars, if possible. But that means each partner ends up being responsible for investing a lot of money. And since one person can only manage so many deals, each deal has to be for multiple millions of dollars.

This turns out to explain nearly all the characteristics of VCs that founders hate.

It explains why VCs take so agonizingly long to make up their minds, and why their due diligence feels like a body cavity search.² With so much at stake, they have to be paranoid.

It explains why they steal your ideas. Every founder knows that VCs will tell your secrets to your competitors if they end up investing in them. It's not unheard of for VCs to meet you when they have no intention of funding you, just to pick your brain for a competitor. This prospect makes naive founders clumsily secretive. Experienced founders treat it as a cost of doing business. Either way it sucks. But again, the only reason VCs are so sneaky is the giant deals they do. With so much at stake, they have to be devious.

It explains why VCs tend to interfere in the companies they invest in. They want to be on your board not just so that they can advise you, but so that they can watch you. Often they even install a new CEO.

²Since most VCs aren't tech guys, the technology side of their due diligence tends to be like a body cavity search by someone with a faulty knowledge of human anatomy. After a while we were quite sore from VCs attempting to probe our nonexistent database orifice.

No, we don't use Oracle. We just store the data in files. Our secret is to use an OS that doesn't lose our data. Which OS? FreeBSD. Why do you use that instead of Windows NT? Because it's better and it doesn't cost anything. What, you're using a *freeware* OS?

How many times that conversation was repeated. Then when we got to Yahoo, we found they used FreeBSD and stored their data in files too.

Yes, he may have extensive business experience. But he's also their man: these newly installed CEOs always play something of the role of a political commissar in a Red Army unit. With so much at stake, VCs can't resist micromanaging you.

The huge investments themselves are something founders would dislike, if they realized how damaging they can be. VCs don't invest \$x million because that's the amount you need, but because that's the amount the structure of their business requires them to invest. Like steroids, these sudden huge investments can do more harm than good. Google survived enormous VC funding because it could legitimately absorb large amounts of money. They had to buy a lot of servers and a lot of bandwidth to crawl the whole Web. Less fortunate startups just end up hiring armies of people to sit around having meetings.

In principle you could take a huge VC investment, put it in treasury bills, and continue to operate frugally. You just try it.

And of course giant investments mean giant valuations. They have to, or there's not enough stock left to keep the founders interested. You might think a high valuation is a great thing. Many founders do. But you can't eat paper. You can't benefit from a high valuation unless you can somehow achieve what those in the business call a "liquidity event," and the higher your valuation, the narrower your options for doing that. Many a founder would be happy to sell his company for \$15 million, but VCs who've just invested at a pre-money valuation of \$8 million won't hear of that. You're rolling the dice again, whether you like it or not.

Back in 1997, one of our competitors raised \$20 million in a single round of VC funding. This was at the time more than the valuation of our entire company. Was I worried? Not at all: I was delighted. It was like watching a car you're chasing turn down a street that you know has no outlet.

Their smartest move at that point would have been to take every penny of the \$20 million and use it to buy us. We would have sold. Their

investors would have been furious of course. But I think the main reason they never considered this was that they never imagined we could be had so cheap. They probably assumed we were on the same VC gravy train they were.

In fact we only spent about \$2 million in our entire existence. And that gave us flexibility. We could sell ourselves to Yahoo for \$50 million, and everyone was delighted. If our competitor had done that, the last round of investors would presumably have lost money. I assume they could have vetoed such a deal. But no one those days was paying a lot more than Yahoo. So unless their founders could pull off an IPO (which would be difficult with Yahoo as a competitor), they had no choice but to ride the thing down.

The puffed-up companies that went public during the Bubble didn't do it just because they were pulled into it by unscrupulous investment bankers. Most were pushed just as hard from the other side by VCs who'd invested at high valuations, leaving an IPO as the only way out. The only people dumber were retail investors. So it was literally IPO or bust. Or rather, IPO then bust, or just bust.

Add up all the evidence of VCs' behavior, and the resulting personality is not attractive. In fact, it's the classic villain: alternately cowardly, greedy, sneaky, and overbearing.

I used to take it for granted that VCs were like this. Complaining that VCs were jerks used to seem as naive to me as complaining that users didn't read the reference manual. Of course VCs were jerks. How could it be otherwise?

But I realize now that they're not intrinsically jerks. VCs are like car salesmen or bureaucrats: the nature of their work turns them into jerks.

I've met a few VCs I like. Mike Moritz seems a good guy. He even has a sense of humor, which is almost unheard of among VCs. From what I've read about John Doerr, he sounds like a good guy too, almost a hacker. But they work for the very best VC funds. And my theory

explains why they'd tend to be different: just as the very most popular kids don't have to persecute nerds, the very best VCs don't have to act like VCs. They get the pick of all the best deals. So they don't have to be so paranoid and sneaky, and they can choose those rare companies, like Google, that will actually benefit from the giant sums they're compelled to invest.

VCs often complain that in their business there's too much money chasing too few deals. Few realize that this also describes a flaw in the way funding works at the level of individual firms.

Perhaps this was the sort of strategic insight I was supposed to come up with as a "technologist in residence." If so, the good news is that they're getting it for free. The bad news is it means that if you're not one of the very top funds, you're condemned to be the bad guys.

Chapter 8

Undergraduation



March 2005

(Parts of this essay began as replies to students who wrote to me with questions.)

Recently I've had several emails from computer science undergrads asking what to do in college. I might not be the best source of advice, because I was a philosophy major in college. But I took so many CS classes that most CS majors thought I was one. I was certainly a hacker, at least.

Hacking

What should you do in college to become a good hacker? There are two main things you can do: become very good at programming, and learn a lot about specific, cool problems. These turn out to be equivalent, because each drives you to do the other.

The way to be good at programming is to work (a) a lot (b) on hard problems. And the way to make yourself work on hard problems is to

work on some very engaging project.

Odds are this project won't be a class assignment. My friend Robert learned a lot by writing network software when he was an undergrad. One of his projects was to connect Harvard to the Arpanet; it had been one of the original nodes, but by 1984 the connection had died.

¹ Not only was this work not for a class, but because he spent all his time on it and neglected his studies, he was kicked out of school for a year. ² It all evened out in the end, and now he's a professor at MIT. But you'll probably be happier if you don't go to that extreme; it caused him a lot of worry at the time.

Another way to be good at programming is to find other people who are good at it, and learn what they know. Programmers tend to sort themselves into tribes according to the type of work they do and the tools they use, and some tribes are smarter than others. Look around you and see what the smart people seem to be working on; there's usually a reason.

Some of the smartest people around you are professors. So one way to find interesting work is to volunteer as a research assistant. Professors are especially interested in people who can solve tedious system-administration type problems for them, so that is a way to get a foot in the door. What they fear are flakes and resume padders. It's all too

¹No one seems to have minded, which shows how unimportant the Arpanet (which became the Internet) was as late as 1984.

²This is why, when I became an employer, I didn't care about GPAs. In fact, we actively sought out people who'd failed out of school. We once put up posters around Harvard saying "Did you just get kicked out for doing badly in your classes because you spent all your time working on some project of your own? Come work for us!" We managed to find a kid who had been, and he was a great hacker.

When Harvard kicks undergrads out for a year, they have to get jobs. The idea is to show them how awful the real world is, so they'll understand how lucky they are to be in college. This plan backfired with the guy who came to work for us, because he had more fun than he'd had in school, and made more that year from stock options than any of his professors did in salary. So instead of crawling back repentant at the end of the year, he took another year off and went to Europe. He did eventually graduate at about 26.

common for an assistant to result in a net increase in work. So you have to make it clear you'll mean a net decrease.

Don't be put off if they say no. Rejection is almost always less personal than the rejectee imagines. Just move on to the next. (This applies to dating too.)

Beware, because although most professors are smart, not all of them work on interesting stuff. Professors have to publish novel results to advance their careers, but there is more competition in more interesting areas of research. So what less ambitious professors do is turn out a series of papers whose conclusions are novel because no one else cares about them. You're better off avoiding these.

I never worked as a research assistant, so I feel a bit dishonest recommending that route. I learned to program by writing stuff of my own, particularly by trying to reverse-engineer Winograd's SHRDLU. I was as obsessed with that program as a mother with a new baby.

Whatever the disadvantages of working by yourself, the advantage is that the project is all your own. You never have to compromise or ask anyone's permission, and if you have a new idea you can just sit down and start implementing it.

In your own projects you don't have to worry about novelty (as professors do) or profitability (as businesses do). All that matters is how hard the project is technically, and that has no correlation to the nature of the application. "Serious" applications like databases are often trivial and dull technically (if you ever suffer from insomnia, try reading the technical literature about databases) while "frivolous" applications like games are often very sophisticated. I'm sure there are game companies out there working on products with more intellectual content than the research at the bottom nine tenths of university CS departments.

If I were in college now I'd probably work on graphics: a network game, for example, or a tool for 3D animation. When I was an undergrad there weren't enough cycles around to make graphics interesting,

but it's hard to imagine anything more fun to work on now.

Math

When I was in college, a lot of the professors believed (or at least wished) that computer science was a branch of math. This idea was strongest at Harvard, where there wasn't even a CS major till the 1980s; till then one had to major in applied math. But it was nearly as bad at Cornell. When I told the fearsome Professor Conway that I was interested in AI (a hot topic then), he told me I should major in math. I'm still not sure whether he thought AI required math, or whether he thought AI was nonsense and that majoring in something rigorous would cure me of such stupid ambitions.

In fact, the amount of math you need as a hacker is a lot less than most university departments like to admit. I don't think you need much more than high school math plus a few concepts from the theory of computation. (You have to know what an n^2 algorithm is if you want to avoid writing them.) Unless you're planning to write math applications, of course. Robotics, for example, is all math.

But while you don't literally need math for most kinds of hacking, in the sense of knowing 1001 tricks for differentiating formulas, math is very much worth studying for its own sake. It's a valuable source of metaphors for almost any kind of work.³ I wish I'd studied more math in college for that reason.

Like a lot of people, I was mathematically abused as a child. I learned to think of math as a collection of formulas that were neither beautiful nor had any relation to my life (despite attempts to translate them into "word problems"), but had to be memorized in order to do well on tests.

³Eric Raymond says the best metaphors for hackers are in set theory, combinatorics, and graph theory.

Trevor Blackwell reminds you to take math classes intended for math majors. "Math for engineers' classes sucked mightily. In fact any 'x for engineers' sucks, where x includes math, law, writing and visual design."

One of the most valuable things you could do in college would be to learn what math is really about. This may not be easy, because a lot of good mathematicians are bad teachers. And while there are many popular books on math, few seem good. The best I can think of are W. W. Sawyer's. And of course Euclid.⁴

Everything

Thomas Huxley said "Try to learn something about everything and everything about something." Most universities aim at this ideal.

But what's everything? To me it means, all that people learn in the course of working honestly on hard problems. All such work tends to be related, in that ideas and techniques from one field can often be transplanted successfully to others. Even others that seem quite distant. For example, I write essays the same way I write software: I sit down and blow out a lame version 1 as fast as I can type, then spend several weeks rewriting it.

Working on hard problems is not, by itself, enough. Medieval alchemists were working on a hard problem, but their approach was so bogus that there was little to learn from studying it, except possibly about people's ability to delude themselves. Unfortunately the sort of AI I was trying to learn in college had the same flaw: a very hard problem, blithely approached with hopelessly inadequate techniques. Bold? Closer to fraudulent.

The social sciences are also fairly bogus, because they're so much influenced by intellectual fashions. If a physicist met a colleague from 100 years ago, he could teach him some new things; if a psychologist met a colleague from 100 years ago, they'd just get into an ideological argument. Yes, of course, you'll learn something by taking a psychology class. The point is, you'll learn more by taking a class in another department.

⁴Other highly recommended books: *What is Mathematics?*, by Courant and Robbins; *Geometry and the Imagination* by Hilbert and Cohn-Vossen. And for those interested in graphic design, Byrne's Euclid.

The worthwhile departments, in my opinion, are math, the hard sciences, engineering, history (especially economic and social history, and the history of science), architecture, and the classics. A survey course in art history may be worthwhile. Modern literature is important, but the way to learn about it is just to read. I don't know enough about music to say.

You can skip the social sciences, philosophy, and the various departments created recently in response to political pressures. Many of these fields talk about important problems, certainly. But the way they talk about them is useless. For example, philosophy talks, among other things, about our obligations to one another; but you can learn more about this from a wise grandmother or E. B. White than from an academic philosopher.

I speak here from experience. I should probably have been offended when people laughed at Clinton for saying "It depends on what the meaning of the word 'is' is." I took about five classes in college on what the meaning of "is" is.

Another way to figure out which fields are worth studying is to create the *dropout graph*. For example, I know many people who switched from math to computer science because they found math too hard, and no one who did the opposite. People don't do hard things gratuitously; no one will work on a harder problem unless it is proportionately (or at least $\log(n)$) more rewarding. So probably math is more worth studying than computer science. By similar comparisons you can make a graph of all the departments in a university. At the bottom you'll find the subjects with least intellectual content.

If you use this method, you'll get roughly the same answer I just gave.

Language courses are an anomaly. I think they're better considered as extracurricular activities, like pottery classes. They'd be far more useful when combined with some time living in a country where the language is spoken. On a whim I studied Arabic as a freshman. It was a lot of work, and the only lasting benefits were a weird ability

to identify semitic roots and some insights into how people recognize words.

Studio art and creative writing courses are wildcards. Usually you don't get taught much: you just work (or don't work) on whatever you want, and then sit around offering "crits" of one another's creations under the vague supervision of the teacher. But writing and art are both very hard problems that (some) people work honestly at, so they're worth doing, especially if you can find a good teacher.

Jobs

Of course college students have to think about more than just learning. There are also two practical problems to consider: jobs, and graduate school.

In theory a liberal education is not supposed to supply job training. But everyone knows this is a bit of a fib. Hackers at every college learn practical skills, and not by accident.

What you should learn to get a job depends on the kind you want. If you want to work in a big company, learn how to hack Blub on Windows. If you want to work at a cool little company or research lab, you'll do better to learn Ruby on Linux. And if you want to start your own company, which I think will be more and more common, master the most powerful tools you can find, because you're going to be in a race against your competitors, and they'll be your horse.

There is not a direct correlation between the skills you should learn in college and those you'll use in a job. You should aim slightly high in college.

In workouts a football player may bench press 300 pounds, even though he may never have to exert anything like that much force in the course of a game. Likewise, if your professors try to make you learn stuff that's more advanced than you'll need in a job, it may not just be because they're academics, detached from the real world. They may be trying to make you lift weights with your brain.

The programs you write in classes differ in three critical ways from the ones you'll write in the real world: they're small; you get to start from scratch; and the problem is usually artificial and predetermined. In the real world, programs are bigger, tend to involve existing code, and often require you to figure out what the problem is before you can solve it.

You don't have to wait to leave (or even enter) college to learn these skills. If you want to learn how to deal with existing code, for example, you can contribute to open-source projects. The sort of employer you want to work for will be as impressed by that as good grades on class assignments.

In existing open-source projects you don't get much practice at the third skill, deciding what problems to solve. But there's nothing to stop you starting new projects of your own. And good employers will be even more impressed with that.

What sort of problem should you try to solve? One way to answer that is to ask what you need as a user. For example, I stumbled on a good algorithm for spam filtering because I wanted to stop getting spam. Now what I wish I had was a mail reader that somehow prevented my inbox from filling up. I tend to use my inbox as a todo list. But that's like using a screwdriver to open bottles; what one really wants is a bottle opener.

Grad School

What about grad school? Should you go? And how do you get into a good one?

In principle, grad school is professional training in research, and you shouldn't go unless you want to do research as a career. And yet half the people who get PhDs in CS don't go into research. I didn't go to grad school to become a professor. I went because I wanted to learn more.

So if you're mainly interested in hacking and you go to grad school,

you'll find a lot of other people who are similarly out of their element. And if half the people around you are out of their element in the same way you are, are you really out of your element?

There's a fundamental problem in "computer science," and it surfaces in situations like this. No one is sure what "research" is supposed to be. A lot of research is hacking that had to be crammed into the form of an academic paper to yield one more quantum of publication.

So it's kind of misleading to ask whether you'll be at home in grad school, because very few people are quite at home in computer science. The whole field is uncomfortable in its own skin. So the fact that you're mainly interested in hacking shouldn't deter you from going to grad school. Just be warned you'll have to do a lot of stuff you don't like.

Number one will be your dissertation. Almost everyone hates their dissertation by the time they're done with it. The process inherently tends to produce an unpleasant result, like a cake made out of whole wheat flour and baked for twelve hours. Few dissertations are read with pleasure, especially by their authors.

But thousands before you have suffered through writing a dissertation. And aside from that, grad school is close to paradise. Many people remember it as the happiest time of their lives. And nearly all the rest, including me, remember it as a period that would have been, if they hadn't had to write a dissertation.⁵

The danger with grad school is that you don't see the scary part up-front. PhD programs start out as college part 2, with several years of classes. So by the time you face the horror of writing a dissertation, you're already several years in. If you quit now, you'll be a grad-school dropout, and you probably won't like that idea. When Robert

⁵If you wanted to have the perfect life, the thing to do would be to go to grad school, secretly write your dissertation in the first year or two, and then just enjoy yourself for the next three years, dribbling out a chapter at a time. This prospect will make grad students' mouths water, but I know of no one who's had the discipline to pull it off.

got kicked out of grad school for writing the Internet worm of 1988, I envied him enormously for finding a way out without the stigma of failure.

On the whole, grad school is probably better than most alternatives. You meet a lot of smart people, and your glum procrastination will at least be a powerful common bond. And of course you have a PhD at the end. I forgot about that. I suppose that's worth something.

The greatest advantage of a PhD (besides being the union card of academia, of course) may be that it gives you some baseline confidence. For example, the Honeywell thermostats in my house have the most atrocious UI. My mother, who has the same model, diligently spent a day reading the user's manual to learn how to operate hers. She assumed the problem was with her. But I can think to myself "If someone with a PhD in computer science can't understand this thermostat, it *must* be badly designed."

If you still want to go to grad school after this equivocal recommendation, I can give you solid advice about how to get in. A lot of my friends are CS professors now, so I have the inside story about admissions. It's quite different from college. At most colleges, admissions officers decide who gets in. For PhD programs, the professors do. And they try to do it well, because the people they admit are going to be working for them.

Apparently only recommendations really matter at the best schools. Standardized tests count for nothing, and grades for little. The essay is mostly an opportunity to disqualify yourself by saying something stupid. The only thing professors trust is recommendations, preferably from people they know.⁶

⁶One professor friend says that 15-20% of the grad students they admit each year are "long shots." But what he means by long shots are people whose applications are perfect in every way, except that no one on the admissions committee knows the professors who wrote the recommendations.

So if you want to get into grad school in the sciences, you need to go to college somewhere with real research professors. Otherwise you'll seem a risky bet to ad-

So if you want to get into a PhD program, the key is to impress your professors. And from my friends who are professors I know what impresses them: not merely trying to impress them. They're not impressed by students who get good grades or want to be their research assistants so they can get into grad school. They're impressed by students who get good grades and want to be their research assistants because they're genuinely interested in the topic.

So the best thing you can do in college, whether you want to get into grad school or just be good at hacking, is figure out what you truly like. It's hard to trick professors into letting you into grad school, and impossible to trick problems into letting you solve them. College is where faking stops working. From this point, unless you want to go work for a big company, which is like reverting to high school, the only way forward is through doing what you love.

Thanks to Trevor Blackwell, Alex Lewin, Jessica Livingston, Robert Morris, Eric Raymond, and several anonymous CS professors for reading drafts of this, and to the students whose questions began it.

missions committees, no matter how good you are.

Which implies a surprising but apparently inevitable consequence: little liberal arts colleges are doomed. Most smart high school kids at least consider going into the sciences, even if they ultimately choose not to. Why go to a college that limits their options?

Chapter 9

Writing, Briefly

March 2005

(In the process of answering an email, I accidentally wrote a tiny essay about writing. I usually spend weeks on an essay. This one took 67 minutes—23 of writing, and 44 of rewriting.)

I think it's far more important to write well than most people realize. Writing doesn't just communicate ideas; it generates them. If you're bad at writing and don't like to do it, you'll miss out on most of the ideas writing would have generated.

As for how to write well, here's the short version: Write a bad version 1 as fast as you can; rewrite it over and over; cut out everything unnecessary; write in a conversational tone; develop a nose for bad writing, so you can see and fix it in yours; imitate writers you like; if you can't get started, tell someone what you plan to write about, then write down what you said; expect 80% of the ideas in an essay to happen after you start writing it, and 50% of those you start with to be wrong; be confident enough to cut; have friends you trust read your stuff and tell you which bits are confusing or drag; don't (always) make detailed outlines; mull ideas over for a few days before writing; carry a small notebook or scrap paper with you; start writing when you think of the first sentence; if a deadline forces you to start before that, just say the most important sentence first; write about stuff you like; don't

try to sound impressive; don't hesitate to change the topic on the fly; use footnotes to contain digressions; use anaphora to knit sentences together; read your essays out loud to see (a) where you stumble over awkward phrases and (b) which bits are boring (the paragraphs you dread reading); try to tell the reader something new and useful; work in fairly big quanta of time; when you restart, begin by rereading what you have so far; when you finish, leave yourself something easy to start with; accumulate notes for topics you plan to cover at the bottom of the file; don't feel obliged to cover any of them; write for a reader who won't read the essay as carefully as you do, just as pop songs are designed to sound ok on crappy car radios; if you say anything mistaken, fix it immediately; ask friends which sentence you'll regret most; go back and tone down harsh remarks; publish stuff online, because an audience makes you write more, and thus generate more ideas; print out drafts instead of just looking at them on the screen; use simple, germanic words; learn to distinguish surprises from digressions; learn to recognize the approach of an ending, and when one appears, grab it.

Chapter 10

Return of the Mac

March 2005

All the best hackers I know are gradually switching to Macs. My friend Robert said his whole research group at MIT recently bought themselves Powerbooks. These guys are not the graphic designers and grandmas who were buying Macs at Apple's low point in the mid 1990s. They're about as hardcore OS hackers as you can get.

The reason, of course, is OS X. Powerbooks are beautifully designed and run FreeBSD. What more do you need to know?

I got a Powerbook at the end of last year. When my IBM Thinkpad's hard disk died soon after, it became my only laptop. And when my friend Trevor showed up at my house recently, he was carrying a Powerbook identical to mine.

For most of us, it's not a switch to Apple, but a return. Hard as this was to believe in the mid 90s, the Mac was in its time the canonical hacker's computer.

In the fall of 1983, the professor in one of my college CS classes got up and announced, like a prophet, that there would soon be a computer with half a MIPS of processing power that would fit under an airline seat and cost so little that we could save enough to buy one from a summer job. The whole room gasped. And when the Mac appeared, it was even better than we'd hoped. It was small and powerful and

cheap, as promised. But it was also something we'd never considered a computer could be: fabulously well designed.

I had to have one. And I wasn't alone. In the mid to late 1980s, all the hackers I knew were either writing software for the Mac, or wanted to. Every futon sofa in Cambridge seemed to have the same fat white book lying open on it. If you turned it over, it said "Inside Macintosh."

Then came Linux and FreeBSD, and hackers, who follow the most powerful OS wherever it leads, found themselves switching to Intel boxes. If you cared about design, you could buy a Thinkpad, which was at least not actively repellent, if you could get the Intel and Microsoft stickers off the front.¹

With OS X, the hackers are back. When I walked into the Apple store in Cambridge, it was like coming home. Much was changed, but there was still that Apple coolness in the air, that feeling that the show was being run by someone who really cared, instead of random corporate deal-makers.

So what, the business world may say. Who cares if hackers like Apple again? How big is the hacker market, after all?

Quite small, but important out of proportion to its size. When it comes to computers, what hackers are doing now, everyone will be doing in ten years. Almost all technology, from Unix to bitmapped displays to the Web, became popular first within CS departments and research labs, and gradually spread to the rest of the world.

I remember telling my father back in 1986 that there was a new kind of computer called a Sun that was a serious Unix machine, but so small and cheap that you could have one of your own to sit in front of, instead of sitting in front of a VT100 connected to a single central Vax. Maybe, I suggested, he should buy some stock in this company. I think he really wishes he'd listened.

¹These horrible stickers are much like the intrusive ads popular on pre-Google search engines. They say to the customer: you are unimportant. We care about Intel and Microsoft, not you.

In 1994 my friend Koling wanted to talk to his girlfriend in Taiwan, and to save long-distance bills he wrote some software that would convert sound to data packets that could be sent over the Internet. We weren't sure at the time whether this was a proper use of the Internet, which was still then a quasi-government entity. What he was doing is now called VoIP, and it is a huge and rapidly growing business.

If you want to know what ordinary people will be doing with computers in ten years, just walk around the CS department at a good university. Whatever they're doing, you'll be doing.

In the matter of "platforms" this tendency is even more pronounced, because novel software originates with great hackers, and they tend to write it first for whatever computer they personally use. And software sells hardware. Many if not most of the initial sales of the Apple II came from people who bought one to run VisiCalc. And why did Bricklin and Frankston write VisiCalc for the Apple II? Because they personally liked it. They could have chosen any machine to make into a star.

If you want to attract hackers to write software that will sell your hardware, you have to make it something that they themselves use. It's not enough to make it "open." It has to be open and good.

And open and good is what Macs are again, finally. The intervening years have created a situation that is, as far as I know, without precedent: Apple is popular at the low end and the high end, but not in the middle. My seventy year old mother has a Mac laptop. My friends with PhDs in computer science have Mac laptops.² And yet Apple's overall market share is still small.

Though unprecedented, I predict this situation is also temporary.

So Dad, there's this company called Apple. They make a new kind of computer that's as well designed as a Bang & Olufsen stereo system,

²Y Combinator is (we hope) visited mostly by hackers. The proportions of OSes are: Windows 66.4%, Macintosh 18.8%, Linux 11.4%, and FreeBSD 1.5%. The Mac number is a big change from what it would have been five years ago.

and underneath is the best Unix machine you can buy. Yes, the price to earnings ratio is kind of high, but I think a lot of people are going to want these.

Chapter 11

Why Smart People Have Bad Ideas

April 2005

This summer, as an experiment, some friends and I are giving seed funding to a bunch of new startups. It's an experiment because we're prepared to fund younger founders than most investors would. That's why we're doing it during the summer—so even college students can participate.

We know from Google and Yahoo that grad students can start successful startups. And we know from experience that some undergrads are as capable as most grad students. The accepted age for startup founders has been creeping downward. We're trying to find the lower bound.

The deadline has now passed, and we're sifting through 227 applications. We expected to divide them into two categories, promising and unpromising. But we soon saw we needed a third: promising people with unpromising ideas.¹

The Artix Phase

¹SFP applicants: please don't assume that not being accepted means we think your idea is bad. Because we want to keep the number of startups small this first summer, we're going to have to turn down some good proposals too.

We should have expected this. It's very common for a group of founders to go through one lame idea before realizing that a startup has to make something people will pay for. In fact, we ourselves did.

Viaweb wasn't the first startup Robert Morris and I started. In January 1995, we and a couple friends started a company called Artix. The plan was to put art galleries on the Web. In retrospect, I wonder how we could have wasted our time on anything so stupid. Galleries are not especially excited about being on the Web even now, ten years later. They don't want to have their stock visible to any random visitor, like an antique store.²

Besides which, art dealers are the most technophobic people on earth. They didn't become art dealers after a difficult choice between that and a career in the hard sciences. Most of them had never seen the Web before we came to tell them why they should be on it. Some didn't even have computers. It doesn't do justice to the situation to describe it as a hard *sell*; we soon sank to building sites for free, and it was hard to convince galleries even to do that.

Gradually it dawned on us that instead of trying to make Web sites for people who didn't want them, we could make sites for people who did. In fact, software that would let people who wanted sites make their own. So we ditched Artix and started a new company, Viaweb, to make software for building online stores. That one succeeded.

We're in good company here. Microsoft was not the first company Paul Allen and Bill Gates started either. The first was called Traf-o-data. It does not seem to have done as well as Micro-soft.

In Robert's defense, he was skeptical about Artix. I dragged him into it.³ But there were moments when he was optimistic. And if we, who

²Dealers try to give each customer the impression that the stuff they're showing him is something special that only a few people have seen, when in fact it may have been sitting in their racks for years while they tried to unload it on buyer after buyer.

³On the other hand, he was skeptical about Viaweb too. I have a precise measure of that, because at one point in the first couple months we made a bet: if he ever made a million dollars out of Viaweb, he'd get his ear pierced. We didn't let him off,

were 29 and 30 at the time, could get excited about such a thoroughly boneheaded idea, we should not be surprised that hackers aged 21 or 22 are pitching us ideas with little hope of making money.

The Still Life Effect

Why does this happen? Why do good hackers have bad business ideas?

Let's look at our case. One reason we had such a lame idea was that it was the first thing we thought of. I was in New York trying to be a starving artist at the time (the starving part is actually quite easy), so I was haunting galleries anyway. When I learned about the Web, it seemed natural to mix the two. Make Web sites for galleries—that's the ticket!

If you're going to spend years working on something, you'd think it might be wise to spend at least a couple days considering different ideas, instead of going with the first that comes into your head. You'd think. But people don't. In fact, this is a constant problem when you're painting still lifes. You plonk down a bunch of stuff on a table, and maybe spend five or ten minutes rearranging it to look interesting. But you're so impatient to get started painting that ten minutes of rearranging feels very long. So you start painting. Three days later, having spent twenty hours staring at it, you're kicking yourself for having set up such an awkward and boring composition, but by then it's too late.

Part of the problem is that big projects tend to grow out of small ones. You set up a still life to make a quick sketch when you have a spare hour, and days later you're still working on it. I once spent a month painting three versions of a still life I set up in about four minutes. At each point (a day, a week, a month) I thought I'd already put in so much time that it was too late to change.

So the biggest cause of bad ideas is the still life effect: you come up with a random idea, plunge into it, and then at each point (a day, a

either.

week, a month) feel you've put so much time into it that this must be *the* idea.

How do we fix that? I don't think we should discard plunging. Plunging into an idea is a good thing. The solution is at the other end: to realize that having invested time in something doesn't make it good.

This is clearest in the case of names. Viaweb was originally called Webgen, but we discovered someone else had a product called that. We were so attached to our name that we offered him *5% of the company* if he'd let us have it. But he wouldn't, so we had to think of another.⁴ The best we could do was Viaweb, which we disliked at first. It was like having a new mother. But within three days we loved it, and Webgen sounded lame and old-fashioned.

If it's hard to change something so simple as a name, imagine how hard it is to garbage-collect an idea. A name only has one point of attachment into your head. An idea for a company gets woven into your thoughts. So you must consciously discount for that. Plunge in, by all means, but remember later to look at your idea in the harsh light of morning and ask: is this something people will pay for? Is this, of all the things we could make, the thing people will pay most for?

Muck

The second mistake we made with Artix is also very common. Putting galleries on the Web seemed cool.

One of the most valuable things my father taught me is an old Yorkshire saying: where there's muck, there's brass. Meaning that unpleasant work pays. And more to the point here, vice versa. Work people like doesn't pay well, for reasons of supply and demand. The most extreme case is developing programming languages, which doesn't pay at all, because people like it so much they do it for free.

⁴I wrote a program to generate all the combinations of "Web" plus a three letter word. I learned from this that most three letter words are bad: Webpig, Webdog, Webfat, Webzit, Webfug. But one of them was Webvia; I swapped them to make Viaweb.

When we started Artix, I was still ambivalent about business. I wanted to keep one foot in the art world. Big, big, mistake. Going into business is like a hang-glider launch: you'd better do it wholeheartedly, or not at all. The purpose of a company, and a startup especially, is to make money. You can't have divided loyalties.

Which is not to say that you have to do the most disgusting sort of work, like spamming, or starting a company whose only purpose is patent litigation. What I mean is, if you're starting a company that will do something cool, the aim had better be to make money and maybe be cool, not to be cool and maybe make money.

It's hard enough to make money that you can't do it by accident. Unless it's your first priority, it's unlikely to happen at all.

Hyenas

When I probe our motives with Artix, I see a third mistake: timidity. If you'd proposed at the time that we go into the e-commerce business, we'd have found the idea terrifying. Surely a field like that would be dominated by fearsome startups with five million dollars of VC money each. Whereas we felt pretty sure that we could hold our own in the slightly less competitive business of generating Web sites for art galleries.

We erred ridiculously far on the side of safety. As it turns out, VC-backed startups are not that fearsome. They're too busy trying to spend all that money to get software written. In 1995, the e-commerce business was very competitive as measured in press releases, but not as measured in software. And really it never was. The big fish like Open Market (rest their souls) were just consulting companies pretending to be product companies ⁵, and the offerings at our end of the market were a couple hundred lines of Perl scripts. Or could have been

⁵It's much easier to sell services than a product, just as it's easier to make a living playing at weddings than by selling recordings. But the margins are greater on products. So during the Bubble a lot of companies used consulting to generate revenues they could attribute to the sale of products, because it made a better story for an IPO.

implemented as a couple hundred lines of Perl; in fact they were probably tens of thousands of lines of C++ or Java. Once we actually took the plunge into e-commerce, it turned out to be surprisingly easy to compete.

So why were we afraid? We felt we were good at programming, but we lacked confidence in our ability to do a mysterious, undifferentiated thing we called “business.” In fact there is no such thing as “business.” There’s selling, promotion, figuring out what people want, deciding how much to charge, customer support, paying your bills, getting customers to pay you, getting incorporated, raising money, and so on. And the combination is not as hard as it seems, because some tasks (like raising money and getting incorporated) are an $O(1)$ pain in the ass, whether you’re big or small, and others (like selling and promotion) depend more on energy and imagination than any kind of special training.

Artix was like a hyena, content to survive on carrion because we were afraid of the lions. Except the lions turned out not to have any teeth, and the business of putting galleries online barely qualified as carrion.

A Familiar Problem

Sum up all these sources of error, and it’s no wonder we had such a bad idea for a company. We did the first thing we thought of; we were ambivalent about being in business at all; and we deliberately chose an impoverished market to avoid competition.

Looking at the applications for the Summer Founders Program, I see signs of all three. But the first is by far the biggest problem. Most of the groups applying have not stopped to ask: of all the things we could do, *is this* the one with the best chance of making money?

If they’d already been through their Artix phase, they’d have learned to ask that. After the reception we got from art dealers, we were ready to. This time, we thought, let’s make something people want.

Reading the *Wall Street Journal* for a week should give anyone ideas

for two or three new startups. The articles are full of descriptions of problems that need to be solved. But most of the applicants don't seem to have looked far for ideas.

We expected the most common proposal to be for multiplayer games. We were not far off: this was the second most common. The most common was some combination of a blog, a calendar, a dating site, and Friendster. Maybe there is some new killer app to be discovered here, but it seems perverse to go poking around in this fog when there are valuable, unsolved problems lying about in the open for anyone to see. Why did no one propose a new scheme for micropayments? An ambitious project, perhaps, but I can't believe we've considered every alternative. And newspapers and magazines are (literally) dying for a solution.

Why did so few applicants really think about what customers want? I think the problem with many, as with people in their early twenties generally, is that they've been trained their whole lives to jump through predefined hoops. They've spent 15-20 years solving problems other people have set for them. And how much time deciding what problems would be good to solve? Two or three course projects? They're good at solving problems, but bad at choosing them.

But that, I'm convinced, is just the effect of training. Or more precisely, the effect of grading. To make grading efficient, everyone has to solve the same problem, and that means it has to be decided in advance. It would be great if schools taught students how to choose problems as well as how to solve them, but I don't know how you'd run such a class in practice.

Copper and Tin

The good news is, choosing problems is something that can be learned. I know that from experience. Hackers can learn to make things customers want.⁶

⁶Trevor Blackwell presents the following recipe for a startup: "Watch people who have money to spend, see what they're wasting their time on, cook up a solution, and

This is a controversial view. One expert on “entrepreneurship” told me that any startup had to include business people, because only they could focus on what customers wanted. I’ll probably alienate this guy forever by quoting him, but I have to risk it, because his email was such a perfect example of this view:

80% of MIT spinoffs succeed *provided* they have at least one management person in the team at the start. The business person represents the “voice of the customer” and that’s what keeps the engineers and product development on track.

This is, in my opinion, a crock. Hackers are perfectly capable of hearing the voice of the customer without a business person to amplify the signal for them. Larry Page and Sergey Brin were grad students in computer science, which presumably makes them “engineers.” Do you suppose Google is only good because they had some business guy whispering in their ears what customers wanted? It seems to me the business guys who did the most for Google were the ones who obligingly flew Altavista into a hillside just as Google was getting started.

The hard part about figuring out what customers want is figuring out that you need to figure it out. But that’s something you can learn quickly. It’s like seeing the other interpretation of an ambiguous picture. As soon as someone tells you there’s a rabbit as well as a duck, it’s hard not to see it.

And compared to the sort of problems hackers are used to solving, giving customers what they want is easy. Anyone who can write an optimizing compiler can design a UI that doesn’t confuse users, once they *choose* to focus on that problem. And once you apply that kind of brain power to petty but profitable questions, you can create wealth very rapidly.

try selling it to them. It’s surprising how small a problem can be and still provide a profitable market for a solution.”

That's the essence of a startup: having brilliant people do work that's beneath them. Big companies try to hire the right person for the job. Startups win because they don't—because they take people so smart that they would in a big company be doing “research,” and set them to work instead on problems of the most immediate and mundane sort. Think Einstein designing refrigerators.⁷

If you want to learn what people want, read Dale Carnegie's *How to Win Friends and Influence People*.⁸ When a friend recommended this book, I couldn't believe he was serious. But he insisted it was good, so I read it, and he was right. It deals with the most difficult problem in human experience: how to see things from other people's point of view, instead of thinking only of yourself.

Most smart people don't do that very well. But adding this ability to raw brainpower is like adding tin to copper. The result is bronze, which is so much harder that it seems a different metal.

A hacker who has learned what to make, and not just how to make, is extraordinarily powerful. And not just at making money: look what a small group of volunteers has achieved with Firefox.

Doing an Artix teaches you to make something people want in the same way that not drinking anything would teach you how much you depend on water. But it would be more convenient for all involved if the Summer Founders didn't learn this on our dime—if they could skip the Artix phase and go right on to make something customers wanted. That, I think, is going to be the real experiment this summer. How long will it take them to grasp this?

We decided we ought to have T-Shirts for the SFP, and we'd been thinking about what to print on the back. Till now we'd been planning

⁷You need to offer especially large rewards to get great people to do tedious work. That's why startups always pay equity rather than just salary.

⁸Buy an old copy from the 1940s or 50s instead of the current edition, which has been rewritten to suit present fashions. The original edition contained a few unPC ideas, but it's always better to read an original book, bearing in mind that it's a book from a past era, than to read a new version sanitized for your protection.

to use

If you can read this, I should be working.

but now we've decided it's going to be

Make something people want.

Thanks to Bill Birch, Trevor Blackwell, Jessica Livingston, and Robert Morris for reading drafts of this.

Chapter 12

The Submarine

April 2005

“Suits make a corporate comeback,” says the *New York Times*. Why does this sound familiar? Maybe because the suit was also back in February, September 2004, June 2004, March 2004, September 2003, November 2002, April 2002, and February 2002.

Why do the media keep running stories saying suits are back? Because PR firms tell them to. One of the most surprising things I discovered during my brief business career was the existence of the PR industry, lurking like a huge, quiet submarine beneath the news. Of the stories you read in traditional media that aren’t about politics, crimes, or disasters, more than half probably come from PR firms.

I know because I spent years hunting such “press hits.” Our startup spent its entire marketing budget on PR: at a time when we were assembling our own computers to save money, we were paying a PR firm \$16,000 a month. And they were worth it. PR is the news equivalent of search engine optimization; instead of buying ads, which readers ignore, you get yourself inserted directly into the stories.¹

Our PR firm was one of the best in the business. In 18 months, they

¹PR has at least one beneficial feature: it favors small companies. If PR didn’t work, the only alternative would be to advertise, and only big companies can afford that.

got press hits in over 60 different publications. And we weren't the only ones they did great things for. In 1997 I got a call from another startup founder considering hiring them to promote his company. I told him they were PR gods, worth every penny of their outrageous fees. But I remember thinking his company's name was odd. Why call an auction site "eBay"?

Symbiosis

PR is not dishonest. Not quite. In fact, the reason the best PR firms are so effective is precisely that they aren't dishonest. They give reporters genuinely valuable information. A good PR firm won't bug reporters just because the client tells them to; they've worked hard to build their credibility with reporters, and they don't want to destroy it by feeding them mere propaganda.

If anyone is dishonest, it's the reporters. The main reason PR firms exist is that reporters are lazy. Or, to put it more nicely, overworked. Really they ought to be out there digging up stories for themselves. But it's so tempting to sit in their offices and let PR firms bring the stories to them. After all, they know good PR firms won't lie to them.

A good flatterer doesn't lie, but tells his victim selective truths (what a nice color your eyes are). Good PR firms use the same strategy: they give reporters stories that are true, but whose truth favors their clients.

For example, our PR firm often pitched stories about how the Web let small merchants compete with big ones. This was perfectly true. But the reason reporters ended up writing stories about this particular truth, rather than some other one, was that small merchants were our target market, and we were paying the piper.

Different publications vary greatly in their reliance on PR firms. At the bottom of the heap are the trade press, who make most of their money from advertising and would give the magazines away for free if advertisers would let them.² The average trade publication is a

²Advertisers pay less for ads in free publications, because they assume readers

bunch of ads, glued together by just enough articles to make it look like a magazine. They're so desperate for "content" that some will print your press releases almost verbatim, if you take the trouble to write them to read like articles.

At the other extreme are publications like the *New York Times* and the *Wall Street Journal*. Their reporters do go out and find their own stories, at least some of the time. They'll listen to PR firms, but briefly and skeptically. We managed to get press hits in almost every publication we wanted, but we never managed to crack the print edition of the *Times*.³

The weak point of the top reporters is not laziness, but vanity. You don't pitch stories to them. You have to approach them as if you were a specimen under their all-seeing microscope, and make it seem as if the story you want them to run is something they thought of themselves.

Our greatest PR coup was a two-part one. We estimated, based on some fairly informal math, that there were about 5000 stores on the Web. We got one paper to print this number, which seemed neutral enough. But once this "fact" was out there in print, we could quote it to other publications, and claim that with 1000 users we had 20% of the online store market.

This was roughly true. We really did have the biggest share of the online store market, and 5000 was our best guess at its size. But the way the story appeared in the press sounded a lot more definite.

Reporters like definitive statements. For example, many of the stories about Jeremy Jaynes's conviction say that he was one of the 10 worst spammers. This "fact" originated in Spamhaus's ROKSO list, which I think even Spamhaus would admit is a rough guess at the top spammers. The first stories about Jaynes cited this source, but now

ignore something they get for free. This is why so many trade publications nominally have a cover price and yet give away free subscriptions with such abandon.

³Different sections of the *Times* vary so much in their standards that they're practically different papers. Whoever fed the style section reporter this story about suits coming back would have been sent packing by the regular news reporters.

it's simply repeated as if it were part of the indictment.⁴

All you can say with certainty about Jaynes is that he was a fairly big spammer. But reporters don't want to print vague stuff like "fairly big." They want statements with punch, like "top ten." And PR firms give them what they want. Wearing suits, we're told, will make us 3.6 percent more productive.

Buzz

Where the work of PR firms really does get deliberately misleading is in the generation of "buzz." They usually feed the same story to several different publications at once. And when readers see similar stories in multiple places, they think there is some important trend afoot. Which is exactly what they're supposed to think.

When Windows 95 was launched, people waited outside stores at midnight to buy the first copies. None of them would have been there without PR firms, who generated such a buzz in the news media that it became self-reinforcing, like a nuclear chain reaction.

I doubt PR firms realize it yet, but the Web makes it possible to track them at work. If you search for the obvious phrases, you turn up several efforts over the years to place stories about the return of the suit. For example, the Reuters article that got picked up by USA Today in September 2004. "The suit is back," it begins.

Trend articles like this are almost always the work of PR firms. Once you know how to read them, it's straightforward to figure out who the client is. With trend stories, PR firms usually line up one or more "experts" to talk about the industry generally. In this case we get three:

⁴The most striking example I know of this type is the "fact" that the Internet worm of 1988 infected 6000 computers. I was there when it was cooked up, and this was the recipe: someone guessed that there were about 60,000 computers attached to the Internet, and that the worm might have infected ten percent of them.

Actually no one knows how many computers the worm infected, because the remedy was to reboot them, and this destroyed all traces. But people like numbers. And so this one is now replicated all over the Internet, like a little worm of its own.

the NPD Group, the creative director of GQ, and a research director at Smith Barney.⁵ When you get to the end of the experts, look for the client. And bingo, there it is: The Men's Wearhouse.

Not surprising, considering The Men's Wearhouse was at that moment running ads saying "The Suit is Back." Talk about a successful press hit— a wire service article whose first sentence is your own ad copy.

The secret to finding other press hits from a given pitch is to realize that they all started from the same document back at the PR firm. Search for a few key phrases and the names of the clients and the experts, and you'll turn up other variants of this story.

Casual fridays are out and dress codes are in writes Diane E. Lewis in *The Boston Globe*. In a remarkable coincidence, Ms. Lewis's industry contacts also include the creative director of GQ.

Ripped jeans and T-shirts are out, writes Mary Kathleen Flynn in *US News & World Report*. And *she too* knows the creative director of GQ.

Men's suits are back writes Nicole Ford in Sexbuzz.Com ("the ultimate men's entertainment magazine").

Dressing down loses appeal as men suit up at the office writes Tenisha Mercer of *The Detroit News*.

Now that so many news articles are online, I suspect you could find a similar pattern for most trend stories placed by PR firms. I propose we call this new sport "PR diving," and I'm sure there are far more striking examples out there than this clump of five stories.

Online

After spending years chasing them, it's now second nature to me to recognize press hits for what they are. But before we hired a PR firm

⁵Not all were necessarily supplied by the PR firm. Reporters sometimes call a few additional sources on their own, like someone adding a few fresh vegetables to a can of soup.

I had no idea where articles in the mainstream media came from. I could tell a lot of them were crap, but I didn't realize why.

Remember the exercises in critical reading you did in school, where you had to look at a piece of writing and step back and ask whether the author was telling the whole truth? If you really want to be a critical reader, it turns out you have to step back one step further, and ask not just whether the author is telling the truth, but *why he's writing about this subject at all*.

Online, the answer tends to be a lot simpler. Most people who publish online write what they write for the simple reason that they want to. You can't see the fingerprints of PR firms all over the articles, as you can in so many print publications— which is one of the reasons, though they may not consciously realize it, that readers trust bloggers more than *Business Week*.

I was talking recently to a friend who works for a big newspaper. He thought the print media were in serious trouble, and that they were still mostly in denial about it. "They think the decline is cyclic," he said. "Actually it's structural."

In other words, the readers are leaving, and they're not coming back.

Why? I think the main reason is that the writing online is more honest. Imagine how incongruous the *New York Times* article about suits would sound if you read it in a blog:

The urge to look corporate— sleek, commanding, prudent, yet with just a touch of hubris on your well-cut sleeve— is an unexpected development in a time of business disgrace.

The problem with this article is not just that it originated in a PR firm. The whole tone is bogus. This is the tone of someone writing down to their audience.

Whatever its flaws, the writing you find online is authentic. It's not

mystery meat cooked up out of scraps of pitch letters and press releases, and pressed into molds of zippy journalese. It's people writing what they think.

I didn't realize, till there was an alternative, just how artificial most of the writing in the mainstream media was. I'm not saying I used to believe what I read in *Time* and *Newsweek*. Since high school, at least, I've thought of magazines like that more as guides to what ordinary people were being told to think than as sources of information. But I didn't realize till the last few years that writing for publication didn't have to mean writing that way. I didn't realize you could write as candidly and informally as you would if you were writing to a friend.

Readers aren't the only ones who've noticed the change. The PR industry has too. A hilarious article on the site of the PR Society of America gets to the heart of the matter:

Bloggers are sensitive about becoming mouthpieces for other organizations and companies, which is the reason they began blogging in the first place.

PR people fear bloggers for the same reason readers like them. And that means there may be a struggle ahead. As this new kind of writing draws readers away from traditional media, we should be prepared for whatever PR mutates into to compensate. When I think how hard PR firms work to score press hits in the traditional media, I can't imagine they'll work any less hard to feed stories to bloggers, if they can figure out how.

Thanks to Ingrid Basset, Trevor Blackwell, Sarah Harlin, Jessica Livingston, Jackie McDonough, Robert Morris, and Aaron Swartz (who also found the PRSA article) for reading drafts of this.

Correction: Earlier versions used a recent *Business Week* article mentioning del.icio.us as an example of a press hit, but Joshua Schachter tells me it was spontaneous.

Chapter 13

Hiring is Obsolete



May 2005

(This essay is derived from a talk at the Berkeley CSUA.)

The three big powers on the Internet now are Yahoo, Google, and Microsoft. Average age of their founders: 24. So it is pretty well established now that grad students can start successful companies. And if grad students can do it, why not undergrads?

Like everything else in technology, the cost of starting a startup has decreased dramatically. Now it's so low that it has disappeared into the noise. The main cost of starting a Web-based startup is food and rent. Which means it doesn't cost much more to start a company than to be a total slacker. You can probably start a startup on ten thousand dollars of seed funding, if you're prepared to live on ramen.

The less it costs to start a company, the less you need the permission of investors to do it. So a lot of people will be able to start companies now who never could have before.

The most interesting subset may be those in their early twenties. I'm not so excited about founders who have everything investors want except intelligence, or everything except energy. The most promising group to be liberated by the new, lower threshold are those who have everything investors want except experience.

Market Rate

I once claimed that nerds were unpopular in secondary school mainly because they had better things to do than work full-time at being popular. Some said I was just telling people what they wanted to hear. Well, I'm now about to do that in a spectacular way: I think undergraduates are undervalued.

Or more precisely, I think few realize the huge spread in the value of 20 year olds. Some, it's true, are not very capable. But others are more capable than all but a handful of 30 year olds.¹

Till now the problem has always been that it's difficult to pick them out. Every VC in the world, if they could go back in time, would try to invest in Microsoft. But which would have then? How many would have understood that this particular 19 year old was Bill Gates?

It's hard to judge the young because (a) they change rapidly, (b) there is great variation between them, and (c) they're individually inconsistent. That last one is a big problem. When you're young, you occasionally say and do stupid things even when you're smart. So if the algorithm is to filter out people who say stupid things, as many investors and employers unconsciously do, you're going to get a lot of false positives.

Most organizations who hire people right out of college are only aware of the average value of 22 year olds, which is not that high. And so the idea for most of the twentieth century was that everyone had to begin as a trainee in some entry-level job. Organizations realized there was a lot of variation in the incoming stream, but instead of pursuing this thought they tended to suppress it, in the belief that it was good for

¹The average B-17 pilot in World War II was in his early twenties. (Thanks to Tad Marko for pointing this out.)

even the most promising kids to start at the bottom, so they didn't get swelled heads.

The most productive young people will *always* be undervalued by large organizations, because the young have no performance to measure yet, and any error in guessing their ability will tend toward the mean.

What's an especially productive 22 year old to do? One thing you can do is go over the heads of organizations, directly to the users. Any company that hires you is, economically, acting as a proxy for the customer. The rate at which they value you (though they may not consciously realize it) is an attempt to guess your value to the user. But there's a way to appeal their judgement. If you want, you can opt to be valued directly by users, by starting your own company.

The market is a lot more discerning than any employer. And it is completely non-discriminatory. On the Internet, nobody knows you're a dog. And more to the point, nobody knows you're 22. All users care about is whether your site or software gives them what they want. They don't care if the person behind it is a high school kid.

If you're really productive, why not make employers pay market rate for you? Why go work as an ordinary employee for a big company, when you could start a startup and make them buy it to get you?

When most people hear the word "startup," they think of the famous ones that have gone public. But most startups that succeed do it by getting bought. And usually the acquirer doesn't just want the technology, but the people who created it as well.

Often big companies buy startups before they're profitable. Obviously in such cases they're not after revenues. What they want is the development team and the software they've built so far. When a startup gets bought for 2 or 3 million six months in, it's really more of a hiring bonus than an acquisition.

I think this sort of thing will happen more and more, and that it will be better for everyone. It's obviously better for the people who start the

startup, because they get a big chunk of money up front. But I think it will be better for the acquirers too. The central problem in big companies, and the main reason they're so much less productive than small companies, is the difficulty of valuing each person's work. Buying larval startups solves that problem for them: the acquirer doesn't pay till the developers have proven themselves. Acquirers are protected on the downside, but still get most of the upside.

Product Development

Buying startups also solves another problem afflicting big companies: they can't do product development. Big companies are good at extracting the value from existing products, but bad at creating new ones.

Why? It's worth studying this phenomenon in detail, because this is the *raison d'être* of startups.

To start with, most big companies have some kind of turf to protect, and this tends to warp their development decisions. For example, Web-based applications are hot now, but within Microsoft there must be a lot of ambivalence about them, because the very idea of Web-based software threatens the desktop. So any Web-based application that Microsoft ends up with, will probably, like Hotmail, be something developed outside the company.

Another reason big companies are bad at developing new products is that the kind of people who do that tend not to have much power in big companies (unless they happen to be the CEO). Disruptive technologies are developed by disruptive people. And they either don't work for the big company, or have been outmaneuvered by yes-men and have comparatively little influence.

Big companies also lose because they usually only build one of each thing. When you only have one Web browser, you can't do anything really risky with it. If ten different startups design ten different Web browsers and you take the best, you'll probably get something better.

The more general version of this problem is that there are too many new ideas for companies to explore them all. There might be 500 startups right now who think they're making something Microsoft might buy. Even Microsoft probably couldn't manage 500 development projects in-house.

Big companies also don't pay people the right way. People developing a new product at a big company get paid roughly the same whether it succeeds or fails. People at a startup expect to get rich if the product succeeds, and get nothing if it fails.² So naturally the people at the startup work a lot harder.

The mere bigness of big companies is an obstacle. In startups, developers are often forced to talk directly to users, whether they want to or not, because there is no one else to do sales and support. It's painful doing sales, but you learn much more from trying to sell people something than reading what they said in focus groups.

And then of course, big companies are bad at product development because they're bad at everything. Everything happens slower in big companies than small ones, and product development is something that has to happen fast, because you have to go through a lot of iterations to get something good.

Trend

I think the trend of big companies buying startups will only accelerate. One of the biggest remaining obstacles is pride. Most companies, at least unconsciously, feel they ought to be able to develop stuff in house, and that buying startups is to some degree an admission of failure. And so, as people generally do with admissions of failure, they put it off for as long as possible. That makes the acquisition very expensive when it finally happens.

What companies should do is go out and discover startups when they're young, before VCs have puffed them up into something that

²If a company tried to pay employees this way, they'd be called unfair. And yet when they buy some startups and not others, no one thinks of calling that unfair.

costs hundreds of millions to acquire. Much of what VCs add, the acquirer doesn't need anyway.

Why don't acquirers try to predict the companies they're going to have to buy for hundreds of millions, and grab them early for a tenth or a twentieth of that? Because they can't predict the winners in advance? If they're only paying a twentieth as much, they only have to predict a twentieth as well. Surely they can manage that.

I think companies that acquire technology will gradually learn to go after earlier stage startups. They won't necessarily buy them outright. The solution may be some hybrid of investment and acquisition: for example, to buy a chunk of the company and get an option to buy the rest later.

When companies buy startups, they're effectively fusing recruiting and product development. And I think that's more efficient than doing the two separately, because you always get people who are really committed to what they're working on.

Plus this method yields teams of developers who already work well together. Any conflicts between them have been ironed out under the very hot iron of running a startup. By the time the acquirer gets them, they're finishing one another's sentences. That's valuable in software, because so many bugs occur at the boundaries between different people's code.

Investors

The increasing cheapness of starting a company doesn't just give hackers more power relative to employers. It also gives them more power relative to investors.

The conventional wisdom among VCs is that hackers shouldn't be allowed to run their own companies. The founders are supposed to accept MBAs as their bosses, and themselves take on some title like Chief Technical Officer. There may be cases where this is a good idea. But I think founders will increasingly be able to push back in the

matter of control, because they just don't need the investors' money as much as they used to.

Startups are a comparatively new phenomenon. Fairchild Semiconductor is considered the first VC-backed startup, and they were founded in 1959, less than fifty years ago. Measured on the time scale of social change, what we have now is pre-beta. So we shouldn't assume the way startups work now is the way they have to work.

Fairchild needed a lot of money to get started. They had to build actual factories. What does the first round of venture funding for a Web-based startup get spent on today? More money can't get software written faster; it isn't needed for facilities, because those can now be quite cheap; all money can really buy you is sales and marketing. A sales force is worth something, I'll admit. But marketing is increasingly irrelevant. On the Internet, anything genuinely good will spread by word of mouth.

Investors' power comes from money. When startups need less money, investors have less power over them. So future founders may not have to accept new CEOs if they don't want them. The VCs will have to be dragged kicking and screaming down this road, but like many things people have to be dragged kicking and screaming toward, it may actually be good for them.

Google is a sign of the way things are going. As a condition of funding, their investors insisted they hire someone old and experienced as CEO. But from what I've heard the founders didn't just give in and take whoever the VCs wanted. They delayed for an entire year, and when they did finally take a CEO, they chose a guy with a PhD in computer science.

It sounds to me as if the founders are still the most powerful people in the company, and judging by Google's performance, their youth and inexperience doesn't seem to have hurt them. Indeed, I suspect Google has done better than they would have if the founders had given the VCs what they wanted, when they wanted it, and let some MBA

take over as soon as they got their first round of funding.

I'm not claiming the business guys installed by VCs have no value. Certainly they have. But they don't need to become the founders' bosses, which is what that title CEO means. I predict that in the future the executives installed by VCs will increasingly be COOs rather than CEOs. The founders will run engineering directly, and the rest of the company through the COO.

The Open Cage

With both employers and investors, the balance of power is slowly shifting towards the young. And yet they seem the last to realize it. Only the most ambitious undergrads even consider starting their own company when they graduate. Most just want to get a job.

Maybe this is as it should be. Maybe if the idea of starting a startup is intimidating, you filter out the uncommitted. But I suspect the filter is set a little too high. I think there are people who could, if they tried, start successful startups, and who instead let themselves be swept into the intake ducts of big companies.

Have you ever noticed that when animals are let out of cages, they don't always realize at first that the door's open? Often they have to be poked with a stick to get them out. Something similar happened with blogs. People could have been publishing online in 1995, and yet blogging has only really taken off in the last couple years. In 1995 we thought only professional writers were entitled to publish their ideas, and that anyone else who did was a crank. Now publishing online is becoming so popular that everyone wants to do it, even print journalists. But blogging has not taken off recently because of any technical innovation; it just took eight years for everyone to realize the cage was open.

I think most undergrads don't realize yet that the economic cage is open. A lot have been told by their parents that the route to success is to get a good job. This was true when their parents were in college, but it's less true now. The route to success is to build something valuable,

and you don't have to be working for an existing company to do that. Indeed, you can often do it better if you're not.

When I talk to undergrads, what surprises me most about them is how conservative they are. Not politically, of course. I mean they don't seem to want to take risks. This is a mistake, because the younger you are, the more risk you can take.

Risk

Risk and reward are always proportionate. For example, stocks are riskier than bonds, and over time always have greater returns. So why does anyone invest in bonds? The catch is that phrase "over time." Stocks will generate greater returns over thirty years, but they might lose value from year to year. So what you should invest in depends on how soon you need the money. If you're young, you should take the riskiest investments you can find.

All this talk about investing may seem very theoretical. Most undergrads probably have more debts than assets. They may feel they have nothing to invest. But that's not true: they have their time to invest, and the same rule about risk applies there. Your early twenties are exactly the time to take insane career risks.

The reason risk is always proportionate to reward is that market forces make it so. People will pay extra for stability. So if you choose stability—by buying bonds, or by going to work for a big company—it's going to cost you.

Riskier career moves pay better on average, because there is less demand for them. Extreme choices like starting a startup are so frightening that most people won't even try. So you don't end up having as much competition as you might expect, considering the prizes at stake.

The math is brutal. While perhaps 9 out of 10 startups fail, the one that succeeds will pay the founders more than 10 times what they

would have made in an ordinary job.³ That's the sense in which startups pay better "on average."

Remember that. If you start a startup, you'll probably fail. Most startups fail. It's the nature of the business. But it's not necessarily a mistake to try something that has a 90% chance of failing, if you can afford the risk. Failing at 40, when you have a family to support, could be serious. But if you fail at 22, so what? If you try to start a startup right out of college and it tanks, you'll end up at 23 broke and a lot smarter. Which, if you think about it, is roughly what you hope to get from a graduate program.

Even if your startup does tank, you won't harm your prospects with employers. To make sure I asked some friends who work for big companies. I asked managers at Yahoo, Google, Amazon, Cisco and Microsoft how they'd feel about two candidates, both 24, with equal ability, one who'd tried to start a startup that tanked, and another who'd spent the two years since college working as a developer at a big company. Every one responded that they'd prefer the guy who'd tried to start his own company. Zed Nazem, who's in charge of engineering at Yahoo, said:

I actually put more value on the guy with the failed startup. And you can quote me!

So there you have it. Want to get hired by Yahoo? Start your own company.

The Man is the Customer

If even big employers think highly of young hackers who start companies, why don't more do it? Why are undergrads so conservative? I think it's because they've spent so much time in institutions.

The first twenty years of everyone's life consists of being piped from

³The 1/10 success rate for startups is a bit of an urban legend. It's suspiciously neat. My guess is the odds are slightly worse.

one institution to another. You probably didn't have much choice about the secondary schools you went to. And after high school it was probably understood that you were supposed to go to college. You may have had a few different colleges to choose between, but they were probably pretty similar. So by this point you've been riding on a subway line for twenty years, and the next stop seems to be a job.

Actually college is where the line ends. Superficially, going to work for a company may feel like just the next in a series of institutions, but underneath, everything is different. The end of school is the fulcrum of your life, the point where you go from net consumer to net producer.

The other big change is that now, you're steering. You can go anywhere you want. So it may be worth standing back and understanding what's going on, instead of just doing the default thing.

All through college, and probably long before that, most undergrads have been thinking about what employers want. But what really matters is what customers want, because they're the ones who give employers the money to pay you.

So instead of thinking about what employers want, you're probably better off thinking directly about what users want. To the extent there's any difference between the two, you can even use that to your advantage if you start a company of your own. For example, big companies like docile conformists. But this is merely an artifact of their bigness, not something customers need.

Grad School

I didn't consciously realize all this when I was graduating from college—partly because I went straight to grad school. Grad school can be a pretty good deal, even if you think of one day starting a startup. You can start one when you're done, or even pull the ripcord part way through, like the founders of Yahoo and Google.

Grad school makes a good launch pad for startups, because you're collected together with a lot of smart people, and you have bigger chunks

of time to work on your own projects than an undergrad or corporate employee would. As long as you have a fairly tolerant advisor, you can take your time developing an idea before turning it into a company. David Filo and Jerry Yang started the Yahoo directory in February 1994 and were getting a million hits a day by the fall, but they didn't actually drop out of grad school and start a company till March 1995.

You could also try the startup first, and if it doesn't work, then go to grad school. When startups tank they usually do it fairly quickly. Within a year you'll know if you're wasting your time.

If it fails, that is. If it succeeds, you may have to delay grad school a little longer. But you'll have a much more enjoyable life once there than you would on a regular grad student stipend.

Experience

Another reason people in their early twenties don't start startups is that they feel they don't have enough experience. Most investors feel the same.

I remember hearing a lot of that word "experience" when I was in college. What do people really mean by it? Obviously it's not the experience itself that's valuable, but something it changes in your brain. What's different about your brain after you have "experience," and can you make that change happen faster?

I now have some data on this, and I can tell you what tends to be missing when people lack experience. I've said that every startup needs three things: to start with good people, to make something users want, and not to spend too much money. It's the middle one you get wrong when you're inexperienced. There are plenty of undergrads with enough technical skill to write good software, and undergrads are not especially prone to waste money. If they get something wrong, it's usually not realizing they have to make something people want.

This is not exclusively a failing of the young. It's common for startup founders of all ages to build things no one wants.

Fortunately, this flaw should be easy to fix. If undergrads were all bad programmers, the problem would be a lot harder. It can take years to learn how to program. But I don't think it takes years to learn how to make things people want. My hypothesis is that all you have to do is smack hackers on the side of the head and tell them: Wake up. Don't sit here making up a priori theories about what users need. Go find some users and see what they need.

Most successful startups not only do something very specific, but solve a problem people already know they have.

The big change that "experience" causes in your brain is learning that you need to solve people's problems. Once you grasp that, you advance quickly to the next step, which is figuring out what those problems are. And that takes some effort, because the way software actually gets used, especially by the people who pay the most for it, is not at all what you might expect. For example, the stated purpose of Powerpoint is to present ideas. Its real role is to overcome people's fear of public speaking. It allows you to give an impressive-looking talk about nothing, and it causes the audience to sit in a dark room looking at slides, instead of a bright one looking at you.

This kind of thing is out there for anyone to see. The key is to know to look for it— to realize that having an idea for a startup is not like having an idea for a class project. The goal in a startup is not to write a cool piece of software. It's to make something people want. And to do that you have to look at users— forget about hacking, and just look at users. This can be quite a mental adjustment, because little if any of the software you write in school even has users.

A few steps before a Rubik's Cube is solved, it still looks like a mess. I think there are a lot of undergrads whose brains are in a similar position: they're only a few steps away from being able to start successful startups, if they wanted to, but they don't realize it. They have more than enough technical skill. They just haven't realized yet that the way to create wealth is to make what users want, and that employers are just proxies for users in which risk is pooled.

If you're young and smart, you don't need either of those. You don't need someone else to tell you what users want, because you can figure it out yourself. And you don't want to pool risk, because the younger you are, the more risk you should take.

A Public Service Message

I'd like to conclude with a joint message from me and your parents. Don't drop out of college to start a startup. There's no rush. There will be plenty of time to start companies after you graduate. In fact, it may be just as well to go work for an existing company for a couple years after you graduate, to learn how companies work.

And yet, when I think about it, I can't imagine telling Bill Gates at 19 that he should wait till he graduated to start a company. He'd have told me to get lost. And could I have honestly claimed that he was harming his future— that he was learning less by working at ground zero of the microcomputer revolution than he would have if he'd been taking classes back at Harvard? No, probably not.

And yes, while it is probably true that you'll learn some valuable things by going to work for an existing company for a couple years before starting your own, you'd learn a thing or two running your own company during that time too.

The advice about going to work for someone else would get an even colder reception from the 19 year old Bill Gates. So I'm supposed to finish college, then go work for another company for two years, and then I can start my own? I have to wait till I'm 23? That's *four years*. That's more than twenty percent of my life so far. Plus in four years it will be way too late to make money writing a Basic interpreter for the Altair.

And he'd be right. The Apple II was launched just two years later. In fact, if Bill had finished college and gone to work for another company as we're suggesting, he might well have gone to work for Apple. And while that would probably have been better for all of us, it wouldn't have been better for him.

So while I stand by our responsible advice to finish college and then go work for a while before starting a startup, I have to admit it's one of those things the old tell the young, but don't expect them to listen to. We say this sort of thing mainly so we can claim we warned you. So don't say I didn't warn you.

Thanks to Jessica Livingston for reading drafts of this, to the friends I promised anonymity to for their opinions about hiring, and to Karen Nguyen and the Berkeley CSUA for organizing this talk.

Chapter 14

What Business Can Learn from Open Source

August 2005

(This essay is derived from a talk at Oscon 2005.)

Lately companies have been paying more attention to open source. Ten years ago there seemed a real danger Microsoft would extend its monopoly to servers. It seems safe to say now that open source has prevented that. A recent survey found 52% of companies are replacing Windows servers with Linux servers.¹

More significant, I think, is *which* 52% they are. At this point, anyone proposing to run Windows on servers should be prepared to explain what they know about servers that Google, Yahoo, and Amazon don't.

But the biggest thing business has to learn from open source is not about Linux or Firefox, but about the forces that produced them. Ultimately these will affect a lot more than what software you use.

We may be able to get a fix on these underlying forces by triangulating from open source and blogging. As you've probably noticed, they have a lot in common.

¹Survey by Forrester Research reported in the cover story of Business Week, 31 Jan 2005. Apparently someone believed you have to replace the actual server in order to switch the operating system.

Like open source, blogging is something people do themselves, for free, because they enjoy it. Like open source hackers, bloggers compete with people working for money, and often win. The method of ensuring quality is also the same: Darwinian. Companies ensure quality through rules to prevent employees from screwing up. But you don't need that when the audience can communicate with one another. People just produce whatever they want; the good stuff spreads, and the bad gets ignored. And in both cases, feedback from the audience improves the best work.

Another thing blogging and open source have in common is the Web. People have always been willing to do great work for free, but before the Web it was harder to reach an audience or collaborate on projects.

Amateurs

I think the most important of the new principles business has to learn is that people work a lot harder on stuff they like. Well, that's news to no one. So how can I claim business has to learn it? When I say business doesn't know this, I mean the structure of business doesn't reflect it.

Business still reflects an older model, exemplified by the French word for working: *travailler*. It has an English cousin, travail, and what it means is torture.²

This turns out not to be the last word on work, however. As societies get richer, they learn something about work that's a lot like what they learn about diet. We know now that the healthiest diet is the one our peasant ancestors were forced to eat because they were poor. Like rich food, idleness only seems desirable when you don't get enough of it. I think we were designed to work, just as we were designed to eat a certain amount of fiber, and we feel bad if we don't.

There's a name for people who work for the love of it: amateurs. The

²It derives from the late Latin *tripalium*, a torture device so called because it consisted of three stakes. I don't know how the stakes were used. "Travel" has the same root.

word now has such bad connotations that we forget its etymology, though it's staring us in the face. "Amateur" was originally rather a complimentary word. But the thing to be in the twentieth century was professional, which amateurs, by definition, are not.

That's why the business world was so surprised by one lesson from open source: that people working for love often surpass those working for money. Users don't switch from Explorer to Firefox because they want to hack the source. They switch because it's a better browser.

It's not that Microsoft isn't trying. They know controlling the browser is one of the keys to retaining their monopoly. The problem is the same they face in operating systems: they can't pay people enough to build something better than a group of inspired hackers will build for free.

I suspect professionalism was always overrated— not just in the literal sense of working for money, but also connotations like formality and detachment. Inconceivable as it would have seemed in, say, 1970, I think professionalism was largely a fashion, driven by conditions that happened to exist in the twentieth century.

One of the most powerful of those was the existence of "channels." Revealingly, the same term was used for both products and information: there were distribution channels, and TV and radio channels.

It was the narrowness of such channels that made professionals seem so superior to amateurs. There were only a few jobs as professional journalists, for example, so competition ensured the average journalist was fairly good. Whereas anyone can express opinions about current events in a bar. And so the average person expressing his opinions in a bar sounds like an idiot compared to a journalist writing about the subject.

On the Web, the barrier for publishing your ideas is even lower. You don't have to buy a drink, and they even let kids in. Millions of people are publishing online, and the average level of what they're writing, as you might expect, is not very good. This has led some in the media

to conclude that blogs don't present much of a threat— that blogs are just a fad.

Actually, the fad is the word “blog,” at least the way the print media now use it. What they mean by “blogger” is not someone who publishes in a weblog format, but anyone who publishes online. That's going to become a problem as the Web becomes the default medium for publication. So I'd like to suggest an alternative word for someone who publishes online. How about “writer?”

Those in the print media who dismiss the writing online because of its low average quality are missing an important point: no one reads the *average* blog. In the old world of channels, it meant something to talk about average quality, because that's what you were getting whether you liked it or not. But now you can read any writer you want. So the average quality of writing online isn't what the print media are competing against. They're competing against the best writing online. And, like Microsoft, they're losing.

I know that from my own experience as a reader. Though most print publications are online, I probably read two or three articles on individual people's sites for every one I read on the site of a newspaper or magazine.

And when I read, say, New York Times stories, I never reach them through the Times front page. Most I find through aggregators like Google News or Slashdot or Delicious. Aggregators show how much better you can do than the channel. The New York Times front page is a list of articles written by people who work for the New York Times. Delicious is a list of articles that are interesting. And it's only now that you can see the two side by side that you notice how little overlap there is.

Most articles in the print media are boring. For example, the president notices that a majority of voters now think invading Iraq was a mistake, so he makes an address to the nation to drum up support. Where is the man bites dog in that? I didn't hear the speech, but I could probably

tell you exactly what he said. A speech like that is, in the most literal sense, not news: there is nothing *new* in it.³

Nor is there anything new, except the names and places, in most “news” about things going wrong. A child is abducted; there’s a tornado; a ferry sinks; someone gets bitten by a shark; a small plane crashes. And what do you learn about the world from these stories? Absolutely nothing. They’re outlying data points; what makes them gripping also makes them irrelevant.

As in software, when professionals produce such crap, it’s not surprising if amateurs can do better. Live by the channel, die by the channel: if you depend on an oligopoly, you sink into bad habits that are hard to overcome when you suddenly get competition.⁴

Workplaces

Another thing blogs and open source software have in common is that they’re often made by people working at home. That may not seem surprising. But it should be. It’s the architectural equivalent of a home-made aircraft shooting down an F-18. Companies spend millions to build office buildings for a single purpose: to be a place to work. And yet people working in their own homes, which aren’t even designed to be workplaces, end up being more productive.

This proves something a lot of us have suspected. The average office is a miserable place to get work done. And a lot of what makes offices bad are the very qualities we associate with professionalism. The sterility of offices is supposed to suggest efficiency. But suggesting efficiency is a different thing from actually being efficient.

The atmosphere of the average workplace is to productivity what flames painted on the side of a car are to speed. And it’s not just the way offices look that’s bleak. The way people act is just as bad.

³It would be much bigger news, in that sense, if the president faced unscripted questions by giving a press conference.

⁴One measure of the incompetence of newspapers is that so many still make you register to read stories. I have yet to find a blog that tried that.

Things are different in a startup. Often as not a startup begins in an apartment. Instead of matching beige cubicles they have an assortment of furniture they bought used. They work odd hours, wearing the most casual of clothing. They look at whatever they want online without worrying whether it's "work safe." The cheery, bland language of the office is replaced by wicked humor. And you know what? The company at this stage is probably the most productive it's ever going to be.

Maybe it's not a coincidence. Maybe some aspects of professionalism are actually a net lose.

To me the most demoralizing aspect of the traditional office is that you're supposed to be there at certain times. There are usually a few people in a company who really have to, but the reason most employees work fixed hours is that the company can't measure their productivity.

The basic idea behind office hours is that if you can't make people work, you can at least prevent them from having fun. If employees have to be in the building a certain number of hours a day, and are forbidden to do non-work things while there, then they must be working. In theory. In practice they spend a lot of their time in a no-man's land, where they're neither working nor having fun.

If you could measure how much work people did, many companies wouldn't need any fixed workday. You could just say: this is what you have to do. Do it whenever you like, wherever you like. If your work requires you to talk to other people in the company, then you may need to be here a certain amount. Otherwise we don't care.

That may seem utopian, but it's what we told people who came to work for our company. There were no fixed office hours. I never showed up before 11 in the morning. But we weren't saying this to be benevolent. We were saying: if you work here we expect you to get a lot done. Don't try to fool us just by being here a lot.

The problem with the facetime model is not just that it's demoralizing,

but that the people pretending to work interrupt the ones actually working. I'm convinced the facetime model is the main reason large organizations have so many meetings. Per capita, large organizations accomplish very little. And yet all those people have to be on site at least eight hours a day. When so much time goes in one end and so little achievement comes out the other, something has to give. And meetings are the main mechanism for taking up the slack.

For one year I worked at a regular nine to five job, and I remember well the strange, cozy feeling that comes over one during meetings. I was very aware, because of the novelty, that I was being paid for programming. It seemed just amazing, as if there was a machine on my desk that spat out a dollar bill every two minutes no matter what I did. Even while I was in the bathroom! But because the imaginary machine was always running, I felt I always ought to be working. And so meetings felt wonderfully relaxing. They counted as work, just like programming, but they were so much easier. All you had to do was sit and look attentive.

Meetings are like an opiate with a network effect. So is email, on a smaller scale. And in addition to the direct cost in time, there's the cost in fragmentation—breaking people's day up into bits too small to be useful.

You can see how dependent you've become on something by removing it suddenly. So for big companies I propose the following experiment. Set aside one day where meetings are forbidden—where everyone has to sit at their desk all day and work without interruption on things they can do without talking to anyone else. Some amount of communication is necessary in most jobs, but I'm sure many employees could find eight hours worth of stuff they could do by themselves. You could call it “Work Day.”

The other problem with pretend work is that it often looks better than real work. When I'm writing or hacking I spend as much time just thinking as I do actually typing. Half the time I'm sitting drinking a cup of tea, or walking around the neighborhood. This is a critical

phase— this is where ideas come from— and yet I'd feel guilty doing this in most offices, with everyone else looking busy.

It's hard to see how bad some practice is till you have something to compare it to. And that's one reason open source, and even blogging in some cases, are so important. They show us what real work looks like.

We're funding eight new startups at the moment. A friend asked what they were doing for office space, and seemed surprised when I said we expected them to work out of whatever apartments they found to live in. But we didn't propose that to save money. We did it because we want their software to be good. Working in crappy informal spaces is one of the things startups do right without realizing it. As soon as you get into an office, work and life start to drift apart.

That is one of the key tenets of professionalism. Work and life are supposed to be separate. But that part, I'm convinced, is a mistake.

Bottom-Up

The third big lesson we can learn from open source and blogging is that ideas can bubble up from the bottom, instead of flowing down from the top. Open source and blogging both work bottom-up: people make what they want, and the best stuff prevails.

Does this sound familiar? It's the principle of a market economy. Ironically, though open source and blogs are done for free, those worlds resemble market economies, while most companies, for all their talk about the value of free markets, are run internally like communist states.

There are two forces that together steer design: ideas about what to do next, and the enforcement of quality. In the channel era, both flowed down from the top. For example, newspaper editors assigned stories to reporters, then edited what they wrote.

Open source and blogging show us things don't have to work that way. Ideas and even the enforcement of quality can flow bottom-up. And

in both cases the results are not merely acceptable, but better. For example, open source software is more reliable precisely because it's open source; anyone can find mistakes.

The same happens with writing. As we got close to publication, I found I was very worried about the essays in *Hackers & Painters* that hadn't been online. Once an essay has had a couple thousand page views I feel reasonably confident about it. But these had had literally orders of magnitude less scrutiny. It felt like releasing software without testing it.

That's what all publishing used to be like. If you got ten people to read a manuscript, you were lucky. But I'd become so used to publishing online that the old method now seemed alarmingly unreliable, like navigating by dead reckoning once you'd gotten used to a GPS.

The other thing I like about publishing online is that you can write what you want and publish when you want. Earlier this year I wrote something that seemed suitable for a magazine, so I sent it to an editor I know. As I was waiting to hear back, I found to my surprise that I was hoping they'd reject it. Then I could put it online right away. If they accepted it, it wouldn't be read by anyone for months, and in the meantime I'd have to fight word-by-word to save it from being mangled by some twenty five year old copy editor.⁵

Many employees would *like* to build great things for the companies they work for, but more often than not management won't let them. How many of us have heard stories of employees going to management and saying, please let us build this thing to make money for you—and the company saying no? The most famous example is probably Steve Wozniak, who originally wanted to build microcomputers for his then-employer, HP. And they turned him down. On the blunderometer, this episode ranks with IBM accepting a non-exclusive license for DOS. But I think this happens all the time. We just don't hear about

⁵They accepted the article, but I took so long to send them the final version that by the time I did the section of the magazine they'd accepted it for had disappeared in a reorganization.

it usually, because to prove yourself right you have to quit and start your own company, like Wozniak did.

Startups

So these, I think, are the three big lessons open source and blogging have to teach business: (1) that people work harder on stuff they like, (2) that the standard office environment is very unproductive, and (3) that bottom-up often works better than top-down.

I can imagine managers at this point saying: what is this guy talking about? What good does it do me to know that my programmers would be more productive working at home on their own projects? I need their asses in here working on version 3.2 of our software, or we're never going to make the release date.

And it's true, the benefit that specific manager could derive from the forces I've described is near zero. When I say business can learn from open source, I don't mean any specific business can. I mean business can learn about new conditions the same way a gene pool does. I'm not claiming companies can get smarter, just that dumb ones will die.

So what will business look like when it has assimilated the lessons of open source and blogging? I think the big obstacle preventing us from seeing the future of business is the assumption that people working for you have to be employees. But think about what's going on underneath: the company has some money, and they pay it to the employee in the hope that he'll make something worth more than they paid him. Well, there are other ways to arrange that relationship. Instead of paying the guy money as a salary, why not give it to him as investment? Then instead of coming to your office to work on your projects, he can work wherever he wants on projects of his own.

Because few of us know any alternative, we have no idea how much better we could do than the traditional employer-employee relationship. Such customs evolve with glacial slowness. Our employer-employee relationship still retains a big chunk of master-servant DNA.

I dislike being on either end of it. I'll work my ass off for a customer, but I resent being told what to do by a boss. And being a boss is also horribly frustrating; half the time it's easier just to do stuff yourself than to get someone else to do it for you. I'd rather do almost anything than give or receive a performance review.

On top of its unpromising origins, employment has accumulated a lot of cruft over the years. The list of what you can't ask in job interviews is now so long that for convenience I assume it's infinite. Within the office you now have to walk on eggshells lest anyone say or do something that makes the company prey to a lawsuit. And God help you if you fire anyone.

Nothing shows more clearly that employment is not an ordinary economic relationship than companies being sued for firing people. In any purely economic relationship you're free to do what you want. If you want to stop buying steel pipe from one supplier and start buying it from another, you don't have to explain why. No one can accuse you of *unjustly* switching pipe suppliers. Justice implies some kind of paternal obligation that isn't there in transactions between equals.

Most of the legal restrictions on employers are intended to protect employees. But you can't have action without an equal and opposite reaction. You can't expect employers to have some kind of paternal responsibility toward employees without putting employees in the position of children. And that seems a bad road to go down.

Next time you're in a moderately large city, drop by the main post office and watch the body language of the people working there. They have the same sullen resentment as children made to do something they don't want to. Their union has exacted pay increases and work restrictions that would have been the envy of previous generations of postal workers, and yet they don't seem any happier for it. It's demoralizing to be on the receiving end of a paternalistic relationship,

⁶The word "boss" is derived from the Dutch *baas*, meaning "master."

no matter how cozy the terms. Just ask any teenager.

I see the disadvantages of the employer-employee relationship because I've been on both sides of a better one: the investor-founder relationship. I wouldn't claim it's painless. When I was running a startup, the thought of our investors used to keep me up at night. And now that I'm an investor, the thought of our startups keeps me up at night. All the pain of whatever problem you're trying to solve is still there. But the pain hurts less when it isn't mixed with resentment.

I had the misfortune to participate in what amounted to a controlled experiment to prove that. After Yahoo bought our startup I went to work for them. I was doing exactly the same work, except with bosses. And to my horror I started acting like a child. The situation pushed buttons I'd forgotten I had.

The big advantage of investment over employment, as the examples of open source and blogging suggest, is that people working on projects of their own are enormously more productive. And a startup is a project of one's own in two senses, both of them important: it's creatively one's own, and also economically one's own.

Google is a rare example of a big company in tune with the forces I've described. They've tried hard to make their offices less sterile than the usual cube farm. They give employees who do great work large grants of stock to simulate the rewards of a startup. They even let hackers spend 20% of their time on their own projects.

Why not let people spend 100% of their time on their own projects, and instead of trying to approximate the value of what they create, give them the actual market value? Impossible? That is in fact what venture capitalists do.

So am I claiming that no one is going to be an employee anymore—that everyone should go and start a startup? Of course not. But more people could do it than do it now. At the moment, even the smartest students leave school thinking they have to get a job. Actually what they need to do is make something valuable. A job is one way to do

that, but the more ambitious ones will ordinarily be better off taking money from an investor than an employer.

Hackers tend to think business is for MBAs. But business administration is not what you're doing in a startup. What you're doing is business *creation*. And the first phase of that is mostly product creation—that is, hacking. That's the hard part. It's a lot harder to create something people love than to take something people love and figure out how to make money from it.

Another thing that keeps people away from starting startups is the risk. Someone with kids and a mortgage should think twice before doing it. But most young hackers have neither.

And as the example of open source and blogging suggests, you'll enjoy it more, even if you fail. You'll be working on your own thing, instead of going to some office and doing what you're told. There may be more pain in your own company, but it won't hurt as much.

That may be the greatest effect, in the long run, of the forces underlying open source and blogging: finally ditching the old paternalistic employer-employee relationship, and replacing it with a purely economic one, between equals.

Thanks to Sarah Harlin, Jessica Livingston, and Robert Morris for reading drafts of this.

Chapter 15

After the Ladder

August 2005

Thirty years ago, one was supposed to work one's way up the corporate ladder. That's less the rule now. Our generation wants to get paid up front. Instead of developing a product for some big company in the expectation of getting job security in return, we develop the product ourselves, in a startup, and sell it to the big company. At the very least we want options.

Among other things, this shift has created the appearance of a rapid increase in economic inequality. But really the two cases are not as different as they look in economic statistics.

Economic statistics are misleading because they ignore the value of safe jobs. An easy job from which one can't be fired is worth money; exchanging the two is one of the commonest forms of corruption. A sinecure is, in effect, an annuity. Except sinecures don't appear in economic statistics. If they did, it would be clear that in practice socialist countries have nontrivial disparities of wealth, because they usually have a class of powerful bureaucrats who are paid mostly by seniority and can never be fired.

While not a sinecure, a position on the corporate ladder was genuinely valuable, because big companies tried not to fire people, and promoted from within based largely on seniority. A position on the

corporate ladder had a value analogous to the “goodwill” that is a very real element in the valuation of companies. It meant one could expect future high paying jobs.

One of main causes of the decay of the corporate ladder is the trend for takeovers that began in the 1980s. Why waste your time climbing a ladder that might disappear before you reach the top?

And, by no coincidence, the corporate ladder was one of the reasons the early corporate raiders were so successful. It’s not only economic statistics that ignore the value of safe jobs. Corporate balance sheets do too. One reason it was profitable to carve up 1980s companies and sell them for parts was that they hadn’t formally acknowledged their implicit debt to employees who had done good work and expected to be rewarded with high-paying executive jobs when their time came.

In the movie *Wall Street*, Gordon Gekko ridicules a company overloaded with vice presidents. But the company may not be as corrupt as it seems; those VPs’ cushy jobs were probably payment for work done earlier.

I like the new model better. For one thing, it seems a bad plan to treat jobs as rewards. Plenty of good engineers got made into bad managers that way. And the old system meant people had to deal with a lot more corporate politics, in order to protect the work they’d invested in a position on the ladder.

The big disadvantage of the new system is that it involves more risk. If you develop ideas in a startup instead of within a big company, any number of random factors could sink you before you can finish. But maybe the older generation would laugh at me for saying that the way we do things is riskier. After all, projects within big companies were always getting cancelled as a result of arbitrary decisions from higher up. My father’s entire industry (breeder reactors) disappeared that way.

For better or worse, the idea of the corporate ladder is probably gone for good. The new model seems more liquid, and more efficient. But

it is less of a change, financially, than one might think. Our fathers weren't *that* stupid.

Chapter 16

Inequality and Risk

August 2005

(This essay is derived from a talk at Defcon 2005.)

Suppose you wanted to get rid of economic inequality. There are two ways to do it: give money to the poor, or take it away from the rich. But they amount to the same thing, because if you want to give money to the poor, you have to get it from somewhere. You can't get it from the poor, or they just end up where they started. You have to get it from the rich.

There is of course a way to make the poor richer without simply shifting money from the rich. You could help the poor become more productive — for example, by improving access to education. Instead of taking money from engineers and giving it to checkout clerks, you could enable people who would have become checkout clerks to become engineers.

This is an excellent strategy for making the poor richer. But the evidence of the last 200 years shows that it doesn't reduce economic inequality, because it makes the rich richer too. If there are more engineers, then there are more opportunities to hire them and to sell them things. Henry Ford couldn't have made a fortune building cars in a society in which most people were still subsistence farmers; he would have had neither workers nor customers.

If you want to reduce economic inequality instead of just improving the overall standard of living, it's not enough just to raise up the poor. What if one of your newly minted engineers gets ambitious and goes on to become another Bill Gates? Economic inequality will be as bad as ever. If you actually want to compress the gap between rich and poor, you have to push down on the top as well as pushing up on the bottom.

How do you push down on the top? You could try to decrease the productivity of the people who make the most money: make the best surgeons operate with their left hands, force popular actors to overeat, and so on. But this approach is hard to implement. The only practical solution is to let people do the best work they can, and then (either by taxation or by limiting what they can charge) to confiscate whatever you deem to be surplus.

So let's be clear what reducing economic inequality means. It is identical with taking money from the rich.

When you transform a mathematical expression into another form, you often notice new things. So it is in this case. Taking money from the rich turns out to have consequences one might not foresee when one phrases the same idea in terms of "reducing inequality."

The problem is, risk and reward have to be proportionate. A bet with only a 10% chance of winning has to pay more than one with a 50% chance of winning, or no one will take it. So if you lop off the top of the possible rewards, you thereby decrease people's willingness to take risks.

Transposing into our original expression, we get: decreasing economic inequality means decreasing the risk people are willing to take.

There are whole classes of risks that are no longer worth taking if the maximum return is decreased. One reason high tax rates are disastrous is that this class of risks includes starting new companies.

Investors

Startups are intrinsically risky. A startup is like a small boat in the open sea. One big wave and you're sunk. A competing product, a downturn in the economy, a delay in getting funding or regulatory approval, a patent suit, changing technical standards, the departure of a key employee, the loss of a big account — any one of these can destroy you overnight. It seems only about 1 in 10 startups succeeds.¹

Our startup paid its first round of outside investors 36x. Which meant, with current US tax rates, that it made sense to invest in us if we had better than a 1 in 24 chance of succeeding. That sounds about right. That's probably roughly how we looked when we were a couple of nerds with no business experience operating out of an apartment.

If that kind of risk doesn't pay, venture investing, as we know it, doesn't happen.

That might be ok if there were other sources of capital for new companies. Why not just have the government, or some large almost-government organization like Fannie Mae, do the venture investing instead of private funds?

I'll tell you why that wouldn't work. Because then you're asking government or almost-government employees to do the one thing they are least able to do: take risks.

As anyone who has worked for the government knows, the important thing is not to make the right choices, but to make choices that can be justified later if they fail. If there is a safe option, that's the one a bureaucrat will choose. But that is exactly the wrong way to do venture investing. The nature of the business means that you want to make terribly risky choices, if the upside looks good enough.

VCS are currently paid in a way that makes them focus on the upside:

¹Success here is defined from the initial investors' point of view: either an IPO, or an acquisition for more than the valuation at the last round of funding. The conventional 1 in 10 success rate is suspiciously neat, but conversations with VCs suggest it's roughly correct for startups overall. Top VC firms expect to do better.

they get a percentage of the fund's gains. And that helps overcome their understandable fear of investing in a company run by nerds who look like (and perhaps are) college students.

If VCs weren't allowed to get rich, they'd behave like bureaucrats. Without hope of gain, they'd have only fear of loss. And so they'd make the wrong choices. They'd turn down the nerds in favor of the smooth-talking MBA in a suit, because that investment would be easier to justify later if it failed.

Founders

But even if you could somehow redesign venture funding to work without allowing VCs to become rich, there's another kind of investor you simply cannot replace: the startups' founders and early employees.

What they invest is their time and ideas. But these are equivalent to money; the proof is that investors are willing (if forced) to treat them as interchangeable, granting the same status to "sweat equity" and the equity they've purchased with cash.

The fact that you're investing time doesn't change the relationship between risk and reward. If you're going to invest your time in something with a small chance of succeeding, you'll only do it if there is a proportionately large payoff.² If large payoffs aren't allowed, you may as well play it safe.

Like many startup founders, I did it to get rich. But not because I wanted to buy expensive things. What I wanted was security. I wanted to make enough money that I didn't have to worry about money. If I'd been forbidden to make enough from a startup to do this, I would have sought security by some other means: for example, by going to work for a big, stable organization from which it would be hard to get fired. Instead of busting my ass in a startup, I would have tried to get a nice, low-stress job at a big research lab, or tenure at a university.

²I'm not claiming founders sit down and calculate the expected after-tax return from a startup. They're motivated by examples of other people who did it. And those examples do reflect after-tax returns.

That's what everyone does in societies where risk isn't rewarded. If you can't ensure your own security, the next best thing is to make a nest for yourself in some large organization where your status depends mostly on seniority.³

Even if we could somehow replace investors, I don't see how we could replace founders. Investors mainly contribute money, which in principle is the same no matter what the source. But the founders contribute ideas. You can't replace those.

Let's rehearse the chain of argument so far. I'm heading for a conclusion to which many readers will have to be dragged kicking and screaming, so I've tried to make each link unbreakable. Decreasing economic inequality means taking money from the rich. Since risk and reward are equivalent, decreasing potential rewards automatically decreases people's appetite for risk. Startups are intrinsically risky. Without the prospect of rewards proportionate to the risk, founders will not invest their time in a startup. Founders are irreplaceable. So eliminating economic inequality means eliminating startups.

Economic inequality is not just a consequence of startups. It's the engine that drives them, in the same way a fall of water drives a water mill. People start startups in the hope of becoming much richer than they were before. And if your society tries to prevent anyone from being much richer than anyone else, it will also prevent one person from being much richer at t_2 than t_1 .

Growth

This argument applies proportionately. It's not just that if you eliminate economic inequality, you get no startups. To the extent you reduce economic inequality, you decrease the number of startups.⁴

³Conjecture: The variation in wealth in a (non-corrupt) country or organization will be inversely proportional to the prevalence of systems of seniority. So if you suppress variation in wealth, seniority will become correspondingly more important. So far, I know of no counterexamples, though in very corrupt countries you may get both simultaneously. (Thanks to Daniel Sobral for pointing this out.)

⁴In a country with a truly feudal economy, you might be able to redistribute

Increase taxes, and willingness to take risks decreases in proportion.

And that seems bad for everyone. New technology and new jobs both come disproportionately from new companies. Indeed, if you don't have startups, pretty soon you won't have established companies either, just as, if you stop having kids, pretty soon you won't have any adults.

It sounds benevolent to say we ought to reduce economic inequality. When you phrase it that way, who can argue with you? *Inequality* has to be bad, right? It sounds a good deal less benevolent to say we ought to reduce the rate at which new companies are founded. And yet the one implies the other.

Indeed, it may be that reducing investors' appetite for risk doesn't merely kill off larval startups, but kills off the most promising ones especially. Startups yield faster growth at greater risk than established companies. Does this trend also hold among startups? That is, are the riskiest startups the ones that generate most growth if they succeed? I suspect the answer is yes. And that's a chilling thought, because it means that if you cut investors' appetite for risk, the most beneficial startups are the first to go.

Not all rich people got that way from startups, of course. What if we let people get rich by starting startups, but taxed away all other surplus wealth? Wouldn't that at least decrease inequality?

Less than you might think. If you made it so that people could only get rich by starting startups, people who wanted to get rich would all start startups. And that might be a great thing. But I don't think it would have much effect on the distribution of wealth. People who want to get rich will do whatever they have to. If startups are the only way to do it, you'll just get far more people starting startups. (If you write the laws very carefully, that is. More likely, you'll just get a lot of people doing things that can be made to look on paper like startups.)

If we're determined to eliminate economic inequality, there is still one

wealth successfully, because there are no startups to kill.

way out: we could say that we're willing to go ahead and do without startups. What would happen if we did?

At a minimum, we'd have to accept lower rates of technological growth. If you believe that large, established companies could somehow be made to develop new technology as fast as startups, the ball is in your court to explain how. (If you can come up with a remotely plausible story, you can make a fortune writing business books and consulting for large companies.)⁵

Ok, so we get slower growth. Is that so bad? Well, one reason it's bad in practice is that other countries might not agree to slow down with us. If you're content to develop new technologies at a slower rate than the rest of the world, what happens is that you don't invent anything at all. Anything you might discover has already been invented elsewhere. And the only thing you can offer in return is raw materials and cheap labor. Once you sink that low, other countries can do whatever they like with you: install puppet governments, siphon off your best workers, use your women as prostitutes, dump their toxic waste on your territory — all the things we do to poor countries now. The only defense is to isolate yourself, as communist countries did in the twentieth century. But the problem then is, you have to become a police state to enforce it.

Wealth and Power

I realize startups are not the main target of those who want to eliminate economic inequality. What they really dislike is the sort of wealth that becomes self-perpetuating through an alliance with power. For example, construction firms that fund politicians' campaigns in return for government contracts, or rich parents who get their children into good colleges by sending them to expensive schools designed for that purpose. But if you try to attack this type of wealth through *economic*

⁵The speed at which startups develop new technology is the other reason they pay so well. As I explained in "How to Make Wealth", what you do in a startup is compress a lifetime's worth of work into a few years. It seems as dumb to discourage that as to discourage risk-taking.

policy, it's hard to hit without destroying startups as collateral damage.

The problem here is not wealth, but corruption. So why not go after corruption?

We don't need to prevent people from being rich if we can prevent wealth from translating into power. And there has been progress on that front. Before he died of drink in 1925, Commodore Vanderbilt's wastrel grandson Reggie ran down pedestrians on five separate occasions, killing two of them. By 1969, when Ted Kennedy drove off the bridge at Chappaquiddick, the limit seemed to be down to one. Today it may well be zero. But what's changed is not variation in wealth. What's changed is the ability to translate wealth into power.

How do you break the connection between wealth and power? Demand transparency. Watch closely how power is exercised, and demand an account of how decisions are made. Why aren't all police interrogations videotaped? Why did 36% of Princeton's class of 2007 come from prep schools, when only 1.7% of American kids attend them? Why did the US really invade Iraq? Why don't government officials disclose more about their finances, and why only during their term of office?

A friend of mine who knows a lot about computer security says the single most important step is to log everything. Back when he was a kid trying to break into computers, what worried him most was the idea of leaving a trail. He was more inconvenienced by the need to avoid that than by any obstacle deliberately put in his path.

Like all illicit connections, the connection between wealth and power flourishes in secret. Expose all transactions, and you will greatly reduce it. Log everything. That's a strategy that already seems to be working, and it doesn't have the side effect of making your whole country poor.

I don't think many people realize there is a connection between economic inequality and risk. I didn't fully grasp it till recently. I'd known

for years of course that if one didn't score in a startup, the other alternative was to get a cozy, tenured research job. But I didn't understand the equation governing my behavior. Likewise, it's obvious empirically that a country that doesn't let people get rich is headed for disaster, whether it's Diocletian's Rome or Harold Wilson's Britain. But I did not till recently understand the role risk played.

If you try to attack wealth, you end up nailing risk as well, and with it growth. If we want a fairer world, I think we're better off attacking one step downstream, where wealth turns into power.

Thanks to Chris Anderson, Trevor Blackwell, Dan Giffin, Jessica Livingston, and Evan Williams for reading drafts of this essay, and to Langley Steinert, Sangam Pant, and Mike Moritz for information about venture investing.

Chapter 17

What I Did this Summer



October 2005

The first Summer Founders Program has just finished. We were surprised how well it went. Overall only about 10% of startups succeed, but if I had to guess now, I'd predict three or four of the eight startups we funded will make it.

Of the startups that needed further funding, I believe all have either closed a round or are likely to soon. Two have already turned down (lowball) acquisition offers.

We would have been happy if just one of the eight seemed promising by the end of the summer. What's going on? Did some kind of anomaly make this summer's applicants especially good? We worry about that, but we can't think of one. We'll find out this winter.

The whole summer was full of surprises. The best was that the hypoth-

esis we were testing seems to be correct. Young hackers can start viable companies. This is good news for two reasons: (a) it's an encouraging thought, and (b) it means that Y Combinator, which is predicated on the idea, is not hosed.

Age

More precisely, the hypothesis was that success in a startup depends mainly on how smart and energetic you are, and much less on how old you are or how much business experience you have. The results so far bear this out. The 2005 summer founders ranged in age from 18 to 28 (average 23), and there is no correlation between their ages and how well they're doing.

This should not really be surprising. Bill Gates and Michael Dell were both 19 when they started the companies that made them famous. Young founders are not a new phenomenon: the trend began as soon as computers got cheap enough for college kids to afford them.

Another of our hypotheses was that you can start a startup on less money than most people think. Other investors were surprised to hear the most we gave any group was \$20,000. But we knew it was possible to start on that little because we started Viaweb on \$10,000.

And so it proved this summer. Three months' funding is enough to get into second gear. We had a demo day for potential investors ten weeks in, and seven of the eight groups had a prototype ready by that time. One, Reddit, had already launched, and were able to give a demo of their live site.

A researcher who studied the SFP startups said the one thing they had in common was that they all worked ridiculously hard. People this age are commonly seen as lazy. I think in some cases it's not so much that they lack the appetite for work, but that the work they're offered is unappetizing.

The experience of the SFP suggests that if you let motivated people do real work, they work hard, whatever their age. As one of the founders

said “I’d read that starting a startup consumed your life, but I had no idea what that meant until I did it.”

I’d feel guilty if I were a boss making people work this hard. But we’re not these people’s bosses. They’re working on their own projects. And what makes them work is not us but their competitors. Like good athletes, they don’t work hard because the coach yells at them, but because they want to win.

We have less power than bosses, and yet the founders work harder than employees. It seems like a win for everyone. The only catch is that we get on average only about 5-7% of the upside, while an employer gets nearly all of it. (We’re counting on it being 5-7% of a much larger number.)

As well as working hard, the groups all turned out to be extraordinarily responsible. I can’t think of a time when one failed to do something they’d promised to, even by being late for an appointment. This is another lesson the world has yet to learn. One of the founders discovered that the hardest part of arranging a meeting with executives at a big cell phone carrier was getting a rental company to rent him a car, because he was too young.

I think the problem here is much the same as with the apparent laziness of people this age. They seem lazy because the work they’re given is pointless, and they act irresponsible because they’re not given any power. Some of them, anyway. We only have a sample size of about twenty, but it seems so far that if you let people in their early twenties be their own bosses, they rise to the occasion.

Morale

The summer founders were as a rule very idealistic. They also wanted very much to get rich. These qualities might seem incompatible, but they’re not. These guys want to get rich, but they want to do it by changing the world. They wouldn’t (well, seven of the eight groups wouldn’t) be interested in making money by speculating in stocks. They want to make something people use.

I think this makes them more effective as founders. As hard as people will work for money, they'll work harder for a cause. And since success in a startup depends so much on motivation, the paradoxical result is that the people likely to make the most money are those who aren't in it just for the money.

The founders of Kiko, for example, are working on an Ajax calendar. They want to get rich, but they pay more attention to design than they would if that were their only motivation. You can tell just by looking at it.

I never considered it till this summer, but this might be another reason startups run by hackers tend to do better than those run by MBAs. Perhaps it's not just that hackers understand technology better, but that they're driven by more powerful motivations. Microsoft, as I've said before, is a dangerously misleading example. Their mean corporate culture only works for monopolies. Google is a better model.

Considering that the summer founders are the sharks in this ocean, we were surprised how frightened most of them were of competitors. But now that I think of it, we were just as frightened when we started Viaweb. For the first year, our initial reaction to news of a competitor was always: we're doomed. Just as a hypochondriac magnifies his symptoms till he's convinced he has some terrible disease, when you're not used to competitors you magnify them into monsters.

Here's a handy rule for startups: competitors are rarely as dangerous as they seem. Most will self-destruct before you can destroy them. And it certainly doesn't matter how many of them there are, any more than it matters to the winner of a marathon how many runners are behind him.

"It's a crowded market," I remember one founder saying worriedly.

"Are you the current leader?" I asked.

"Yes."

"Is anyone able to develop software faster than you?"

“Probably not.”

“Well, if you’re ahead now, and you’re the fastest, then you’ll stay ahead. What difference does it make how many others there are?”

Another group was worried when they realized they had to rewrite their software from scratch. I told them it would be a bad sign if they didn’t. The main function of your initial version is to be rewritten.

That’s why we advise groups to ignore issues like scalability, internationalization, and heavy-duty security at first.¹ I can imagine an advocate of “best practices” saying these ought to be considered from the start. And he’d be right, except that they interfere with the primary function of software in a startup: to be a vehicle for experimenting with its own design. Having to retrofit internationalization or scalability is a pain, certainly. The only bigger pain is not needing to, because your initial version was too big and rigid to evolve into something users wanted.

I suspect this is another reason startups beat big companies. Startups can be irresponsible and release version 1s that are light enough to evolve. In big companies, all the pressure is in the direction of over-engineering.

What Got Learned

One thing we were curious about this summer was where these groups would need help. That turned out to vary a lot. Some we helped with technical advice— for example, about how to set up an application to run on multiple servers. Most we helped with strategy questions, like what to patent, and what to charge for and what to give away. Nearly all wanted advice about dealing with future investors: how much money should they take and what kind of terms should they

¹By heavy-duty security I mean efforts to protect against truly determined attackers.

The image shows us, the 2005 summer founders, and Smartleaf co-founders Mark Nitzberg and Olin Shivers at the 30-foot table Kate Courteau designed for us. Photo by Alex Lewin.

expect?

However, all the groups quickly learned how to deal with stuff like patents and investors. These problems aren't intrinsically difficult, just unfamiliar.

It was surprising—slightly frightening even—how fast they learned. The weekend before the demo day for investors, we had a practice session where all the groups gave their presentations. They were all terrible. We tried to explain how to make them better, but we didn't have much hope. So on demo day I told the assembled angels and VCs that these guys were hackers, not MBAs, and so while their software was good, we should not expect slick presentations from them.

The groups then proceeded to give fabulously slick presentations. Gone were the mumbling recitations of lists of features. It was as if they'd spent the past week at acting school. I still don't know how they did it.

Perhaps watching each others' presentations helped them see what they'd been doing wrong. Just as happens in college, the summer founders learned a lot from one another—maybe more than they learned from us. A lot of the problems they face are the same, from dealing with investors to hacking Javascript.

I don't want to give the impression there were no problems this summer. A lot went wrong, as usually happens with startups. One group got an “exploding term-sheet” from some VCs. Pretty much all the groups who had dealings with big companies found that big companies do everything infinitely slowly. (This is to be expected. If big companies weren't incapable, there would be no room for startups to exist.) And of course there were the usual nightmares associated with servers.

In short, the disasters this summer were just the usual childhood diseases. Some of this summer's eight startups will probably die eventually; it would be extraordinary if all eight succeeded. But what kills them will not be dramatic, external threats, but a mundane, internal

one: not getting enough done.

So far, though, the news is all good. In fact, we were surprised how much fun the summer was for us. The main reason was how much we liked the founders. They're so earnest and hard-working. They seem to like us too. And this illustrates another advantage of investing over hiring: our relationship with them is way better than it would be between a boss and an employee. Y Combinator ends up being more like an older brother than a parent.

I was surprised how much time I spent making introductions. Fortunately I discovered that when a startup needed to talk to someone, I could usually get to the right person by at most one hop. I remember wondering, how did my friends get to be so eminent? and a second later realizing: shit, I'm forty.

Another surprise was that the three-month batch format, which we were forced into by the constraints of the summer, turned out to be an advantage. When we started Y Combinator, we planned to invest the way other venture firms do: as proposals came in, we'd evaluate them and decide yes or no. The SFP was just an experiment to get things started. But it worked so well that we plan to do all our investing this way, one cycle in the summer and one in winter. It's more efficient for us, and better for the startups too.

Several groups said our weekly dinners saved them from a common problem afflicting startups: working so hard that one has no social life. (I remember that part all too well.) This way, they were guaranteed a social event at least once a week.

Independence

I've heard Y Combinator described as an "incubator." Actually we're the opposite: incubators exert more control than ordinary VCs, and we make a point of exerting less. Among other things, incubators usually make you work in their office— that's where the word "incubator" comes from. That seems the wrong model. If investors get too involved, they smother one of the most powerful forces in a startup: the

feeling that it's your own company.

Incubators were conspicuous failures during the Bubble. There's still debate about whether this was because of the Bubble, or because they're a bad idea. My vote is they're a bad idea. I think they fail because they select for the wrong people. When we were starting a startup, we would never have taken funding from an "incubator." We can find office space, thanks; just give us the money. And people with that attitude are the ones likely to succeed in startups.

Indeed, one quality all the founders shared this summer was a spirit of independence. I've been wondering about that. Are some people just a lot more independent than others, or would everyone be this way if they were allowed to?

As with most nature/nurture questions, the answer is probably: some of each. But my main conclusion from the summer is that there's more environment in the mix than most people realize. I could see that from how the founders' attitudes *changed* during the summer. Most were emerging from twenty or so years of being told what to do. They seemed a little surprised at having total freedom. But they grew into it really quickly; some of these guys now seem about four inches taller (metaphorically) than they did at the beginning of the summer.

When we asked the summer founders what surprised them most about starting a company, one said "the most shocking thing is that it worked."

It will take more experience to know for sure, but my guess is that a lot of hackers could do this— that if you put people in a position of independence, they develop the qualities they need. Throw them off a cliff, and most will find on the way down that they have wings.

The reason this is news to anyone is that the same forces work in the other direction too. Most hackers are employees, and this molds you into someone to whom starting a startup seems impossible as surely as starting a startup molds you into someone who can handle it.

If I'm right, "hacker" will mean something different in twenty years than it does now. Increasingly it will mean the people who run the company. Y Combinator is just accelerating a process that would have happened anyway. Power is shifting from the people who deal with money to the people who create technology, and if our experience this summer is any guide, this will be a good thing.

Thanks to Sarah Harlin, Steve Huffman, Jessica Livingston, Zak Stone, and Aaron Swartz for reading drafts of this.

Chapter 18

Ideas for Startups

October 2005

(This essay is derived from a talk at the 2005 Startup School.)

How do you get good ideas for startups? That's probably the number one question people ask me.

I'd like to reply with another question: why do people think it's hard to come up with ideas for startups?

That might seem a stupid thing to ask. Why do they *think* it's hard? If people can't do it, then it *is* hard, at least for them. Right?

Well, maybe not. What people usually say is not that they can't think of ideas, but that they don't have any. That's not quite the same thing. It could be the reason they don't have any is that they haven't tried to generate them.

I think this is often the case. I think people believe that coming up with ideas for startups is very hard— that it *must* be very hard— and so they don't try to do it. They assume ideas are like miracles: they either pop into your head or they don't.

I also have a theory about why people think this. They overvalue ideas. They think creating a startup is just a matter of implementing some fabulous initial idea. And since a successful startup is worth millions of dollars, a good idea is therefore a million dollar idea.

If coming up with an idea for a startup equals coming up with a million dollar idea, then of course it's going to seem hard. Too hard to bother trying. Our instincts tell us something so valuable would not be just lying around for anyone to discover.

Actually, startup ideas are not million dollar ideas, and here's an experiment you can try to prove it: just try to sell one. Nothing evolves faster than markets. The fact that there's no market for startup ideas suggests there's no demand. Which means, in the narrow sense of the word, that startup ideas are worthless.

Questions

The fact is, most startups end up nothing like the initial idea. It would be closer to the truth to say the main value of your initial idea is that, in the process of discovering it's broken, you'll come up with your real idea.

The initial idea is just a starting point— not a blueprint, but a question. It might help if they were expressed that way. Instead of saying that your idea is to make a collaborative, web-based spreadsheet, say: could one make a collaborative, web-based spreadsheet? A few grammatical tweaks, and a woefully incomplete idea becomes a promising question to explore.

There's a real difference, because an assertion provokes objections in a way a question doesn't. If you say: I'm going to build a web-based spreadsheet, then critics— the most dangerous of which are in your own head— will immediately reply that you'd be competing with Microsoft, that you couldn't give people the kind of UI they expect, that users wouldn't want to have their data on your servers, and so on.

A question doesn't seem so challenging. It becomes: let's try making a web-based spreadsheet and see how far we get. And everyone knows that if you tried this you'd be able to make *something* useful. Maybe what you'd end up with wouldn't even be a spreadsheet. Maybe it would be some kind of new spreadsheet-like collaboration tool that doesn't even have a name yet. You wouldn't have thought of some-

thing like that except by implementing your way toward it.

Treating a startup idea as a question changes what you're looking for. If an idea is a blueprint, it has to be right. But if it's a question, it can be wrong, so long as it's wrong in a way that leads to more ideas.

One valuable way for an idea to be wrong is to be only a partial solution. When someone's working on a problem that seems too big, I always ask: is there some way to bite off some subset of the problem, then gradually expand from there? That will generally work unless you get trapped on a local maximum, like 1980s-style AI, or C.

Upwind

So far, we've reduced the problem from thinking of a million dollar idea to thinking of a mistaken question. That doesn't seem so hard, does it?

To generate such questions you need two things: to be familiar with promising new technologies, and to have the right kind of friends. New technologies are the ingredients startup ideas are made of, and conversations with friends are the kitchen they're cooked in.

Universities have both, and that's why so many startups grow out of them. They're filled with new technologies, because they're trying to produce research, and only things that are new count as research. And they're full of exactly the right kind of people to have ideas with: the other students, who will be not only smart but elastic-minded to a fault.

The opposite extreme would be a well-paying but boring job at a big company. Big companies are biased against new technologies, and the people you'd meet there would be wrong too.

In an essay I wrote for high school students, I said a good rule of thumb was to stay upwind— to work on things that maximize your future options. The principle applies for adults too, though perhaps it has to be modified to: stay upwind for as long as you can, then cash in the potential energy you've accumulated when you need to pay for

kids.

I don't think people consciously realize this, but one reason downwind jobs like churning out Java for a bank pay so well is precisely that they are downwind. The market price for that kind of work is higher because it gives you fewer options for the future. A job that lets you work on exciting new stuff will tend to pay less, because part of the compensation is in the form of the new skills you'll learn.

Grad school is the other end of the spectrum from a coding job at a big company: the pay's low but you spend most of your time working on new stuff. And of course, it's called "school," which makes that clear to everyone, though in fact all jobs are some percentage school.

The right environment for having startup ideas need not be a university per se. It just has to be a situation with a large percentage of school.

It's obvious why you want exposure to new technology, but why do you need other people? Can't you just think of new ideas yourself? The empirical answer is: no. Even Einstein needed people to bounce ideas off. Ideas get developed in the process of explaining them to the right kind of person. You need that resistance, just as a carver needs the resistance of the wood.

This is one reason Y Combinator has a rule against investing in startups with only one founder. Practically every successful company has at least two. And because startup founders work under great pressure, it's critical they be friends.

I didn't realize it till I was writing this, but that may help explain why there are so few female startup founders. I read on the Internet (so it must be true) that only 1.7% of VC-backed startups are founded by women. The percentage of female hackers is small, but not that small. So why the discrepancy?

When you realize that successful startups tend to have multiple founders who were already friends, a possible explanation emerges.

People's best friends are likely to be of the same sex, and if one group is a minority in some population, *pairs* of them will be a minority squared.

¹

Doodling

What these groups of co-founders do together is more complicated than just sitting down and trying to think of ideas. I suspect the most productive setup is a kind of together-alone-together sandwich. Together you talk about some hard problem, probably getting nowhere. Then, the next morning, one of you has an idea in the shower about how to solve it. He runs eagerly to tell the others, and together they work out the kinks.

What happens in that shower? It seems to me that ideas just pop into my head. But can we say more than that?

Taking a shower is like a form of meditation. You're alert, but there's nothing to distract you. It's in a situation like this, where your mind is free to roam, that it bumps into new ideas.

What happens when your mind wanders? It may be like doodling. Most people have characteristic ways of doodling. This habit is unconscious, but not random: I found my doodles changed after I started studying painting. I started to make the kind of gestures I'd make if I were drawing from life. They were atoms of drawing, but arranged randomly. ²

Perhaps letting your mind wander is like doodling with ideas. You have certain mental gestures you've learned in your work, and when you're not paying attention, you keep making these same gestures, but

¹This phenomenon may account for a number of discrepancies currently blamed on various forbidden isms. Never attribute to malice what can be explained by math.

²A lot of classic abstract expressionism is doodling of this type: artists trained to paint from life using the same gestures but without using them to represent anything. This explains why such paintings are (slightly) more interesting than random marks would be.

somewhat randomly. In effect, you call the same functions on random arguments. That's what a metaphor is: a function applied to an argument of the wrong type.

Conveniently, as I was writing this, my mind wandered: would it be useful to have metaphors in a programming language? I don't know; I don't have time to think about this. But it's convenient because this is an example of what I mean by habits of mind. I spend a lot of time thinking about language design, and my habit of always asking "would x be useful in a programming language" just got invoked.

If new ideas arise like doodles, this would explain why you have to work at something for a while before you have any. It's not just that you can't judge ideas till you're an expert in a field. You won't even generate ideas, because you won't have any habits of mind to invoke.

Of course the habits of mind you invoke on some field don't have to be derived from working in that field. In fact, it's often better if they're not. You're not just looking for good ideas, but for good *new* ideas, and you have a better chance of generating those if you combine stuff from distant fields. As hackers, one of our habits of mind is to ask, could one open-source x? For example, what if you made an open-source operating system? A fine idea, but not very novel. Whereas if you ask, could you make an open-source play? you might be onto something.

Are some kinds of work better sources of habits of mind than others? I suspect harder fields may be better sources, because to attack hard problems you need powerful solvents. I find math is a good source of metaphors—good enough that it's worth studying just for that. Related fields are also good sources, especially when they're related in unexpected ways. Everyone knows computer science and electrical engineering are related, but precisely because everyone knows it, importing ideas from one to the other doesn't yield great profits. It's like importing something from Wisconsin to Michigan. Whereas (I claim) hacking and painting are also related, in the sense that hackers and painters are both makers, and this source of new ideas is practically virgin territory.

Problems

In theory you could stick together ideas at random and see what you came up with. What if you built a peer-to-peer dating site? Would it be useful to have an automatic book? Could you turn theorems into a commodity? When you assemble ideas at random like this, they may not be just stupid, but semantically ill-formed. What would it even mean to make theorems a commodity? You got me. I didn't think of that idea, just its name.

You might come up with something useful this way, but I never have. It's like knowing a fabulous sculpture is hidden inside a block of marble, and all you have to do is remove the marble that isn't part of it. It's an encouraging thought, because it reminds you there is an answer, but it's not much use in practice because the search space is too big.

I find that to have good ideas I need to be working on some problem. You can't start with randomness. You have to start with a problem, then let your mind wander just far enough for new ideas to form.

In a way, it's harder to see problems than their solutions. Most people prefer to remain in denial about problems. It's obvious why: problems are irritating. They're problems! Imagine if people in 1700 saw their lives the way we'd see them. It would have been unbearable. This denial is such a powerful force that, even when presented with possible solutions, people often prefer to believe they wouldn't work.

I saw this phenomenon when I worked on spam filters. In 2002, most people preferred to ignore spam, and most of those who didn't preferred to believe the heuristic filters then available were the best you could do.

I found spam intolerable, and I felt it had to be possible to recognize it statistically. And it turns out that was all you needed to solve the problem. The algorithm I used was ridiculously simple. Anyone who'd really tried to solve the problem would have found it. It was

just that no one had really tried to solve the problem.³

Let me repeat that recipe: finding the problem intolerable and feeling it must be possible to solve it. Simple as it seems, that's the recipe for a lot of startup ideas.

Wealth

So far most of what I've said applies to ideas in general. What's special about startup ideas? Startup ideas are ideas for companies, and companies have to make money. And the way to make money is to make something people want.

Wealth is what people want. I don't mean that as some kind of philosophical statement; I mean it as a tautology.

So an idea for a startup is an idea for something people want. Wouldn't any good idea be something people want? Unfortunately not. I think new theorems are a fine thing to create, but there is no great demand for them. Whereas there appears to be great demand for celebrity gossip magazines. Wealth is defined democratically. Good ideas and valuable ideas are not quite the same thing; the difference is individual tastes.

But valuable ideas are very close to good ideas, especially in technology. I think they're so close that you can get away with working as if the goal were to discover good ideas, so long as, in the final stage, you stop and ask: will people actually pay for this? Only a few ideas are likely to make it that far and then get shot down; RPN calculators might be one example.

One way to make something people want is to look at stuff people use now that's broken. Dating sites are a prime example. They have millions of users, so they must be promising something people want. And yet they work horribly. Just ask anyone who uses them. It's as if they used the worse-is-better approach but stopped after the first stage

³Bill Yerazunis had solved the problem, but he got there by another path. He made a general-purpose file classifier so good that it also worked for spam.

and handed the thing over to marketers.

Of course, the most obvious breakage in the average computer user's life is Windows itself. But this is a special case: you can't defeat a monopoly by a frontal attack. Windows can and will be overthrown, but not by giving people a better desktop OS. The way to kill it is to redefine the problem as a superset of the current one. The problem is not, what operating system should people use on desktop computers? but how should people use applications? There are answers to that question that don't even involve desktop computers.

Everyone thinks Google is going to solve this problem, but it is a very subtle one, so subtle that a company as big as Google might well get it wrong. I think the odds are better than 50-50 that the Windows killer—or more accurately, Windows transcender—will come from some little startup.

Another classic way to make something people want is to take a luxury and make it into a commodity. People must want something if they pay a lot for it. And it is a very rare product that can't be made dramatically cheaper if you try.

This was Henry Ford's plan. He made cars, which had been a luxury item, into a commodity. But the idea is much older than Henry Ford. Water mills transformed mechanical power from a luxury into a commodity, and they were used in the Roman empire. Arguably pastoralism transformed a luxury into a commodity.

When you make something cheaper you can sell more of them. But if you make something dramatically cheaper you often get qualitative changes, because people start to use it in different ways. For example, once computers get so cheap that most people can have one of their own, you can use them as communication devices.

Often to make something dramatically cheaper you have to redefine the problem. The Model T didn't have all the features previous cars did. It only came in black, for example. But it solved the problem people cared most about, which was getting from place to place.

One of the most useful mental habits I know I learned from Michael Rabin: that the best way to solve a problem is often to redefine it. A lot of people use this technique without being consciously aware of it, but Rabin was spectacularly explicit. You need a big prime number? Those are pretty expensive. How about if I give you a big number that only has a 10 to the minus 100 chance of not being prime? Would that do? Well, probably; I mean, that's probably smaller than the chance that I'm imagining all this anyway.

Redefining the problem is a particularly juicy heuristic when you have competitors, because it's so hard for rigid-minded people to follow. You can work in plain sight and they don't realize the danger. Don't worry about us. We're just working on search. Do one thing and do it well, that's our motto.

Making things cheaper is actually a subset of a more general technique: making things easier. For a long time it was most of making things easier, but now that the things we build are so complicated, there's another rapidly growing subset: making things easier to *use*.

This is an area where there's great room for improvement. What you want to be able to say about technology is: it just works. How often do you say that now?

Simplicity takes effort—genius, even. The average programmer seems to produce UI designs that are almost willfully bad. I was trying to use the stove at my mother's house a couple weeks ago. It was a new one, and instead of physical knobs it had buttons and an LED display. I tried pressing some buttons I thought would cause it to get hot, and you know what it said? “Err.” Not even “Error.” “Err.” You can't just say “Err” to the user of a *stove*. You should design the UI so that errors are impossible. And the boneheads who designed this stove even had an example of such a UI to work from: the old one. You turn one knob to set the temperature and another to set the timer. What was wrong with that? It just worked.

It seems that, for the average engineer, more options just means more

rope to hang yourself. So if you want to start a startup, you can take almost any existing technology produced by a big company, and assume you could build something way easier to use.

Design for Exit

Success for a startup approximately equals getting bought. You need some kind of exit strategy, because you can't get the smartest people to work for you without giving them options likely to be worth something. Which means you either have to get bought or go public, and the number of startups that go public is very small.

If success probably means getting bought, should you make that a conscious goal? The old answer was no: you were supposed to pretend that you wanted to create a giant, public company, and act surprised when someone made you an offer. Really, you want to buy us? Well, I suppose we'd consider it, for the right price.

I think things are changing. If 98% of the time success means getting bought, why not be open about it? If 98% of the time you're doing product development on spec for some big company, why not think of that as your task? One advantage of this approach is that it gives you another source of ideas: look at big companies, think what they should be doing, and do it yourself. Even if they already know it, you'll probably be done faster.

Just be sure to make something multiple acquirers will want. Don't fix Windows, because the only potential acquirer is Microsoft, and when there's only one acquirer, they don't have to hurry. They can take their time and copy you instead of buying you. If you want to get market price, work on something where there's competition.

If an increasing number of startups are created to do product development on spec, it will be a natural counterweight to monopolies. Once some type of technology is captured by a monopoly, it will only evolve at big company rates instead of startup rates, whereas alternatives will evolve with especial speed. A free market interprets monopoly as damage and routes around it.

The Woz Route

The most productive way to generate startup ideas is also the most unlikely-sounding: by accident. If you look at how famous startups got started, a lot of them weren't initially supposed to be startups. Lotus began with a program Mitch Kapor wrote for a friend. Apple got started because Steve Wozniak wanted to build microcomputers, and his employer, Hewlett-Packard, wouldn't let him do it at work. Yahoo began as David Filo's personal collection of links.

This is not the only way to start startups. You can sit down and consciously come up with an idea for a company; we did. But measured in total market cap, the build-stuff-for-yourself model might be more fruitful. It certainly has to be the most fun way to come up with startup ideas. And since a startup ought to have multiple founders who were already friends before they decided to start a company, the rather surprising conclusion is that the best way to generate startup ideas is to do what hackers do for fun: cook up amusing hacks with your friends.

It seems like it violates some kind of conservation law, but there it is: the best way to get a "million dollar idea" is just to do what hackers enjoy doing anyway.

Chapter 19

The Venture Capital Squeeze

November 2005

In the next few years, venture capital funds will find themselves squeezed from four directions. They're already stuck with a seller's market, because of the huge amounts they raised at the end of the Bubble and still haven't invested. This by itself is not the end of the world. In fact, it's just a more extreme version of the norm in the VC business: too much money chasing too few deals.

Unfortunately, those few deals now want less and less money, because it's getting so cheap to start a startup. The four causes: open source, which makes software free; Moore's law, which makes hardware geometrically closer to free; the Web, which makes promotion free if you're good; and better languages, which make development a lot cheaper.

When we started our startup in 1995, the first three were our biggest expenses. We had to pay \$5000 for the Netscape Commerce Server, the only software that then supported secure http connections. We paid \$3000 for a server with a 90 MHz processor and 32 meg of memory. And we paid a PR firm about \$30,000 to promote our launch.

Now you could get all three for nothing. You can get the software for free; people throw away computers more powerful than our first server; and if you make something good you can generate ten times

as much traffic by word of mouth online than our first PR firm got through the print media.

And of course another big change for the average startup is that programming languages have improved— or rather, the median language has. At most startups ten years ago, software development meant ten programmers writing code in C++. Now the same work might be done by one or two using Python or Ruby.

During the Bubble, a lot of people predicted that startups would outsource their development to India. I think a better model for the future is David Heinemeier Hansson, who outsourced his development to a more powerful language instead. A lot of well-known applications are now, like BaseCamp, written by just one programmer. And one guy is more than 10x cheaper than ten, because (a) he won't waste any time in meetings, and (b) since he's probably a founder, he can pay himself nothing.

Because starting a startup is so cheap, venture capitalists now often want to give startups more money than the startups want to take. VCs like to invest several million at a time. But as one VC told me after a startup he funded would only take about half a million, "I don't know what we're going to do. Maybe we'll just have to give some of it back." Meaning give some of the fund back to the institutional investors who supplied it, because it wasn't going to be possible to invest it all.

Into this already bad situation comes the third problem: Sarbanes-Oxley. Sarbanes-Oxley is a law, passed after the Bubble, that drastically increases the regulatory burden on public companies. And in addition to the cost of compliance, which is at least two million dollars a year, the law introduces frightening legal exposure for corporate officers. An experienced CFO I know said flatly: "I would not want to be CFO of a public company now."

You might think that responsible corporate governance is an area where you can't go too far. But you can go too far in any law, and this remark convinced me that Sarbanes-Oxley must have. This CFO

is both the smartest and the most upstanding money guy I know. If Sarbanes-Oxley deters people like him from being CFOs of public companies, that's proof enough that it's broken.

Largely because of Sarbanes-Oxley, few startups go public now. For all practical purposes, succeeding now equals getting bought. Which means VCs are now in the business of finding promising little 2-3 man startups and pumping them up into companies that cost \$100 million to acquire. They didn't mean to be in this business; it's just what their business has evolved into.

Hence the fourth problem: the acquirers have begun to realize they can buy wholesale. Why should they wait for VCs to make the startups they want more expensive? Most of what the VCs add, acquirers don't want anyway. The acquirers already have brand recognition and HR departments. What they really want is the software and the developers, and that's what the startup is in the early phase: concentrated software and developers.

Google, typically, seems to have been the first to figure this out. "Bring us your startups early," said Google's speaker at the Startup School. They're quite explicit about it: they like to acquire startups at just the point where they would do a Series A round. (The Series A round is the first round of real VC funding; it usually happens in the first year.) It is a brilliant strategy, and one that other big technology companies will no doubt try to duplicate. Unless they want to have still more of their lunch eaten by Google.

Of course, Google has an advantage in buying startups: a lot of the people there are rich, or expect to be when their options vest. Ordinary employees find it very hard to recommend an acquisition; it's just too annoying to see a bunch of twenty year olds get rich when you're still working for salary. Even if it's the right thing for your company to do.

The Solution(s)

Bad as things look now, there is a way for VCs to save themselves.

They need to do two things, one of which won't surprise them, and another that will seem an anathema.

Let's start with the obvious one: lobby to get Sarbanes-Oxley loosened. This law was created to prevent future Enrons, not to destroy the IPO market. Since the IPO market was practically dead when it passed, few saw what bad effects it would have. But now that technology has recovered from the last bust, we can see clearly what a bottleneck Sarbanes-Oxley has become.

Startups are fragile plants—seedlings, in fact. These seedlings are worth protecting, because they grow into the trees of the economy. Much of the economy's growth is their growth. I think most politicians realize that. But they don't realize just how fragile startups are, and how easily they can become collateral damage of laws meant to fix some other problem.

Still more dangerously, when you destroy startups, they make very little noise. If you step on the toes of the coal industry, you'll hear about it. But if you inadvertantly squash the startup industry, all that happens is that the founders of the next Google stay in grad school instead of starting a company.

My second suggestion will seem shocking to VCs: let founders cash out partially in the Series A round. At the moment, when VCs invest in a startup, all the stock they get is newly issued and all the money goes to the company. They could buy some stock directly from the founders as well.

Most VCs have an almost religious rule against doing this. They don't want founders to get a penny till the company is sold or goes public. VCs are obsessed with control, and they worry that they'll have less leverage over the founders if the founders have any money.

This is a dumb plan. In fact, letting the founders sell a little stock early would generally be better for the company, because it would cause the founders' attitudes toward risk to be aligned with the VCs'. As things currently work, their attitudes toward risk tend to be diametrically op-

posed: the founders, who have nothing, would prefer a 100% chance of \$1 million to a 20% chance of \$10 million, while the VCs can afford to be “rational” and prefer the latter.

Whatever they say, the reason founders are selling their companies early instead of doing Series A rounds is that they get paid up front. That first million is just worth so much more than the subsequent ones. If founders could sell a little stock early, they’d be happy to take VC money and bet the rest on a bigger outcome.

So why not let the founders have that first million, or at least half million? The VCs would get same number of shares for the money. So what if some of the money would go to the founders instead of the company?

Some VCs will say this is unthinkable—that they want all their money to be put to work growing the company. But the fact is, the huge size of current VC investments is dictated by the structure of VC funds, not the needs of startups. Often as not these large investments go to work destroying the company rather than growing it.

The angel investors who funded our startup let the founders sell some stock directly to them, and it was a good deal for everyone. The angels made a huge return on that investment, so they’re happy. And for us founders it blunted the terrifying all-or-nothingness of a startup, which in its raw form is more a distraction than a motivator.

If VCs are frightened at the idea of letting founders partially cash out, let me tell them something still more frightening: you are now competing directly with Google.

Thanks to Trevor Blackwell, Sarah Harlin, Jessica Livingston, and Robert Morris for reading drafts of this.

Chapter 20

How to Fund a Startup

November 2005

Venture funding works like gears. A typical startup goes through several rounds of funding, and at each round you want to take just enough money to reach the speed where you can shift into the next gear.

Few startups get it quite right. Many are underfunded. A few are overfunded, which is like trying to start driving in third gear.

I think it would help founders to understand funding better—not just the mechanics of it, but what investors are thinking. I was surprised recently when I realized that all the worst problems we faced in our startup were due not to competitors, but investors. Dealing with competitors was easy by comparison.

I don't mean to suggest that our investors were nothing but a drag on us. They were helpful in negotiating deals, for example. I mean more that conflicts with investors are particularly nasty. Competitors punch you in the jaw, but investors have you by the balls.

Apparently our situation was not unusual. And if trouble with investors is one of the biggest threats to a startup, managing them is one of the most important skills founders need to learn.

Let's start by talking about the five sources of startup funding. Then we'll trace the life of a hypothetical (very fortunate) startup as it shifts

gears through successive rounds.

Friends and Family

A lot of startups get their first funding from friends and family. Excite did, for example: after the founders graduated from college, they borrowed \$15,000 from their parents to start a company. With the help of some part-time jobs they made it last 18 months.

If your friends or family happen to be rich, the line blurs between them and angel investors. At Viaweb we got our first \$10,000 of seed money from our friend Julian, but he was sufficiently rich that it's hard to say whether he should be classified as a friend or angel. He was also a lawyer, which was great, because it meant we didn't have to pay legal bills out of that initial small sum.

The advantage of raising money from friends and family is that they're easy to find. You already know them. There are three main disadvantages: you mix together your business and personal life; they will probably not be as well connected as angels or venture firms; and they may not be accredited investors, which could complicate your life later.

The SEC defines an "accredited investor" as someone with over a million dollars in liquid assets or an income of over \$200,000 a year. The regulatory burden is much lower if a company's shareholders are all accredited investors. Once you take money from the general public you're more restricted in what you can do.¹

A startup's life will be more complicated, legally, if any of the investors aren't accredited. In an IPO, it might not merely add expense, but change the outcome. A lawyer I asked about it said:

When the company goes public, the SEC will carefully study all prior issuances of stock by the company and de-

¹The aim of such regulations is to protect widows and orphans from crooked investment schemes; people with a million dollars in liquid assets are assumed to be able to protect themselves. The unintended consequence is that the investments that generate the highest returns, like hedge funds, are available only to the rich.

mand that it take immediate action to cure any past violations of securities laws. Those remedial actions can delay, stall or even kill the IPO.

Of course the odds of any given startup doing an IPO are small. But not as small as they might seem. A lot of startups that end up going public didn't seem likely to at first. (Who could have guessed that the company Wozniak and Jobs started in their spare time selling plans for microcomputers would yield one of the biggest IPOs of the decade?) Much of the value of a startup consists of that tiny probability multiplied by the huge outcome.

It wasn't because they weren't accredited investors that I didn't ask my parents for seed money, though. When we were starting Viaweb, I didn't know about the concept of an accredited investor, and didn't stop to think about the value of investors' connections. The reason I didn't take money from my parents was that I didn't want them to lose it.

Consulting

Another way to fund a startup is to get a job. The best sort of job is a consulting project in which you can build whatever software you wanted to sell as a startup. Then you can gradually transform yourself from a consulting company into a product company, and have your clients pay your development expenses.

This is a good plan for someone with kids, because it takes most of the risk out of starting a startup. There never has to be a time when you have no revenues. Risk and reward are usually proportionate, however: you should expect a plan that cuts the risk of starting a startup also to cut the average return. In this case, you trade decreased financial risk for increased risk that your company won't succeed as a startup.

But isn't the consulting company itself a startup? No, not generally. A company has to be more than small and newly founded to be a

startup. There are millions of small businesses in America, but only a few thousand are startups. To be a startup, a company has to be a product business, not a service business. By which I mean not that it has to make something physical, but that it has to have one thing it sells to many people, rather than doing custom work for individual clients. Custom work doesn't scale. To be a startup you need to be the band that sells a million copies of a song, not the band that makes money by playing at individual weddings and bar mitzvahs.

The trouble with consulting is that clients have an awkward habit of calling you on the phone. Most startups operate close to the margin of failure, and the distraction of having to deal with clients could be enough to put you over the edge. Especially if you have competitors who get to work full time on just being a startup.

So you have to be very disciplined if you take the consulting route. You have to work actively to prevent your company growing into a "weed tree," dependent on this source of easy but low-margin money.
2

Indeed, the biggest danger of consulting may be that it gives you an excuse for failure. In a startup, as in grad school, a lot of what ends up driving you are the expectations of your family and friends. Once you start a startup and tell everyone that's what you're doing, you're now on a path labelled "get rich or bust." You now have to get rich, or you've failed.

Fear of failure is an extraordinarily powerful force. Usually it prevents people from starting things, but once you publish some definite ambition, it switches directions and starts working in your favor. I think it's a pretty clever piece of jiu-jitsu to set this irresistible force against the slightly less immovable object of becoming rich. You won't have it driving you if your stated ambition is merely to start a consulting company that you will one day morph into a startup.

²Consulting is where product companies go to die. IBM is the most famous example. So starting as a consulting company is like starting out in the grave and trying to work your way up into the world of the living.

An advantage of consulting, as a way to develop a product, is that you know you're making something at least one customer wants. But if you have what it takes to start a startup you should have sufficient vision not to need this crutch.

Angel Investors

Angels are individual rich people. The word was first used for backers of Broadway plays, but now applies to individual investors generally. Angels who've made money in technology are preferable, for two reasons: they understand your situation, and they're a source of contacts and advice.

The contacts and advice can be more important than the money. When *del.icio.us* took money from investors, they took money from, among others, Tim O'Reilly. The amount he put in was small compared to the VCs who led the round, but Tim is a smart and influential guy and it's good to have him on your side.

You can do whatever you want with money from consulting or friends and family. With angels we're now talking about venture funding proper, so it's time to introduce the concept of *exit strategy*. Younger would-be founders are often surprised that investors expect them either to sell the company or go public. The reason is that investors need to get their capital back. They'll only consider companies that have an exit strategy—meaning companies that could get bought or go public.

This is not as selfish as it sounds. There are few large, private technology companies. Those that don't fail all seem to get bought or go public. The reason is that employees are investors too—of their time—and they want just as much to be able to cash out. If your competitors offer employees stock options that might make them rich, while you make it clear you plan to stay private, your competitors will get the best people. So the principle of an “exit” is not just something forced on startups by investors, but part of what it means to be a startup.

Another concept we need to introduce now is valuation. When someone buys shares in a company, that implicitly establishes a value for it. If someone pays \$20,000 for 10% of a company, the company is in theory worth \$200,000. I say “in theory” because in early stage investing, valuations are voodoo. As a company gets more established, its valuation gets closer to an actual market value. But in a newly founded startup, the valuation number is just an artifact of the respective contributions of everyone involved.

Startups often “pay” investors who will help the company in some way by letting them invest at low valuations. If I had a startup and Steve Jobs wanted to invest in it, I’d give him the stock for \$10, just to be able to brag that he was an investor. Unfortunately, it’s impractical (if not illegal) to adjust the valuation of the company up and down for each investor. Startups’ valuations are supposed to rise over time. So if you’re going to sell cheap stock to eminent angels, do it early, when it’s natural for the company to have a low valuation.

Some angel investors join together in syndicates. Any city where people start startups will have one or more of them. In Boston the biggest is the Common Angels. In the Bay Area it’s the Band of Angels. You can find groups near you through the Angel Capital Association.³ However, most angel investors don’t belong to these groups. In fact, the more prominent the angel, the less likely they are to belong to a group.

Some angel groups charge you money to pitch your idea to them. Needless to say, you should never do this.

One of the dangers of taking investment from individual angels, rather than through an angel group or investment firm, is that they have less reputation to protect. A big-name VC firm will not screw you too outrageously, because other founders would avoid them if word got out. With individual angels you don’t have this protection, as we

³If “near you” doesn’t mean the Bay Area, Boston, or Seattle, consider moving. It’s not a coincidence you haven’t heard of many startups from Philadelphia.

found to our dismay in our own startup. In many startups' lives there comes a point when you're at the investors' mercy—when you're out of money and the only place to get more is your existing investors. When we got into such a scrape, our investors took advantage of it in a way that a name-brand VC probably wouldn't have.

Angels have a corresponding advantage, however: they're also not bound by all the rules that VC firms are. And so they can, for example, allow founders to cash out partially in a funding round, by selling some of their stock directly to the investors. I think this will become more common; the average founder is eager to do it, and selling, say, half a million dollars worth of stock will not, as VCs fear, cause most founders to be any less committed to the business.

The same angels who tried to screw us also let us do this, and so on balance I'm grateful rather than angry. (As in families, relations between founders and investors can be complicated.)

The best way to find angel investors is through personal introductions. You could try to cold-call angel groups near you, but angels, like VCs, will pay more attention to deals recommended by someone they respect.

Deal terms with angels vary a lot. There are no generally accepted standards. Sometimes angels' deal terms are as fearsome as VCs'. Other angels, particularly in the earliest stages, will invest based on a two-page agreement.

Angels who only invest occasionally may not themselves know what terms they want. They just want to invest in this startup. What kind of anti-dilution protection do they want? Hell if they know. In these situations, the deal terms tend to be random: the angel asks his lawyer to create a vanilla agreement, and the terms end up being whatever the lawyer considers vanilla. Which in practice usually means, whatever existing agreement he finds lying around his firm. (Few legal documents are created from scratch.)

These heaps o' boilerplate are a problem for small startups, because

they tend to grow into the union of all preceding documents. I know of one startup that got from an angel investor what amounted to a five hundred pound handshake: after deciding to invest, the angel presented them with a 70-page agreement. The startup didn't have enough money to pay a lawyer even to read it, let alone negotiate the terms, so the deal fell through.

One solution to this problem would be to have the startup's lawyer produce the agreement, instead of the angel's. Some angels might balk at this, but others would probably welcome it.

Inexperienced angels often get cold feet when the time comes to write that big check. In our startup, one of the two angels in the initial round took months to pay us, and only did after repeated nagging from our lawyer, who was also, fortunately, his lawyer.

It's obvious why investors delay. Investing in startups is risky! When a company is only two months old, every *day* you wait gives you 1.7% more data about their trajectory. But the investor is already being compensated for that risk in the low price of the stock, so it is unfair to delay.

Fair or not, investors do it if you let them. Even VCs do it. And funding delays are a big distraction for founders, who ought to be working on their company, not worrying about investors. What's a startup to do? With both investors and acquirers, the only leverage you have is competition. If an investor knows you have other investors lined up, he'll be a lot more eager to close— and not just because he'll worry about losing the deal, but because if other investors are interested, you must be worth investing in. It's the same with acquisitions. No one wants to buy you till someone else wants to buy you, and then everyone wants to buy you.

The key to closing deals is never to stop pursuing alternatives. When an investor says he wants to invest in you, or an acquirer says they want to buy you, *don't believe it till you get the check*. Your natural tendency when an investor says yes will be to relax and go back to writing code.

Alas, you can't; you have to keep looking for more investors, if only to get this one to act.⁴

Seed Funding Firms

Seed firms are like angels in that they invest relatively small amounts at early stages, but like VCs in that they're companies that do it as a business, rather than individuals making occasional investments on the side.

Till now, nearly all seed firms have been so-called "incubators," so Y Combinator gets called one too, though the only thing we have in common is that we invest in the earliest phase.

According to the National Association of Business Incubators, there are about 800 incubators in the US. This is an astounding number, because I know the founders of a lot of startups, and I can't think of one that began in an incubator.

What is an incubator? I'm not sure myself. The defining quality seems to be that you work in their space. That's where the name "incubator" comes from. They seem to vary a great deal in other respects. At one extreme is the sort of pork-barrel project where a town gets money from the state government to renovate a vacant building as a "high-tech incubator," as if it were merely lack of the right sort of office space that had till now prevented the town from becoming a startup hub. At the other extreme are places like Idealab, which generates ideas for new startups internally and hires people to work for them.

The classic Bubble incubators, most of which now seem to be dead, were like VC firms except that they took a much bigger role in the startups they funded. In addition to working in their space, you were supposed to use their office staff, lawyers, accountants, and so on.

⁴Investors are often compared to sheep. And they are like sheep, but that's a rational response to their situation. Sheep act the way they do for a reason. If all the other sheep head for a certain field, it's probably good grazing. And when a wolf appears, is he going to eat a sheep in the middle of the flock, or one near the edge?

Whereas incubators tend (or tended) to exert more control than VCs, Y Combinator exerts less. And we think it's better if startups operate out of their own premises, however crappy, than the offices of their investors. So it's annoying that we keep getting called an "incubator," but perhaps inevitable, because there's only one of us so far and no word yet for what we are. If we have to be called something, the obvious name would be "excubator." (The name is more excusable if one considers it as meaning that we enable people to escape cubicles.)

Because seed firms are companies rather than individual people, reaching them is easier than reaching angels. Just go to their web site and send them an email. The importance of personal introductions varies, but is less than with angels or VCs.

The fact that seed firms are companies also means the investment process is more standardized. (This is generally true with angel groups too.) Seed firms will probably have set deal terms they use for every startup they fund. The fact that the deal terms are standard doesn't mean they're favorable to you, but if other startups have signed the same agreements and things went well for them, it's a sign the terms are reasonable.

Seed firms differ from angels and VCs in that they invest exclusively in the earliest phases—often when the company is still just an idea. Angels and even VC firms occasionally do this, but they also invest at later stages.

The problems are different in the early stages. For example, in the first couple months a startup may completely redefine their idea. So seed investors usually care less about the idea than the people. This is true of all venture funding, but especially so in the seed stage.

Like VCs, one of the advantages of seed firms is the advice they offer. But because seed firms operate in an earlier phase, they need to offer different kinds of advice. For example, a seed firm should be able to give advice about how to approach VCs, which VCs obviously don't need to do; whereas VCs should be able to give advice about how to

hire an “executive team,” which is not an issue in the seed stage.

In the earliest phases, a lot of the problems are technical, so seed firms should be able to help with technical as well as business problems.

Seed firms and angel investors generally want to invest in the initial phases of a startup, then hand them off to VC firms for the next round. Occasionally startups go from seed funding direct to acquisition, however, and I expect this to become increasingly common.

Google has been aggressively pursuing this route, and now Yahoo is too. Both now compete directly with VCs. And this is a smart move. Why wait for further funding rounds to jack up a startup’s price? When a startup reaches the point where VCs have enough information to invest in it, the acquirer should have enough information to buy it. More information, in fact; with their technical depth, the acquirers should be better at picking winners than VCs.

Venture Capital Funds

VC firms are like seed firms in that they’re actual companies, but they invest other people’s money, and much larger amounts of it. VC investments average several million dollars. So they tend to come later in the life of a startup, are harder to get, and come with tougher terms.

The word “venture capitalist” is sometimes used loosely for any venture investor, but there is a sharp difference between VCs and other investors: VC firms are organized as *funds*, much like hedge funds or mutual funds. The fund managers, who are called “general partners,” get about 2% of the fund annually as a management fee, plus about 20% of the fund’s gains.

There is a very sharp dropoff in performance among VC firms, because in the VC business both success and failure are self-perpetuating. When an investment scores spectacularly, as Google did for Kleiner and Sequoia, it generates a lot of good publicity for the VCs. And many founders prefer to take money from successful VC firms, because of the legitimacy it confers. Hence a vicious (for the losers) cy-

cle: VC firms that have been doing badly will only get the deals the bigger fish have rejected, causing them to continue to do badly.

As a result, of the thousand or so VC funds in the US now, only about 50 are likely to make money, and it is very hard for a new fund to break into this group.

In a sense, the lower-tier VC firms are a bargain for founders. They may not be quite as smart or as well connected as the big-name firms, but they are much hungrier for deals. This means you should be able to get better terms from them.

Better how? The most obvious is valuation: they'll take less of your company. But as well as money, there's power. I think founders will increasingly be able to stay on as CEO, and on terms that will make it fairly hard to fire them later.

The most dramatic change, I predict, is that VCs will allow founders to cash out partially by selling some of their stock direct to the VC firm. VCs have traditionally resisted letting founders get anything before the ultimate "liquidity event." But they're also desperate for deals. And since I know from my own experience that the rule against buying stock from founders is a stupid one, this is a natural place for things to give as venture funding becomes more and more a seller's market.

The disadvantage of taking money from less known firms is that people will assume, correctly or not, that you were turned down by the more exalted ones. But, like where you went to college, the name of your VC stops mattering once you have some performance to measure. So the more confident you are, the less you need a brand-name VC. We funded Viaweb entirely with angel money; it never occurred to us that the backing of a well known VC firm would make us seem more impressive.⁵

⁵This was partly confidence, and partly simple ignorance. We didn't know ourselves which VC firms were the impressive ones. We thought software was all that mattered. But that turned out to be the right direction to be naive in: it's much better to overestimate than underestimate the importance of making a good product.

Another danger of less known firms is that, like angels, they have less reputation to protect. I suspect it's the lower-tier firms that are responsible for most of the tricks that have given VCs such a bad reputation among hackers. They are doubly hosed: the general partners themselves are less able, and yet they have harder problems to solve, because the top VCs skim off all the best deals, leaving the lower-tier firms exactly the startups that are likely to blow up.

For example, lower-tier firms are much more likely to pretend to want to do a deal with you just to lock you up while they decide if they really want to. One experienced CFO said:

The better ones usually will not give a term sheet unless they really want to do a deal. The second or third tier firms have a much higher break rate—it could be as high as 50%.

It's obvious why: the lower-tier firms' biggest fear, when chance throws them a bone, is that one of the big dogs will notice and take it away. The big dogs don't have to worry about that.

Falling victim to this trick could really hurt you. As one VC told me:

If you were talking to four VCs, told three of them that you accepted a term sheet, and then have to call them back to tell them you were just kidding, you are absolutely damaged goods.

Here's a partial solution: when a VC offers you a term sheet, ask how many of their last 10 term sheets turned into deals. This will at least force them to lie outright if they want to mislead you.

Not all the people who work at VC firms are partners. Most firms also have a handful of junior employees called something like associates or analysts. If you get a call from a VC firm, go to their web site and check whether the person you talked to is a partner. Odds are it will

be a junior person; they scour the web looking for startups their bosses could invest in. The junior people will tend to seem very positive about your company. They're not pretending; they *want* to believe you're a hot prospect, because it would be a huge coup for them if their firm invested in a company they discovered. Don't be misled by this optimism. It's the partners who decide, and they view things with a colder eye.

Because VCs invest large amounts, the money comes with more restrictions. Most only come into effect if the company gets into trouble. For example, VCs generally write it into the deal that in any sale, they get their investment back first. So if the company gets sold at a low price, the founders could get nothing. Some VCs now require that in any sale they get 4x their investment back before the common stock holders (that is, you) get anything, but this is an abuse that should be resisted.

Another difference with large investments is that the founders are usually required to accept "vesting"—to surrender their stock and earn it back over the next 4-5 years. VCs don't want to invest millions in a company the founders could just walk away from. Financially, vesting has little effect, but in some situations it could mean founders will have less power. If VCs got de facto control of the company and fired one of the founders, he'd lose any unvested stock unless there was specific protection against this. So vesting would in that situation force founders to toe the line.

The most noticeable change when a startup takes serious funding is that the founders will no longer have complete control. Ten years ago VCs used to insist that founders step down as CEO and hand the job over to a business guy they supplied. This is less the rule now, partly because the disasters of the Bubble showed that generic business guys don't make such great CEOs.

But while founders will increasingly be able to stay on as CEO, they'll have to cede some power, because the board of directors will become more powerful. In the seed stage, the board is generally a formality;

if you want to talk to the other board members, you just yell into the next room. This stops with VC-scale money. In a typical VC funding deal, the board of directors might be composed of two VCs, two founders, and one outside person acceptable to both. The board will have ultimate power, which means the founders now have to convince instead of commanding.

This is not as bad as it sounds, however. Bill Gates is in the same position; he doesn't have majority control of Microsoft; in principle he also has to convince instead of commanding. And yet he seems pretty commanding, doesn't he? As long as things are going smoothly, boards don't interfere much. The danger comes when there's a bump in the road, as happened to Steve Jobs at Apple.

Like angels, VCs prefer to invest in deals that come to them through people they know. So while nearly all VC funds have some address you can send your business plan to, VCs privately admit the chance of getting funding by this route is near zero. One recently told me that he did not know a single startup that got funded this way.

I suspect VCs accept business plans "over the transom" more as a way to keep tabs on industry trends than as a source of deals. In fact, I would strongly advise against mailing your business plan randomly to VCs, because they treat this as evidence of laziness. Do the extra work of getting personal introductions. As one VC put it:

I'm not hard to find. I know a lot of people. If you can't find some way to reach me, how are you going to create a successful company?

One of the most difficult problems for startup founders is deciding when to approach VCs. You really only get one chance, because they rely heavily on first impressions. And you can't approach some and save others for later, because (a) they ask who else you've talked to and when and (b) they talk among themselves. If you're talking to one VC and he finds out that you were rejected by another several months ago, you'll definitely seem shopworn.

So when do you approach VCs? When you can convince them. If the founders have impressive resumes and the idea isn't hard to understand, you could approach VCs quite early. Whereas if the founders are unknown and the idea is very novel, you might have to launch the thing and show that users loved it before VCs would be convinced.

If several VCs are interested in you, they will sometimes be willing to split the deal between them. They're more likely to do this if they're close in the VC pecking order. Such deals may be a net win for founders, because you get multiple VCs interested in your success, and you can ask each for advice about the other. One founder I know wrote:

Two-firm deals are great. It costs you a little more equity, but being able to play the two firms off each other (as well as ask one if the other is being out of line) is invaluable.

When you do negotiate with VCs, remember that they've done this a lot more than you have. They've invested in dozens of startups, whereas this is probably the first you've founded. But don't let them or the situation intimidate you. The average founder is smarter than the average VC. So just do what you'd do in any complex, unfamiliar situation: proceed deliberately, and question anything that seems odd.

It is, unfortunately, common for VCs to put terms in an agreement whose consequences surprise founders later, and also common for VCs to defend things they do by saying that they're standard in the industry. Standard, schmandard; the whole industry is only a few decades old, and rapidly evolving. The concept of "standard" is a useful one when you're operating on a small scale (Y Combinator uses identical terms for every deal because for tiny seed-stage investments it's not worth the overhead of negotiating individual deals), but it doesn't apply at the VC level. On that scale, every negotiation is unique.

Most successful startups get money from more than one of the preceding five sources.⁶ And, confusingly, the names of funding sources also tend to be used as the names of different rounds. The best way to explain how it all works is to follow the case of a hypothetical startup.

Stage 1: Seed Round

Our startup begins when a group of three friends have an idea—either an idea for something they might build, or simply the idea “let’s start a company.” Presumably they already have some source of food and shelter. But if you have food and shelter, you probably also have something you’re supposed to be working on: either classwork, or a job. So if you want to work full-time on a startup, your money situation will probably change too.

A lot of startup founders say they started the company without any idea of what they planned to do. This is actually less common than it seems: many have to claim they thought of the idea after quitting because otherwise their former employer would own it.

The three friends decide to take the leap. Since most startups are in competitive businesses, you not only want to work full-time on them, but more than full-time. So some or all of the friends quit their jobs or leave school. (Some of the founders in a startup can stay in grad school, but at least one has to make the company his full-time job.)

⁶I’ve omitted one source: government grants. I don’t think these are even worth thinking about for the average startup. Governments may mean well when they set up grant programs to encourage startups, but what they give with one hand they take away with the other: the process of applying is inevitably so arduous, and the restrictions on what you can do with the money so burdensome, that it would be easier to take a job to get the money.

You should be especially suspicious of grants whose purpose is some kind of social engineering—e.g. to encourage more startups to be started in Mississippi. Free money to start a startup in a place where few succeed is hardly free.

Some government agencies run venture funding groups, which make investments rather than giving grants. For example, the CIA runs a venture fund called In-Q-Tel that is modelled on private sector funds and apparently generates good returns. They would probably be worth approaching—if you don’t mind taking money from the CIA.

They're going to run the company out of one of their apartments at first, and since they don't have any users they don't have to pay much for infrastructure. Their main expenses are setting up the company, which costs a couple thousand dollars in legal work and registration fees, and the living expenses of the founders.

The phrase "seed investment" covers a broad range. To some VC firms it means \$500,000, but to most startups it means several months' living expenses. We'll suppose our group of friends start with \$15,000 from their friend's rich uncle, who they give 5% of the company in return. There's only common stock at this stage. They leave 20% as an options pool for later employees (but they set things up so that they can issue this stock to themselves if they get bought early and most is still unissued), and the three founders each get 25%.

By living really cheaply they think they can make the remaining money last five months. When you have five months' runway left, how soon do you need to start looking for your next round? Answer: immediately. It takes time to find investors, and time (always more than you expect) for the deal to close even after they say yes. So if our group of founders know what they're doing they'll start sniffing around for angel investors right away. But of course their main job is to build version 1 of their software.

The friends might have liked to have more money in this first phase, but being slightly underfunded teaches them an important lesson. For a startup, cheapness is power. The lower your costs, the more options you have—not just at this stage, but at every point till you're profitable. When you have a high "burn rate," you're always under time pressure, which means (a) you don't have time for your ideas to evolve, and (b) you're often forced to take deals you don't like.

Every startup's rule should be: spend little, and work fast.

After ten weeks' work the three friends have built a prototype that gives one a taste of what their product will do. It's not what they originally set out to do—in the process of writing it, they had some new ideas.

And it only does a fraction of what the finished product will do, but that fraction includes stuff that no one else has done before.

They've also written at least a skeleton business plan, addressing the five fundamental questions: what they're going to do, why users need it, how large the market is, how they'll make money, and who the competitors are and why this company is going to beat them. (That last has to be more specific than "they suck" or "we'll work really hard.")

If you have to choose between spending time on the demo or the business plan, spend most on the demo. Software is not only more convincing, but a better way to explore ideas.

Stage 2: Angel Round

While writing the prototype, the group has been traversing their network of friends in search of angel investors. They find some just as the prototype is demoable. When they demo it, one of the angels is willing to invest. Now the group is looking for more money: they want enough to last for a year, and maybe to hire a couple friends. So they're going to raise \$200,000.

The angel agrees to invest at a pre-money valuation of \$1 million. The company issues \$200,000 worth of new shares to the angel; if there were 1000 shares before the deal, this means 200 additional shares. The angel now owns 200/1200 shares, or a sixth of the company, and all the previous shareholders' percentage ownership is diluted by a sixth. After the deal, the capitalization table looks like this:

Shareholder	Shares	Percent
angel	200	16.7
uncle	50	4.2
each founder	250	20.8
option pool	200	16.7
total	1200	

To keep things simple, I had the angel do a straight cash for stock deal. In reality the angel might be more likely to make the investment in the form of a convertible loan. A convertible loan is a loan that can be converted into stock later; it works out the same as a stock purchase in the end, but gives the angel more protection against being squashed by VCs in future rounds.

Who pays the legal bills for this deal? The startup, remember, only has a couple thousand left. In practice this turns out to be a sticky problem that usually gets solved in some improvised way. Maybe the startup can find lawyers who will do it cheaply in the hope of future work if the startup succeeds. Maybe someone has a lawyer friend. Maybe the angel pays for his lawyer to represent both sides. (Make sure if you take the latter route that the lawyer is *representing* you rather than merely advising you, or his only duty is to the investor.)

An angel investing \$200k would probably expect a seat on the board of directors. He might also want preferred stock, meaning a special class of stock that has some additional rights over the common stock everyone else has. Typically these rights include vetoes over major strategic decisions, protection against being diluted in future rounds, and the right to get one's investment back first if the company is sold.

Some investors might expect the founders to accept vesting for a sum this size, and others wouldn't. VCs are more likely to require vesting than angels. At Viaweb we managed to raise \$2.5 million from angels without ever accepting vesting, largely because we were so inexperienced that we were appalled at the idea. In practice this turned out to be good, because it made us harder to push around.

Our experience was unusual; vesting is the norm for amounts that size. Y Combinator doesn't require vesting, because (a) we invest such small amounts, and (b) we think it's unnecessary, and that the hope of getting rich is enough motivation to keep founders at work. But maybe if we were investing millions we would think differently.

I should add that vesting is also a way for founders to protect themselves against one another. It solves the problem of what to do if one of the founders quits. So some founders impose it on themselves when they start the company.

The angel deal takes two weeks to close, so we are now three months into the life of the company.

The point after you get the first big chunk of angel money will usually

be the happiest phase in a startup's life. It's a lot like being a postdoc: you have no immediate financial worries, and few responsibilities. You get to work on juicy kinds of work, like designing software. You don't have to spend time on bureaucratic stuff, because you haven't hired any bureaucrats yet. Enjoy it while it lasts, and get as much done as you can, because you will never again be so productive.

With an apparently inexhaustible sum of money sitting safely in the bank, the founders happily set to work turning their prototype into something they can release. They hire one of their friends—at first just as a consultant, so they can try him out—and then a month later as employee #1. They pay him the smallest salary he can live on, plus 3% of the company in restricted stock, vesting over four years. (So after this the option pool is down to 13.7%).⁷ They also spend a little money on a freelance graphic designer.

How much stock do you give early employees? That varies so much that there's no conventional number. If you get someone really good, really early, it might be wise to give him as much stock as the founders. The one universal rule is that the amount of stock an employee gets decreases polynomially with the age of the company. In other words, you get rich as a power of how early you were. So if some friends want you to come work for their startup, don't wait several months before deciding.

A month later, at the end of month four, our group of founders have something they can launch. Gradually through word of mouth they start to get users. Seeing the system in use by real users—people they don't know—gives them lots of new ideas. Also they find they now worry obsessively about the status of their server. (How relaxing founders' lives must have been when startups wrote VisiCalc.)

By the end of month six, the system is starting to have a solid core

⁷Options have largely been replaced with restricted stock, which amounts to the same thing. Instead of earning the right to buy stock, the employee gets the stock up front, and earns the right not to have to give it back. The shares set aside for this purpose are still called the "option pool."

of features, and a small but devoted following. People start to write about it, and the founders are starting to feel like experts in their field.

We'll assume that their startup is one that could put millions more to use. Perhaps they need to spend a lot on marketing, or build some kind of expensive infrastructure, or hire highly paid salesmen. So they decide to start talking to VCs. They get introductions to VCs from various sources: their angel investor connects them with a couple; they meet a few at conferences; a couple VCs call them after reading about them.

Step 3: Series A Round

Armed with their now somewhat fleshed-out business plan and able to demo a real, working system, the founders visit the VCs they have introductions to. They find the VCs intimidating and inscrutable. They all ask the same question: who else have you pitched to? (VCs are like high school girls: they're acutely aware of their position in the VC pecking order, and their interest in a company is a function of the interest other VCs show in it.)

One of the VC firms says they want to invest and offers the founders a term sheet. A term sheet is a summary of what the deal terms will be when and if they do a deal; lawyers will fill in the details later. By accepting the term sheet, the startup agrees to turn away other VCs for some set amount of time while this firm does the "due diligence" required for the deal. Due diligence is the corporate equivalent of a background check: the purpose is to uncover any hidden bombs that might sink the company later, like serious design flaws in the product, pending lawsuits against the company, intellectual property issues, and so on. VCs' legal and financial due diligence is pretty thorough, but the technical due diligence is generally a joke.⁸

The due diligence discloses no ticking bombs, and six weeks later they

⁸First-rate technical people do not generally hire themselves out to do due diligence for VCs. So the most difficult part for startup founders is often responding politely to the inane questions of the "expert" they send to look you over.

go ahead with the deal. Here are the terms: a \$2 million investment at a pre-money valuation of \$4 million, meaning that after the deal closes the VCs will own a third of the company ($2 / (4 + 2)$). The VCs also insist that prior to the deal the option pool be enlarged by an additional hundred shares. So the total number of new shares issued is 750, and the cap table becomes: shareholder shares percent —

————— - VCs 650 33.3 angel 200 10.3 uncle 50 2.6 each founder 250 12.8 employee 36* 1.8 *unvested option pool 264 13.5

— - — total 1950 100 This picture is unrealistic in several respects. For example, while the percentages might end up looking like this, it's unlikely that the VCs would keep the existing numbers of shares. In fact, every bit of the startup's paperwork would probably be replaced, as if the company were being founded anew. Also, the money might come in several tranches, the later ones subject to various conditions—though this is apparently more common in deals with lower-tier VCs (whose lot in life is to fund more dubious startups) than with the top firms.

And of course any VCs reading this are probably rolling on the floor laughing at how my hypothetical VCs let the angel keep his 10.3 of the company. I admit, this is the Bambi version; in simplifying the picture, I've also made everyone nicer. In the real world, VCs regard angels the way a jealous husband feels about his wife's previous boyfriends. To them the company didn't exist before they invested in it.⁹

I don't want to give the impression you have to do an angel round before going to VCs. In this example I stretched things out to show multiple sources of funding in action. Some startups could go directly from seed funding to a VC round; several of the companies we've

⁹VCs regularly wipe out angels by issuing arbitrary amounts of new stock. They seem to have a standard piece of casuistry for this situation: that the angels are no longer working to help the company, and so don't deserve to keep their stock. This of course reflects a willful misunderstanding of what investment means; like any investor, the angel is being compensated for risks he took earlier. By a similar logic, one could argue that the VCs should be deprived of their shares when the company goes public.

funded have.

The founders are required to vest their shares over four years, and the board is now reconstituted to consist of two VCs, two founders, and a fifth person acceptable to both. The angel investor cheerfully surrenders his board seat.

At this point there is nothing new our startup can teach us about funding—or at least, nothing good.¹⁰ The startup will almost certainly hire more people at this point; those millions must be put to work, after all. The company may do additional funding rounds, presumably at higher valuations. They may if they are extraordinarily fortunate do an IPO, which we should remember is also in principle a round of funding, regardless of its *de facto* purpose. But that, if not beyond the bounds of possibility, is beyond the scope of this article.

Deals Fall Through

Anyone who's been through a startup will find the preceding portrait to be missing something: disasters. If there's one thing all startups have in common, it's that something is always going wrong. And nowhere more than in matters of funding.

For example, our hypothetical startup never spent more than half of one round before securing the next. That's more ideal than typical. Many startups—even successful ones—come close to running out of money at some point. Terrible things happen to startups when they run out of money, because they're designed for growth, not adversity.

¹⁰One new thing the company might encounter is a *down round*, or a funding round at valuation lower than the previous round. Down rounds are bad news; it is generally the common stock holders who take the hit. Some of the most fearsome provisions in VC deal terms have to do with down rounds—like “full ratchet anti-dilution,” which is as frightening as it sounds.

Founders are tempted to ignore these clauses, because they think the company will either be a big success or a complete bust. VCs know otherwise: it's not uncommon for startups to have moments of adversity before they ultimately succeed. So it's worth negotiating anti-dilution provisions, even though you don't think you need to, and VCs will try to make you feel that you're being gratuitously troublesome.

But the most unrealistic thing about the series of deals I've described is that they all closed. In the startup world, closing is not what deals do. What deals do is fall through. If you're starting a startup you would do well to remember that. Birds fly; fish swim; deals fall through.

Why? Partly the reason deals seem to fall through so often is that you lie to yourself. You want the deal to close, so you start to believe it will. But even correcting for this, startup deals fall through alarmingly often—far more often than, say, deals to buy real estate. The reason is that it's such a risky environment. People about to fund or acquire a startup are prone to wicked cases of buyer's remorse. They don't really grasp the risk they're taking till the deal's about to close. And then they panic. And not just inexperienced angel investors, but big companies too.

So if you're a startup founder wondering why some angel investor isn't returning your phone calls, you can at least take comfort in the thought that the same thing is happening to other deals a hundred times the size.

The example of a startup's history that I've presented is like a skeleton—accurate so far as it goes, but needing to be fleshed out to be a complete picture. To get a complete picture, just add in every possible disaster.

A frightening prospect? In a way. And yet also in a way encouraging. The very uncertainty of startups frightens away almost everyone. People overvalue stability—especially young people, who ironically need it least. And so in starting a startup, as in any really bold undertaking, merely deciding to do it gets you halfway there. On the day of the race, most of the other runners won't show up.

Thanks to Sam Altman, Hutch Fishman, Steve Huffman, Jessica Livingston, Sessa Pratap, Stan Reiss, Andy Singleton, Zak Stone, and Aaron Swartz for reading drafts of this.

Chapter 21

Web 2.0

November 2005

Does “Web 2.0” mean anything? Till recently I thought it didn’t, but the truth turns out to be more complicated. Originally, yes, it was meaningless. Now it seems to have acquired a meaning. And yet those who dislike the term are probably right, because if it means what I think it does, we don’t need it.

I first heard the phrase “Web 2.0” in the name of the Web 2.0 conference in 2004. At the time it was supposed to mean using “the web as a platform,” which I took to refer to web-based applications.¹

So I was surprised at a conference this summer when Tim O’Reilly led a session intended to figure out a definition of “Web 2.0.” Didn’t it already mean using the web as a platform? And if it didn’t already mean something, why did we need the phrase at all?

Origins

Tim says the phrase “Web 2.0” first arose in “a brainstorming session between O’Reilly and Medialive International.” What is Medialive International? “Producers of technology tradeshows and confer-

¹From the conference site, June 2004: “While the first wave of the Web was closely tied to the browser, the second wave extends applications across the web and enables a new generation of services and business opportunities.” To the extent this means anything, it seems to be about web-based applications.

ences,” according to their site. So presumably that’s what this brainstorming session was about. O’Reilly wanted to organize a conference about the web, and they were wondering what to call it.

I don’t think there was any deliberate plan to suggest there was a new *version* of the web. They just wanted to make the point that the web mattered again. It was a kind of semantic deficit spending: they knew new things were coming, and the “2.0” referred to whatever those might turn out to be.

And they were right. New things were coming. But the new version number led to some awkwardness in the short term. In the process of developing the pitch for the first conference, someone must have decided they’d better take a stab at explaining what that “2.0” referred to. Whatever it meant, “the web as a platform” was at least not too constricting.

The story about “Web 2.0” meaning the web as a platform didn’t live much past the first conference. By the second conference, what “Web 2.0” seemed to mean was something about democracy. At least, it did when people wrote about it online. The conference itself didn’t seem very grassroots. It cost \$2800, so the only people who could afford to go were VCs and people from big companies.

And yet, oddly enough, Ryan Singel’s article about the conference in *Wired News* spoke of “throngs of geeks.” When a friend of mine asked Ryan about this, it was news to him. He said he’d originally written something like “throngs of VCs and biz dev guys” but had later shortened it just to “throngs,” and that this must have in turn been expanded by the editors into “throngs of geeks.” After all, a Web 2.0 conference would presumably be full of geeks, right?

Well, no. There were about 7. Even Tim O’Reilly was wearing a suit, a sight so alien I couldn’t parse it at first. I saw him walk by and said to one of the O’Reilly people “that guy looks just like Tim.”

“Oh, that’s Tim. He bought a suit.” I ran after him, and sure enough, it was. He explained that he’d just bought it in Thailand.

The 2005 Web 2.0 conference reminded me of Internet trade shows during the Bubble, full of prowling VCs looking for the next hot startup. There was that same odd atmosphere created by a large number of people determined not to miss out. Miss out on what? They didn't know. Whatever was going to happen—whatever Web 2.0 turned out to be.

I wouldn't quite call it "Bubble 2.0" just because VCs are eager to invest again. The Internet is a genuinely big deal. The bust was as much an overreaction as the boom. It's to be expected that once we started to pull out of the bust, there would be a lot of growth in this area, just as there was in the industries that spiked the sharpest before the Depression.

The reason this won't turn into a second Bubble is that the IPO market is gone. Venture investors are driven by exit strategies. The reason they were funding all those laughable startups during the late 90s was that they hoped to sell them to gullible retail investors; they hoped to be laughing all the way to the bank. Now that route is closed. Now the default exit strategy is to get bought, and acquirers are less prone to irrational exuberance than IPO investors. The closest you'll get to Bubble valuations is Rupert Murdoch paying \$580 million for Myspace. That's only off by a factor of 10 or so.

1. Ajax

Does "Web 2.0" mean anything more than the name of a conference yet? I don't like to admit it, but it's starting to. When people say "Web 2.0" now, I have some idea what they mean. And the fact that I both despise the phrase and understand it is the surest proof that it has started to mean something.

One ingredient of its meaning is certainly Ajax, which I can still only just bear to use without scare quotes. Basically, what "Ajax" means is "Javascript now works." And that in turn means that web-based applications can now be made to work much more like desktop ones.

As you read this, a whole new generation of software is being written

to take advantage of Ajax. There hasn't been such a wave of new applications since microcomputers first appeared. Even Microsoft sees it, but it's too late for them to do anything more than leak "internal" documents designed to give the impression they're on top of this new trend.

In fact the new generation of software is being written way too fast for Microsoft even to channel it, let alone write their own in house. Their only hope now is to buy all the best Ajax startups before Google does. And even that's going to be hard, because Google has as big a head start in buying microstartups as it did in search a few years ago. After all, Google Maps, the canonical Ajax application, was the result of a startup they bought.

So ironically the original description of the Web 2.0 conference turned out to be partially right: web-based applications are a big component of Web 2.0. But I'm convinced they got this right by accident. The Ajax boom didn't start till early 2005, when Google Maps appeared and the term "Ajax" was coined.

2. Democracy

The second big element of Web 2.0 is democracy. We now have several examples to prove that amateurs can surpass professionals, when they have the right kind of system to channel their efforts. Wikipedia may be the most famous. Experts have given Wikipedia middling reviews, but they miss the critical point: it's good enough. And it's free, which means people actually read it. On the web, articles you have to pay for might as well not exist. Even if you were willing to pay to read them yourself, you can't link to them. They're not part of the conversation.

Another place democracy seems to win is in deciding what counts as news. I never look at any news site now except Reddit.² I know

²Disclosure: Reddit was funded by Y Combinator. But although I started using it out of loyalty to the home team, I've become a genuine addict. While we're at it, I'm also an investor in MSFT, having sold all my shares earlier this year.

if something major happens, or someone writes a particularly interesting article, it will show up there. Why bother checking the front page of any specific paper or magazine? Reddit's like an RSS feed for the whole web, with a filter for quality. Similar sites include Digg, a technology news site that's rapidly approaching Slashdot in popularity, and del.icio.us, the collaborative bookmarking network that set off the "tagging" movement. And whereas Wikipedia's main appeal is that it's good enough and free, these sites suggest that voters do a significantly better job than human editors.

The most dramatic example of Web 2.0 democracy is not in the selection of ideas, but their production. I've noticed for a while that the stuff I read on individual people's sites is as good as or better than the stuff I read in newspapers and magazines. And now I have independent evidence: the top links on Reddit are generally links to individual people's sites rather than to magazine articles or news stories.

My experience of writing for magazines suggests an explanation. Editors. They control the topics you can write about, and they can generally rewrite whatever you produce. The result is to damp extremes. Editing yields 95th percentile writing—95% of articles are improved by it, but 5% are dragged down. 5% of the time you get "throngs of geeks."

On the web, people can publish whatever they want. Nearly all of it falls short of the editor-damped writing in print publications. But the pool of writers is very, very large. If it's large enough, the lack of damping means the best writing online should surpass the best in print.³ And now that the web has evolved mechanisms for selecting good stuff, the web wins net. Selection beats damping, for the same reason market economies beat centrally planned ones.

Even the startups are different this time around. They are to the startups of the Bubble what bloggers are to the print media. During the

³I'm not against editing. I spend more time editing than writing, and I have a group of picky friends who proofread almost everything I write. What I dislike is editing done after the fact by someone else.

Bubble, a startup meant a company headed by an MBA that was blowing through several million dollars of VC money to “get big fast” in the most literal sense. Now it means a smaller, younger, more technical group that just decided to make something great. They’ll decide later if they want to raise VC-scale funding, and if they take it, they’ll take it on their terms.

3. Don’t Maltreat Users

I think everyone would agree that democracy and Ajax are elements of “Web 2.0.” I also see a third: not to maltreat users. During the Bubble a lot of popular sites were quite high-handed with users. And not just in obvious ways, like making them register, or subjecting them to annoying ads. The very design of the average site in the late 90s was an abuse. Many of the most popular sites were loaded with obtrusive branding that made them slow to load and sent the user the message: this is our site, not yours. (There’s a physical analog in the Intel and Microsoft stickers that come on some laptops.)

I think the root of the problem was that sites felt they were giving something away for free, and till recently a company giving anything away for free could be pretty high-handed about it. Sometimes it reached the point of economic sadism: site owners assumed that the more pain they caused the user, the more benefit it must be to them. The most dramatic remnant of this model may be at salon.com, where you can read the beginning of a story, but to get the rest you have sit through a *movie*.

At Y Combinator we advise all the startups we fund never to lord it over users. Never make users register, unless you need to in order to store something for them. If you do make users register, never make them wait for a confirmation link in an email; in fact, don’t even ask for their email address unless you need it for some reason. Don’t ask them any unnecessary questions. Never send them email unless they explicitly ask for it. Never frame pages you link to, or open them in new windows. If you have a free version and a pay version, don’t make the free version too restricted. And if you find yourself asking “should

we allow users to do x?” just answer “yes” whenever you’re unsure. Err on the side of generosity.

In *How to Start a Startup* I advised startups never to let anyone fly under them, meaning never to let any other company offer a cheaper, easier solution. Another way to fly low is to give users more power. Let users do what they want. If you don’t and a competitor does, you’re in trouble.

iTunes is Web 2.0ish in this sense. Finally you can buy individual songs instead of having to buy whole albums. The recording industry hated the idea and resisted it as long as possible. But it was obvious what users wanted, so Apple flew under the labels.⁴ Though really it might be better to describe iTunes as Web 1.5. Web 2.0 applied to music would probably mean individual bands giving away DRMless songs for free.

The ultimate way to be nice to users is to give them something for free that competitors charge for. During the 90s a lot of people probably thought we’d have some working system for micropayments by now. In fact things have gone in the other direction. The most successful sites are the ones that figure out new ways to give stuff away for free. Craigslist has largely destroyed the classified ad sites of the 90s, and OkCupid looks likely to do the same to the previous generation of dating sites.

Serving web pages is very, very cheap. If you can make even a fraction of a cent per page view, you can make a profit. And technology for targeting ads continues to improve. I wouldn’t be surprised if ten years from now eBay had been supplanted by an ad-supported freeBay (or, more likely, gBay).

Odd as it might sound, we tell startups that they should try to make as little money as possible. If you can figure out a way to turn a billion dollar industry into a fifty million dollar industry, so much the better,

⁴Obvious is an understatement. Users had been climbing in through the window for years before Apple finally moved the door.

if all fifty million go to you. Though indeed, making things cheaper often turns out to generate more money in the end, just as automating things often turns out to generate more jobs.

The ultimate target is Microsoft. What a bang that balloon is going to make when someone pops it by offering a free web-based alternative to MS Office.⁵ Who will? Google? They seem to be taking their time. I suspect the pin will be wielded by a couple of 20 year old hackers who are too naive to be intimidated by the idea. (How hard can it be?)

The Common Thread

Ajax, democracy, and not dissing users. What do they all have in common? I didn't realize they had anything in common till recently, which is one of the reasons I disliked the term "Web 2.0" so much. It seemed that it was being used as a label for whatever happened to be new—that it didn't predict anything.

But there is a common thread. Web 2.0 means using the web the way it's meant to be used. The "trends" we're seeing now are simply the inherent nature of the web emerging from under the broken models that got imposed on it during the Bubble.

I realized this when I read an interview with Joe Kraus, the co-founder of Excite.⁶

Excite really never got the business model right at all. We fell into the classic problem of how when a new medium comes out it adopts the practices, the content, the business models of the old medium—which fails, and then the more appropriate models get figured out.

It may have seemed as if not much was happening during the years

⁵Hint: the way to create a web-based alternative to Office may not be to write every component yourself, but to establish a protocol for web-based apps to share a virtual home directory spread across multiple servers. Or it may be to write it all yourself.

⁶In Jessica Livingston's *Founders at Work*.

after the Bubble burst. But in retrospect, something was happening: the web was finding its natural angle of repose. The democracy component, for example—that’s not an innovation, in the sense of something someone made happen. That’s what the web naturally tends to produce.

Ditto for the idea of delivering desktop-like applications over the web. That idea is almost as old as the web. But the first time around it was co-opted by Sun, and we got Java applets. Java has since been remade into a generic replacement for C++, but in 1996 the story about Java was that it represented a new model of software. Instead of desktop applications, you’d run Java “applets” delivered from a server.

This plan collapsed under its own weight. Microsoft helped kill it, but it would have died anyway. There was no uptake among hackers. When you find PR firms promoting something as the next development platform, you can be sure it’s not. If it were, you wouldn’t need PR firms to tell you, because hackers would already be writing stuff on top of it, the way sites like Busmonster used Google Maps as a platform before Google even meant it to be one.

The proof that Ajax is the next hot platform is that thousands of hackers have spontaneously started building things on top of it. Mikey likes it.

There’s another thing all three components of Web 2.0 have in common. Here’s a clue. Suppose you approached investors with the following idea for a Web 2.0 startup:

Sites like del.icio.us and flickr allow users to “tag” content with descriptive tokens. But there is also huge source of *implicit* tags that they ignore: the text within web links. Moreover, these links represent a social network connecting the individuals and organizations who created the pages, and by using graph theory we can compute from this network an estimate of the reputation of each member. We plan to mine the web for these implicit tags, and

use them together with the reputation hierarchy they embody to enhance web searches.

How long do you think it would take them on average to realize that it was a description of Google?

Google was a pioneer in all three components of Web 2.0: their core business sounds crushingly hip when described in Web 2.0 terms, “Don’t maltreat users” is a subset of “Don’t be evil,” and of course Google set off the whole Ajax boom with Google Maps.

Web 2.0 means using the web as it was meant to be used, and Google does. That’s their secret. They’re sailing with the wind, instead of sitting becalmed praying for a business model, like the print media, or trying to tack upwind by suing their customers, like Microsoft and the record labels.⁷

Google doesn’t try to force things to happen their way. They try to figure out what’s going to happen, and arrange to be standing there when it does. That’s the way to approach technology—and as business includes an ever larger technological component, the right way to do business.

The fact that Google is a “Web 2.0” company shows that, while meaningful, the term is also rather bogus. It’s like the word “allopathic.” It just means doing things right, and it’s a bad sign when you have a special word for that.

Thanks to Trevor Blackwell, Sarah Harlin, Jessica Livingston, Peter Norvig, Aaron Swartz, and Jeff Weiner for reading drafts of this, and to the guys at O’Reilly and Adaptive Path for answering my questions.

⁷Microsoft didn’t sue their customers directly, but they seem to have done all they could to help SCO sue them.

Chapter 22

Good and Bad Procrastination

December 2005

The most impressive people I know are all terrible procrastinators. So could it be that procrastination isn't always bad?

Most people who write about procrastination write about how to cure it. But this is, strictly speaking, impossible. There are an infinite number of things you could be doing. No matter what you work on, you're not working on everything else. So the question is not how to avoid procrastination, but how to procrastinate well.

There are three variants of procrastination, depending on what you do instead of working on something: you could work on (a) nothing, (b) something less important, or (c) something more important. That last type, I'd argue, is good procrastination.

That's the "absent-minded professor," who forgets to shave, or eat, or even perhaps look where he's going while he's thinking about some interesting question. His mind is absent from the everyday world because it's hard at work in another.

That's the sense in which the most impressive people I know are all procrastinators. They're type-C procrastinators: they put off working on small stuff to work on big stuff.

What's "small stuff?" Roughly, work that has zero chance of being mentioned in your obituary. It's hard to say at the time what will turn out to be your best work (will it be your magnum opus on Sumerian temple architecture, or the detective thriller you wrote under a pseudonym?), but there's a whole class of tasks you can safely rule out: shaving, doing your laundry, cleaning the house, writing thank-you notes—anything that might be called an errand.

Good procrastination is avoiding errands to do real work.

Good in a sense, at least. The people who want you to do the errands won't think it's good. But you probably have to annoy them if you want to get anything done. The mildest seeming people, if they want to do real work, all have a certain degree of ruthlessness when it comes to avoiding errands.

Some errands, like replying to letters, go away if you ignore them (perhaps taking friends with them). Others, like mowing the lawn, or filing tax returns, only get worse if you put them off. In principle it shouldn't work to put off the second kind of errand. You're going to have to do whatever it is eventually. Why not (as past-due notices are always saying) do it now?

The reason it pays to put off even those errands is that real work needs two things errands don't: big chunks of time, and the right mood. If you get inspired by some project, it can be a net win to blow off everything you were supposed to do for the next few days to work on it. Yes, those errands may cost you more time when you finally get around to them. But if you get a lot done during those few days, you will be net more productive.

In fact, it may not be a difference in degree, but a difference in kind. There may be types of work that can only be done in long, uninterrupted stretches, when inspiration hits, rather than dutifully in scheduled little slices. Empirically it seems to be so. When I think of the people I know who've done great things, I don't imagine them dutifully crossing items off to-do lists. I imagine them sneaking off to work

on some new idea.

Conversely, forcing someone to perform errands synchronously is bound to limit their productivity. The cost of an interruption is not just the time it takes, but that it breaks the time on either side in half. You probably only have to interrupt someone a couple times a day before they're unable to work on hard problems at all.

I've wondered a lot about why startups are most productive at the very beginning, when they're just a couple guys in an apartment. The main reason may be that there's no one to interrupt them yet. In theory it's good when the founders finally get enough money to hire people to do some of the work for them. But it may be better to be overworked than interrupted. Once you dilute a startup with ordinary office workers—with type-B procrastinators—the whole company starts to resonate at their frequency. They're interrupt-driven, and soon you are too.

Errands are so effective at killing great projects that a lot of people use them for that purpose. Someone who has decided to write a novel, for example, will suddenly find that the house needs cleaning. People who fail to write novels don't do it by sitting in front of a blank page for days without writing anything. They do it by feeding the cat, going out to buy something they need for their apartment, meeting a friend for coffee, checking email. "I don't have time to work," they say. And they don't; they've made sure of that.

(There's also a variant where one has no place to work. The cure is to visit the places where famous people worked, and see how unsuitable they were.)

I've used both these excuses at one time or another. I've learned a lot of tricks for making myself work over the last 20 years, but even now I don't win consistently. Some days I get real work done. Other days are eaten up by errands. And I know it's usually my fault: I *let* errands eat up the day, to avoid facing some hard problem.

The most dangerous form of procrastination is unacknowledged type-B procrastination, because it doesn't feel like procrastination. You're

“getting things done.” Just the wrong things.

Any advice about procrastination that concentrates on crossing things off your to-do list is not only incomplete, but positively misleading, if it doesn't consider the possibility that the to-do list is itself a form of type-B procrastination. In fact, possibility is too weak a word. Nearly everyone's is. Unless you're working on the biggest things you could be working on, you're type-B procrastinating, no matter how much you're getting done.

In his famous essay *You and Your Research* (which I recommend to anyone ambitious, no matter what they're working on), Richard Hamming suggests that you ask yourself three questions:

1. What are the most important problems in your field?
2. Are you working on one of them?
3. Why not?

Hamming was at Bell Labs when he started asking such questions. In principle anyone there ought to have been able to work on the most important problems in their field. Perhaps not everyone can make an equally dramatic mark on the world; I don't know; but whatever your capacities, there are projects that stretch them. So Hamming's exercise can be generalized to:

What's the best thing you could be working on, and why aren't you?

Most people will shy away from this question. I shy away from it myself; I see it there on the page and quickly move on to the next sentence. Hamming used to go around actually asking people this, and it didn't make him popular. But it's a question anyone ambitious should face.

The trouble is, you may end up hooking a very big fish with this bait. To do good work, you need to do more than find good projects. Once you've found them, you have to get yourself to work on them, and that can be hard. The bigger the problem, the harder it is to get yourself to work on it.

Of course, the main reason people find it difficult to work on a particular problem is that they don't enjoy it. When you're young, especially, you often find yourself working on stuff you don't really like—because it seems impressive, for example, or because you've been assigned to work on it. Most grad students are stuck working on big problems they don't really like, and grad school is thus synonymous with procrastination.

But even when you like what you're working on, it's easier to get yourself to work on small problems than big ones. Why? Why is it so hard to work on big problems? One reason is that you may not get any reward in the foreseeable future. If you work on something you can finish in a day or two, you can expect to have a nice feeling of accomplishment fairly soon. If the reward is indefinitely far in the future, it seems less real.

Another reason people don't work on big projects is, ironically, fear of wasting time. What if they fail? Then all the time they spent on it will be wasted. (In fact it probably won't be, because work on hard projects almost always leads somewhere.)

But the trouble with big problems can't be just that they promise no immediate reward and might cause you to waste a lot of time. If that were all, they'd be no worse than going to visit your in-laws. There's more to it than that. Big problems are *terrifying*. There's an almost physical pain in facing them. It's like having a vacuum cleaner hooked up to your imagination. All your initial ideas get sucked out immediately, and you don't have any more, and yet the vacuum cleaner is still sucking.

You can't look a big problem too directly in the eye. You have to

approach it somewhat obliquely. But you have to adjust the angle just right: you have to be facing the big problem directly enough that you catch some of the excitement radiating from it, but not so much that it paralyzes you. You can tighten the angle once you get going, just as a sailboat can sail closer to the wind once it gets underway.

If you want to work on big things, you seem to have to trick yourself into doing it. You have to work on small things that could grow into big things, or work on successively larger things, or split the moral load with collaborators. It's not a sign of weakness to depend on such tricks. The very best work has been done this way.

When I talk to people who've managed to make themselves work on big things, I find that all blow off errands, and all feel guilty about it. I don't think they should feel guilty. There's more to do than anyone could. So someone doing the best work they can is inevitably going to leave a lot of errands undone. It seems a mistake to feel bad about that.

I think the way to "solve" the problem of procrastination is to let delight pull you instead of making a to-do list push you. Work on an ambitious project you really enjoy, and sail as close to the wind as you can, and you'll leave the right things undone.

Thanks to Trevor Blackwell, Jessica Livingston, and Robert Morris for reading drafts of this.

Chapter 23

How to Do What You Love

January 2006

To do something well you have to like it. That idea is not exactly novel. We've got it down to four words: "Do what you love." But it's not enough just to tell people that. Doing what you love is complicated.

The very idea is foreign to what most of us learn as kids. When I was a kid, it seemed as if work and fun were opposites by definition. Life had two states: some of the time adults were making you do things, and that was called work; the rest of the time you could do what you wanted, and that was called playing. Occasionally the things adults made you do were fun, just as, occasionally, playing wasn't — for example, if you fell and hurt yourself. But except for these few anomalous cases, work was pretty much defined as not-fun.

And it did not seem to be an accident. School, it was implied, was tedious *because* it was preparation for grownup work.

The world then was divided into two groups, grownups and kids. Grownups, like some kind of cursed race, had to work. Kids didn't, but they did have to go to school, which was a dilute version of work meant to prepare us for the real thing. Much as we disliked school, the grownups all agreed that grownup work was worse, and that we had it easy.

Teachers in particular all seemed to believe implicitly that work was

not fun. Which is not surprising: work wasn't fun for most of them. Why did we have to memorize state capitals instead of playing dodgeball? For the same reason they had to watch over a bunch of kids instead of lying on a beach. You couldn't just do what you wanted.

I'm not saying we should let little kids do whatever they want. They may have to be made to work on certain things. But if we make kids work on dull stuff, it might be wise to tell them that tediousness is not the defining quality of work, and indeed that the reason they have to work on dull stuff now is so they can work on more interesting stuff later.¹

Once, when I was about 9 or 10, my father told me I could be whatever I wanted when I grew up, so long as I enjoyed it. I remember that precisely because it seemed so anomalous. It was like being told to use dry water. Whatever I thought he meant, I didn't think he meant work could *literally* be fun — fun like playing. It took me years to grasp that.

Jobs

By high school, the prospect of an actual job was on the horizon. Adults would sometimes come to speak to us about their work, or we would go to see them at work. It was always understood that they enjoyed what they did. In retrospect I think one may have: the private jet pilot. But I don't think the bank manager really did.

The main reason they all acted as if they enjoyed their work was presumably the upper-middle class convention that you're supposed to. It would not merely be bad for your career to say that you despised your job, but a social faux-pas.

Why is it conventional to pretend to like what you do? The first sentence of this essay explains that. If you have to like something to do it well, then the most successful people will all like what they do. That's

¹Currently we do the opposite: when we make kids do boring work, like arithmetic drills, instead of admitting frankly that it's boring, we try to disguise it with superficial decorations.

where the upper-middle class tradition comes from. Just as houses all over America are full of chairs that are, without the owners even knowing it, nth-degree imitations of chairs designed 250 years ago for French kings, conventional attitudes about work are, without the owners even knowing it, nth-degree imitations of the attitudes of people who've done great things.

What a recipe for alienation. By the time they reach an age to think about what they'd like to do, most kids have been thoroughly misled about the idea of loving one's work. School has trained them to regard work as an unpleasant duty. Having a job is said to be even more onerous than schoolwork. And yet all the adults claim to like what they do. You can't blame kids for thinking "I am not like these people; I am not suited to this world."

Actually they've been told three lies: the stuff they've been taught to regard as work in school is not real work; grownup work is not (necessarily) worse than schoolwork; and many of the adults around them are lying when they say they like what they do.

The most dangerous liars can be the kids' own parents. If you take a boring job to give your family a high standard of living, as so many people do, you risk infecting your kids with the idea that work is boring.² Maybe it would be better for kids in this one case if parents were not so unselfish. A parent who set an example of loving their work might help their kids more than an expensive house.³

It was not till I was in college that the idea of work finally broke free from the idea of making a living. Then the important question became not how to make money, but what to work on. Ideally these coincided,

²One father told me about a related phenomenon: he found himself concealing from his family how much he liked his work. When he wanted to go to work on a saturday, he found it easier to say that it was because he "had to" for some reason, rather than admitting he preferred to work than stay home with them.

³Something similar happens with suburbs. Parents move to suburbs to raise their kids in a safe environment, but suburbs are so dull and artificial that by the time they're fifteen the kids are convinced the whole world is boring.

but some spectacular boundary cases (like Einstein in the patent office) proved they weren't identical.

The definition of work was now to make some original contribution to the world, and in the process not to starve. But after the habit of so many years my idea of work still included a large component of pain. Work still seemed to require discipline, because only hard problems yielded grand results, and hard problems couldn't literally be fun. Surely one had to force oneself to work on them.

If you think something's supposed to hurt, you're less likely to notice if you're doing it wrong. That about sums up my experience of graduate school.

Bounds

How much are you supposed to like what you do? Unless you know that, you don't know when to stop searching. And if, like most people, you underestimate it, you'll tend to stop searching too early. You'll end up doing something chosen for you by your parents, or the desire to make money, or prestige — or sheer inertia.

Here's an upper bound: Do what you love doesn't mean, do what you would like to do most *this second*. Even Einstein probably had moments when he wanted to have a cup of coffee, but told himself he ought to finish what he was working on first.

It used to perplex me when I read about people who liked what they did so much that there was nothing they'd rather do. There didn't seem to be any sort of work I liked *that* much. If I had a choice of (a) spending the next hour working on something or (b) be teleported to Rome and spend the next hour wandering about, was there any sort of work I'd prefer? Honestly, no.

But the fact is, almost anyone would rather, at any given moment, float about in the Carribbean, or have sex, or eat some delicious food, than work on hard problems. The rule about doing what you love assumes a certain length of time. It doesn't mean, do what will make

you happiest this second, but what will make you happiest over some longer period, like a week or a month.

Unproductive pleasures pall eventually. After a while you get tired of lying on the beach. If you want to stay happy, you have to do something.

As a lower bound, you have to like your work more than any unproductive pleasure. You have to like what you do enough that the concept of “spare time” seems mistaken. Which is not to say you have to spend all your time working. You can only work so much before you get tired and start to screw up. Then you want to do something else — even something mindless. But you don’t regard this time as the prize and the time you spend working as the pain you endure to earn it.

I put the lower bound there for practical reasons. If your work is not your favorite thing to do, you’ll have terrible problems with procrastination. You’ll have to force yourself to work, and when you resort to that the results are distinctly inferior.

To be happy I think you have to be doing something you not only enjoy, but admire. You have to be able to say, at the end, wow, that’s pretty cool. This doesn’t mean you have to make something. If you learn how to hang glide, or to speak a foreign language fluently, that will be enough to make you say, for a while at least, wow, that’s pretty cool. What there has to be is a test.

So one thing that falls just short of the standard, I think, is reading books. Except for some books in math and the hard sciences, there’s no test of how well you’ve read a book, and that’s why merely reading books doesn’t quite feel like work. You have to do something with what you’ve read to feel productive.

I think the best test is one Gino Lee taught me: to try to do things that would make your friends say wow. But it probably wouldn’t start to work properly till about age 22, because most people haven’t had a big enough sample to pick friends from before then.

Sirens

What you should not do, I think, is worry about the opinion of anyone beyond your friends. You shouldn't worry about prestige. Prestige is the opinion of the rest of the world. When you can ask the opinions of people whose judgement you respect, what does it add to consider the opinions of people you don't even know? ⁴

This is easy advice to give. It's hard to follow, especially when you're young. ⁵ Prestige is like a powerful magnet that warps even your beliefs about what you enjoy. It causes you to work not on what you like, but what you'd like to like.

That's what leads people to try to write novels, for example. They like reading novels. They notice that people who write them win Nobel prizes. What could be more wonderful, they think, than to be a novelist? But liking the idea of being a novelist is not enough; you have to like the actual work of novel-writing if you're going to be good at it; you have to like making up elaborate lies.

Prestige is just fossilized inspiration. If you do anything well enough, you'll *make* it prestigious. Plenty of things we now consider prestigious were anything but at first. Jazz comes to mind — though almost any established art form would do. So just do what you like, and let prestige take care of itself.

Prestige is especially dangerous to the ambitious. If you want to make ambitious people waste their time on errands, the way to do it is to bait the hook with prestige. That's the recipe for getting people to give

⁴I'm not saying friends should be the only audience for your work. The more people you can help, the better. But friends should be your compass.

⁵Donald Hall said young would-be poets were mistaken to be so obsessed with being published. But you can imagine what it would do for a 24 year old to get a poem published in *The New Yorker*. Now to people he meets at parties he's a real poet. Actually he's no better or worse than he was before, but to a clueless audience like that, the approval of an official authority makes all the difference. So it's a harder problem than Hall realizes. The reason the young care so much about prestige is that the people they want to impress are not very discerning.

talks, write forewords, serve on committees, be department heads, and so on. It might be a good rule simply to avoid any prestigious task. If it didn't suck, they wouldn't have had to make it prestigious.

Similarly, if you admire two kinds of work equally, but one is more prestigious, you should probably choose the other. Your opinions about what's admirable are always going to be slightly influenced by prestige, so if the two seem equal to you, you probably have more genuine admiration for the less prestigious one.

The other big force leading people astray is money. Money by itself is not that dangerous. When something pays well but is regarded with contempt, like telemarketing, or prostitution, or personal injury litigation, ambitious people aren't tempted by it. That kind of work ends up being done by people who are "just trying to make a living." (Tip: avoid any field whose practitioners say this.) The danger is when money is combined with prestige, as in, say, corporate law, or medicine. A comparatively safe and prosperous career with some automatic baseline prestige is dangerously tempting to someone young, who hasn't thought much about what they really like.

The test of whether people love what they do is whether they'd do it even if they weren't paid for it — even if they had to work at another job to make a living. How many corporate lawyers would do their current work if they had to do it for free, in their spare time, and take day jobs as waiters to support themselves?

This test is especially helpful in deciding between different kinds of academic work, because fields vary greatly in this respect. Most good mathematicians would work on math even if there were no jobs as math professors, whereas in the departments at the other end of the spectrum, the availability of teaching jobs is the driver: people would rather be English professors than work in ad agencies, and publishing papers is the way you compete for such jobs. Math would happen without math departments, but it is the existence of English majors, and therefore jobs teaching them, that calls into being all those thousands of dreary papers about gender and identity in the novels of Con-

rad. No one does that kind of thing for fun.

The advice of parents will tend to err on the side of money. It seems safe to say there are more undergrads who want to be novelists and whose parents want them to be doctors than who want to be doctors and whose parents want them to be novelists. The kids think their parents are “materialistic.” Not necessarily. All parents tend to be more conservative for their kids than they would for themselves, simply because, as parents, they share risks more than rewards. If your eight year old son decides to climb a tall tree, or your teenage daughter decides to date the local bad boy, you won’t get a share in the excitement, but if your son falls, or your daughter gets pregnant, you’ll have to deal with the consequences.

Discipline

With such powerful forces leading us astray, it’s not surprising we find it so hard to discover what we like to work on. Most people are doomed in childhood by accepting the axiom that work = pain. Those who escape this are nearly all lured onto the rocks by prestige or money. How many even discover something they love to work on? A few hundred thousand, perhaps, out of billions.

It’s hard to find work you love; it must be, if so few do. So don’t underestimate this task. And don’t feel bad if you haven’t succeeded yet. In fact, if you admit to yourself that you’re discontented, you’re a step ahead of most people, who are still in denial. If you’re surrounded by colleagues who claim to enjoy work that you find contemptible, odds are they’re lying to themselves. Not necessarily, but probably.

Although doing great work takes less discipline than people think — because the way to do great work is to find something you like so much that you don’t have to force yourself to do it — *finding* work you love does usually require discipline. Some people are lucky enough to know what they want to do when they’re 12, and just glide along as if they were on railroad tracks. But this seems the exception. More often people who do great things have careers with the trajectory of a

ping-pong ball. They go to school to study A, drop out and get a job doing B, and then become famous for C after taking it up on the side.

Sometimes jumping from one sort of work to another is a sign of energy, and sometimes it's a sign of laziness. Are you dropping out, or boldly carving a new path? You often can't tell yourself. Plenty of people who will later do great things seem to be disappointments early on, when they're trying to find their niche.

Is there some test you can use to keep yourself honest? One is to try to do a good job at whatever you're doing, even if you don't like it. Then at least you'll know you're not using dissatisfaction as an excuse for being lazy. Perhaps more importantly, you'll get into the habit of doing things well.

Another test you can use is: always produce. For example, if you have a day job you don't take seriously because you plan to be a novelist, are you producing? Are you writing pages of fiction, however bad? As long as you're producing, you'll know you're not merely using the hazy vision of the grand novel you plan to write one day as an opiate. The view of it will be obstructed by the all too palpably flawed one you're actually writing.

"Always produce" is also a heuristic for finding the work you love. If you subject yourself to that constraint, it will automatically push you away from things you think you're supposed to work on, toward things you actually like. "Always produce" will discover your life's work the way water, with the aid of gravity, finds the hole in your roof.

Of course, figuring out what you like to work on doesn't mean you get to work on it. That's a separate question. And if you're ambitious you have to keep them separate: you have to make a conscious effort to keep your ideas about what you want from being contaminated by what seems possible.⁶

⁶This is isomorphic to the principle that you should prevent your beliefs about how things are from being contaminated by how you wish they were. Most people let them mix pretty promiscuously. The continuing popularity of religion is the most

It's painful to keep them apart, because it's painful to observe the gap between them. So most people pre-emptively lower their expectations. For example, if you asked random people on the street if they'd like to be able to draw like Leonardo, you'd find most would say something like "Oh, I can't draw." This is more a statement of intention than fact; it means, I'm not going to try. Because the fact is, if you took a random person off the street and somehow got them to work as hard as they possibly could at drawing for the next twenty years, they'd get surprisingly far. But it would require a great moral effort; it would mean staring failure in the eye every day for years. And so to protect themselves people say "I can't."

Another related line you often hear is that not everyone can do work they love — that someone has to do the unpleasant jobs. Really? How do you make them? In the US the only mechanism for forcing people to do unpleasant jobs is the draft, and that hasn't been invoked for over 30 years. All we can do is encourage people to do unpleasant work, with money and prestige.

If there's something people still won't do, it seems as if society just has to make do without. That's what happened with domestic servants. For millennia that was the canonical example of a job "someone had to do." And yet in the mid twentieth century servants practically disappeared in rich countries, and the rich have just had to do without.

So while there may be some things someone has to do, there's a good chance anyone saying that about any particular job is mistaken. Most unpleasant jobs would either get automated or go undone if no one were willing to do them.

Two Routes

There's another sense of "not everyone can do work they love" that's all too true, however. One has to make a living, and it's hard to get paid for doing work you love. There are two routes to that destination:

visible index of that.

The organic route: as you become more eminent, gradually to increase the parts of your job that you like at the expense of those you don't.

The two-job route: to work at things you don't like to get money to work on things you do.

The organic route is more common. It happens naturally to anyone who does good work. A young architect has to take whatever work he can get, but if he does well he'll gradually be in a position to pick and choose among projects. The disadvantage of this route is that it's slow and uncertain. Even tenure is not real freedom.

The two-job route has several variants depending on how long you work for money at a time. At one extreme is the "day job," where you work regular hours at one job to make money, and work on what you love in your spare time. At the other extreme you work at something till you make enough not to have to work for money again.

The two-job route is less common than the organic route, because it requires a deliberate choice. It's also more dangerous. Life tends to get more expensive as you get older, so it's easy to get sucked into working longer than you expected at the money job. Worse still, anything you work on changes you. If you work too long on tedious stuff, it will rot your brain. And the best paying jobs are most dangerous, because they require your full attention.

The advantage of the two-job route is that it lets you jump over obstacles. The landscape of possible jobs isn't flat; there are walls of varying heights between different kinds of work.⁷ The trick of maximizing the parts of your job that you like can get you from architecture to product design, but not, probably, to music. If you make money doing one thing and then work on another, you have more freedom of choice.

⁷A more accurate metaphor would be to say that the graph of jobs is not very well connected.

Which route should you take? That depends on how sure you are of what you want to do, how good you are at taking orders, how much risk you can stand, and the odds that anyone will pay (in your lifetime) for what you want to do. If you're sure of the general area you want to work in and it's something people are likely to pay you for, then you should probably take the organic route. But if you don't know what you want to work on, or don't like to take orders, you may want to take the two-job route, if you can stand the risk.

Don't decide too soon. Kids who know early what they want to do seem impressive, as if they got the answer to some math question before the other kids. They have an answer, certainly, but odds are it's wrong.

A friend of mine who is a quite successful doctor complains constantly about her job. When people applying to medical school ask her for advice, she wants to shake them and yell "Don't do it!" (But she never does.) How did she get into this fix? In high school she already wanted to be a doctor. And she is so ambitious and determined that she overcame every obstacle along the way — including, unfortunately, not liking it.

Now she has a life chosen for her by a high-school kid.

When you're young, you're given the impression that you'll get enough information to make each choice before you need to make it. But this is certainly not so with work. When you're deciding what to do, you have to operate on ridiculously incomplete information. Even in college you get little idea what various types of work are like. At best you may have a couple internships, but not all jobs offer internships, and those that do don't teach you much more about the work than being a batboy teaches you about playing baseball.

In the design of lives, as in the design of most other things, you get better results if you use flexible media. So unless you're fairly sure what you want to do, your best bet may be to choose a type of work that could turn into either an organic or two-job career. That was

probably part of the reason I chose computers. You can be a professor, or make a lot of money, or morph it into any number of other kinds of work.

It's also wise, early on, to seek jobs that let you do many different things, so you can learn faster what various kinds of work are like. Conversely, the extreme version of the two-job route is dangerous because it teaches you so little about what you like. If you work hard at being a bond trader for ten years, thinking that you'll quit and write novels when you have enough money, what happens when you quit and then discover that you don't actually like writing novels?

Most people would say, I'd take that problem. Give me a million dollars and I'll figure out what to do. But it's harder than it looks. Constraints give your life shape. Remove them and most people have no idea what to do: look at what happens to those who win lotteries or inherit money. Much as everyone thinks they want financial security, the happiest people are not those who have it, but those who like what they do. So a plan that promises freedom at the expense of knowing what to do with it may not be as good as it seems.

Whichever route you take, expect a struggle. Finding work you love is very difficult. Most people fail. Even if you succeed, it's rare to be free to work on what you want till your thirties or forties. But if you have the destination in sight you'll be more likely to arrive at it. If you know you can love work, you're in the home stretch, and if you know what work you love, you're practically there.

Thanks to Trevor Blackwell, Dan Friedman, Sarah Harlin, Jessica Livingston, Jackie McDonough, Robert Morris, Peter Norvig, David Sloo, and Aaron Swartz for reading drafts of this.

Chapter 24

Why YC

March 2006, rev August 2009

Yesterday one of the founders we funded asked me why we started Y Combinator. Or more precisely, he asked if we'd started YC mainly for fun.

Kind of, but not quite. It is enormously fun to be able to work with Rtm and Trevor again. I missed that after we sold Viaweb, and for all the years after I always had a background process running, looking for something we could do together. There is definitely an aspect of a band reunion to Y Combinator. Every couple days I slip and call it "Viaweb."

Viaweb we started very explicitly to make money. I was sick of living from one freelance project to the next, and decided to just work as hard as I could till I'd made enough to solve the problem once and for all. Viaweb was sometimes fun, but it wasn't designed for fun, and mostly it wasn't. I'd be surprised if any startup is. All startups are mostly schleps.

The real reason we started Y Combinator is neither selfish nor virtuous. We didn't start it mainly to make money; we have no idea what our average returns might be, and won't know for years. Nor did we start YC mainly to help out young would-be founders, though we do like the idea, and comfort ourselves occasionally with the thought that

if all our investments tank, we will thus have been doing something unselfish. (It's oddly nondeterministic.)

The real reason we started Y Combinator is one probably only a hacker would understand. We did it because it seems such a great hack. There are thousands of smart people who could start companies and don't, and with a relatively small amount of force applied at just the right place, we can spring on the world a stream of new startups that might otherwise not have existed.

In a way this is virtuous, because I think startups are a good thing. But really what motivates us is the completely amoral desire that would motivate any hacker who looked at some complex device and realized that with a tiny tweak he could make it run more efficiently. In this case, the device is the world's economy, which fortunately happens to be open source.

Chapter 25

6,631,372

March 2006, rev August 2009

A couple days ago I found to my surprise that I'd been granted a patent. It issued in 2003, but no one told me. I wouldn't know about it now except that a few months ago, while visiting Yahoo, I happened to run into a Big Cheese I knew from working there in the late nineties. He brought up something called Revenue Loop, which Viaweb had been working on when they bought us.

The idea is basically that you sort search results not in order of textual “relevance” (as search engines did then) nor in order of how much advertisers bid (as Overture did) but in order of the bid times the number of transactions. Ordinarily you'd do this for shopping searches, though in fact one of the features of our scheme is that it automatically detects which searches are shopping searches.

If you just order the results in order of bids, you can make the search results useless, because the first results could be dominated by lame sites that had bid the most. But if you order results by bid multiplied by transactions, far from selling out, you're getting a *better* measure of relevance. What could be a better sign that someone was satisfied with a search result than going to the site and buying something?

And, of course, this algorithm automatically maximizes the revenue of the search engine.

Everyone is focused on this type of approach now, but few were in 1998. In 1998 it was all about selling banner ads. We didn't know that, so we were pretty excited when we figured out what seemed to us the optimal way of doing shopping searches.

When Yahoo was thinking of buying us, we had a meeting with Jerry Yang in New York. For him, I now realize, this was supposed to be one of those meetings when you check out a company you've pretty much decided to buy, just to make sure they're ok guys. We weren't expected to do more than chat and seem smart and reasonable. He must have been dismayed when I jumped up to the whiteboard and launched into a presentation of our exciting new technology.

I was just as dismayed when he didn't seem to care at all about it. At the time I thought, "boy, is this guy poker-faced. We present to him what has to be the optimal way of sorting product search results, and he's not even curious." I didn't realize till much later why he didn't care. In 1998, advertisers were overpaying enormously for ads on web sites. In 1998, if advertisers paid the maximum that traffic was worth to them, Yahoo's revenues would have *decreased*.

Things are different now, of course. Now this sort of thing is all the rage. So when I ran into the Yahoo exec I knew from the old days in the Yahoo cafeteria a few months ago, the first thing he remembered was not (fortunately) all the fights I had with him, but Revenue Loop.

"Well," I said, "I think we actually applied for a patent on it. I'm not sure what happened to the application after I left."

"Really? That would be an important patent."

So someone investigated, and sure enough, that patent application had continued in the pipeline for several years after, and finally issued in 2003.

The main thing that struck me on reading it, actually, is that lawyers at some point messed up my nice clear writing. Some clever person with a spell checker reduced one section to Zen-like incomprehensibility:

Also, common spelling errors will tend to get fixed. For example, if users searching for “compact disc player” end up spending considerable money at sites offering compact disc players, then those pages will have a higher relevance for that search phrase, even though the phrase “compact disc player” is not present on those pages.

(That “compat disc player” wasn’t a typo, guys.)

For the fine prose of the original, see the provisional application of February 1998, back when we were still Viaweb and couldn’t afford to pay lawyers to turn every “a lot of” into “considerable.”

Chapter 26

Are Software Patents Evil?

March 2006

(This essay is derived from a talk at Google.)

A few weeks ago I found to my surprise that I'd been granted four patents. This was all the more surprising because I'd only applied for three. The patents aren't mine, of course. They were assigned to Viaweb, and became Yahoo's when they bought us. But the news set me thinking about the question of software patents generally.

Patents are a hard problem. I've had to advise most of the startups we've funded about them, and despite years of experience I'm still not always sure I'm giving the right advice.

One thing I do feel pretty certain of is that if you're against software patents, you're against patents in general. Gradually our machines consist more and more of software. Things that used to be done with levers and cams and gears are now done with loops and trees and closures. There's nothing special about physical embodiments of control systems that should make them patentable, and the software equivalent not.

Unfortunately, patent law is inconsistent on this point. Patent law in most countries says that algorithms aren't patentable. This rule is left over from a time when "algorithm" meant something like the Sieve of Eratosthenes. In 1800, people could not see as readily as we can

that a great many patents on mechanical objects were really patents on the algorithms they embodied.

Patent lawyers still have to pretend that's what they're doing when they patent algorithms. You must not use the word "algorithm" in the title of a patent application, just as you must not use the word "essays" in the title of a book. If you want to patent an algorithm, you have to frame it as a computer system executing that algorithm. Then it's mechanical; phew. The default euphemism for algorithm is "system and method." Try a patent search for that phrase and see how many results you get.

Since software patents are no different from hardware patents, people who say "software patents are evil" are saying simply "patents are evil." So why do so many people complain about software patents specifically?

I think the problem is more with the patent office than the concept of software patents. Whenever software meets government, bad things happen, because software changes fast and government changes slow. The patent office has been overwhelmed by both the volume and the novelty of applications for software patents, and as a result they've made a lot of mistakes.

The most common is to grant patents that shouldn't be granted. To be patentable, an invention has to be more than new. It also has to be non-obvious. And this, especially, is where the USPTO has been dropping the ball. Slashdot has an icon that expresses the problem vividly: a knife and fork with the words "patent pending" superimposed.

The scary thing is, this is the *only* icon they have for patent stories. Slashdot readers now take it for granted that a story about a patent will be about a bogus patent. That's how bad the problem has become.

The problem with Amazon's notorious one-click patent, for example, is not that it's a software patent, but that it's obvious. Any online store that kept people's shipping addresses would have implemented this. The reason Amazon did it first was not that they were especially

smart, but because they were one of the earliest sites with enough clout to force customers to log in before they could buy something.¹

We, as hackers, know the USPTO is letting people patent the knives and forks of our world. The problem is, the USPTO are not hackers. They're probably good at judging new inventions for casting steel or grinding lenses, but they don't understand software yet.

At this point an optimist would be tempted to add "but they will eventually." Unfortunately that might not be true. The problem with software patents is an instance of a more general one: the patent office takes a while to understand new technology. If so, this problem will only get worse, because the rate of technological change seems to be increasing. In thirty years, the patent office may understand the sort of things we now patent as software, but there will be other new types of inventions they understand even less.

Applying for a patent is a negotiation. You generally apply for a broader patent than you think you'll be granted, and the examiners reply by throwing out some of your claims and granting others. So I don't really blame Amazon for applying for the one-click patent. The big mistake was the patent office's, for not insisting on something narrower, with real technical content. By granting such an over-broad patent, the USPTO in effect slept with Amazon on the first date. Was Amazon supposed to say no?

Where Amazon went over to the dark side was not in applying for the patent, but in enforcing it. A lot of companies (Microsoft, for example) have been granted large numbers of preposterously over-broad patents, but they keep them mainly for defensive purposes. Like nuclear weapons, the main role of big companies' patent portfolios is to threaten anyone who attacks them with a counter-suit. Amazon's suit against Barnes & Noble was thus the equivalent of a nuclear first strike.

¹You have to be careful here, because a great discovery often seems obvious in retrospect. One-click ordering, however, is not such a discovery.

That suit probably hurt Amazon more than it helped them. Barnes & Noble was a lame site; Amazon would have crushed them anyway. To attack a rival they could have ignored, Amazon put a lasting black mark on their own reputation. Even now I think if you asked hackers to free-associate about Amazon, the one-click patent would turn up in the first ten topics.

Google clearly doesn't feel that merely holding patents is evil. They've applied for a lot of them. Are they hypocrites? Are patents evil?

There are really two variants of that question, and people answering it often aren't clear in their own minds which they're answering. There's a narrow variant: is it bad, given the current legal system, to apply for patents? and also a broader one: is it bad that the current legal system allows patents?

These are separate questions. For example, in preindustrial societies like medieval Europe, when someone attacked you, you didn't call the police. There were no police. When attacked, you were supposed to fight back, and there were conventions about how to do it. Was this wrong? That's two questions: was it wrong to take justice into your own hands, and was it wrong that you had to? We tend to say yes to the second, but no to the first. If no one else will defend you, you have to defend yourself. ²

The situation with patents is similar. Business is a kind of ritualized warfare. Indeed, it evolved from actual warfare: most early traders switched on the fly from merchants to pirates depending on how strong you seemed. In business there are certain rules describing how companies may and may not compete with one another, and someone deciding that they're going to play by their own rules is missing the point. Saying "I'm not going to apply for patents just because everyone else does" is not like saying "I'm not going to lie just because everyone else does." It's more like saying "I'm not going to use

²"Turn the other cheek" skirts the issue; the critical question is not how to deal with slaps, but sword thrusts.

TCP/IP just because everyone else does.” Oh yes you are.

A closer comparison might be someone seeing a hockey game for the first time, realizing with shock that the players were *deliberately* bumping into one another, and deciding that one would on no account be so rude when playing hockey oneself.

Hockey allows checking. It’s part of the game. If your team refuses to do it, you simply lose. So it is in business. Under the present rules, patents are part of the game.

What does that mean in practice? We tell the startups we fund not to worry about infringing patents, because startups rarely get sued for patent infringement. There are only two reasons someone might sue you: for money, or to prevent you from competing with them. Startups are too poor to be worth suing for money. And in practice they don’t seem to get sued much by competitors, either. They don’t get sued by other startups because (a) patent suits are an expensive distraction, and (b) since the other startups are as young as they are, their patents probably haven’t issued yet.³ Nor do startups, at least in the software business, seem to get sued much by established competitors. Despite all the patents Microsoft holds, I don’t know of an instance where they sued a startup for patent infringement. Companies like Microsoft and Oracle don’t win by winning lawsuits. That’s too uncertain. They win by locking competitors out of their sales channels. If you do manage to threaten them, they’re more likely to buy you than sue you.

When you read of big companies filing patent suits against smaller ones, it’s usually a big company on the way down, grasping at straws. For example, Unisys’s attempts to enforce their patent on LZW compression. When you see a big company threatening patent suits, sell. When a company starts fighting over IP, it’s a sign they’ve lost the real battle, for users.

³Applying for a patent is now very slow, but it might actually be bad if that got fixed. At the moment the time it takes to get a patent is conveniently just longer than the time it takes a startup to succeed or fail.

A company that sues competitors for patent infringement is like a defender who has been beaten so thoroughly that he turns to plead with the referee. You don't do that if you can still reach the ball, even if you genuinely believe you've been fouled. So a company threatening patent suits is a company in trouble.

When we were working on Viaweb, a bigger company in the e-commerce business was granted a patent on online ordering, or something like that. I got a call from a VP there asking if we'd like to license it. I replied that I thought the patent was completely bogus, and would never hold up in court. "Ok," he replied. "So, are you guys hiring?"

If your startup grows big enough, however, you'll start to get sued, no matter what you do. If you go public, for example, you'll be sued by multiple patent trolls who hope you'll pay them off to go away. More on them later.

In other words, no one will sue you for patent infringement till you have money, and once you have money, people will sue you whether they have grounds to or not. So I advise fatalism. Don't waste your time worrying about patent infringement. You're probably violating a patent every time you tie your shoelaces. At the start, at least, just worry about making something great and getting lots of users. If you grow to the point where anyone considers you worth attacking, you're doing well.

We do advise the companies we fund to apply for patents, but not so they can sue competitors. Successful startups either get bought or grow into big companies. If a startup wants to grow into a big company, they should apply for patents to build up the patent portfolio they'll need to maintain an armed truce with other big companies. If they want to get bought, they should apply for patents because patents are part of the mating dance with acquirers.

Most startups that succeed do it by getting bought, and most acquirers care about patents. Startup acquisitions are usually a build-vs-buy decision for the acquirer. Should we buy this little startup or build our

own? And two things, especially, make them decide not to build their own: if you already have a large and rapidly growing user base, and if you have a fairly solid patent application on critical parts of your software.

There's a third reason big companies should prefer buying to building: that if they built their own, they'd screw it up. But few big companies are smart enough yet to admit this to themselves. It's usually the acquirer's engineers who are asked how hard it would be for the company to build their own, and they overestimate their abilities.⁴ A patent seems to change the balance. It gives the acquirer an excuse to admit they couldn't copy what you're doing. It may also help them to grasp what's special about your technology.

Frankly, it surprises me how small a role patents play in the software business. It's kind of ironic, considering all the dire things experts say about software patents stifling innovation, but when one looks closely at the software business, the most striking thing is how little patents seem to matter.

In other fields, companies regularly sue competitors for patent infringement. For example, the airport baggage scanning business was for many years a cozy duopoly shared between two companies, InVision and L-3. In 2002 a startup called Reveal appeared, with new technology that let them build scanners a third the size. They were sued for patent infringement before they'd even released a product.

You rarely hear that kind of story in our world. The one example I've found is, embarrassingly enough, Yahoo, which filed a patent suit against a gaming startup called Xfire in 2005. Xfire doesn't seem to be a very big deal, and it's hard to say why Yahoo felt threatened. Xfire's VP of engineering had worked at Yahoo on similar stuff— in fact, he was listed as an inventor on the patent Yahoo sued over— so perhaps there was something personal about it. My guess is that someone at

⁴Instead of the canonical “could you build this?” maybe the corp dev guys should be asking “will you build this?” or even “why haven't you already built this?”

Yahoo goofed. At any rate they didn't pursue the suit very vigorously.

Why do patents play so small a role in software? I can think of three possible reasons.

One is that software is so complicated that patents by themselves are not worth very much. I may be maligning other fields here, but it seems that in most types of engineering you can hand the details of some new technique to a group of medium-high quality people and get the desired result. For example, if someone develops a new process for smelting ore that gets a better yield, and you assemble a team of qualified experts and tell them about it, they'll be able to get the same yield. This doesn't seem to work in software. Software is so subtle and unpredictable that "qualified experts" don't get you very far.

That's why we rarely hear phrases like "qualified expert" in the software business. What that level of ability can get you is, say, to make your software compatible with some other piece of software—in eight months, at enormous cost. To do anything harder you need individual brilliance. If you assemble a team of qualified experts and tell them to make a new web-based email program, they'll get their asses kicked by a team of inspired nineteen year olds.

Experts can implement, but they can't design. Or rather, expertise in implementation is the only kind most people, including the experts themselves, can measure.⁵

But design is a definite skill. It's not just an airy intangible. Things always seem intangible when you don't understand them. Electricity seemed an airy intangible to most people in 1800. Who knew there was so much to know about it? So it is with design. Some people are good at it and some people are bad at it, and there's something very tangible they're good or bad at.

⁵Design ability is so hard to measure that you can't even trust the design world's internal standards. You can't assume that someone with a degree in design is any good at design, or that an eminent designer is any better than his peers. If that worked, any company could build products as good as Apple's just by hiring sufficiently qualified designers.

The reason design counts so much in software is probably that there are fewer constraints than on physical things. Building physical things is expensive and dangerous. The space of possible choices is smaller; you tend to have to work as part of a larger group; and you're subject to a lot of regulations. You don't have any of that if you and a couple friends decide to create a new web-based application.

Because there's so much scope for design in software, a successful application tends to be way more than the sum of its patents. What protects little companies from being copied by bigger competitors is not just their patents, but the thousand little things the big company will get wrong if they try.

The second reason patents don't count for much in our world is that startups rarely attack big companies head-on, the way Reveal did. In the software business, startups beat established companies by transcending them. Startups don't build desktop word processing programs to compete with Microsoft Word.⁶ They build Writely. If this paradigm is crowded, just wait for the next one; they run pretty frequently on this route.

Fortunately for startups, big companies are extremely good at denial. If you take the trouble to attack them from an oblique angle, they'll meet you half-way and maneuver to keep you in their blind spot. To sue a startup would mean admitting it was dangerous, and that often means seeing something the big company doesn't want to see. IBM used to sue its mainframe competitors regularly, but they didn't bother much about the microcomputer industry because they didn't want to see the threat it posed. Companies building web based apps are similarly protected from Microsoft, which even now doesn't want to imagine a world in which Windows is irrelevant.

The third reason patents don't seem to matter very much in software is public opinion—or rather, hacker opinion. In a recent interview, Steve

⁶If anyone wanted to try, we'd be interested to hear from them. I suspect it's one of those things that's not as hard as everyone assumes.

Ballmer coyly left open the possibility of attacking Linux on patent grounds. But I doubt Microsoft would ever be so stupid. They'd face the mother of all boycotts. And not just from the technical community in general; a lot of their own people would rebel.

Good hackers care a lot about matters of principle, and they are highly mobile. If a company starts misbehaving, smart people won't work there. For some reason this seems to be more true in software than other businesses. I don't think it's because hackers have intrinsically higher principles so much as that their skills are easily transferrable. Perhaps we can split the difference and say that mobility gives hackers the luxury of being principled.

Google's "don't be evil" policy may for this reason be the most valuable thing they've discovered. It's very constraining in some ways. If Google does do something evil, they get doubly whacked for it: once for whatever they did, and again for hypocrisy. But I think it's worth it. It helps them to hire the best people, and it's better, even from a purely selfish point of view, to be constrained by principles than by stupidity.

(I wish someone would get this point across to the present administration.)

I'm not sure what the proportions are of the preceding three ingredients, but the custom among the big companies seems to be not to sue the small ones, and the startups are mostly too busy and too poor to sue one another. So despite the huge number of software patents there's not a lot of suing going on. With one exception: patent trolls.

Patent trolls are companies consisting mainly of lawyers whose whole business is to accumulate patents and threaten to sue companies who actually make things. Patent trolls, it seems safe to say, are evil. I feel a bit stupid saying that, because when you're saying something that Richard Stallman and Bill Gates would both agree with, you must be perilously close to tautologies.

The CEO of Forgent, one of the most notorious patent trolls, says that

what his company does is “the American way.” Actually that’s not true. The American way is to make money by creating wealth, not by suing people.⁷ What companies like Forgent do is actually the proto-industrial way. In the period just before the industrial revolution, some of the greatest fortunes in countries like England and France were made by courtiers who extracted some lucrative right from the crown—like the right to collect taxes on the import of silk—and then used this to squeeze money from the merchants in that business. So when people compare patent trolls to the mafia, they’re more right than they know, because the mafia too are not merely bad, but bad specifically in the sense of being an obsolete business model.

Patent trolls seem to have caught big companies by surprise. In the last couple years they’ve extracted hundreds of millions of dollars from them. Patent trolls are hard to fight precisely because they create nothing. Big companies are safe from being sued by other big companies because they can threaten a counter-suit. But because patent trolls don’t make anything, there’s nothing they can be sued for. I predict this loophole will get closed fairly quickly, at least by legal standards. It’s clearly an abuse of the system, and the victims are powerful.⁸

But evil as patent trolls are, I don’t think they hamper innovation much. They don’t sue till a startup has made money, and by that point the innovation that generated it has already happened. I can’t think of a startup that avoided working on some problem because of patent trolls.

So much for hockey as the game is played now. What about the more theoretical question of whether hockey would be a better game with-

⁷Patent trolls can’t even claim, like speculators, that they “create” liquidity.

⁸If big companies don’t want to wait for the government to take action, there is a way to fight back themselves. For a long time I thought there wasn’t, because there was nothing to grab onto. But there is one resource patent trolls need: lawyers. Big technology companies between them generate a lot of legal business. If they agreed among themselves never to do business with any firm employing anyone who had worked for a patent troll, either as an employee or as outside counsel, they could probably starve the trolls of the lawyers they need.

out checking? Do patents encourage or discourage innovation?

This is a very hard question to answer in the general case. People write whole books on the topic. One of my main hobbies is the history of technology, and even though I've studied the subject for years, it would take me several weeks of research to be able to say whether patents have in general been a net win.

One thing I can say is that 99.9% of the people who express opinions on the subject do it not based on such research, but out of a kind of religious conviction. At least, that's the polite way of putting it; the colloquial version involves speech coming out of organs not designed for that purpose.

Whether they encourage innovation or not, patents were at least intended to. You don't get a patent for nothing. In return for the exclusive right to use an idea, you have to *publish* it, and it was largely to encourage such openness that patents were established.

Before patents, people protected ideas by keeping them secret. With patents, central governments said, in effect, if you tell everyone your idea, we'll protect it for you. There is a parallel here to the rise of civil order, which happened at roughly the same time. Before central governments were powerful enough to enforce order, rich people had private armies. As governments got more powerful, they gradually compelled magnates to cede most responsibility for protecting them. (Magnates still have bodyguards, but no longer to protect them from other magnates.)

Patents, like police, are involved in many abuses. But in both cases the default is something worse. The choice is not "patents or freedom?" any more than it is "police or freedom?" The actual questions are respectively "patents or secrecy?" and "police or gangs?"

As with gangs, we have some idea what secrecy would be like, because that's how things used to be. The economy of medieval Europe was divided up into little tribes, each jealously guarding their privileges and secrets. In Shakespeare's time, "mystery" was synonymous with

“craft.” Even today we can see an echo of the secrecy of medieval guilds, in the now pointless secrecy of the Masons.

The most memorable example of medieval industrial secrecy is probably Venice, which forbade glassblowers to leave the city, and sent assassins after those who tried. We might like to think we wouldn’t go so far, but the movie industry has already tried to pass laws prescribing three year prison terms just for putting movies on public networks. Want to try a frightening thought experiment? If the movie industry could have any law they wanted, where would they stop? Short of the death penalty, one assumes, but how close would they get?

Even worse than the spectacular abuses might be the overall decrease in efficiency that would accompany increased secrecy. As anyone who has dealt with organizations that operate on a “need to know” basis can attest, dividing information up into little cells is terribly inefficient. The flaw in the “need to know” principle is that you don’t *know* who needs to know something. An idea from one area might spark a great discovery in another. But the discoverer doesn’t know he needs to know it.

If secrecy were the only protection for ideas, companies wouldn’t just have to be secretive with other companies; they’d have to be secretive internally. This would encourage what is already the worst trait of big companies.

I’m not saying secrecy would be worse than patents, just that we couldn’t discard patents for free. Businesses would become more secretive to compensate, and in some fields this might get ugly. Nor am I defending the current patent system. There is clearly a lot that’s broken about it. But the breakage seems to affect software less than most other fields.

In the software business I know from experience whether patents encourage or discourage innovation, and the answer is the type that people who like to argue about public policy least like to hear: they don’t affect innovation much, one way or the other. Most innovation in

the software business happens in startups, and startups should simply ignore other companies' patents. At least, that's what we advise, and we bet money on that advice.

The only real role of patents, for most startups, is as an element of the mating dance with acquirers. There patents do help a little. And so they do encourage innovation indirectly, in that they give more power to startups, which is where, pound for pound, the most innovation happens. But even in the mating dance, patents are of secondary importance. It matters more to make something great and get a lot of users.

Thanks to Dan Bloomberg, Paul Buchheit, Sarah Harlin, Jessica Livingston, and Peter Norvig for reading drafts of this, to Joel Lehrer and Peter Eng for answering my questions about patents, and to Ankur Pansari for inviting me to speak.

Chapter 27

See Randomness

April 2006, rev August 2009

Plato quotes Socrates as saying “the unexamined life is not worth living.” Part of what he meant was that the proper role of humans is to think, just as the proper role of anteaters is to poke their noses into anthills.

A lot of ancient philosophy had the quality — and I don’t mean this in an insulting way — of the kind of conversations freshmen have late at night in common rooms:

What is our purpose? Well, we humans are as conspicuously different from other animals as the anteater. In our case the distinguishing feature is the ability to reason. So obviously that is what we should be doing, and a human who doesn’t is doing a bad job of being human — is no better than an animal.

Now we’d give a different answer. At least, someone Socrates’s age would. We’d ask why we even suppose we have a “purpose” in life. We may be better adapted for some things than others; we may be happier doing things we’re adapted for; but why assume purpose?

The history of ideas is a history of gradually discarding the assumption that it’s all about us. No, it turns out, the earth is not the center

of the universe — not even the center of the solar system. No, it turns out, humans are not created by God in his own image; they're just one species among many, descended not merely from apes, but from microorganisms. Even the concept of "me" turns out to be fuzzy around the edges if you examine it closely.

The idea that we're the center of things is difficult to discard. So difficult that there's probably room to discard more. Richard Dawkins made another step in that direction only in the last several decades, with the idea of the selfish gene. No, it turns out, we're not even the protagonists: we're just the latest model vehicle our genes have constructed to travel around in. And having kids is our genes heading for the lifeboats. Reading that book snapped my brain out of its previous way of thinking the way Darwin's must have when it first appeared.

(Few people can experience now what Darwin's contemporaries did when *The Origin of Species* was first published, because everyone now is raised either to take evolution for granted, or to regard it as a heresy. No one encounters the idea of natural selection for the first time as an adult.)

So if you want to discover things that have been overlooked till now, one really good place to look is in our blind spot: in our natural, naive belief that it's all about us. And expect to encounter ferocious opposition if you do.

Conversely, if you have to choose between two theories, prefer the one that doesn't center on you.

This principle isn't only for big ideas. It works in everyday life, too. For example, suppose you're saving a piece of cake in the fridge, and you come home one day to find your housemate has eaten it. Two possible theories:

- a) Your housemate did it deliberately to upset you. He *knew* you were saving that piece of cake.
- b) Your housemate was hungry.

I say pick b. No one knows who said “never attribute to malice what can be explained by incompetence,” but it is a powerful idea. Its more general version is our answer to the Greeks:

Don't see purpose where there isn't.

Or better still, the positive version:

See randomness.

Chapter 28

The Hardest Lessons for Startups to Learn

April 2006

(This essay is derived from a talk at the 2006 Startup School.)

The startups we've funded so far are pretty quick, but they seem quicker to learn some lessons than others. I think it's because some things about startups are kind of counterintuitive.

We've now invested in enough companies that I've learned a trick for determining which points are the counterintuitive ones: they're the ones I have to keep repeating.

So I'm going to number these points, and maybe with future startups I'll be able to pull off a form of Huffman coding. I'll make them all read this, and then instead of nagging them in detail, I'll just be able to say: *number four!*

1. Release Early.

The thing I probably repeat most is this recipe for a startup: get a version 1 out fast, then improve it based on users' reactions.

By "release early" I don't mean you should release something full of bugs, but that you should release something minimal. Users hate bugs, but they don't seem to mind a minimal version 1, if there's more com-

ing soon.

There are several reasons it pays to get version 1 done fast. One is that this is simply the right way to write software, whether for a startup or not. I've been repeating that since 1993, and I haven't seen much since to contradict it. I've seen a lot of startups die because they were too slow to release stuff, and none because they were too quick.¹

One of the things that will surprise you if you build something popular is that you won't know your users. Reddit now has almost half a million unique visitors a month. Who are all those people? They have no idea. No web startup does. And since you don't know your users, it's dangerous to guess what they'll like. Better to release something and let them tell you.

Wufoo took this to heart and released their form-builder before the underlying database. You can't even drive the thing yet, but 83,000 people came to sit in the driver's seat and hold the steering wheel. And Wufoo got valuable feedback from it: Linux users complained they used too much Flash, so they rewrote their software not to. If they'd waited to release everything at once, they wouldn't have discovered this problem till it was more deeply wired in.

Even if you had no users, it would still be important to release quickly, because for a startup the initial release acts as a shakedown cruise. If anything major is broken— if the idea's no good, for example, or the founders hate one another— the stress of getting that first version out will expose it. And if you have such problems you want to find them early.

Perhaps the most important reason to release early, though, is that it makes you work harder. When you're working on something that isn't released, problems are intriguing. In something that's out there, problems are alarming. There is a lot more urgency once you release.

¹Startups can die from releasing something full of bugs, and not fixing them fast enough, but I don't know of any that died from releasing something stable but minimal very early, then promptly improving it.

And I think that's precisely why people put it off. They know they'll have to work a lot harder once they do.²

2. Keep Pumping Out Features.

Of course, “release early” has a second component, without which it would be bad advice. If you're going to start with something that doesn't do much, you better improve it fast.

What I find myself repeating is “pump out features.” And this rule isn't just for the initial stages. This is something all startups should do for as long as they want to be considered startups.

I don't mean, of course, that you should make your application ever more complex. By “feature” I mean one unit of hacking— one quantum of making users' lives better.

As with exercise, improvements beget improvements. If you run every day, you'll probably feel like running tomorrow. But if you skip running for a couple weeks, it will be an effort to drag yourself out. So it is with hacking: the more ideas you implement, the more ideas you'll have. You should make your system better at least in some small way every day or two.

This is not just a good way to get development done; it is also a form of marketing. Users love a site that's constantly improving. In fact, users expect a site to improve. Imagine if you visited a site that seemed very good, and then returned two months later and not one thing had changed. Wouldn't it start to seem lame?³

They'll like you even better when you improve in response to their comments, because customers are used to companies ignoring them. If you're the rare exception— a company that actually listens— you'll

²I know this is why I haven't released Arc. The moment I do, I'll have people nagging me for features.

³A web site is different from a book or movie or desktop application in this respect. Users judge a site not as a single snapshot, but as an animation with multiple frames. Of the two, I'd say the rate of improvement is more important to users than where you currently are.

generate fanatical loyalty. You won't need to advertise, because your users will do it for you.

This seems obvious too, so why do I have to keep repeating it? I think the problem here is that people get used to how things are. Once a product gets past the stage where it has glaring flaws, you start to get used to it, and gradually whatever features it happens to have become its identity. For example, I doubt many people at Yahoo (or Google for that matter) realized how much better web mail could be till Paul Buchheit showed them.

I think the solution is to assume that anything you've made is far short of what it could be. Force yourself, as a sort of intellectual exercise, to keep thinking of improvements. Ok, sure, what you have is perfect. But if you had to change something, what would it be?

If your product seems finished, there are two possible explanations: (a) it is finished, or (b) you lack imagination. Experience suggests (b) is a thousand times more likely.

3. Make Users Happy.

Improving constantly is an instance of a more general rule: make users happy. One thing all startups have in common is that they can't force anyone to do anything. They can't force anyone to use their software, and they can't force anyone to do deals with them. A startup has to sing for its supper. That's why the successful ones make great things. They have to, or die.

When you're running a startup you feel like a little bit of debris blown about by powerful winds. The most powerful wind is users. They can either catch you and loft you up into the sky, as they did with Google, or leave you flat on the pavement, as they do with most startups. Users are a fickle wind, but more powerful than any other. If they take you up, no competitor can keep you down.

As a little piece of debris, the rational thing for you to do is not to lie flat, but to curl yourself into a shape the wind will catch.

I like the wind metaphor because it reminds you how impersonal the stream of traffic is. The vast majority of people who visit your site will be casual visitors. It's them you have to design your site for. The people who really care will find what they want by themselves.

The median visitor will arrive with their finger poised on the Back button. Think about your own experience: most links you follow lead to something lame. Anyone who has used the web for more than a couple weeks has been *trained* to click on Back after following a link. So your site has to say "Wait! Don't click on Back. This site isn't lame. Look at this, for example."

There are two things you have to do to make people pause. The most important is to explain, as concisely as possible, what the hell your site is about. How often have you visited a site that seemed to assume you already knew what they did? For example, the corporate site that says the company makes

enterprise content management solutions for business
that enable organizations to unify people, content and
processes to minimize business risk, accelerate time-to-
value and sustain lower total cost of ownership.

An established company may get away with such an opaque description, but no startup can. A startup should be able to explain in one or two sentences exactly what it does.⁴ And not just to users. You need this for everyone: investors, acquirers, partners, reporters, potential employees, and even current employees. You probably shouldn't even start a company to do something that can't be described compellingly in one or two sentences.

The other thing I repeat is to give people everything you've got, right away. If you have something impressive, try to put it on the front

⁴It should not always tell this to users, however. For example, MySpace is basically a replacement mall for mallrats. But it was wiser for them, initially, to pretend that the site was about bands.

page, because that's the only one most visitors will see. Though indeed there's a paradox here: the more you push the good stuff toward the front, the more likely visitors are to explore further.⁵

In the best case these two suggestions get combined: you tell visitors what your site is about by *showing* them. One of the standard pieces of advice in fiction writing is "show, don't tell." Don't say that a character's angry; have him grind his teeth, or break his pencil in half. Nothing will explain what your site does so well as using it.

The industry term here is "conversion." The job of your site is to convert casual visitors into users—whatever your definition of a user is. You can measure this in your growth rate. Either your site is catching on, or it isn't, and you must know which. If you have decent growth, you'll win in the end, no matter how obscure you are now. And if you don't, you need to fix something.

4. Fear the Right Things.

Another thing I find myself saying a lot is "don't worry." Actually, it's more often "don't worry about this; worry about that instead." Startups are right to be paranoid, but they sometimes fear the wrong things.

Most visible disasters are not so alarming as they seem. Disasters are normal in a startup: a founder quits, you discover a patent that covers what you're doing, your servers keep crashing, you run into an insoluble technical problem, you have to change your name, a deal falls through—these are all par for the course. They won't kill you unless you let them.

Nor will most competitors. A lot of startups worry "what if Google builds something like us?" Actually big companies are not the ones you have to worry about— not even Google. The people at Google

⁵Similarly, don't make users register to try your site. Maybe what you have is so valuable that visitors should gladly register to get at it. But they've been trained to expect the opposite. Most of the things they've tried on the web have sucked— and probably especially those that made them register.

are smart, but no smarter than you; they're not as motivated, because Google is not going to go out of business if this one product fails; and even at Google they have a lot of bureaucracy to slow them down.

What you should fear, as a startup, is not the established players, but other startups you don't know exist yet. They're way more dangerous than Google because, like you, they're cornered animals.

Looking just at existing competitors can give you a false sense of security. You should compete against what someone else *could* be doing, not just what you can see people doing. A corollary is that you shouldn't relax just because you have no visible competitors yet. No matter what your idea, there's someone else out there working on the same thing.

That's the downside of it being easier to start a startup: more people are doing it. But I disagree with Caterina Fake when she says that makes this a bad time to start a startup. More people are starting startups, but not as many more as could. Most college graduates still think they have to get a job. The average person can't ignore something that's been beaten into their head since they were three just because serving web pages recently got a lot cheaper.

And in any case, competitors are not the biggest threat. Way more startups hose themselves than get crushed by competitors. There are a lot of ways to do it, but the three main ones are internal disputes, inertia, and ignoring users. Each is, by itself, enough to kill you. But if I had to pick the worst, it would be ignoring users. If you want a recipe for a startup that's going to die, here it is: a couple of founders who have some great idea they know everyone is going to love, and that's what they're going to build, no matter what.

Almost everyone's initial plan is broken. If companies stuck to their initial plans, Microsoft would be selling programming languages, and Apple would be selling printed circuit boards. In both cases their customers told them what their business should be— and they were smart enough to listen.

As Richard Feynman said, the imagination of nature is greater than the imagination of man. You'll find more interesting things by looking at the world than you could ever produce just by thinking. This principle is very powerful. It's why the best abstract painting still falls short of Leonardo, for example. And it applies to startups too. No idea for a product could ever be so clever as the ones you can discover by smashing a beam of prototypes into a beam of users.

5. Commitment Is a Self-Fulfilling Prophecy.

I now have enough experience with startups to be able to say what the most important quality is in a startup founder, and it's not what you might think. The most important quality in a startup founder is determination. Not intelligence—determination.

This is a little depressing. I'd like to believe Viaweb succeeded because we were smart, not merely determined. A lot of people in the startup world want to believe that. Not just founders, but investors too. They like the idea of inhabiting a world ruled by intelligence. And you can tell they really believe this, because it affects their investment decisions.

Time after time VCs invest in startups founded by eminent professors. This may work in biotech, where a lot of startups simply commercialize existing research, but in software you want to invest in students, not professors. Microsoft, Yahoo, and Google were all founded by people who dropped out of school to do it. What students lack in experience they more than make up in dedication.

Of course, if you want to get rich, it's not enough merely to be determined. You have to be smart too, right? I'd like to think so, but I've had an experience that convinced me otherwise: I spent several years living in New York.

You can lose quite a lot in the brains department and it won't kill you. But lose even a little bit in the commitment department, and that will kill you very rapidly.

Running a startup is like walking on your hands: it's possible, but it

requires extraordinary effort. If an ordinary employee were asked to do the things a startup founder has to, he'd be very indignant. Imagine if you were hired at some big company, and in addition to writing software ten times faster than you'd ever had to before, they expected you to answer support calls, administer the servers, design the web site, cold-call customers, find the company office space, and go out and get everyone lunch.

And to do all this not in the calm, womb-like atmosphere of a big company, but against a backdrop of constant disasters. That's the part that really demands determination. In a startup, there's always some disaster happening. So if you're the least bit inclined to find an excuse to quit, there's always one right there.

But if you lack commitment, chances are it will have been hurting you long before you actually quit. Everyone who deals with startups knows how important commitment is, so if they sense you're ambivalent, they won't give you much attention. If you lack commitment, you'll just find that for some mysterious reason good things happen to your competitors but not to you. If you lack commitment, it will seem to you that you're unlucky.

Whereas if you're determined to stick around, people will pay attention to you, because odds are they'll have to deal with you later. You're a local, not just a tourist, so everyone has to come to terms with you.

At Y Combinator we sometimes mistakenly fund teams who have the attitude that they're going to give this startup thing a shot for three months, and if something great happens, they'll stick with it—“something great” meaning either that someone wants to buy them or invest millions of dollars in them. But if this is your attitude, “something great” is very unlikely to happen to you, because both acquirers and investors judge you by your level of commitment.

If an acquirer thinks you're going to stick around no matter what, they'll be more likely to buy you, because if they don't and you stick around, you'll probably grow, your price will go up, and they'll be left

wishing they'd bought you earlier. Ditto for investors. What really motivates investors, even big VCs, is not the hope of good returns, but the fear of missing out.⁶ So if you make it clear you're going to succeed no matter what, and the only reason you need them is to make it happen a little faster, you're much more likely to get money.

You can't fake this. The only way to convince everyone that you're ready to fight to the death is actually to be ready to.

You have to be the right kind of determined, though. I carefully chose the word determined rather than stubborn, because stubbornness is a disastrous quality in a startup. You have to be determined, but flexible, like a running back. A successful running back doesn't just put his head down and try to run through people. He improvises: if someone appears in front of him, he runs around them; if someone tries to grab him, he spins out of their grip; he'll even run in the wrong direction briefly if that will help. The one thing he'll never do is stand still.⁷

6. There Is Always Room.

I was talking recently to a startup founder about whether it might be good to add a social component to their software. He said he didn't think so, because the whole social thing was tapped out. Really? So in a hundred years the only social networking sites will be the Facebook, MySpace, Flickr, and Del.icio.us? Not likely.

There is always room for new stuff. At every point in history, even the darkest bits of the dark ages, people were discovering things that made everyone say "why didn't anyone think of that before?" We know this continued to be true up till 2004, when the Facebook was founded—

⁶VCs have rational reasons for behaving this way. They don't make their money (if they make money) off their median investments. In a typical fund, half the companies fail, most of the rest generate mediocre returns, and one or two "make the fund" by succeeding spectacularly. So if they miss just a few of the most promising opportunities, it could hose the whole fund.

⁷The attitude of a running back doesn't translate to soccer. Though it looks great when a forward dribbles past multiple defenders, a player who persists in trying such things will do worse in the long term than one who passes.

though strictly speaking someone else did think of that.

The reason we don't see the opportunities all around us is that we adjust to however things are, and assume that's how things have to be. For example, it would seem crazy to most people to try to make a better search engine than Google. Surely that field, at least, is tapped out. Really? In a hundred years— or even twenty— are people still going to search for information using something like the current Google? Even Google probably doesn't think that.

In particular, I don't think there's any limit to the number of startups. Sometimes you hear people saying "All these guys starting startups now are going to be disappointed. How many little startups are Google and Yahoo going to buy, after all?" That sounds cleverly skeptical, but I can prove it's mistaken. No one proposes that there's some limit to the number of people who can be employed in an economy consisting of big, slow-moving companies with a couple thousand people each. Why should there be any limit to the number who could be employed by small, fast-moving companies with ten each? It seems to me the only limit would be the number of people who want to work that hard.

The limit on the number of startups is not the number that can get acquired by Google and Yahoo— though it seems even that should be unlimited, if the startups were actually worth buying— but the amount of wealth that can be created. And I don't think there's any limit on that, except cosmological ones.

So for all practical purposes, there is no limit to the number of startups. Startups make wealth, which means they make things people want, and if there's a limit on the number of things people want, we are nowhere near it. I still don't even have a flying car.

7. Don't Get Your Hopes Up.

This is another one I've been repeating since long before Y Combinator. It was practically the corporate motto at Viaweb.

Startup founders are naturally optimistic. They wouldn't do it otherwise. But you should treat your optimism the way you'd treat the core of a nuclear reactor: as a source of power that's also very dangerous. You have to build a shield around it, or it will fry you.

The shielding of a reactor is not uniform; the reactor would be useless if it were. It's pierced in a few places to let pipes in. An optimism shield has to be pierced too. I think the place to draw the line is between what you expect of yourself, and what you expect of other people. It's ok to be optimistic about what you can do, but assume the worst about machines and other people.

This is particularly necessary in a startup, because you tend to be pushing the limits of whatever you're doing. So things don't happen in the smooth, predictable way they do in the rest of the world. Things change suddenly, and usually for the worse.

Shielding your optimism is nowhere more important than with deals. If your startup is doing a deal, just assume it's not going to happen. The VCs who say they're going to invest in you aren't. The company that says they're going to buy you isn't. The big customer who wants to use your system in their whole company won't. Then if things work out you can be pleasantly surprised.

The reason I warn startups not to get their hopes up is not to save them from being *disappointed* when things fall through. It's for a more practical reason: to prevent them from leaning their company against something that's going to fall over, taking them with it.

For example, if someone says they want to invest in you, there's a natural tendency to stop looking for other investors. That's why people proposing deals seem so positive: they *want* you to stop looking. And you want to stop too, because doing deals is a pain. Raising money, in particular, is a huge time sink. So you have to consciously force yourself to keep looking.

Even if you ultimately do the first deal, it will be to your advantage to have kept looking, because you'll get better terms. Deals are dynamic;

unless you're negotiating with someone unusually honest, there's not a single point where you shake hands and the deal's done. There are usually a lot of subsidiary questions to be cleared up after the handshake, and if the other side senses weakness— if they sense you need this deal— they will be very tempted to screw you in the details.

Vcs and corp dev guys are professional negotiators. They're trained to take advantage of weakness.⁸ So while they're often nice guys, they just can't help it. And as pros they do this more than you. So don't even try to bluff them. The only way a startup can have any leverage in a deal is genuinely not to need it. And if you don't believe in a deal, you'll be less likely to depend on it.

So I want to plant a hypnotic suggestion in your heads: when you hear someone say the words “we want to invest in you” or “we want to acquire you,” I want the following phrase to appear automatically in your head: *don't get your hopes up*. Just continue running your company as if this deal didn't exist. Nothing is more likely to make it close.

The way to succeed in a startup is to focus on the goal of getting lots of users, and keep walking swiftly toward it while investors and acquirers scurry alongside trying to wave money in your face.

Speed, not Money

The way I've described it, starting a startup sounds pretty stressful. It is. When I talk to the founders of the companies we've funded, they all say the same thing: I knew it would be hard, but I didn't realize it would be this hard.

So why do it? It would be worth enduring a lot of pain and stress to do something grand or heroic, but just to make money? Is making money really that important?

No, not really. It seems ridiculous to me when people take business too seriously. I regard making money as a boring errand to be got out

⁸The reason Y Combinator never negotiates valuations is that we're not professional negotiators, and don't want to turn into them.

of the way as soon as possible. There is nothing grand or heroic about starting a startup per se.

So why do I spend so much time thinking about startups? I'll tell you why. Economically, a startup is best seen not as a way to get rich, but as a way to work faster. You have to make a living, and a startup is a way to get that done quickly, instead of letting it drag on through your whole life.⁹

We take it for granted most of the time, but human life is fairly miraculous. It is also palpably short. You're given this marvellous thing, and then poof, it's taken away. You can see why people invent gods to explain it. But even to people who don't believe in gods, life commands respect. There are times in most of our lives when the days go by in a blur, and almost everyone has a sense, when this happens, of wasting something precious. As Ben Franklin said, if you love life, don't waste time, because time is what life is made of.

So no, there's nothing particularly grand about making money. That's not what makes startups worth the trouble. What's important about startups is the speed. By compressing the dull but necessary task of making a living into the smallest possible time, you show respect for life, and there is something grand about that.

Thanks to Sam Altman, Trevor Blackwell, Beau Hartshorne, Jessica Livingston, and Robert Morris for reading drafts of this.

⁹There are two ways to do work you love: (a) to make money, then work on what you love, or (b) to get a job where you get paid to work on stuff you love. In practice the first phases of both consist mostly of unedifying schleps, and in (b) the second phase is less secure.

Chapter 29

How to Be Silicon Valley

May 2006

(This essay is derived from a keynote at Xtech.)

Could you reproduce Silicon Valley elsewhere, or is there something unique about it?

It wouldn't be surprising if it were hard to reproduce in other countries, because you couldn't reproduce it in most of the US either. What does it take to make a silicon valley even here?

What it takes is the right people. If you could get the right ten thousand people to move from Silicon Valley to Buffalo, Buffalo would become Silicon Valley.¹

That's a striking departure from the past. Up till a couple decades ago, geography was destiny for cities. All great cities were located on waterways, because cities made money by trade, and water was the only economical way to ship.

Now you could make a great city anywhere, if you could get the right people to move there. So the question of how to make a silicon valley

¹It's interesting to consider how low this number could be made. I suspect five hundred would be enough, even if they could bring no assets with them. Probably just thirty, if I could pick them, would be enough to turn Buffalo into a significant startup hub.

becomes: who are the right people, and how do you get them to move?

Two Types

I think you only need two kinds of people to create a technology hub: rich people and nerds. They're the limiting reagents in the reaction that produces startups, because they're the only ones present when startups get started. Everyone else will move.

Observation bears this out: within the US, towns have become startup hubs if and only if they have both rich people and nerds. Few startups happen in Miami, for example, because although it's full of rich people, it has few nerds. It's not the kind of place nerds like.

Whereas Pittsburgh has the opposite problem: plenty of nerds, but no rich people. The top US Computer Science departments are said to be MIT, Stanford, Berkeley, and Carnegie-Mellon. MIT yielded Route 128. Stanford and Berkeley yielded Silicon Valley. But Carnegie-Mellon? The record skips at that point. Lower down the list, the University of Washington yielded a high-tech community in Seattle, and the University of Texas at Austin yielded one in Austin. But what happened in Pittsburgh? And in Ithaca, home of Cornell, which is also high on the list?

I grew up in Pittsburgh and went to college at Cornell, so I can answer for both. The weather is terrible, particularly in winter, and there's no interesting old city to make up for it, as there is in Boston. Rich people don't want to live in Pittsburgh or Ithaca. So while there are plenty of hackers who could start startups, there's no one to invest in them.

Not Bureaucrats

Do you really need the rich people? Wouldn't it work to have the government invest in the nerds? No, it would not. Startup investors are a distinct type of rich people. They tend to have a lot of experience themselves in the technology business. This (a) helps them pick the right startups, and (b) means they can supply advice and connections as well as money. And the fact that they have a personal stake in the

outcome makes them really pay attention.

Bureaucrats by their nature are the exact opposite sort of people from startup investors. The idea of them making startup investments is comic. It would be like mathematicians running *Vogue*-- or perhaps more accurately, *Vogue* editors running a math journal.²

Though indeed, most things bureaucrats do, they do badly. We just don't notice usually, because they only have to compete against other bureaucrats. But as startup investors they'd have to compete against pros with a great deal more experience and motivation.

Even corporations that have in-house VC groups generally forbid them to make their own investment decisions. Most are only allowed to invest in deals where some reputable private VC firm is willing to act as lead investor.

Not Buildings

If you go to see Silicon Valley, what you'll see are buildings. But it's the people that make it Silicon Valley, not the buildings. I read occasionally about attempts to set up "technology parks" in other places, as if the active ingredient of Silicon Valley were the office space. An article about Sophia Antipolis bragged that companies there included Cisco, Compaq, IBM, NCR, and Nortel. Don't the French realize these aren't startups?

Building office buildings for technology companies won't get you a silicon valley, because the key stage in the life of a startup happens before they want that kind of space. The key stage is when they're three guys operating out of an apartment. Wherever the startup is when it gets funded, it will stay. The defining quality of Silicon Valley is not that Intel or Apple or Google have offices there, but that they

²Bureaucrats manage to allocate research funding moderately well, but only because (like an in-house VC fund) they outsource most of the work of selection. A professor at a famous university who is highly regarded by his peers will get funding, pretty much regardless of the proposal. That wouldn't work for startups, whose founders aren't sponsored by organizations, and are often unknowns.

were *started* there.

So if you want to reproduce Silicon Valley, what you need to reproduce is those two or three founders sitting around a kitchen table deciding to start a company. And to reproduce that you need those people.

Universities

The exciting thing is, *all* you need are the people. If you could attract a critical mass of nerds and investors to live somewhere, you could reproduce Silicon Valley. And both groups are highly mobile. They'll go where life is good. So what makes a place good to them?

What nerds like is other nerds. Smart people will go wherever other smart people are. And in particular, to great universities. In theory there could be other ways to attract them, but so far universities seem to be indispensable. Within the US, there are no technology hubs without first-rate universities— or at least, first-rate computer science departments.

So if you want to make a silicon valley, you not only need a university, but one of the top handful in the world. It has to be good enough to act as a magnet, drawing the best people from thousands of miles away. And that means it has to stand up to existing magnets like MIT and Stanford.

This sounds hard. Actually it might be easy. My professor friends, when they're deciding where they'd like to work, consider one thing above all: the quality of the other faculty. What attracts professors is good colleagues. So if you managed to recruit, en masse, a significant number of the best young researchers, you could create a first-rate university from nothing overnight. And you could do that for surprisingly little. If you paid 200 people hiring bonuses of \$3 million apiece, you could put together a faculty that would bear comparison with any in the world. And from that point the chain reaction would be self-sustaining. So whatever it costs to establish a mediocre university, for

an additional half billion or so you could have a great one.³

Personality

However, merely creating a new university would not be enough to start a silicon valley. The university is just the seed. It has to be planted in the right soil, or it won't germinate. Plant it in the wrong place, and you just create Carnegie-Mellon.

To spawn startups, your university has to be in a town that has attractions other than the university. It has to be a place where investors want to live, and students want to stay after they graduate.

The two like much the same things, because most startup investors are nerds themselves. So what do nerds look for in a town? Their tastes aren't completely different from other people's, because a lot of the towns they like most in the US are also big tourist destinations: San Francisco, Boston, Seattle. But their tastes can't be quite mainstream either, because they dislike other big tourist destinations, like New York, Los Angeles, and Las Vegas.

There has been a lot written lately about the "creative class." The thesis seems to be that as wealth derives increasingly from ideas, cities will prosper only if they attract those who have them. That is certainly true; in fact it was the basis of Amsterdam's prosperity 400 years ago.

A lot of nerd tastes they share with the creative class in general. For example, they like well-preserved old neighborhoods instead of cookie-cutter suburbs, and locally-owned shops and restaurants instead of national chains. Like the rest of the creative class, they want to live somewhere with personality.

What exactly is personality? I think it's the feeling that each building is the work of a distinct group of people. A town with personality is one

³You'd have to do it all at once, or at least a whole department at a time, because people would be more likely to come if they knew their friends were. And you should probably start from scratch, rather than trying to upgrade an existing university, or much energy would be lost in friction.

that doesn't feel mass-produced. So if you want to make a startup hub—or any town to attract the “creative class”—you probably have to ban large development projects. When a large tract has been developed by a single organization, you can always tell.⁴

Most towns with personality are old, but they don't have to be. Old towns have two advantages: they're denser, because they were laid out before cars, and they're more varied, because they were built one building at a time. You could have both now. Just have building codes that ensure density, and ban large scale developments.

A corollary is that you have to keep out the biggest developer of all: the government. A government that asks “How can we build a silicon valley?” has probably ensured failure by the way they framed the question. You don't build a silicon valley; you let one grow.

Nerds

If you want to attract nerds, you need more than a town with personality. You need a town with the right personality. Nerds are a distinct subset of the creative class, with different tastes from the rest. You can see this most clearly in New York, which attracts a lot of creative people, but few nerds.⁵

What nerds like is the kind of town where people walk around smiling. This excludes LA, where no one walks at all, and also New York, where people walk, but not smiling. When I was in grad school in Boston, a friend came to visit from New York. On the subway back from the airport she asked “Why is everyone smiling?” I looked and they weren't smiling. They just looked like they were compared to the facial expressions she was used to.

⁴Hypothesis: Any plan in which multiple independent buildings are gutted or demolished to be “redeveloped” as a single project is a net loss of personality for the city, with the exception of the conversion of buildings not previously public, like warehouses.

⁵A few startups get started in New York, but less than a tenth as many per capita as in Boston, and mostly in less nerdy fields like finance and media.

If you've lived in New York, you know where these facial expressions come from. It's the kind of place where your mind may be excited, but your body knows it's having a bad time. People don't so much enjoy living there as endure it for the sake of the excitement. And if you like certain kinds of excitement, New York is incomparable. It's a hub of glamour, a magnet for all the shorter half-life isotopes of style and fame.

Nerds don't care about glamour, so to them the appeal of New York is a mystery. People who like New York will pay a fortune for a small, dark, noisy apartment in order to live in a town where the cool people are really cool. A nerd looks at that deal and sees only: pay a fortune for a small, dark, noisy apartment.

Nerds *will* pay a premium to live in a town where the smart people are really smart, but you don't have to pay as much for that. It's supply and demand: glamour is popular, so you have to pay a lot for it.

Most nerds like quieter pleasures. They like cafes instead of clubs; used bookshops instead of fashionable clothing shops; hiking instead of dancing; sunlight instead of tall buildings. A nerd's idea of paradise is Berkeley or Boulder.

Youth

It's the young nerds who start startups, so it's those specifically the city has to appeal to. The startup hubs in the US are all young-feeling towns. This doesn't mean they have to be new. Cambridge has the oldest town plan in America, but it feels young because it's full of students.

What you can't have, if you want to create a silicon valley, is a large, existing population of stodgy people. It would be a waste of time to try to reverse the fortunes of a declining industrial town like Detroit or Philadelphia by trying to encourage startups. Those places have too much momentum in the wrong direction. You're better off starting with a blank slate in the form of a small town. Or better still, if there's a town young people already flock to, that one.

The Bay Area was a magnet for the young and optimistic for decades before it was associated with technology. It was a place people went in search of something new. And so it became synonymous with California nuttiness. There's still a lot of that there. If you wanted to start a new fad— a new way to focus one's "energy," for example, or a new category of things not to eat— the Bay Area would be the place to do it. But a place that tolerates oddness in the search for the new is exactly what you want in a startup hub, because economically that's what startups are. Most good startup ideas seem a little crazy; if they were obviously good ideas, someone would have done them already.

(How many people are going to want computers in their *houses*? What, *another* search engine?)

That's the connection between technology and liberalism. Without exception the high-tech cities in the US are also the most liberal. But it's not because liberals are smarter that this is so. It's because liberal cities tolerate odd ideas, and smart people by definition have odd ideas.

Conversely, a town that gets praised for being "solid" or representing "traditional values" may be a fine place to live, but it's never going to succeed as a startup hub. The 2004 presidential election, though a disaster in other respects, conveniently supplied us with a county-by-county map of such places.⁶

To attract the young, a town must have an intact center. In most American cities the center has been abandoned, and the growth, if any, is in the suburbs. Most American cities have been turned inside out. But none of the startup hubs has: not San Francisco, or Boston, or Seattle. They all have intact centers.⁷ My guess is that no city

⁶Some blue counties are false positives (reflecting the remaining power of Democratic party machines), but there are no false negatives. You can safely write off all the red counties.

⁷Some "urban renewal" experts took a shot at destroying Boston's in the 1960s, leaving the area around city hall a bleak wasteland, but most neighborhoods successfully resisted them.

with a dead center could be turned into a startup hub. Young people don't want to live in the suburbs.

Within the US, the two cities I think could most easily be turned into new silicon valleys are Boulder and Portland. Both have the kind of effervescent feel that attracts the young. They're each only a great university short of becoming a silicon valley, if they wanted to.

Time

A great university near an attractive town. Is that all it takes? That was all it took to make the original Silicon Valley. Silicon Valley traces its origins to William Shockley, one of the inventors of the transistor. He did the research that won him the Nobel Prize at Bell Labs, but when he started his own company in 1956 he moved to Palo Alto to do it. At the time that was an odd thing to do. Why did he? Because he had grown up there and remembered how nice it was. Now Palo Alto is suburbia, but then it was a charming college town— a charming college town with perfect weather and San Francisco only an hour away.

The companies that rule Silicon Valley now are all descended in various ways from Shockley Semiconductor. Shockley was a difficult man, and in 1957 his top people— “the traitorous eight”— left to start a new company, Fairchild Semiconductor. Among them were Gordon Moore and Robert Noyce, who went on to found Intel, and Eugene Kleiner, who founded the VC firm Kleiner Perkins. Forty-two years later, Kleiner Perkins funded Google, and the partner responsible for the deal was John Doerr, who came to Silicon Valley in 1974 to work for Intel.

So although a lot of the newest companies in Silicon Valley don't make anything out of silicon, there always seem to be multiple links back to Shockley. There's a lesson here: startups beget startups. People who work for startups start their own. People who get rich from startups fund new ones. I suspect this kind of organic growth is the only way to produce a startup hub, because it's the only way to grow the expertise

you need.

That has two important implications. The first is that you need time to grow a silicon valley. The university you could create in a couple years, but the startup community around it has to grow organically. The cycle time is limited by the time it takes a company to succeed, which probably averages about five years.

The other implication of the organic growth hypothesis is that you can't be somewhat of a startup hub. You either have a self-sustaining chain reaction, or not. Observation confirms this too: cities either have a startup scene, or they don't. There is no middle ground. Chicago has the third largest metropolitan area in America. As a source of startups it's negligible compared to Seattle, number 15.

The good news is that the initial seed can be quite small. Shockley Semiconductor, though itself not very successful, was big enough. It brought a critical mass of experts in an important new technology together in a place they liked enough to stay.

Competing

Of course, a would-be silicon valley faces an obstacle the original one didn't: it has to compete with Silicon Valley. Can that be done? Probably.

One of Silicon Valley's biggest advantages is its venture capital firms. This was not a factor in Shockley's day, because VC funds didn't exist. In fact, Shockley Semiconductor and Fairchild Semiconductor were not startups at all in our sense. They were subsidiaries—of Beckman Instruments and Fairchild Camera and Instrument respectively. Those companies were apparently willing to establish subsidiaries wherever the experts wanted to live.

Venture investors, however, prefer to fund startups within an hour's drive. For one, they're more likely to notice startups nearby. But when they do notice startups in other towns they prefer them to move. They don't want to have to travel to attend board meetings, and in any case

the odds of succeeding are higher in a startup hub.

The centralizing effect of venture firms is a double one: they cause startups to form around them, and those draw in more startups through acquisitions. And although the first may be weakening because it's now so cheap to start some startups, the second seems as strong as ever. Three of the most admired "Web 2.0" companies were started outside the usual startup hubs, but two of them have already been reeled in through acquisitions.

Such centralizing forces make it harder for new silicon valleys to get started. But by no means impossible. Ultimately power rests with the founders. A startup with the best people will beat one with funding from famous VCs, and a startup that was sufficiently successful would never have to move. So a town that could exert enough pull over the right people could resist and perhaps even surpass Silicon Valley.

For all its power, Silicon Valley has a great weakness: the paradise Shockley found in 1956 is now one giant parking lot. San Francisco and Berkeley are great, but they're forty miles away. Silicon Valley proper is soul-crushing suburban sprawl. It has fabulous weather, which makes it significantly better than the soul-crushing sprawl of most other American cities. But a competitor that managed to avoid sprawl would have real leverage. All a city needs is to be the kind of place the next traitorous eight look at and say "I want to stay here," and that would be enough to get the chain reaction started.

Thanks to Chris Anderson, Trevor Blackwell, Marc Hedlund, Jessica Livingston, Robert Morris, Greg Mcadoo, Fred Wilson, and Stephen Wolfram for reading drafts of this, and to Ed Dumbill for inviting me to speak.

(The second part of this talk became *Why Startups Condense in America*.)

Chapter 30

Why Startups Condense in America

May 2006

(This essay is derived from a keynote at Xtech.)

Startups happen in clusters. There are a lot of them in Silicon Valley and Boston, and few in Chicago or Miami. A country that wants startups will probably also have to reproduce whatever makes these clusters form.

I've claimed that the recipe is a great university near a town smart people like. If you set up those conditions within the US, startups will form as inevitably as water droplets condense on a cold piece of metal. But when I consider what it would take to reproduce Silicon Valley in another country, it's clear the US is a particularly humid environment. Startups condense more easily here.

It is by no means a lost cause to try to create a silicon valley in another country. There's room not merely to equal Silicon Valley, but to surpass it. But if you want to do that, you have to understand the advantages startups get from being in America.

1. The US Allows Immigration.

For example, I doubt it would be possible to reproduce Silicon Val-

ley in Japan, because one of Silicon Valley's most distinctive features is immigration. Half the people there speak with accents. And the Japanese don't like immigration. When they think about how to make a Japanese silicon valley, I suspect they unconsciously frame it as how to make one consisting only of Japanese people. This way of framing the question probably guarantees failure.

A silicon valley has to be a mecca for the smart and the ambitious, and you can't have a mecca if you don't let people into it.

Of course, it's not saying much that America is more open to immigration than Japan. Immigration policy is one area where a competitor could do better.

2. The US Is a Rich Country.

I could see India one day producing a rival to Silicon Valley. Obviously they have the right people: you can tell that by the number of Indians in the current Silicon Valley. The problem with India itself is that it's still so poor.

In poor countries, things we take for granted are missing. A friend of mine visiting India sprained her ankle falling down the steps in a railway station. When she turned to see what had happened, she found the steps were all different heights. In industrialized countries we walk down steps our whole lives and never think about this, because there's an infrastructure that prevents such a staircase from being built.

The US has never been so poor as some countries are now. There have never been swarms of beggars in the streets of American cities. So we have no data about what it takes to get from the swarms-of-beggars stage to the silicon-valley stage. Could you have both at once, or does there have to be some baseline prosperity before you get a silicon valley?

I suspect there is some speed limit to the evolution of an economy. Economies are made out of people, and attitudes can only change a

certain amount per generation.¹

3. The US Is Not (Yet) a Police State.

Another country I could see wanting to have a silicon valley is China. But I doubt they could do it yet either. China still seems to be a police state, and although present rulers seem enlightened compared to the last, even enlightened despotism can probably only get you part way toward being a great economic power.

It can get you factories for building things designed elsewhere. Can it get you the designers, though? Can imagination flourish where people can't criticize the government? Imagination means having odd ideas, and it's hard to have odd ideas about technology without also having odd ideas about politics. And in any case, many technical ideas do have political implications. So if you squash dissent, the back pressure will propagate into technical fields.²

Singapore would face a similar problem. Singapore seems very aware of the importance of encouraging startups. But while energetic government intervention may be able to make a port run efficiently, it can't coax startups into existence. A state that bans chewing gum has a long way to go before it could create a San Francisco.

Do you need a San Francisco? Might there not be an alternate route to innovation that goes through obedience and cooperation instead of individualism? Possibly, but I'd bet not. Most imaginative people seem to share a certain prickly independence, whenever and wherever they lived. You see it in Diogenes telling Alexander to get out of his light and two thousand years later in Feynman breaking into safes

¹On the verge of the Industrial Revolution, England was already the richest country in the world. As far as such things can be compared, per capita income in England in 1750 was higher than India's in 1960.

Deane, Phyllis, *The First Industrial Revolution*, Cambridge University Press, 1965.

²This has already happened once in China, during the Ming Dynasty, when the country turned its back on industrialization at the command of the court. One of Europe's advantages was that it had no government powerful enough to do that.

at Los Alamos. ³ Imaginative people don't want to follow or lead. They're most productive when everyone gets to do what they want.

Ironically, of all rich countries the US has lost the most civil liberties recently. But I'm not too worried yet. I'm hoping once the present administration is out, the natural openness of American culture will reassert itself.

4. American Universities Are Better.

You need a great university to seed a silicon valley, and so far there are few outside the US. I asked a handful of American computer science professors which universities in Europe were most admired, and they all basically said "Cambridge" followed by a long pause while they tried to think of others. There don't seem to be many universities elsewhere that compare with the best in America, at least in technology.

In some countries this is the result of a deliberate policy. The German and Dutch governments, perhaps from fear of elitism, try to ensure that all universities are roughly equal in quality. The downside is that none are especially good. The best professors are spread out, instead of being concentrated as they are in the US. This probably makes them less productive, because they don't have good colleagues to inspire them. It also means no one university will be good enough to act as a mecca, attracting talent from abroad and causing startups to form around it.

The case of Germany is a strange one. The Germans invented the modern university, and up till the 1930s theirs were the best in the world. Now they have none that stand out. As I was mulling this over, I found myself thinking: "I can understand why German universities declined in the 1930s, after they excluded Jews. But surely they should have bounced back by now." Then I realized: maybe not. There are few Jews left in Germany and most Jews I know would not want

³Of course, Feynman and Diogenes were from adjacent traditions, but Confucius, though more polite, was no more willing to be told what to think.

to move there. And if you took any great American university and removed the Jews, you'd have some pretty big gaps. So maybe it would be a lost cause trying to create a silicon valley in Germany, because you couldn't establish the level of university you'd need as a seed.⁴

It's natural for US universities to compete with one another because so many are private. To reproduce the quality of American universities you probably also have to reproduce this. If universities are controlled by the central government, log-rolling will pull them all toward the mean: the new Institute of X will end up at the university in the district of a powerful politician, instead of where it should be.

5. You Can Fire People in America.

I think one of the biggest obstacles to creating startups in Europe is the attitude toward employment. The famously rigid labor laws hurt every company, but startups especially, because startups have the least time to spare for bureaucratic hassles.

The difficulty of firing people is a particular problem for startups because they have no redundancy. Every person has to do their job well.

But the problem is more than just that some startup might have a problem firing someone they needed to. Across industries and countries, there's a strong inverse correlation between performance and job security. Actors and directors are fired at the end of each film, so they have to deliver every time. Junior professors are fired by default after a few years unless the university chooses to grant them tenure. Professional athletes know they'll be pulled if they play badly for just a couple games. At the other end of the scale (at least in the US) are auto workers, New York City schoolteachers, and civil servants, who

⁴For similar reasons it might be a lost cause to try to establish a silicon valley in Israel. Instead of no Jews moving there, only Jews would move there, and I don't think you could build a silicon valley out of just Jews any more than you could out of just Japanese.

(This is not a remark about the qualities of these groups, just their sizes. Japanese are only about 2% of the world population, and Jews about .2%.)

are all nearly impossible to fire. The trend is so clear that you'd have to be willfully blind not to see it.

Performance isn't everything, you say? Well, are auto workers, schoolteachers, and civil servants *happier* than actors, professors, and professional athletes?

European public opinion will apparently tolerate people being fired in industries where they really care about performance. Unfortunately the only industry they care enough about so far is soccer. But that is at least a precedent.

6. In America Work Is Less Identified with Employment.

The problem in more traditional places like Europe and Japan goes deeper than the employment laws. More dangerous is the attitude they reflect: that an employee is a kind of servant, whom the employer has a duty to protect. It used to be that way in America too. In 1970 you were still supposed to get a job with a big company, for whom ideally you'd work your whole career. In return the company would take care of you: they'd try not to fire you, cover your medical expenses, and support you in old age.

Gradually employment has been shedding such paternalistic overtones and becoming simply an economic exchange. But the importance of the new model is not just that it makes it easier for startups to grow. More important, I think, is that it makes it easier for people to *start* startups.

Even in the US most kids graduating from college still think they're supposed to get jobs, as if you couldn't be productive without being someone's employee. But the less you identify work with employment, the easier it becomes to start a startup. When you see your career as a series of different types of work, instead of a lifetime's service to a single employer, there's less risk in starting your own company, because you're only replacing one segment instead of discarding the whole thing.

The old ideas are so powerful that even the most successful startup founders have had to struggle against them. A year after the founding of Apple, Steve Wozniak still hadn't quit HP. He still planned to work there for life. And when Jobs found someone to give Apple serious venture funding, on the condition that Woz quit, he initially refused, arguing that he'd designed both the Apple I and the Apple II while working at HP, and there was no reason he couldn't continue.

7. America Is Not Too Fussy.

If there are any laws regulating businesses, you can assume larval startups will break most of them, because they don't know what the laws are and don't have time to find out.

For example, many startups in America begin in places where it's not really legal to run a business. Hewlett-Packard, Apple, and Google were all run out of garages. Many more startups, including ours, were initially run out of apartments. If the laws against such things were actually enforced, most startups wouldn't happen.

That could be a problem in fussier countries. If Hewlett and Packard tried running an electronics company out of their garage in Switzerland, the old lady next door would report them to the municipal authorities.

But the worst problem in other countries is probably the effort required just to start a company. A friend of mine started a company in Germany in the early 90s, and was shocked to discover, among many other regulations, that you needed \$20,000 in capital to incorporate. That's one reason I'm not typing this on an Apfel laptop. Jobs and Wozniak couldn't have come up with that kind of money in a company financed by selling a VW bus and an HP calculator. We couldn't have started Viaweb either. ⁵

Here's a tip for governments that want to encourage startups: read

⁵According to the World Bank, the initial capital requirement for German companies is 47.6% of the per capita income. Doh.

World Bank, *Doing Business in 2006*, <http://doingbusiness.org>

the stories of existing startups, and then try to simulate what would have happened in your country. When you hit something that would have killed Apple, prune it off.

Startups are marginal. They're started by the poor and the timid; they begin in marginal space and spare time; they're started by people who are supposed to be doing something else; and though businesses, their founders often know nothing about business. Young startups are fragile. A society that trims its margins sharply will kill them all.

8. America Has a Large Domestic Market.

What sustains a startup in the beginning is the prospect of getting their initial product out. The successful ones therefore make the first version as simple as possible. In the US they usually begin by making something just for the local market.

This works in America, because the local market is 300 million people. It wouldn't work so well in Sweden. In a small country, a startup has a harder task: they have to sell internationally from the start.

The EU was designed partly to simulate a single, large domestic market. The problem is that the inhabitants still speak many different languages. So a software startup in Sweden is still at a disadvantage relative to one in the US, because they have to deal with internationalization from the beginning. It's significant that the most famous recent startup in Europe, Skype, worked on a problem that was intrinsically international.

However, for better or worse it looks as if Europe will in a few decades speak a single language. When I was a student in Italy in 1990, few Italians spoke English. Now all educated people seem to be expected to— and Europeans do not like to seem uneducated. This is presumably a taboo subject, but if present trends continue, French and German will eventually go the way of Irish and Luxembourgish: they'll be spoken in homes and by eccentric nationalists.

9. America Has Venture Funding.

Startups are easier to start in America because funding is easier to get. There are now a few VC firms outside the US, but startup funding doesn't only come from VC firms. A more important source, because it's more personal and comes earlier in the process, is money from individual angel investors. Google might never have got to the point where they could raise millions from VC funds if they hadn't first raised a hundred thousand from Andy Bechtolsheim. And he could help them because he was one of the founders of Sun. This pattern is repeated constantly in startup hubs. It's this pattern that *makes* them startup hubs.

The good news is, all you have to do to get the process rolling is get those first few startups successfully launched. If they stick around after they get rich, startup founders will almost automatically fund and encourage new startups.

The bad news is that the cycle is slow. It probably takes five years, on average, before a startup founder can make angel investments. And while governments *might* be able to set up local VC funds by supplying the money themselves and recruiting people from existing firms to run them, only organic growth can produce angel investors.

Incidentally, America's private universities are one reason there's so much venture capital. A lot of the money in VC funds comes from their endowments. So another advantage of private universities is that a good chunk of the country's wealth is managed by enlightened investors.

10. America Has Dynamic Typing for Careers.

Compared to other industrialized countries the US is disorganized about routing people into careers. For example, in America people often don't decide to go to medical school till they've finished college. In Europe they generally decide in high school.

The European approach reflects the old idea that each person has a single, definite occupation— which is not far from the idea that each person has a natural “station” in life. If this were true, the most effi-

cient plan would be to discover each person's station as early as possible, so they could receive the training appropriate to it.

In the US things are more haphazard. But that turns out to be an advantage as an economy gets more liquid, just as dynamic typing turns out to work better than static for ill-defined problems. This is particularly true with startups. "Startup founder" is not the sort of career a high school student would choose. If you ask at that age, people will choose conservatively. They'll choose well-understood occupations like engineer, or doctor, or lawyer.

Startups are the kind of thing people don't plan, so you're more likely to get them in a society where it's ok to make career decisions on the fly.

For example, in theory the purpose of a PhD program is to train you to do research. But fortunately in the US this is another rule that isn't very strictly enforced. In the US most people in CS PhD programs are there simply because they wanted to learn more. They haven't decided what they'll do afterward. So American grad schools spawn a lot of startups, because students don't feel they're failing if they don't go into research.

Those worried about America's "competitiveness" often suggest spending more on public schools. But perhaps America's lousy public schools have a hidden advantage. Because they're so bad, the kids adopt an attitude of waiting for college. I did; I knew I was learning so little that I wasn't even learning what the choices were, let alone which to choose. This is demoralizing, but it does at least make you keep an open mind.

Certainly if I had to choose between bad high schools and good universities, like the US, and good high schools and bad universities, like most other industrialized countries, I'd take the US system. Better to make everyone feel like a late bloomer than a failed child prodigy.

Attitudes

There's one item conspicuously missing from this list: American attitudes. Americans are said to be more entrepreneurial, and less afraid of risk. But America has no monopoly on this. Indians and Chinese seem plenty entrepreneurial, perhaps more than Americans.

Some say Europeans are less energetic, but I don't believe it. I think the problem with Europe is not that they lack balls, but that they lack examples.

Even in the US, the most successful startup founders are often technical people who are quite timid, initially, about the idea of starting their own company. Few are the sort of backslapping extroverts one thinks of as typically American. They can usually only summon up the activation energy to start a startup when they meet people who've done it and realize they could too.

I think what holds back European hackers is simply that they don't meet so many people who've done it. You see that variation even within the US. Stanford students are more entrepreneurial than Yale students, but not because of some difference in their characters; the Yale students just have fewer examples.

I admit there seem to be different attitudes toward ambition in Europe and the US. In the US it's ok to be overtly ambitious, and in most of Europe it's not. But this can't be an intrinsically European quality; previous generations of Europeans were as ambitious as Americans. What happened? My hypothesis is that ambition was discredited by the terrible things ambitious people did in the first half of the twentieth century. Now swagger is out. (Even now the image of a very ambitious German presses a button or two, doesn't it?)

It would be surprising if European attitudes weren't affected by the disasters of the twentieth century. It takes a while to be optimistic after events like that. But ambition is human nature. Gradually it will re-emerge.⁶

⁶For most of the twentieth century, Europeans looked back on the summer of 1914 as if they'd been living in a dream world. It seems more accurate (or at least,

How To Do Better

I don't mean to suggest by this list that America is the perfect place for startups. It's the best place so far, but the sample size is small, and "so far" is not very long. On historical time scales, what we have now is just a prototype.

So let's look at Silicon Valley the way you'd look at a product made by a competitor. What weaknesses could you exploit? How could you make something users would like better? The users in this case are those critical few thousand people you'd like to move to your silicon valley.

To start with, Silicon Valley is too far from San Francisco. Palo Alto, the original ground zero, is about thirty miles away, and the present center more like forty. So people who come to work in Silicon Valley face an unpleasant choice: either live in the boring sprawl of the valley proper, or live in San Francisco and endure an hour commute each way.

The best thing would be if the silicon valley were not merely closer to the interesting city, but interesting itself. And there is a lot of room for improvement here. Palo Alto is not so bad, but everything built since is the worst sort of strip development. You can measure how demoralizing it is by the number of people who will sacrifice two hours a day commuting rather than live there.

Another area in which you could easily surpass Silicon Valley is public transportation. There is a train running the length of it, and by American standards it's not bad. Which is to say that to Japanese or Europeans it would seem like something out of the third world.

The kind of people you want to attract to your silicon valley like to get around by train, bicycle, and on foot. So if you want to beat America, design a town that puts cars last. It will be a while before any

as accurate) to call the years after 1914 a nightmare than to call those before a dream. A lot of the optimism Europeans consider distinctly American is simply what they too were feeling in 1914.

American city can bring itself to do that.

Capital Gains

There are also a couple things you could do to beat America at the national level. One would be to have lower capital gains taxes. It doesn't seem critical to have the lowest *income* taxes, because to take advantage of those, people have to move.⁷ But if capital gains rates vary, you move assets, not yourself, so changes are reflected at market speeds. The lower the rate, the cheaper it is to buy stock in growing companies as opposed to real estate, or bonds, or stocks bought for the dividends they pay.

So if you want to encourage startups you should have a low rate on capital gains. Politicians are caught between a rock and a hard place here, however: make the capital gains rate low and be accused of creating "tax breaks for the rich," or make it high and starve growing companies of investment capital. As Galbraith said, politics is a matter of choosing between the unpalatable and the disastrous. A lot of governments experimented with the disastrous in the twentieth century; now the trend seems to be toward the merely unpalatable.

Oddly enough, the leaders now are European countries like Belgium, which has a capital gains tax rate of zero.

Immigration

The other place you could beat the US would be with smarter immigration policy. There are huge gains to be made here. Silicon valleys are made of people, remember.

⁷The point where things start to go wrong seems to be about 50%. Above that people get serious about tax avoidance. The reason is that the payoff for avoiding tax grows hyperexponentially ($x/1-x$ for $0 < x < 1$). If your income tax rate is 10%, moving to Monaco would only give you 11% more income, which wouldn't even cover the extra cost. If it's 90%, you'd get ten times as much income. And at 98%, as it was briefly in Britain in the 70s, moving to Monaco would give you fifty times as much income. It seems quite likely that European governments of the 70s never drew this curve.

Like a company whose software runs on Windows, those in the current Silicon Valley are all too aware of the shortcomings of the INS, but there's little they can do about it. They're hostages of the platform.

America's immigration system has never been well run, and since 2001 there has been an additional admixture of paranoia. What fraction of the smart people who want to come to America can even get in? I doubt even half. Which means if you made a competing technology hub that let in all smart people, you'd immediately get more than half the world's top talent, for free.

US immigration policy is particularly ill-suited to startups, because it reflects a model of work from the 1970s. It assumes good technical people have college degrees, and that work means working for a big company.

If you don't have a college degree you can't get an H1B visa, the type usually issued to programmers. But a test that excludes Steve Jobs, Bill Gates, and Michael Dell can't be a good one. Plus you can't get a visa for working on your own company, only for working as an employee of someone else's. And if you want to apply for citizenship you daren't work for a startup at all, because if your sponsor goes out of business, you have to start over.

American immigration policy keeps out most smart people, and channels the rest into unproductive jobs. It would be easy to do better. Imagine if, instead, you treated immigration like recruiting— if you made a conscious effort to seek out the smartest people and get them to come to your country.

A country that got immigration right would have a huge advantage. At this point you could become a mecca for smart people simply by having an immigration system that let them in.

A Good Vector

If you look at the kinds of things you have to do to create an environment where startups condense, none are great sacrifices. Great uni-

versities? Livable towns? Civil liberties? Flexible employment laws? Immigration policies that let in smart people? Tax laws that encourage growth? It's not as if you have to risk destroying your country to get a silicon valley; these are all good things in their own right.

And then of course there's the question, can you afford not to? I can imagine a future in which the default choice of ambitious young people is to start their own company rather than work for someone else's. I'm not sure that will happen, but it's where the trend points now. And if that is the future, places that don't have startups will be a whole step behind, like those that missed the Industrial Revolution.

Thanks to Trevor Blackwell, Matthias Felleisen, Jessica Livingston, Robert Morris, Neil Rimer, Hugues Steinier, Brad Templeton, Fred Wilson, and Stephen Wolfram for reading drafts of this, and to Ed Dumbill for inviting me to speak.

Chapter 31

The Power of the Marginal

June 2006

(This essay is derived from talks at Usenix 2006 and Railsconf 2006.)

A couple years ago my friend Trevor and I went to look at the Apple garage. As we stood there, he said that as a kid growing up in Saskatchewan he'd been amazed at the dedication Jobs and Wozniak must have had to work in a garage.

“Those guys must have been freezing!”

That's one of California's hidden advantages: the mild climate means there's lots of marginal space. In cold places that margin gets trimmed off. There's a sharper line between outside and inside, and only projects that are officially sanctioned — by organizations, or parents, or wives, or at least by oneself — get proper indoor space. That raises the activation energy for new ideas. You can't just tinker. You have to justify.

Some of Silicon Valley's most famous companies began in garages: Hewlett-Packard in 1938, Apple in 1976, Google in 1998. In Apple's case the garage story is a bit of an urban legend. Woz says all they did there was assemble some computers, and that he did all the actual design of the Apple I and Apple II in his apartment or his cube at HP.

¹ This was apparently too marginal even for Apple's PR people.

By conventional standards, Jobs and Wozniak were marginal people too. Obviously they were smart, but they can't have looked good on paper. They were at the time a pair of college dropouts with about three years of school between them, and hippies to boot. Their previous business experience consisted of making "blue boxes" to hack into the phone system, a business with the rare distinction of being both illegal and unprofitable.

Outsiders

Now a startup operating out of a garage in Silicon Valley would feel part of an exalted tradition, like the poet in his garret, or the painter who can't afford to heat his studio and thus has to wear a beret indoors. But in 1976 it didn't seem so cool. The world hadn't yet realized that starting a computer company was in the same category as being a writer or a painter. It hadn't been for long. Only in the preceding couple years had the dramatic fall in the cost of hardware allowed outsiders to compete.

In 1976, everyone looked down on a company operating out of a garage, including the founders. One of the first things Jobs did when they got some money was to rent office space. He wanted Apple to seem like a real company.

They already had something few real companies ever have: a fabulously well designed product. You'd think they'd have had more confidence. But I've talked to a lot of startup founders, and it's always this way. They've built something that's going to change the world, and they're worried about some nit like not having proper business cards.

That's the paradox I want to explore: great new things often come from the margins, and yet the people who discover them are looked down on by everyone, including themselves.

¹The facts about Apple's early history are from an interview with Steve Wozniak in Jessica Livingston's *Founders at Work*.

It's an old idea that new things come from the margins. I want to examine its internal structure. Why do great ideas come from the margins? What kind of ideas? And is there anything we can do to encourage the process?

Insiders

One reason so many good ideas come from the margin is simply that there's so much of it. There have to be more outsiders than insiders, if insider means anything. If the number of outsiders is huge it will always seem as if a lot of ideas come from them, even if few do per capita. But I think there's more going on than this. There are real disadvantages to being an insider, and in some kinds of work they can outweigh the advantages.

Imagine, for example, what would happen if the government decided to commission someone to write an official Great American Novel. First there'd be a huge ideological squabble over who to choose. Most of the best writers would be excluded for having offended one side or the other. Of the remainder, the smart ones would refuse such a job, leaving only a few with the wrong sort of ambition. The committee would choose one at the height of his career — that is, someone whose best work was behind him — and hand over the project with copious free advice about how the book should show in positive terms the strength and diversity of the American people, etc, etc.

The unfortunate writer would then sit down to work with a huge weight of expectation on his shoulders. Not wanting to blow such a public commission, he'd play it safe. This book had better command respect, and the way to ensure that would be to make it a tragedy. Audiences have to be enticed to laugh, but if you kill people they feel obliged to take you seriously. As everyone knows, America plus tragedy equals the Civil War, so that's what it would have to be about. When finally completed twelve years later, the book would be a 900-page pastiche of existing popular novels — roughly *Gone with the Wind* plus *Roots*. But its bulk and celebrity would make it a bestseller for a few months, until blown out of the water by a talk-show host's au-

tobiography. The book would be made into a movie and thereupon forgotten, except by the more waspish sort of reviewers, among whom it would be a byword for bogusness like Milli Vanilli or *Battlefield Earth*.

Maybe I got a little carried away with this example. And yet is this not at each point the way such a project would play out? The government knows better than to get into the novel business, but in other fields where they have a natural monopoly, like nuclear waste dumps, aircraft carriers, and regime change, you'd find plenty of projects isomorphic to this one — and indeed, plenty that were less successful.

This little thought experiment suggests a few of the disadvantages of insider projects: the selection of the wrong kind of people, the excessive scope, the inability to take risks, the need to seem serious, the weight of expectations, the power of vested interests, the undiscerning audience, and perhaps most dangerous, the tendency of such work to become a duty rather than a pleasure.

Tests

A world with outsiders and insiders implies some kind of test for distinguishing between them. And the trouble with most tests for selecting elites is that there are two ways to pass them: to be good at what they try to measure, and to be good at hacking the test itself.

So the first question to ask about a field is how honest its tests are, because this tells you what it means to be an outsider. This tells you how much to trust your instincts when you disagree with authorities, whether it's worth going through the usual channels to become one yourself, and perhaps whether you want to work in this field at all.

Tests are least hackable when there are consistent standards for quality, and the people running the test really care about its integrity. Admissions to PhD programs in the hard sciences are fairly honest, for example. The professors will get whoever they admit as their own grad students, so they try hard to choose well, and they have a fair amount of data to go on. Whereas undergraduate admissions seem to be much more hackable.

One way to tell whether a field has consistent standards is the overlap between the leading practitioners and the people who teach the subject in universities. At one end of the scale you have fields like math and physics, where nearly all the teachers are among the best practitioners. In the middle are medicine, law, history, architecture, and computer science, where many are. At the bottom are business, literature, and the visual arts, where there's almost no overlap between the teachers and the leading practitioners. It's this end that gives rise to phrases like "those who can't do, teach."

Incidentally, this scale might be helpful in deciding what to study in college. When I was in college the rule seemed to be that you should study whatever you were most interested in. But in retrospect you're probably better off studying something moderately interesting with someone who's good at it than something very interesting with someone who isn't. You often hear people say that you shouldn't major in business in college, but this is actually an instance of a more general rule: don't learn things from teachers who are bad at them.

How much you should worry about being an outsider depends on the quality of the insiders. If you're an amateur mathematician and think you've solved a famous open problem, better go back and check. When I was in grad school, a friend in the math department had the job of replying to people who sent in proofs of Fermat's last theorem and so on, and it did not seem as if he saw it as a valuable source of tips — more like manning a mental health hotline. Whereas if the stuff you're writing seems different from what English professors are interested in, that's not necessarily a problem.

Anti-Tests

Where the method of selecting the elite is thoroughly corrupt, most of the good people will be outsiders. In art, for example, the image of the poor, misunderstood genius is not just one possible image of a great artist: it's the *standard* image. I'm not saying it's correct, incidentally, but it is telling how well this image has stuck. You couldn't make a

rap like that stick to math or medicine.²

If it's corrupt enough, a test becomes an anti-test, filtering out the people it should select by making them to do things only the wrong people would do. Popularity in high school seems to be such a test. There are plenty of similar ones in the grownup world. For example, rising up through the hierarchy of the average big company demands an attention to politics few thoughtful people could spare.³ Someone like Bill Gates can grow a company under him, but it's hard to imagine him having the patience to climb the corporate ladder at General Electric — or Microsoft, actually.

It's kind of strange when you think about it, because lord-of-the-flies schools and bureaucratic companies are both the default. There are probably a lot of people who go from one to the other and never realize the whole world doesn't work this way.

I think that's one reason big companies are so often blindsided by startups. People at big companies don't realize the extent to which they live in an environment that is one large, ongoing test for the wrong qualities.

If you're an outsider, your best chances for beating insiders are obviously in fields where corrupt tests select a lame elite. But there's a catch: if the tests are corrupt, your victory won't be recognized, at least in your lifetime. You may feel you don't need that, but history suggests it's dangerous to work in fields with corrupt tests. You may beat the insiders, and yet not do as good work, on an absolute scale, as you would in a field that was more honest.

²As usual the popular image is several decades behind reality. Now the misunderstood artist is not a chain-smoking drunk who pours his soul into big, messy canvases that philistines see and say "that's not art" because it isn't a picture of anything. The philistines have now been trained that anything hung on a wall is art. Now the misunderstood artist is a coffee-drinking vegan cartoonist whose work they see and say "that's not art" because it looks like stuff they've seen in the Sunday paper.

³In fact this would do fairly well as a definition of politics: what determines rank in the absence of objective tests.

Standards in art, for example, were almost as corrupt in the first half of the eighteenth century as they are today. This was the era of those fluffy idealized portraits of countesses with their lapdogs. Chardin decided to skip all that and paint ordinary things as he saw them. He's now considered the best of that period — and yet not the equal of Leonardo or Bellini or Memling, who all had the additional encouragement of honest standards.

It can be worth participating in a corrupt contest, however, if it's followed by another that isn't corrupt. For example, it would be worth competing with a company that can spend more than you on marketing, as long as you can survive to the next round, when customers compare your actual products. Similarly, you shouldn't be discouraged by the comparatively corrupt test of college admissions, because it's followed immediately by less hackable tests.⁴

Risk

Even in a field with honest tests, there are still advantages to being an outsider. The most obvious is that outsiders have nothing to lose. They can do risky things, and if they fail, so what? Few will even notice.

The eminent, on the other hand, are weighed down by their eminence. Eminence is like a suit: it impresses the wrong people, and it constrains the wearer.

Outsiders should realize the advantage they have here. Being able to take risks is hugely valuable. Everyone values safety too much, both the obscure and the eminent. No one wants to look like a fool. But it's very useful to be able to. If most of your ideas aren't stupid, you're probably being too conservative. You're not bracketing the problem.

Lord Acton said we should judge talent at its best and character at its

⁴In high school you're led to believe your whole future depends on where you go to college, but it turns out only to buy you a couple years. By your mid-twenties the people worth impressing already judge you more by what you've done than where you went to school.

worst. For example, if you write one great book and ten bad ones, you still count as a great writer — or at least, a better writer than someone who wrote eleven that were merely good. Whereas if you're a quiet, law-abiding citizen most of the time but occasionally cut someone up and bury them in your backyard, you're a bad guy.

Almost everyone makes the mistake of treating ideas as if they were indications of character rather than talent — as if having a stupid idea made you stupid. There's a huge weight of tradition advising us to play it safe. "Even a fool is thought wise if he keeps silent," says the Old Testament (Proverbs 17:28).

Well, that may be fine advice for a bunch of goatherds in Bronze Age Palestine. There conservatism would be the order of the day. But times have changed. It might still be reasonable to stick with the Old Testament in political questions, but materially the world now has a lot more state. Tradition is less of a guide, not just because things change faster, but because the space of possibilities is so large. The more complicated the world gets, the more valuable it is to be willing to look like a fool.

Delegation

And yet the more successful people become, the more heat they get if they screw up — or even seem to screw up. In this respect, as in many others, the eminent are prisoners of their own success. So the best way to understand the advantages of being an outsider may be to look at the disadvantages of being an insider.

If you ask eminent people what's wrong with their lives, the first thing they'll complain about is the lack of time. A friend of mine at Google is fairly high up in the company and went to work for them long before they went public. In other words, he's now rich enough not to have to work. I asked him if he could still endure the annoyances of having a job, now that he didn't have to. And he said that there weren't really any annoyances, except — and he got a wistful look when he said this — that he got *so much email*.

The eminent feel like everyone wants to take a bite out of them. The problem is so widespread that people pretending to be eminent do it by pretending to be overstretched.

The lives of the eminent become scheduled, and that's not good for thinking. One of the great advantages of being an outsider is long, uninterrupted blocks of time. That's what I remember about grad school: apparently endless supplies of time, which I spent worrying about, but not writing, my dissertation. Obscurity is like health food — unpleasant, perhaps, but good for you. Whereas fame tends to be like the alcohol produced by fermentation. When it reaches a certain concentration, it kills off the yeast that produced it.

The eminent generally respond to the shortage of time by turning into managers. They don't have time to work. They're surrounded by junior people they're supposed to help or supervise. The obvious solution is to have the junior people do the work. Some good stuff happens this way, but there are problems it doesn't work so well for: the kind where it helps to have everything in one head.

For example, it recently emerged that the famous glass artist Dale Chihuly hasn't actually blown glass for 27 years. He has assistants do the work for him. But one of the most valuable sources of ideas in the visual arts is the resistance of the medium. That's why oil paintings look so different from watercolors. In principle you could make any mark in any medium; in practice the medium steers you. And if you're no longer doing the work yourself, you stop learning from this.

So if you want to beat those eminent enough to delegate, one way to do it is to take advantage of direct contact with the medium. In the arts it's obvious how: blow your own glass, edit your own films, stage your own plays. And in the process pay close attention to accidents and to new ideas you have on the fly. This technique can be generalized to any sort of work: if you're an outsider, don't be ruled by plans. Planning is often just a weakness forced on those who delegate.

Is there a general rule for finding problems best solved in one head?

Well, you can manufacture them by taking any project usually done by multiple people and trying to do it all yourself. Wozniak's work was a classic example: he did everything himself, hardware and software, and the result was miraculous. He claims not one bug was ever found in the Apple II, in either hardware or software.

Another way to find good problems to solve in one head is to focus on the grooves in the chocolate bar — the places where tasks are divided when they're split between several people. If you want to beat delegation, focus on a vertical slice: for example, be both writer and editor, or both design buildings and construct them.

One especially good groove to span is the one between tools and things made with them. For example, programming languages and applications are usually written by different people, and this is responsible for a lot of the worst flaws in programming languages. I think every language should be designed simultaneously with a large application written in it, the way C was with Unix.

Techniques for competing with delegation translate well into business, because delegation is endemic there. Instead of avoiding it as a drawback of senility, many companies embrace it as a sign of maturity. In big companies software is often designed, implemented, and sold by three separate types of people. In startups one person may have to do all three. And though this feels stressful, it's one reason startups win. The needs of customers and the means of satisfying them are all in one head.

Focus

The very skill of insiders can be a weakness. Once someone is good at something, they tend to spend all their time doing that. This kind of focus is very valuable, actually. Much of the skill of experts is the ability to ignore false trails. But focus has drawbacks: you don't learn from other fields, and when a new approach arrives, you may be the last to notice.

For outsiders this translates into two ways to win. One is to work on a

variety of things. Since you can't derive as much benefit (yet) from a narrow focus, you may as well cast a wider net and derive what benefit you can from similarities between fields. Just as you can compete with delegation by working on larger vertical slices, you can compete with specialization by working on larger horizontal slices — by both writing and illustrating your book, for example.

The second way to compete with focus is to see what focus overlooks. In particular, new things. So if you're not good at anything yet, consider working on something so new that no one else is either. It won't have any prestige yet, if no one is good at it, but you'll have it all to yourself.

The potential of a new medium is usually underestimated, precisely because no one has yet explored its possibilities. Before Durer tried making engravings, no one took them very seriously. Engraving was for making little devotional images — basically fifteenth century baseball cards of saints. Trying to make masterpieces in this medium must have seemed to Durer's contemporaries the way that, say, making masterpieces in comics might seem to the average person today.

In the computer world we get not new mediums but new platforms: the minicomputer, the microprocessor, the web-based application. At first they're always dismissed as being unsuitable for real work. And yet someone always decides to try anyway, and it turns out you can do more than anyone expected. So in the future when you hear people say of a new platform: yeah, it's popular and cheap, but not ready yet for real work, jump on it.

As well as being more comfortable working on established lines, insiders generally have a vested interest in perpetuating them. The professor who made his reputation by discovering some new idea is not likely to be the one to discover its replacement. This is particularly true with companies, who have not only skill and pride anchoring them to the status quo, but money as well. The Achilles heel of successful companies is their inability to cannibalize themselves. Many innovations consist of replacing something with a cheaper alternative, and com-

panies just don't want to see a path whose immediate effect is to cut an existing source of revenue.

So if you're an outsider you should actively seek out contrarian projects. Instead of working on things the eminent have made prestigious, work on things that could steal that prestige.

The really juicy new approaches are not the ones insiders reject as impossible, but those they ignore as undignified. For example, after Wozniak designed the Apple II he offered it first to his employer, HP. They passed. One of the reasons was that, to save money, he'd designed the Apple II to use a TV as a monitor, and HP felt they couldn't produce anything so *declassé*.

Less

Wozniak used a TV as a monitor for the simple reason that he couldn't afford a monitor. Outsiders are not merely free but compelled to make things that are cheap and lightweight. And both are good bets for growth: cheap things spread faster, and lightweight things evolve faster.

The eminent, on the other hand, are almost forced to work on a large scale. Instead of garden sheds they must design huge art museums. One reason they work on big things is that they can: like our hypothetical novelist, they're flattered by such opportunities. They also know that big projects will by their sheer bulk impress the audience. A garden shed, however lovely, would be easy to ignore; a few might even snicker at it. You can't snicker at a giant museum, no matter how much you dislike it. And finally, there are all those people the eminent have working for them; they have to choose projects that can keep them all busy.

Outsiders are free of all this. They can work on small things, and there's something very pleasing about small things. Small things can be perfect; big ones always have something wrong with them. But there's a magic in small things that goes beyond such rational explanations. All kids know it. Small things have more personality.

Plus making them is more fun. You can do what you want; you don't have to satisfy committees. And perhaps most important, small things can be done fast. The prospect of seeing the finished project hangs in the air like the smell of dinner cooking. If you work fast, maybe you could have it done tonight.

Working on small things is also a good way to learn. The most important kinds of learning happen one project at a time. ("Next time, I won't...") The faster you cycle through projects, the faster you'll evolve.

Plain materials have a charm like small scale. And in addition there's the challenge of making do with less. Every designer's ears perk up at the mention of that game, because it's a game you can't lose. Like the JV playing the varsity, if you even tie, you win. So paradoxically there are cases where fewer resources yield better results, because the designers' pleasure at their own ingenuity more than compensates.⁵

So if you're an outsider, take advantage of your ability to make small and inexpensive things. Cultivate the pleasure and simplicity of that kind of work; one day you'll miss it.

Responsibility

When you're old and eminent, what will you miss about being young and obscure? What people seem to miss most is the lack of responsibilities.

Responsibility is an occupational disease of eminence. In principle you could avoid it, just as in principle you could avoid getting fat as you get old, but few do. I sometimes suspect that responsibility is a trap and that the most virtuous route would be to shirk it, but regardless it's certainly constraining.

⁵Managers are presumably wondering, how can I make this miracle happen? How can I make the people working for me do more with less? Unfortunately the constraint probably has to be self-imposed. If you're *expected* to do more with less, then you're being starved, not eating virtuously.

When you're an outsider you're constrained too, of course. You're short of money, for example. But that constrains you in different ways. How does responsibility constrain you? The worst thing is that it allows you not to focus on real work. Just as the most dangerous forms of procrastination are those that seem like work, the danger of responsibilities is not just that they can consume a whole day, but that they can do it without setting off the kind of alarms you'd set off if you spent a whole day sitting on a park bench.

A lot of the pain of being an outsider is being aware of one's own procrastination. But this is actually a good thing. You're at least close enough to work that the smell of it makes you hungry.

As an outsider, you're just one step away from getting things done. A huge step, admittedly, and one that most people never seem to make, but only one step. If you can summon up the energy to get started, you can work on projects with an intensity (in both senses) that few insiders can match. For insiders work turns into a duty, laden with responsibilities and expectations. It's never so pure as it was when they were young.

Work like a dog being taken for a walk, instead of an ox being yoked to the plow. That's what they miss.

Audience

A lot of outsiders make the mistake of doing the opposite; they admire the eminent so much that they copy even their flaws. Copying is a good way to learn, but copy the right things. When I was in college I imitated the pompous diction of famous professors. But this wasn't what *made* them eminent — it was more a flaw their eminence had allowed them to sink into. Imitating it was like pretending to have gout in order to seem rich.

Half the distinguishing qualities of the eminent are actually disadvantages. Imitating these is not only a waste of time, but will make you seem a fool to your models, who are often well aware of it.

What are the genuine advantages of being an insider? The greatest is an audience. It often seems to outsiders that the great advantage of insiders is money — that they have the resources to do what they want. But so do people who inherit money, and that doesn't seem to help, not as much as an audience. It's good for morale to know people want to see what you're making; it draws work out of you.

If I'm right that the defining advantage of insiders is an audience, then we live in exciting times, because just in the last ten years the Internet has made audiences a lot more liquid. Outsiders don't have to content themselves anymore with a proxy audience of a few smart friends. Now, thanks to the Internet, they can start to grow themselves actual audiences. This is great news for the marginal, who retain the advantages of outsiders while increasingly being able to siphon off what had till recently been the prerogative of the elite.

Though the Web has been around for more than ten years, I think we're just beginning to see its democratizing effects. Outsiders are still learning how to steal audiences. But more importantly, audiences are still learning how to be stolen — they're still just beginning to realize how much deeper bloggers can dig than journalists, how much more interesting a democratic news site can be than a front page controlled by editors, and how much funnier a bunch of kids with webcams can be than mass-produced sitcoms.

The big media companies shouldn't worry that people will post their copyrighted material on YouTube. They should worry that people will post their own stuff on YouTube, and audiences will watch that instead.

Hacking

If I had to condense the power of the marginal into one sentence it would be: just try hacking something together. That phrase draws in most threads I've mentioned here. Hacking something together means deciding what to do as you're doing it, not a subordinate executing the vision of his boss. It implies the result won't be pretty,

because it will be made quickly out of inadequate materials. It may work, but it won't be the sort of thing the eminent would want to put their name on. Something hacked together means something that barely solves the problem, or maybe doesn't solve the problem at all, but another you discovered en route. But that's ok, because the main value of that initial version is not the thing itself, but what it leads to. Insiders who daren't walk through the mud in their nice clothes will never make it to the solid ground on the other side.

The word "try" is an especially valuable component. I disagree here with Yoda, who said there is no try. There is try. It implies there's no punishment if you fail. You're driven by curiosity instead of duty. That means the wind of procrastination will be in your favor: instead of avoiding this work, this will be what you do as a way of avoiding other work. And when you do it, you'll be in a better mood. The more the work depends on imagination, the more that matters, because most people have more ideas when they're happy.

If I could go back and redo my twenties, that would be one thing I'd do more of: just try hacking things together. Like many people that age, I spent a lot of time worrying about what I should do. I also spent some time trying to build stuff. I should have spent less time worrying and more time building. If you're not sure what to do, make something.

Raymond Chandler's advice to thriller writers was "When in doubt, have a man come through a door with a gun in his hand." He followed that advice. Judging from his books, he was often in doubt. But though the result is occasionally cheesy, it's never boring. In life, as in books, action is underrated.

Fortunately the number of things you can just hack together keeps increasing. People fifty years ago would be astonished that one could just hack together a movie, for example. Now you can even hack together distribution. Just make stuff and put it online.

Inappropriate

If you really want to score big, the place to focus is the margin of the margin: the territories only recently captured from the insiders. That's where you'll find the juiciest projects still undone, either because they seemed too risky, or simply because there were too few insiders to explore everything.

This is why I spend most of my time writing essays lately. The writing of essays used to be limited to those who could get them published. In principle you could have written them and just shown them to your friends; in practice that didn't work.⁶ An essayist needs the resistance of an audience, just as an engraver needs the resistance of the plate.

Up till a few years ago, writing essays was the ultimate insider's game. Domain experts were allowed to publish essays about their field, but the pool allowed to write on general topics was about eight people who went to the right parties in New York. Now the reconquista has overrun this territory, and, not surprisingly, found it sparsely cultivated. There are so many essays yet unwritten. They tend to be the naughtier ones; the insiders have pretty much exhausted the motherhood and apple pie topics.

This leads to my final suggestion: a technique for determining when you're on the right track. You're on the right track when people complain that you're unqualified, or that you've done something inappropriate. If people are complaining, that means you're doing something rather than sitting around, which is the first step. And if they're driven to such empty forms of complaint, that means you've probably done something good.

If you make something and people complain that it doesn't *work*, that's a problem. But if the worst thing they can hit you with is your own status as an outsider, that implies that in every other respect you've succeeded. Pointing out that someone is unqualified is as desperate as

⁶Without the prospect of publication, the closest most people come to writing essays is to write in a journal. I find I never get as deeply into subjects as I do in proper essays. As the name implies, you don't go back and rewrite journal entries over and over for two weeks.

resorting to racial slurs. It's just a legitimate sounding way of saying: we don't like your type around here.

But the best thing of all is when people call what you're doing inappropriate. I've been hearing this word all my life and I only recently realized that it is, in fact, the sound of the homing beacon. "Inappropriate" is the null criticism. It's merely the adjective form of "I don't like it."

So that, I think, should be the highest goal for the marginal. Be inappropriate. When you hear people saying that, you're golden. And they, incidentally, are busted.

Thanks to Sam Altman, Trevor Blackwell, Paul Buchheit, Sarah Harlin, Jessica Livingston, Jackie McDonough, Robert Morris, Olin Shivers, and Chris Small for reading drafts of this, and to Chris Small and Chad Fowler for inviting me to speak.

Chapter 32

The Island Test



July 2006

I've discovered a handy test for figuring out what you're addicted to. Imagine you were going to spend the weekend at a friend's house on a little island off the coast of Maine. There are no shops on the island and you won't be able to leave while you're there. Also, you've never been to this house before, so you can't assume it will have more than any house might.

What, besides clothes and toiletries, do you make a point of packing? That's what you're addicted to. For example, if you find yourself packing a bottle of vodka (just in case), you may want to stop and think about that.

For me the list is four things: books, earplugs, a notebook, and a pen. There are other things I might bring if I thought of it, like music, or

tea, but I can live without them. I'm not so addicted to caffeine that I wouldn't risk the house not having any tea, just for a weekend.

Quiet is another matter. I realize it seems a bit eccentric to take earplugs on a trip to an island off the coast of Maine. If anywhere should be quiet, that should. But what if the person in the next room snored? What if there was a kid playing basketball? (Thump, thump, thump... thump.) Why risk it? Earplugs are small.

Sometimes I can think with noise. If I already have momentum on some project, I can work in noisy places. I can edit an essay or debug code in an airport. But airports are not so bad: most of the noise is whitish. I couldn't work with the sound of a sitcom coming through the wall, or a car in the street playing thump-thump music.

And of course there's another kind of thinking, when you're starting something new, that requires complete quiet. You never know when this will strike. It's just as well to carry plugs.

The notebook and pen are professional equipment, as it were. Though actually there is something druglike about them, in the sense that their main purpose is to make me feel better. I hardly ever go back and read stuff I write down in notebooks. It's just that if I can't write things down, worrying about remembering one idea gets in the way of having the next. Pen and paper wick ideas.

The best notebooks I've found are made by a company called Miquelrius. I use their smallest size, which is about 2.5 x 4 in. The secret to writing on such narrow pages is to break words only when you run out of space, like a Latin inscription. I use the cheapest plastic Bic ballpoints, partly because their gluey ink doesn't seep through pages, and partly so I don't worry about losing them.

I only started carrying a notebook about three years ago. Before that I used whatever scraps of paper I could find. But the problem with scraps of paper is that they're not ordered. In a notebook you can guess what a scribble means by looking at the pages around it. In the scrap era I was constantly finding notes I'd written years before that

might say something I needed to remember, if I could only figure out what.

As for books, I know the house would probably have something to read. On the average trip I bring four books and only read one of them, because I find new books to read en route. Really bringing books is insurance.

I realize this dependence on books is not entirely good—that what I need them for is distraction. The books I bring on trips are often quite virtuous, the sort of stuff that might be assigned reading in a college class. But I know my motives aren't virtuous. I bring books because if the world gets boring I need to be able to slip into another distilled by some writer. It's like eating jam when you know you should be eating fruit.

There is a point where I'll do without books. I was walking in some steep mountains once, and decided I'd rather just think, if I was bored, rather than carry a single unnecessary ounce. It wasn't so bad. I found I could entertain myself by having ideas instead of reading other people's. If you stop eating jam, fruit starts to taste better.

So maybe I'll try not bringing books on some future trip. They're going to have to pry the plugs out of my cold, dead ears, however.

Chapter 33

Copy What You Like



July 2006

When I was in high school I spent a lot of time imitating bad writers. What we studied in English classes was mostly fiction, so I assumed that was the highest form of writing. Mistake number one. The stories that seemed to be most admired were ones in which people suffered in complicated ways. Anything funny or gripping was ipso facto suspect, unless it was old enough to be hard to understand, like Shakespeare or Chaucer. Mistake number two. The ideal medium seemed the short story, which I've since learned had quite a brief life, roughly coincident with the peak of magazine publishing. But since their size made them perfect for use in high school classes, we read a lot of them, which gave us the impression the short story was flourishing. Mistake number three. And because they were so short, nothing really had to happen; you could just show a randomly truncated slice of life, and

that was considered advanced. Mistake number four. The result was that I wrote a lot of stories in which nothing happened except that someone was unhappy in a way that seemed deep.

For most of college I was a philosophy major. I was very impressed by the papers published in philosophy journals. They were so beautifully typeset, and their tone was just captivating—alternately casual and buffer-overflowingly technical. A fellow would be walking along a street and suddenly modality qua modality would spring upon him. I didn't ever quite understand these papers, but I figured I'd get around to that later, when I had time to reread them more closely. In the meantime I tried my best to imitate them. This was, I can now see, a doomed undertaking, because they weren't really saying anything. No philosopher ever refuted another, for example, because no one said anything definite enough to refute. Needless to say, my imitations didn't say anything either.

In grad school I was still wasting time imitating the wrong things. There was then a fashionable type of program called an expert system, at the core of which was something called an inference engine. I looked at what these things did and thought "I could write that in a thousand lines of code." And yet eminent professors were writing books about them, and startups were selling them for a year's salary a copy. What an opportunity, I thought; these impressive things seem easy to me; I must be pretty sharp. Wrong. It was simply a fad. The books the professors wrote about expert systems are now ignored. They were not even on a *path* to anything interesting. And the customers paying so much for them were largely the same government agencies that paid thousands for screwdrivers and toilet seats.

How do you avoid copying the wrong things? Copy only what you genuinely like. That would have saved me in all three cases. I didn't enjoy the short stories we had to read in English classes; I didn't learn anything from philosophy papers; I didn't use expert systems myself. I believed these things were good because they were admired.

It can be hard to separate the things you like from the things you're

impressed with. One trick is to ignore presentation. Whenever I see a painting impressively hung in a museum, I ask myself: how much would I pay for this if I found it at a garage sale, dirty and frameless, and with no idea who painted it? If you walk around a museum trying this experiment, you'll find you get some truly startling results. Don't ignore this data point just because it's an outlier.

Another way to figure out what you like is to look at what you enjoy as guilty pleasures. Many things people like, especially if they're young and ambitious, they like largely for the feeling of virtue in liking them. 99% of people reading *Ulysses* are thinking "I'm reading *Ulysses*" as they do it. A guilty pleasure is at least a pure one. What do you read when you don't feel up to being virtuous? What kind of book do you read and feel sad that there's only half of it left, instead of being impressed that you're half way through? That's what you really like.

Even when you find genuinely good things to copy, there's another pitfall to be avoided. Be careful to copy what makes them good, rather than their flaws. It's easy to be drawn into imitating flaws, because they're easier to see, and of course easier to copy too. For example, most painters in the eighteenth and nineteenth centuries used brownish colors. They were imitating the great painters of the Renaissance, whose paintings by that time were brown with dirt. Those paintings have since been cleaned, revealing brilliant colors; their imitators are of course still brown.

It was painting, incidentally, that cured me of copying the wrong things. Halfway through grad school I decided I wanted to try being a painter, and the art world was so manifestly corrupt that it snapped the leash of credulity. These people made philosophy professors seem as scrupulous as mathematicians. It was so clearly a choice of doing good work xor being an insider that I was forced to see the distinction. It's there to some degree in almost every field, but I had till then managed to avoid facing it.

That was one of the most valuable things I learned from painting: you have to figure out for yourself what's good. You can't trust authorities.

They'll lie to you on this one.

Chapter 34

How to Present to Investors



August 2006, rev. April 2007, September 2010

In a few days it will be Demo Day, when the startups we funded this summer present to investors. Y Combinator funds startups twice a year, in January and June. Ten weeks later we invite all the investors we know to hear them present what they've built so far.

Ten weeks is not much time. The average startup probably doesn't have much to show for itself after ten weeks. But the average startup fails. When you look at the ones that went on to do great things, you find a lot that began with someone pounding out a prototype in a week or two of nonstop work. Startups are a counterexample to the rule that haste makes waste.

(Too much money seems to be as bad for startups as too much time, so we don't give them much money either.)

A week before Demo Day, we have a dress rehearsal called Rehearsal Day. At other Y Combinator events we allow outside guests, but not at Rehearsal Day. No one except the other founders gets to see the rehearsals.

The presentations on Rehearsal Day are often pretty rough. But this is to be expected. We try to pick founders who are good at building things, not ones who are slick presenters. Some of the founders are just out of college, or even still in it, and have never spoken to a group of people they didn't already know.

So we concentrate on the basics. On Demo Day each startup will only get ten minutes, so we encourage them to focus on just two goals: (a) explain what you're doing, and (b) explain why users will want it.

That might sound easy, but it's not when the speakers have no experience presenting, and they're explaining technical matters to an audience that's mostly non-technical.

This situation is constantly repeated when startups present to investors: people who are bad at explaining, talking to people who are bad at understanding. Practically every successful startup, including stars like Google, presented at some point to investors who didn't get it and turned them down. Was it because the founders were bad at presenting, or because the investors were obtuse? It's probably always some of both.

At the most recent Rehearsal Day, we four Y Combinator partners found ourselves saying a lot of the same things we said at the last two. So at dinner afterward we collected all our tips about presenting to investors. Most startups face similar challenges, so we hope these will be useful to a wider audience.

1. Explain what you're doing.

Investors' main question when judging a very early startup is whether you've made a compelling product. Before they can judge whether you've built a good *x*, they have to understand what kind of *x* you've

built. They will get very frustrated if instead of telling them what you do, you make them sit through some kind of preamble.

Say what you're doing as soon as possible, preferably in the first sentence. "We're Jeff and Bob and we've built an easy to use web-based database. Now we'll show it to you and explain why people need this."

If you're a great public speaker you may be able to violate this rule. Last year one founder spent the whole first half of his talk on a fascinating analysis of the limits of the conventional desktop metaphor. He got away with it, but unless you're a captivating speaker, which most hackers aren't, it's better to play it safe.

2. Get rapidly to demo.

This section is now obsolete for YC founders presenting at Demo Day, because Demo Day presentations are now so short that they rarely include much if any demo. They seem to work just as well without, however, which makes me think I was wrong to emphasize demos so much before.

A demo explains what you've made more effectively than any verbal description. The only thing worth talking about first is the problem you're trying to solve and why it's important. But don't spend more than a tenth of your time on that. Then demo.

When you demo, don't run through a catalog of features. Instead start with the problem you're solving, and then show how your product solves it. Show features in an order driven by some kind of purpose, rather than the order in which they happen to appear on the screen.

If you're demoing something web-based, assume that the network connection will mysteriously die 30 seconds into your presentation, and come prepared with a copy of the server software running on your laptop.

3. Better a narrow description than a vague one.

One reason founders resist describing their projects concisely is that, at this early stage, there are all kinds of possibilities. The most concise

descriptions seem misleadingly narrow. So for example a group that has built an easy web-based database might resist calling their application that, because it could be so much more. In fact, it could be anything...

The problem is, as you approach (in the calculus sense) a description of something that could be anything, the content of your description approaches zero. If you describe your web-based database as “a system to allow people to collaboratively leverage the value of information,” it will go in one investor ear and out the other. They’ll just discard that sentence as meaningless boilerplate, and hope, with increasing impatience, that in the next sentence you’ll actually explain what you’ve made.

Your primary goal is not to describe everything your system might one day become, but simply to convince investors you’re worth talking to further. So approach this like an algorithm that gets the right answer by successive approximations. Begin with a description that’s gripping but perhaps overly narrow, then flesh it out to the extent you can. It’s the same principle as incremental development: start with a simple prototype, then add features, but at every point have working code. In this case, “working code” means a working description in the investor’s head.

4. Don’t talk and drive.

Have one person talk while another uses the computer. If the same person does both, they’ll inevitably mumble downwards at the computer screen instead of talking clearly at the audience.

As long as you’re standing near the audience and looking at them, politeness (and habit) compel them to pay attention to you. Once you stop looking at them to fuss with something on your computer, their minds drift off to the errands they have to run later.

5. Don’t talk about secondary matters at length.

If you only have a few minutes, spend them explaining what your prod-

uct does and why it's great. Second order issues like competitors or resumes should be single slides you go through quickly at the end. If you have impressive resumes, just flash them on the screen for 15 seconds and say a few words. For competitors, list the top 3 and explain in one sentence each what they lack that you have. And put this kind of thing at the end, after you've made it clear what you've built.

6. Don't get too deeply into business models.

It's good to talk about how you plan to make money, but mainly because it shows you care about that and have thought about it. Don't go into detail about your business model, because (a) that's not what smart investors care about in a brief presentation, and (b) any business model you have at this point is probably wrong anyway.

Recently a VC who came to speak at Y Combinator talked about a company he just invested in. He said their business model was wrong and would probably change three times before they got it right. The founders were experienced guys who'd done startups before and who'd just succeeded in getting millions from one of the top VC firms, and even their business model was crap. (And yet he invested anyway, because he expected it to be crap at this stage.)

If you're solving an important problem, you're going to sound a lot smarter talking about that than the business model. The business model is just a bunch of guesses, and guesses about stuff that's probably not your area of expertise. So don't spend your precious few minutes talking about crap when you could be talking about solid, interesting things you know a lot about: the problem you're solving and what you've built so far.

As well as being a bad use of time, if your business model seems spectacularly wrong, that will push the stuff you want investors to remember out of their heads. They'll just remember you as the company with the boneheaded plan for making money, rather than the company that solved that important problem.

7. Talk slowly and clearly at the audience.

Everyone at Rehearsal Day could see the difference between the people who'd been out in the world for a while and had presented to groups, and those who hadn't.

You need to use a completely different voice and manner talking to a roomful of people than you would in conversation. Everyday life gives you no practice in this. If you can't already do it, the best solution is to treat it as a consciously artificial trick, like juggling.

However, that doesn't mean you should talk like some kind of announcer. Audiences tune that out. What you need to do is talk in this artificial way, and yet make it seem conversational. (Writing is the same. Good writing is an elaborate effort to seem spontaneous.)

If you want to write out your whole presentation beforehand and memorize it, that's ok. That has worked for some groups in the past. But make sure to write something that sounds like spontaneous, informal speech, and deliver it that way too.

Err on the side of speaking slowly. At Rehearsal Day, one of the founders mentioned a rule actors use: if you feel you're speaking too slowly, you're speaking at about the right speed.

8. Have one person talk.

Startups often want to show that all the founders are equal partners. This is a good instinct; investors dislike unbalanced teams. But trying to show it by partitioning the presentation is going too far. It's distracting. You can demonstrate your respect for one another in more subtle ways. For example, when one of the groups presented at Demo Day, the more extroverted of the two founders did most of the talking, but he described his co-founder as the best hacker he'd ever met, and you could tell he meant it.

Pick the one or at most two best speakers, and have them do most of the talking.

Exception: If one of the founders is an expert in some specific technical field, it can be good for them to talk about that for a minute or so.

This kind of “expert witness” can add credibility, even if the audience doesn’t understand all the details. If Jobs and Wozniak had 10 minutes to present the Apple II, it might be a good plan to have Jobs speak for 9 minutes and have Woz speak for a minute in the middle about some of the technical feats he’d pulled off in the design. (Though of course if it were actually those two, Jobs would speak for the entire 10 minutes.)

9. Seem confident.

Between the brief time available and their lack of technical background, many in the audience will have a hard time evaluating what you’re doing. Probably the single biggest piece of evidence, initially, will be your own confidence in it. You have to show you’re impressed with what you’ve made.

And I mean show, not tell. Never say “we’re passionate” or “our product is great.” People just ignore that—or worse, write you off as bullshitters. Such messages must be implicit.

What you must not do is seem nervous and apologetic. If you’ve truly made something good, you’re doing investors a *favor* by telling them about it. If you don’t genuinely believe that, perhaps you ought to change what your company is doing. If you don’t believe your startup has such promise that you’d be doing them a favor by letting them invest, why are you investing your time in it?

10. Don’t try to seem more than you are.

Don’t worry if your company is just a few months old and doesn’t have an office yet, or your founders are technical people with no business experience. Google was like that once, and they turned out ok. Smart investors can see past such superficial flaws. They’re not looking for finished, smooth presentations. They’re looking for raw talent. All you need to convince them of is that you’re smart and that you’re onto something good. If you try too hard to conceal your rawness—by trying to seem corporate, or pretending to know about stuff you don’t—you may just conceal your talent.

You can afford to be candid about what you haven't figured out yet. Don't go out of your way to bring it up (e.g. by having a slide about what might go wrong), but don't try to pretend either that you're further along than you are. If you're a hacker and you're presenting to experienced investors, they're probably better at detecting bullshit than you are at producing it.

11. Don't put too many words on slides.

When there are a lot of words on a slide, people just skip reading it. So look at your slides and ask of each word "could I cross this out?" This includes gratuitous clip art. Try to get your slides under 20 words if you can.

Don't read your slides. They should be something in the background as you face the audience and talk to them, not something you face and read to an audience sitting behind you.

Cluttered sites don't do well in demos, especially when they're projected onto a screen. At the very least, crank up the font size big enough to make all the text legible. But cluttered sites are bad anyway, so perhaps you should use this opportunity to make your design simpler.

12. Specific numbers are good.

If you have any kind of data, however preliminary, tell the audience. Numbers stick in people's heads. If you can claim that the median visitor generates 12 page views, that's great.

But don't give them more than four or five numbers, and only give them numbers specific to you. You don't need to tell them the size of the market you're in. Who cares, really, if it's 500 million or 5 billion a year? Talking about that is like an actor at the beginning of his career telling his parents how much Tom Hanks makes. Yeah, sure, but first you have to become Tom Hanks. The important part is not whether he makes ten million a year or a hundred, but how you get there.

13. Tell stories about users.

The biggest fear of investors looking at early stage startups is that you've built something based on your own a priori theories of what the world needs, but that no one will actually want. So it's good if you can talk about problems specific users have and how you solve them.

Greg Mcadoo said one thing Sequoia looks for is the “proxy for demand.” What are people doing now, using inadequate tools, that shows they need what you're making?

Another sign of user need is when people pay a lot for something. It's easy to convince investors there will be demand for a cheaper alternative to something popular, if you preserve the qualities that made it popular.

The best stories about user needs are about your own. A remarkable number of famous startups grew out of some need the founders had: Apple, Microsoft, Yahoo, Google. Experienced investors know that, so stories of this type will get their attention. The next best thing is to talk about the needs of people you know personally, like your friends or siblings.

14. Make a soundbite stick in their heads.

Professional investors hear a lot of pitches. After a while they all blur together. The first cut is simply to be one of those they remember. And the way to ensure that is to create a descriptive phrase about yourself that sticks in their heads.

In Hollywood, these phrases seem to be of the form “x meets y.” In the startup world, they're usually “the x of y” or “the x y.” Viaweb's was “the Microsoft Word of ecommerce.”

Find one and launch it clearly (but apparently casually) in your talk, preferably near the beginning.

It's a good exercise for you, too, to sit down and try to figure out how to describe your startup in one compelling phrase. If you can't, your plans may not be sufficiently focused.

Chapter 35

A Student's Guide to Startups

October 2006

(This essay is derived from a talk at MIT.)

Till recently graduating seniors had two choices: get a job or go to grad school. I think there will increasingly be a third option: to start your own startup. But how common will that be?

I'm sure the default will always be to get a job, but starting a startup could well become as popular as grad school. In the late 90s my professor friends used to complain that they couldn't get grad students, because all the undergrads were going to work for startups. I wouldn't be surprised if that situation returns, but with one difference: this time they'll be starting their own instead of going to work for other people's.

The most ambitious students will at this point be asking: Why wait till you graduate? Why not start a startup while you're in college? In fact, why go to college at all? Why not start a startup instead?

A year and a half ago I gave a talk where I said that the average age of the founders of Yahoo, Google, and Microsoft was 24, and that if grad students could start startups, why not undergrads? I'm glad I phrased that as a question, because now I can pretend it wasn't merely a rhetorical one. At the time I couldn't imagine why there should

be any lower limit for the age of startup founders. Graduation is a bureaucratic change, not a biological one. And certainly there are undergrads as competent technically as most grad students. So why shouldn't undergrads be able to start startups as well as grad students?

I now realize that something does change at graduation: you lose a huge excuse for failing. Regardless of how complex your life is, you'll find that everyone else, including your family and friends, will discard all the low bits and regard you as having a single occupation at any given time. If you're in college and have a summer job writing software, you still read as a student. Whereas if you graduate and get a job programming, you'll be instantly regarded by everyone as a programmer.

The problem with starting a startup while you're still in school is that there's a built-in escape hatch. If you start a startup in the summer between your junior and senior year, it reads to everyone as a summer job. So if it goes nowhere, big deal; you return to school in the fall with all the other seniors; no one regards you as a failure, because your occupation is student, and you didn't fail at that. Whereas if you start a startup just one year later, after you graduate, as long as you're not accepted to grad school in the fall the startup reads to everyone as your occupation. You're now a startup founder, so you have to do well at that.

For nearly everyone, the opinion of one's peers is the most powerful motivator of all—more powerful even than the nominal goal of most startup founders, getting rich.¹ About a month into each funding cycle we have an event called Prototype Day where each startup presents to the others what they've got so far. You might think they wouldn't need any more motivation. They're working on their cool new idea; they have funding for the immediate future; and they're

¹Even the desire to protect one's children seems weaker, judging from things people have historically done to their kids rather than risk their community's disapproval. (I assume we still do things that will be regarded in the future as barbaric, but historical abuses are easier for us to see.)

playing a game with only two outcomes: wealth or failure. You'd think that would be motivation enough. And yet the prospect of a demo pushes most of them into a rush of activity.

Even if you start a startup explicitly to get rich, the money you might get seems pretty theoretical most of the time. What drives you day to day is not wanting to look bad.

You probably can't change that. Even if you could, I don't think you'd want to; someone who really, truly doesn't care what his peers think of him is probably a psychopath. So the best you can do is consider this force like a wind, and set up your boat accordingly. If you know your peers are going to push you in some direction, choose good peers, and position yourself so they push you in a direction you like.

Graduation changes the prevailing winds, and those make a difference. Starting a startup is so hard that it's a close call even for the ones that succeed. However high a startup may be flying now, it probably has a few leaves stuck in the landing gear from those trees it barely cleared at the end of the runway. In such a close game, the smallest increase in the forces against you can be enough to flick you over the edge into failure.

When we first started Y Combinator we encouraged people to start startups while they were still in college. That's partly because Y Combinator began as a kind of summer program. We've kept the program shape—all of us having dinner together once a week turns out to be a good idea—but we've decided now that the party line should be to tell people to wait till they graduate.

Does that mean you can't start a startup in college? Not at all. Sam Altman, the co-founder of Loopt, had just finished his sophomore year when we funded them, and Loopt is probably the most promising of all the startups we've funded so far. But Sam Altman is a very unusual guy. Within about three minutes of meeting him, I remember thinking "Ah, so this is what Bill Gates must have been like when he was 19."

If it can work to start a startup during college, why do we tell people

not to? For the same reason that the probably apocryphal violinist, whenever he was asked to judge someone's playing, would always say they didn't have enough talent to make it as a pro. Succeeding as a musician takes determination as well as talent, so this answer works out to be the right advice for everyone. The ones who are uncertain believe it and give up, and the ones who are sufficiently determined think "screw that, I'll succeed anyway."

So our official policy now is only to fund undergrads we can't talk out of it. And frankly, if you're not certain, you *should* wait. It's not as if all the opportunities to start companies are going to be gone if you don't do it now. Maybe the window will close on some idea you're working on, but that won't be the last idea you'll have. For every idea that times out, new ones become feasible. Historically the opportunities to start startups have only increased with time.

In that case, you might ask, why not wait longer? Why not go work for a while, or go to grad school, and then start a startup? And indeed, that might be a good idea. If I had to pick the sweet spot for startup founders, based on who we're most excited to see applications from, I'd say it's probably the mid-twenties. Why? What advantages does someone in their mid-twenties have over someone who's 21? And why isn't it older? What can 25 year olds do that 32 year olds can't? Those turn out to be questions worth examining.

Plus

If you start a startup soon after college, you'll be a young founder by present standards, so you should know what the relative advantages of young founders are. They're not what you might think. As a young founder your strengths are: stamina, poverty, rootlessness, colleagues, and ignorance.

The importance of stamina shouldn't be surprising. If you've heard anything about startups you've probably heard about the long hours. As far as I can tell these are universal. I can't think of any successful startups whose founders worked 9 to 5. And it's particularly necessary

for younger founders to work long hours because they're probably not as efficient as they'll be later.

Your second advantage, poverty, might not sound like an advantage, but it is a huge one. Poverty implies you can live cheaply, and this is critically important for startups. Nearly every startup that fails, fails by running out of money. It's a little misleading to put it this way, because there's usually some other underlying cause. But regardless of the source of your problems, a low burn rate gives you more opportunity to recover from them. And since most startups make all kinds of mistakes at first, room to recover from mistakes is a valuable thing to have.

Most startups end up doing something different than they planned. The way the successful ones find something that works is by trying things that don't. So the worst thing you can do in a startup is to have a rigid, pre-ordained plan and then start spending a lot of money to implement it. Better to operate cheaply and give your ideas time to evolve.

Recent grads can live on practically nothing, and this gives you an edge over older founders, because the main cost in software startups is people. The guys with kids and mortgages are at a real disadvantage. This is one reason I'd bet on the 25 year old over the 32 year old. The 32 year old probably is a better programmer, but probably also has a much more expensive life. Whereas a 25 year old has some work experience (more on that later) but can live as cheaply as an undergrad.

Robert Morris and I were 29 and 30 respectively when we started Viaweb, but fortunately we still lived like 23 year olds. We both had roughly zero assets. I would have loved to have a mortgage, since that would have meant I had a *house*. But in retrospect having nothing turned out to be convenient. I wasn't tied down and I was used to living cheaply.

Even more important than living cheaply, though, is thinking cheaply.

One reason the Apple II was so popular was that it was cheap. The computer itself was cheap, and it used cheap, off-the-shelf peripherals like a cassette tape recorder for data storage and a TV as a monitor. And you know why? Because Woz designed this computer for himself, and he couldn't afford anything more.

We benefitted from the same phenomenon. Our prices were daringly low for the time. The top level of service was \$300 a month, which was an order of magnitude below the norm. In retrospect this was a smart move, but we didn't do it because we were smart. \$300 a month seemed like a lot of money to us. Like Apple, we created something inexpensive, and therefore popular, simply because we were poor.

A lot of startups have that form: someone comes along and makes something for a tenth or a hundredth of what it used to cost, and the existing players can't follow because they don't even want to think about a world in which that's possible. Traditional long distance carriers, for example, didn't even want to think about VoIP. (It was coming, all the same.) Being poor helps in this game, because your own personal bias points in the same direction technology evolves in.

The advantages of rootlessness are similar to those of poverty. When you're young you're more mobile—not just because you don't have a house or much stuff, but also because you're less likely to have serious relationships. This turns out to be important, because a lot of startups involve someone moving.

The founders of Kiko, for example, are now en route to the Bay Area to start their next startup. It's a better place for what they want to do. And it was easy for them to decide to go, because neither as far as I know has a serious girlfriend, and everything they own will fit in one car—or more precisely, will either fit in one car or is crappy enough that they don't mind leaving it behind.

They at least were in Boston. What if they'd been in Nebraska, like Evan Williams was at their age? Someone wrote recently that the drawback of Y Combinator was that you had to move to participate.

It couldn't be any other way. The kind of conversations we have with founders, we have to have in person. We fund a dozen startups at a time, and we can't be in a dozen places at once. But even if we could somehow magically save people from moving, we wouldn't. We wouldn't be doing founders a favor by letting them stay in Nebraska. Places that aren't startup hubs are toxic to startups. You can tell that from indirect evidence. You can tell how hard it must be to start a startup in Houston or Chicago or Miami from the microscopically small number, per capita, that succeed there. I don't know exactly what's suppressing all the startups in these towns—probably a hundred subtle little things—but something must be.²

Maybe this will change. Maybe the increasing cheapness of startups will mean they'll be able to survive anywhere, instead of only in the most hospitable environments. Maybe 37signals is the pattern for the future. But maybe not. Historically there have always been certain towns that were centers for certain industries, and if you weren't in one of them you were at a disadvantage. So my guess is that 37signals is an anomaly. We're looking at a pattern much older than "Web 2.0" here.

Perhaps the reason more startups per capita happen in the Bay Area than Miami is simply that there are more founder-type people there. Successful startups are almost never started by one person. Usually they begin with a conversation in which someone mentions that something would be a good idea for a company, and his friend says, "Yeah, that is a good idea, let's try it." If you're missing that second person who says "let's try it," the startup never happens. And that is another area where undergrads have an edge. They're surrounded by people willing to say that. At a good college you're concentrated together with a lot of other ambitious and technically minded people—probably more concentrated than you'll ever be again. If your nucleus spits out a neutron, there's a good chance it will hit another nucleus.

²Worrying that Y Combinator makes founders move for 3 months also suggests one underestimates how hard it is to start a startup. You're going to have to put up with much greater inconveniences than that.

The number one question people ask us at Y Combinator is: Where can I find a co-founder? That's the biggest problem for someone starting a startup at 30. When they were in school they knew a lot of good co-founders, but by 30 they've either lost touch with them or these people are tied down by jobs they don't want to leave.

Viaweb was an anomaly in this respect too. Though we were comparatively old, we weren't tied down by impressive jobs. I was trying to be an artist, which is not very constraining, and Robert, though 29, was still in grad school due to a little interruption in his academic career back in 1988. So arguably the Worm made Viaweb possible. Otherwise Robert would have been a junior professor at that age, and he wouldn't have had time to work on crazy speculative projects with me.

Most of the questions people ask Y Combinator we have some kind of answer for, but not the co-founder question. There is no good answer. Co-founders really should be people you already know. And by far the best place to meet them is school. You have a large sample of smart people; you get to compare how they all perform on identical tasks; and everyone's life is pretty fluid. A lot of startups grow out of schools for this reason. Google, Yahoo, and Microsoft, among others, were all founded by people who met in school. (In Microsoft's case, it was high school.)

Many students feel they should wait and get a little more experience before they start a company. All other things being equal, they should. But all other things are not quite as equal as they look. Most students don't realize how rich they are in the scarcest ingredient in startups, co-founders. If you wait too long, you may find that your friends are now involved in some project they don't want to abandon. The better they are, the more likely this is to happen.

One way to mitigate this problem might be to actively plan your startup while you're getting those n years of experience. Sure, go off and get jobs or go to grad school or whatever, but get together regularly to scheme, so the idea of starting a startup stays alive in everyone's

brain. I don't know if this works, but it can't hurt to try.

It would be helpful just to realize what an advantage you have as students. Some of your classmates are probably going to be successful startup founders; at a great technical university, that is a near certainty. So which ones? If I were you I'd look for the people who are not just smart, but incurable builders. Look for the people who keep starting projects, and finish at least some of them. That's what we look for. Above all else, above academic credentials and even the idea you apply with, we look for people who build things.

The other place co-founders meet is at work. Fewer do than at school, but there are things you can do to improve the odds. The most important, obviously, is to work somewhere that has a lot of smart, young people. Another is to work for a company located in a startup hub. It will be easier to talk a co-worker into quitting with you in a place where startups are happening all around you.

You might also want to look at the employment agreement you sign when you get hired. Most will say that any ideas you think of while you're employed by the company belong to them. In practice it's hard for anyone to prove what ideas you had when, so the line gets drawn at code. If you're going to start a startup, don't write any of the code while you're still employed. Or at least discard any code you wrote while still employed and start over. It's not so much that your employer will find out and sue you. It won't come to that; investors or acquirers or (if you're so lucky) underwriters will nail you first. Between $t = 0$ and when you buy that yacht, *someone* is going to ask if any of your code legally belongs to anyone else, and you need to be able to say no.

3

³Most employee agreements say that any idea relating to the company's present or potential future business belongs to them. Often as not the second clause could include any possible startup, and anyone doing due diligence for an investor or acquirer will assume the worst.

To be safe either (a) don't use code written while you were still employed in your previous job, or (b) get your employer to renounce, in writing, any claim to the code you write for your side project. Many will consent to (b) rather than lose a prized

The most overreaching employee agreement I've seen so far is Amazon's. In addition to the usual clauses about owning your ideas, you also can't be a founder of a startup that has another founder who worked at Amazon—even if you didn't know them or even work there at the same time. I suspect they'd have a hard time enforcing this, but it's a bad sign they even try. There are plenty of other places to work; you may as well choose one that keeps more of your options open.

Speaking of cool places to work, there is of course Google. But I notice something slightly frightening about Google: zero startups come out of there. In that respect it's a black hole. People seem to like working at Google too much to leave. So if you hope to start a startup one day, the evidence so far suggests you shouldn't work there.

I realize this seems odd advice. If they make your life so good that you don't want to leave, why not work there? Because, in effect, you're probably getting a local maximum. You need a certain activation energy to start a startup. So an employer who's fairly pleasant to work for can lull you into staying indefinitely, even if it would be a net win for you to leave.⁴

The best place to work, if you want to start a startup, is probably a startup. In addition to being the right sort of experience, one way or another it will be over quickly. You'll either end up rich, in which case problem solved, or the startup will get bought, in which case it will start to suck to work there and it will be easy to leave, or most likely, the thing will blow up and you'll be free again.

Your final advantage, ignorance, may not sound very useful. I deliberately used a controversial word for it; you might equally call it innocence. But it seems to be a powerful force. My Y Combinator co-founder Jessica Livingston is just about to publish a book of interviews with startup founders, and I noticed a remarkable pattern in

employee. The downside is that you'll have to tell them exactly what your project does.

⁴Geshke and Warnock only founded Adobe because Xerox ignored them. If Xerox had used what they built, they would probably never have left PARC.

them. One after another said that if they'd known how hard it would be, they would have been too intimidated to start.

Ignorance can be useful when it's a counterweight to other forms of stupidity. It's useful in starting startups because you're capable of more than you realize. Starting startups is harder than you expect, but you're also capable of more than you expect, so they balance out.

Most people look at a company like Apple and think, how could I ever make such a thing? Apple is an institution, and I'm just a person. But every institution was at one point just a handful of people in a room deciding to start something. Institutions are made up, and made up by people no different from you.

I'm not saying everyone could start a startup. I'm sure most people couldn't; I don't know much about the population at large. When you get to groups I know well, like hackers, I can say more precisely. At the top schools, I'd guess as many as a quarter of the CS majors could make it as startup founders if they wanted.

That "if they wanted" is an important qualification—so important that it's almost cheating to append it like that—because once you get over a certain threshold of intelligence, which most CS majors at top schools are past, the deciding factor in whether you succeed as a founder is how much you want to. You don't have to be that smart. If you're not a genius, just start a startup in some unsexy field where you'll have less competition, like software for human resources departments. I picked that example at random, but I feel safe in predicting that whatever they have now, it wouldn't take genius to do better. There are a lot of people out there working on boring stuff who are desperately in need of better software, so however short you think you fall of Larry and Sergey, you can ratchet down the coolness of the idea far enough to compensate.

As well as preventing you from being intimidated, ignorance can sometimes help you discover new ideas. Steve Wozniak put this very strongly:

All the best things that I did at Apple came from (a) not having money and (b) not having done it before, ever. Every single thing that we came out with that was really great, I'd never once done that thing in my life.

When you know nothing, you have to reinvent stuff for yourself, and if you're smart your reinventions may be better than what preceded them. This is especially true in fields where the rules change. All our ideas about software were developed in a time when processors were slow, and memories and disks were tiny. Who knows what obsolete assumptions are embedded in the conventional wisdom? And the way these assumptions are going to get fixed is not by explicitly deallocating them, but by something more akin to garbage collection. Someone ignorant but smart will come along and reinvent everything, and in the process simply fail to reproduce certain existing ideas.

Minus

So much for the advantages of young founders. What about the disadvantages? I'm going to start with what goes wrong and try to trace it back to the root causes.

What goes wrong with young founders is that they build stuff that looks like class projects. It was only recently that we figured this out ourselves. We noticed a lot of similarities between the startups that seemed to be falling behind, but we couldn't figure out how to put it into words. Then finally we realized what it was: they were building class projects.

But what does that really mean? What's wrong with class projects? What's the difference between a class project and a real startup? If we could answer that question it would be useful not just to would-be startup founders but to students in general, because we'd be a long way toward explaining the mystery of the so-called real world.

There seem to be two big things missing in class projects: (1) an iterative definition of a real problem and (2) intensity.

The first is probably unavoidable. Class projects will inevitably solve fake problems. For one thing, real problems are rare and valuable. If a professor wanted to have students solve real problems, he'd face the same paradox as someone trying to give an example of whatever "paradigm" might succeed the Standard Model of physics. There may well be something that does, but if you could think of an example you'd be entitled to the Nobel Prize. Similarly, good new problems are not to be had for the asking.

In technology the difficulty is compounded by the fact that real startups tend to discover the problem they're solving by a process of evolution. Someone has an idea for something; they build it; and in doing so (and probably only by doing so) they realize the problem they should be solving is another one. Even if the professor let you change your project description on the fly, there isn't time enough to do that in a college class, or a market to supply evolutionary pressures. So class projects are mostly about implementation, which is the least of your problems in a startup.

It's not just that in a startup you work on the idea as well as implementation. The very implementation is different. Its main purpose is to refine the idea. Often the only value of most of the stuff you build in the first six months is that it proves your initial idea was mistaken. And that's extremely valuable. If you're free of a misconception that everyone else still shares, you're in a powerful position. But you're not thinking that way about a class project. Proving your initial plan was mistaken would just get you a bad grade. Instead of building stuff to throw away, you tend to want every line of code to go toward that final goal of showing you did a lot of work.

That leads to our second difference: the way class projects are measured. Professors will tend to judge you by the distance between the starting point and where you are now. If someone has achieved a lot, they should get a good grade. But customers will judge you from the other direction: the distance remaining between where you are now and the features they need. The market doesn't give a shit how hard

you worked. Users just want your software to do what they need, and you get a zero otherwise. That is one of the most distinctive differences between school and the real world: there is no reward for putting in a good effort. In fact, the whole concept of a “good effort” is a fake idea adults invented to encourage kids. It is not found in nature.

Such lies seem to be helpful to kids. But unfortunately when you graduate they don’t give you a list of all the lies they told you during your education. You have to get them beaten out of you by contact with the real world. And this is why so many jobs want work experience. I couldn’t understand that when I was in college. I knew how to program. In fact, I could tell I knew how to program better than most people doing it for a living. So what was this mysterious “work experience” and why did I need it?

Now I know what it is, and part of the confusion is grammatical. Describing it as “work experience” implies it’s like experience operating a certain kind of machine, or using a certain programming language. But really what work experience refers to is not some specific expertise, but the elimination of certain habits left over from childhood.

One of the defining qualities of kids is that they flake. When you’re a kid and you face some hard test, you can cry and say “I can’t” and they won’t make you do it. Of course, no one can make you do anything in the grownup world either. What they do instead is fire you. And when motivated by that you find you can do a lot more than you realized. So one of the things employers expect from someone with “work experience” is the elimination of the flake reflex—the ability to get things done, with no excuses.

The other thing you get from work experience is an understanding of what work is, and in particular, how intrinsically horrible it is. Fundamentally the equation is a brutal one: you have to spend most of your waking hours doing stuff someone else wants, or starve. There are a few places where the work is so interesting that this is concealed, because what other people want done happens to coincide with what you want to work on. But you only have to imagine what would hap-

pen if they diverged to see the underlying reality.

It's not so much that adults lie to kids about this as never explain it. They never explain what the deal is with money. You know from an early age that you'll have some sort of job, because everyone asks what you're going to "be" when you grow up. What they don't tell you is that as a kid you're sitting on the shoulders of someone else who's treading water, and that starting working means you get thrown into the water on your own, and have to start treading water yourself or sink. "Being" something is incidental; the immediate problem is not to drown.

The relationship between work and money tends to dawn on you only gradually. At least it did for me. One's first thought tends to be simply "This sucks. I'm in debt. Plus I have to get up on monday and go to work." Gradually you realize that these two things are as tightly connected as only a market can make them.

So the most important advantage 24 year old founders have over 20 year old founders is that they know what they're trying to avoid. To the average undergrad the idea of getting rich translates into buying Ferraris, or being admired. To someone who has learned from experience about the relationship between money and work, it translates to something way more important: it means you get to opt out of the brutal equation that governs the lives of 99.9% of people. Getting rich means you can stop treading water.

Someone who gets this will work much harder at making a startup succeed—with the proverbial energy of a drowning man, in fact. But understanding the relationship between money and work also changes the way you work. You don't get money just for working, but for doing things other people want. Someone who's figured that out will automatically focus more on the user. And that cures the other half of the class-project syndrome. After you've been working for a while, you yourself tend to measure what you've done the same way the market does.

Of course, you don't have to spend years working to learn this stuff. If you're sufficiently perceptive you can grasp these things while you're still in school. Sam Altman did. He must have, because Loopt is no class project. And as his example suggests, this can be valuable knowledge. At a minimum, if you get this stuff, you already have most of what you gain from the "work experience" employers consider so desirable. But of course if you really get it, you can use this information in a way that's more valuable to you than that.

Now

So suppose you think you might start a startup at some point, either when you graduate or a few years after. What should you do now? For both jobs and grad school, there are ways to prepare while you're in college. If you want to get a job when you graduate, you should get summer jobs at places you'd like to work. If you want to go to grad school, it will help to work on research projects as an undergrad. What's the equivalent for startups? How do you keep your options maximally open?

One thing you can do while you're still in school is to learn how startups work. Unfortunately that's not easy. Few if any colleges have classes about startups. There may be business school classes on entrepreneurship, as they call it over there, but these are likely to be a waste of time. Business schools like to talk about startups, but philosophically they're at the opposite end of the spectrum. Most books on startups also seem to be useless. I've looked at a few and none get it right. Books in most fields are written by people who know the subject from experience, but for startups there's a unique problem: by definition the founders of successful startups don't need to write books to make money. As a result most books on the subject end up being written by people who don't understand it.

So I'd be skeptical of classes and books. The way to learn about startups is by watching them in action, preferably by working at one. How do you do that as an undergrad? Probably by sneaking in through the back door. Just hang around a lot and gradually start doing things

for them. Most startups are (or should be) very cautious about hiring. Every hire increases the burn rate, and bad hires early on are hard to recover from. However, startups usually have a fairly informal atmosphere, and there's always a lot that needs to be done. If you just start doing stuff for them, many will be too busy to shoo you away. You can thus gradually work your way into their confidence, and maybe turn it into an official job later, or not, whichever you prefer. This won't work for all startups, but it would work for most I've known.

Number two, make the most of the great advantage of school: the wealth of co-founders. Look at the people around you and ask yourself which you'd like to work with. When you apply that test, you may find you get surprising results. You may find you'd prefer the quiet guy you've mostly ignored to someone who seems impressive but has an attitude to match. I'm not suggesting you suck up to people you don't really like because you think one day they'll be successful. Exactly the opposite, in fact: you should only start a startup with someone you like, because a startup will put your friendship through a stress test. I'm just saying you should think about who you really admire and hang out with them, instead of whoever circumstances throw you together with.

Another thing you can do is learn skills that will be useful to you in a startup. These may be different from the skills you'd learn to get a job. For example, thinking about getting a job will make you want to learn programming languages you think employers want, like Java and C++. Whereas if you start a startup, you get to pick the language, so you have to think about which will actually let you get the most done. If you use that test you might end up learning Ruby or Python instead.

But the most important skill for a startup founder isn't a programming technique. It's a knack for understanding users and figuring out how to give them what they want. I know I repeat this, but that's because it's so important. And it's a skill you can learn, though perhaps habit might be a better word. Get into the habit of thinking of software as

having users. What do those users want? What would make them say wow?

This is particularly valuable for undergrads, because the concept of users is missing from most college programming classes. The way you get taught programming in college would be like teaching writing as grammar, without mentioning that its purpose is to communicate something to an audience. Fortunately an audience for software is now only an http request away. So in addition to the programming you do for your classes, why not build some kind of website people will find useful? At the very least it will teach you how to write software with users. In the best case, it might not just be preparation for a startup, but the startup itself, like it was for Yahoo and Google.

Thanks to Jessica Livingston and Robert Morris for reading drafts of this, and to Jeff Arnold and the SIPB for inviting me to speak.